Axiom Repository Audit

FILE: package.json

```
{
  "name": "axiom",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint",
    "test": "node --import tsx --test src/lib/**/*.test.ts"
  },
  "dependencies": {
    "@supabase/auth-ui-react": "^0.4.7",
    "@supabase/auth-ui-shared": "^0.1.8",
    "@supabase/ssr": "^0.10.3",
    "@supabase/supabase-js": "^2.105.4",
    "framer-motion": "^12.40.0",
    "lenis": "^1.3.23",
    "lucide-react": "^1.17.0",
    "next": "16.2.6",
    "react": "19.2.4",
    "react-dom": "19.2.4",
    "recharts": "^3.8.1"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/d3-array": "^3.2.2",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "babel-plugin-react-compiler": "1.0.0",
    "eslint": "^9",
    "eslint-config-next": "16.2.6",
    "tailwindcss": "^4",
    "tsx": "^4.21.0",
    "typescript": "^5"
  }
}
```

FILE: tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next",
      },
```

```
    ],
    "paths": {
      "@/*": ["./src/*"],
    },
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts",
  ],
  "exclude": ["node_modules", "mobile"],
}
```

FILE: next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
  reactCompiler: true,
};

export default nextConfig;
```

FILE: postcss.config.mjs

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

export default config;
```

FILE: eslint.config.mjs

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
]);

export default eslintConfig;
```

Axiom Repository Audit

FILE: README.md

AXIOM

Axiom is a behavioral financial intelligence system designed to help users understand how capital is being deployed over time.

It is not a budgeting app.
It is not a passive expense tracker.

Axiom treats money as **deployable strategic capital**.

The system analyzes where capital flows, what behavioral patterns emerge, and whether those deployments are increasing or degrading long-term financial positioning.

? CORE PRINCIPLE

Financial events are truth. Everything else is interpretation.

Axiom is built around three strict layers:

Layer	Responsibility
Financial Truth	Immutable deployment records stored in Supabase
Derived Intelligence	Deterministic analytics (Burn, Runway, Allocation)
Interpretation Layer	Kairos behavioral insights (Strategic signals)

?? SYSTEM ARCHITECTURE (V2.4)

Axiom follows **Architecture A**:

```text
Next.js (UI + Server Actions)
        ?
Application Logic Layer (/lib)
        ?
Supabase (PostgreSQL + Auth)
        ?
Kairos Intelligence Layer
```

? ARCHITECTURAL PHILOSOPHY
The architecture exists to preserve **traceability, determinism, and explainability**.
- The frontend is purely for display and secure command entry.
- The database is the singular source of financial truth.
- Kairos interprets behavior but never mutates financial state.

? RETURN-BASED TAXONOMY SYSTEM

Axiom classifies every dollar by its **expected return profile**.

Category	Strategic Meaning
Asset	Expected future value generation (Equity, Crypto, Real Estate)
Skill	Improves future earning capability (Courses, Books, Coaching)
Leverage	Multiplies output or saves time (Software, Automation, Outsourcing)
Experience	Intentional quality-of-life deployment (Travel, Family)
Leakage	Non-strategic capital drift (Mindless spending without intent)

? LIQUIDITY MANAGEMENT
The system tracks **Total Liquidity** to drive the **Operational Runway** engine.
- **Total Liquidity** represents your total investable liquid capital (cash, near-cash assets).
- Users can update their "Truth" (starting balance) using the **"Set starting liquidity"** action in the header.
- Runway is calculated as: `Total Liquidity / Daily Burn Rate`.

Axiom Repository Audit

? PROJECT STRUCTURE
- `/app` ? Next.js routes + UI rendering
- `/lib/analytics` ? Deterministic metrics + calculations
- `/lib/finance` ? Taxonomy enforcement + validation
- `/lib/db` ? Server-side Supabase access layer
- `/lib/context` ? Strategic data compression for AI

? NON-NEGOTIABLE RULES
1. AI never writes financial truth.
2. Financial logic cannot exist in multiple places.
3. Frontend state is never authoritative.
4. Taxonomy integrity must remain enforced via the **Taxonomy Gate**.

Axiom v1: Stabilized, Coherent, and Strategically Aware. (Production Locked)

FILE: AGENTS.md

AGENTS.md ? AI Workflow & Methodology

This document instructs future AI coding agents on the specific methodologies and UX patterns required to build within the Axiom system.

1. "Build Backwards" Methodology
When implementing new features or insights, work from the end of the pipeline toward the source to ensure architectural alignment:
1. **Insight Registry**: Define the interpretation rules and thresholds in `registry.ts` (Interpretation).
2. **Context Layer**: Define the signals and behavioral markers in `lib/context` (Signals).
3. **Analytics Engine**: Implement the deterministic math and logic in `lib/analytics` (Math).
4. **Database/UI**: Finally, implement the schema changes and the user interface.

This ensures that UI components are always backed by a rigorous, testable analytical foundation.

2. "The Quiet System" UI Philosophy
- **Forgiving Interfaces**: Assume user failure or data gaps. Build elastic interfaces that handle missing telemetry or "Quiet System" states without breaking.
- **Immediate Revalidation**: After any server-side mutation, call `revalidatePath` immediately to drop the cache and ensure the UI reflects "The Source" (Supabase).
- **No Optimistic UI for Totals**: While UI interactions can be snappy, critical financial totals (Net Worth, Runway, Burn) MUST NEVER use optimistic updates. They must only update after a successful round-trip to the server to maintain "Financial Certainty."

3. Core Workflow Priorities
- **Correctness over Speed**: Every financial calculation must be verified against the deterministic analytics engine.
- **Traceability**: Every figure in the UI must be traceable back to a raw value in Supabase.
- **Security**: Always verify RLS and auth boundaries before touching a single line of data-access code.

FILE: CLAUDE.md

CLAUDE.md ? AXIOM Architectural Laws

This document defines the non-negotiable architectural laws of the Axiom system. All development must adhere to these 5 Pillars to prevent logic drift and maintain financial integrity.

1. The Database is Absolute Truth
- **Server-Side Authority**: All data mutations MUST occur through Next.js Server Actions.
- **Identity Enforcement**: Server Actions MUST explicitly call `requireAuth()` to derive the `userId`. Never trust a `userId` passed from the client.
- **RLS is Absolute**: Row Level Security (RLS) must be enabled on every table. Code must be written assuming RLS is the final line of defense.

2. Ledger Immutability

- **Hard Deletes are Banned**: No record representing a financial event or state may be permanently deleted.
- **Soft Delete Pattern**: All transactional tables MUST use a `deleted_at` timestamp column.
- **Read Consistency**: Every read query must filter for active records using `.is("deleted_at", null)`.

3. The Mapping Layer Pattern (Localization)
- **Abstract Schema**: Database enums and types must remain generic (e.g., `"loan"`, `"checking"`, `"mobile_money"`).
- **Taxonomy Translation**: The UI must never display raw database enums. It MUST intercept and translate these using the `taxonomy.ts` mapping dictionary (e.g., mapping `"loan"` to `"Fuliza / SACCO Loan"` based on context).
- **Currency Standard**: All financial values are strictly Kenyan Shillings (KES) and must be formatted accordingly.

4. AI Sandboxing (Kairos)
- **Read-Only Observer**: Kairos AI is a passive intelligence layer. It is strictly prohibited from mutating data or initiating transactions.
- **Deterministic Input**: Kairos must consume the `BehavioralContext` (derived from `analytics`) and map it through the `registry.ts` interpretation layer.
- **The Philosophy of Silence**: Kairos must gracefully handle the "Quiet System" state. If no rules are triggered, it remains silent. Silence is a valid and preferred state of certainty.

5. Telemetry & Observability
- **Non-Blocking Execution**: The engine must log execution latency, rule hits, and pipeline performance via the asynchronous telemetry service.
- **Asynchronous Logging**: Telemetry operations must never block the main UI thread or delay the delivery of financial figures to the user.

FILE: src\proxy.ts

```
import { type NextRequest, NextResponse } from "next/server";
import { updateSession } from "@/lib/supabase/middleware";
import { createServerClient } from "@supabase/ssr";

export async function proxy(request: NextRequest) {
  // Update the session (refreshes the token)
  const response = await updateSession(request);

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll() {
          // No-op for read-only session check
        },
      },
    },
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();

  // ROUTE PROTECTION

  // If user is not logged in and trying to access dashboard, redirect to home
  if (!user && request.nextUrl.pathname.startsWith("/dashboard")) {
    const url = request.nextUrl.clone();
    url.pathname = "/";
    const redirectResponse = NextResponse.redirect(url);

    // Copy all cookies from the session update response to the redirect response
    response.cookies.getAll().forEach((cookie) => {
```

```
      redirectResponse.cookies.set(cookie.name, cookie.value, cookie);
    });

    return redirectResponse;
  }

  // If user is logged in and trying to access home (login page), redirect to dashboard
  if (user && request.nextUrl.pathname === "/") {
    const url = request.nextUrl.clone();
    url.pathname = "/dashboard";
    const redirectResponse = NextResponse.redirect(url);

    // Copy all cookies from the session update response to the redirect response
    response.cookies.getAll().forEach((cookie) => {
      redirectResponse.cookies.set(cookie.name, cookie.value, cookie);
    });

    return redirectResponse;
  }

  return response;
}

export const config = {
  matcher: [
    /*
     * Match all request paths except for the ones starting with:
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico (favicon file)
     * Feel free to modify this pattern to include more paths.
     */
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)",
  ],
};
```

FILE: src\app\globals.css

```
@import "tailwindcss";

@theme inline {
    /* Core Colors */
    --color-background: var(--background);
    --color-foreground: var(--foreground);

    --color-card: var(--card);
    --color-card-foreground: var(--card-foreground);

    --color-popover: var(--popover);
    --color-popover-foreground: var(--popover-foreground);

    --color-primary: var(--primary);
    --color-primary-foreground: var(--primary-foreground);

    --color-secondary: var(--secondary);
    --color-secondary-foreground: var(--secondary-foreground);

    --color-muted: var(--muted);
    --color-muted-foreground: var(--muted-foreground);

    --color-accent: var(--accent);
    --color-accent-foreground: var(--accent-foreground);
```

```
    --color-destructive: var(--destructive);
    --color-destructive-foreground: var(--destructive-foreground);

    --color-border: var(--border);
    --color-input: var(--input);
    --color-ring: var(--ring);

    /* Functional Axiom Accents */
    --color-truth: var(--truth);
    --color-warning: var(--warning);
    --color-opportunity: var(--opportunity);
    --color-leakage: var(--leakage);

    /* Typography */
    --font-sans: var(--font-geist-sans);
    --font-mono: var(--font-geist-mono);

    /* Radius */
    --radius-none: 0px;
    --radius-xs: 0.125rem;
    --radius-default: 0.375rem;
    --radius-2xl: 1.5rem;
    --radius-3xl: 1.875rem;
    --radius-full: 9999px;

    /* Custom Utilities */
    --color-button-ripple-color: var(--button-ripple-color);
}

:root {
    /* Light Mode (Fallback/Future) */
    --background: hsl(0 0% 100%);
    --foreground: hsl(240 10% 3.9%);
    --card: hsl(0 0% 100%);
    --card-foreground: hsl(240 10% 3.9%);
    --popover: hsl(0 0% 100%);
    --popover-foreground: hsl(240 10% 3.9%);
    --primary: hsl(240 5.9% 10%);
    --primary-foreground: hsl(0 0% 98%);
    --secondary: hsl(240 4.8% 95.9%);
    --secondary-foreground: hsl(240 5.9% 10%);
    --muted: hsl(240 4.8% 95.9%);
    --muted-foreground: hsl(240 3.8% 46.1%);
    --accent: hsl(240 4.8% 95.9%);
    --accent-foreground: hsl(240 5.9% 10%);
    --destructive: hsl(0 84.2% 60.2%);
    --destructive-foreground: hsl(0 0% 98%);
    --border: hsl(240 5.9% 90%);
    --input: hsl(240 5.9% 90%);
    --ring: hsl(240 10% 3.9%);

    --truth: hsl(221.2 83.2% 53.3%);
    --warning: hsl(38 92% 50%);
    --opportunity: hsl(142.1 76.2% 36.3%);
    --leakage: hsl(0 84.2% 60.2%);

    --button-ripple-color: oklch(0.145 0 0 / 0.3);
}

.dark {
    /* Axiom Midnight - Deeper, more nuanced blacks */
    --background: hsl(240 10% 2%);
    --foreground: hsl(0 0% 98%);

    /* Layered Depth with subtle color tint */
```

```
    --card: hsl(240 10% 4%);
    --card-foreground: hsl(0 0% 98%);

    --popover: hsl(240 10% 3%);
    --popover-foreground: hsl(0 0% 98%);

    --primary: hsl(0 0% 100%);
    --primary-foreground: hsl(240 10% 2%);

    --secondary: hsl(240 5% 10%);
    --secondary-foreground: hsl(0 0% 98%);

    --muted: hsl(240 5% 10%);
    --muted-foreground: hsl(240 5% 75%);

    --accent: hsl(240 5% 20%);
    --accent-foreground: hsl(0 0% 100%);

    --destructive: hsl(0 84% 45%);
    --destructive-foreground: hsl(0 0% 98%);

    --border: hsl(240 5% 12%);
    --input: hsl(240 5% 12%);
    --ring: hsl(240 10% 90%);

    /* Axiom Functional Lighting - More vibrant in dark mode */
    --truth: hsl(210 100% 60%);
    --warning: hsl(35 100% 55%);
    --opportunity: hsl(155 100% 50%);
    --leakage: hsl(0 100% 60%);

    --button-ripple-color: oklch(0.985 0 0 / 0.4);
}

body {
    background-color: var(--background);
    color: var(--foreground);
    font-family: var(--font-sans), ui-sans-serif, system-ui, sans-serif;
    -webkit-font-smoothing: antialiased;
}

/* Axiom Advanced UI Utilities */
@utility bg-noise {
       background-image:  url("data:image/svg+xml,%3Csvg  viewBox='0  0  200  200'  xmlns='http://www.w3.org/2000/svg'%3%3Ffilter
id='noiseFilter'%3E%3FfeTurbulence             type='fractalNoise'             baseFrequency='0.65'             numOctaves='3'
stitchTiles='stitch'/%3E%3C/filter%3E%3Crect width='100%25' height='100%25' filter='url(%23noiseFilter)'/%3E%3C/svg%3E");
    opacity: 0.03;
}

@utility bg-grid-white {
    background-image: radial-gradient(circle, rgba(255, 255, 255, 0.05) 1px, transparent 1px);
    background-size: 24px 24px;
}

/* Range Inputs */
input[type="range"] {
    -webkit-appearance: none;
    appearance: none;
    background: transparent;
    cursor: pointer;
    height: 32px;
}

/* Recharts / SVG Fixes */
svg stop {
```

```
    stop-color: currentColor;
}

.recharts-cartesian-axis-tick-value {
    fill: var(--muted-foreground);
    font-family: var(--font-mono);
    font-size: 9px;
}

input[type="number"]::-webkit-inner-spin-button,
input[type="number"]::-webkit-outer-spin-button {
    -webkit-appearance: none;
    margin: 0;
}

input[type="number"] {
    -moz-appearance: textfield;
    appearance: textfield;
}

input[type="range"]::-webkit-slider-runnable-track {
    background: var(--border);
    height: 2px;
    border-radius: 9999px;
}

input[type="range"]::-webkit-slider-thumb {
    -webkit-appearance: none;
    appearance: none;
    margin-top: -11px;
    background-color: var(--foreground);
    height: 24px;
    width: 24px;
    border-radius: 9999px;
    box-shadow: 0 0 20px rgba(255, 255, 255, 0.1);
    transition:
        transform 0.2s ease,
        box-shadow 0.2s ease;
}

input[type="range"]:active::-webkit-slider-thumb {
    transform: scale(1.1);
    box-shadow: 0 0 30px rgba(255, 255, 255, 0.2);
}
```

FILE: src\app\layout.tsx

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "AXIOM // Strategic Wealth Intelligence",
```

```
  description: "Bespoke Liquidity Architecture & Structural Solvency Engine",
};

import { SmoothScrollProvider } from "@/components/ui/smooth-scroll-provider";

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} h-full antialiased dark`}
    >
      <body className="min-h-full flex flex-col bg-black text-foreground selection:bg-white selection:text-black relative">
        {/* Global Texture Overlays */}
        <div className="fixed inset-0 pointer-events-none z-50 bg-noise mix-blend-overlay opacity-[0.03]" />
        <div className="fixed inset-0 pointer-events-none z-0 bg-grid-white opacity-[0.02]" />

        <SmoothScrollProvider>{children}</SmoothScrollProvider>
      </body>
    </html>
  );
}
```

FILE: src\app\page.tsx

```
"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";
import { supabase } from "@/lib/supabase";
import { LandingTerminal } from "@/components/landing/LandingTerminal";
import { SolvencyCalculator } from "@/components/landing/SolvencyCalculator";
import { StrategicPillars } from "@/components/landing/StrategicPillars";
import { ProcessFlow } from "@/components/landing/ProcessFlow";
import { BrandMark } from "@/components/ui/brand-mark";
import { Zap, Activity, Cpu, ShieldCheck } from "lucide-react";

export default function LandingPage() {
  const router = useRouter();
  const [isScrolled, setIsScrolled] = useState(false);

  useEffect(() => {
    const handleScroll = () => {
      setIsScrolled(window.scrollY > 20);
    };

    window.addEventListener("scroll", handleScroll);

    const {
      data: { subscription },
    } = supabase.auth.onAuthStateChange((event, session) => {
      if (session) {
        router.push("/dashboard");
      }
    });

    return () => {
      window.removeEventListener("scroll", handleScroll);
      subscription.unsubscribe();
```

```
    };
  }, [router]);

  return (
      <main className="bg-[#000000] text-foreground min-h-screen selection:bg-white selection:text-black overflow-x-hidden antialiased font-mono">
      {/* Editorial Navigation - Clinical & Functional */}
      <nav
         className={`fixed top-0 left-0 w-full z-50 px-4 md:px-8 flex justify-between items-center transition-all duration-300 ${
          isScrolled
            ? "py-4 bg-black/90 border-b border-white/10 backdrop-blur-md"
            : "py-6 bg-transparent"
        }`}
      >
        <div className="flex items-center gap-3 md:gap-4 cursor-pointer">
          <BrandMark className="w-8 h-8 md:w-10 md:h-10 text-white" />
          <div className="flex flex-col">
            <span className="font-bold text-[10px] tracking-[0.6em] uppercase text-white">
              AXIOM
            </span>
            <span className="text-[7px] tracking-[0.3em] uppercase text-zinc-500">
              Sovereign Intelligence Unit
            </span>
          </div>
        </div>
        <div className="flex gap-4 items-center">
          <Link href="/signup">
                  <button className="px-6 py-2 border border-white/10 text-[10px] tracking-widest uppercase hover:bg-white hover:text-black transition-all flex items-center gap-2">
              <Zap size={12} />
              Initialize
            </button>
          </Link>
        </div>
      </nav>

      {/* ASSEMBLY */}
      <div className="relative z-10 pt-32">
        {/* HERO TERMINAL SECTION */}
        <section className="min-h-[80vh] flex flex-col justify-center items-center px-6 text-center space-y-16">
          <div className="space-y-6">
            <div className="inline-flex items-center gap-4 text-zinc-500">
              <div className="h-px w-8 bg-zinc-800" />
              <span className="text-[9px] tracking-[0.8em] uppercase">
                System Status: Operational
              </span>
              <div className="h-px w-8 bg-zinc-800" />
            </div>
            <h1 className="text-4xl md:text-7xl font-bold tracking-tighter text-white uppercase leading-none">
              Deterministic <br />
              <span className="text-zinc-500">Wealth Architecture.</span>
            </h1>
            <p className="max-w-xl mx-auto text-zinc-400 text-sm md:text-base tracking-tight leading-relaxed">
              Clinical financial intelligence for elite operators.
              Secure your structural solvency with sovereign precision.
            </p>
          </div>

          <LandingTerminal />

          <div className="flex flex-col md:flex-row gap-4">
            <Link href="/signup">
                        <button className="px-12 py-4 bg-white text-black font-bold text-[10px] tracking-widest uppercase hover:bg-zinc-200 transition-all">
```

```
                Establish Baseline
              </button>
            </Link>
            <button
              onClick={() => document.getElementById("architecture")?.scrollIntoView({ behavior: "smooth" })}
                    className="px-12 py-4 border border-white/10 text-white font-bold text-[10px] tracking-widest uppercase
hover:bg-white/5 transition-all"
            >
              System Specs ?
            </button>
          </div>
        </section>

        <div id="architecture">
          <SolvencyCalculator />
          <StrategicPillars />
          <ProcessFlow />
        </div>

        {/* FINAL CONVERSION SECTION - CLINICAL */}
        <section className="py-40 md:py-60 px-6 text-center bg-black border-t border-white/5">
          <div className="max-w-4xl mx-auto space-y-12">
            <div className="space-y-4">
              <span className="text-[9px] tracking-[1em] text-zinc-500 uppercase">
                End of Transmission
              </span>
              <h2 className="text-4xl md:text-6xl font-bold text-white uppercase tracking-tighter">
                Deploy Your <br />
                <span className="text-zinc-600">Sovereign Protocol.</span>
              </h2>
            </div>

            <Link href="/signup">
                      <button  className="px-16  py-6  bg-white  text-black  font-bold  text-[11px]  tracking-[0.2em]  uppercase
hover:bg-zinc-200 transition-all">
                Initialize Access Portal
              </button>
            </Link>
          </div>
        </section>

        {/* TERMINAL FOOTER */}
        <footer className="py-20 px-6 md:px-24 border-t border-white/10 bg-black">
          <div className="max-w-7xl mx-auto grid grid-cols-1 md:grid-cols-3 gap-16">
            <div className="space-y-8">
              <div className="flex items-center gap-4">
                <BrandMark className="w-8 h-8 text-white" />
                <span className="text-xs tracking-[0.5em] uppercase font-bold text-white">
                  AXIOM
                </span>
              </div>
              <div className="space-y-4 text-zinc-500 text-[11px] leading-relaxed uppercase tracking-tight">
                <p>Node: Nairobi // Instance: 2026.06.12</p>
                <p>Deterministic Financial Intelligence for the Sovereign Individual.</p>
              </div>
            </div>

            <div className="space-y-6">
              <h4 className="text-[10px] tracking-[0.4em] uppercase text-white font-bold">
                Protocols
              </h4>
              <div className="flex flex-col gap-4 text-[10px] tracking-widest uppercase text-zinc-600">
                <Link href="#" className="hover:text-white transition-colors">Methodology</Link>
                <Link href="#" className="hover:text-white transition-colors">Taxonomy</Link>
                <Link href="#" className="hover:text-white transition-colors">Risk Engine</Link>
```

```
            </div>
          </div>

          <div className="space-y-6">
            <h4 className="text-[10px] tracking-[0.4em] uppercase text-white font-bold">
              Verification
            </h4>
            <div className="flex flex-col gap-4 text-[10px] tracking-widest uppercase text-zinc-600">
              <Link href="#" className="hover:text-white transition-colors">Privacy</Link>
              <Link href="#" className="hover:text-white transition-colors">Terms</Link>
              <Link href="#" className="hover:text-white transition-colors">Audit log</Link>
            </div>
          </div>
        </div>
           <div className="mt-20 pt-8 border-t border-white/5 flex flex-col md:flex-row justify-between items-center gap-4
text-[9px] tracking-[0.3em] uppercase text-zinc-700 font-mono">
          <span>© 2026 Axiom Strategic Units</span>
          <div className="flex gap-8">
            <span className="flex items-center gap-2"><Activity size={10} /> Network: Stable</span>
            <span className="flex items-center gap-2"><ShieldCheck size={10} /> Encryption: active</span>
            <span className="flex items-center gap-2"><Cpu size={10} /> Engine: Kairos v1.1</span>
          </div>
        </div>
      </footer>
    </div>
  </main>
 );
}
```

FILE: src\app\auth\auth-code-error\page.tsx

```
"use client";

import Link from "next/link";

export default function AuthCodeError() {
 return (
  <main className="min-h-screen flex items-center justify-center bg-background text-foreground p-6">
   <div className="w-full max-w-md bg-background border-2 border-red-500/20 rounded-[2.5rem] p-10 shadow-2xl text-center">
    <div className="w-20 h-20 bg-red-500/10 rounded-3xl flex items-center justify-center mx-auto mb-6">
     <svg
      width="40"
      height="40"
      viewBox="0 0 24 24"
      fill="none"
      stroke="currentColor"
      strokeWidth="3"
      className="text-red-500"
     >
      <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z" />
      <line x1="12" y1="9" x2="12" y2="13" />
      <line x1="12" y1="17" x2="12.01" y2="17" />
     </svg>
    </div>

    <h1 className="text-3xl font-black tracking-tighter uppercase mb-4">
     Auth Failed
    </h1>

    <p className="text-gray-500 text-sm font-bold uppercase tracking-widest mb-8 leading-relaxed">
     The secure handshake with the authentication provider was interrupted.
    </p>
```

```
        <Link
          href="/"
                classNameName="w-full bg-foreground text-background px-4 py-5 rounded-2xl font-black text-lg hover:scale-[1.02]
active:scale-[0.98] transition-all shadow-xl shadow-foreground/10 block uppercase text-center"
        >
          Return to Login
        </Link>
      </div>
    </main>
  );
}
```

FILE: src\app\auth\callback\route.ts

```
import { NextResponse } from "next/server";
// The client you created in Step 1
import { createClient } from "@/lib/supabase/server";

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url);
  const code = searchParams.get("code");
  // if "next" is in search params, use it as the redirection URL
  const next = searchParams.get("next") ?? "/dashboard";

  if (code) {
    const supabase = await createClient();
    const { error } = await supabase.auth.exchangeCodeForSession(code);
    if (!error) {
      return NextResponse.redirect(`${origin}${next}`);
    }
  }

  // return the user to an error page with instructions
  return NextResponse.redirect(`${origin}/auth/auth-code-error`);
}
```

FILE: src\app\dashboard\AccountSection.tsx

```
"use client";

import { useState, useSyncExternalStore } from "react";
import { ACCOUNT_TYPES, type Account } from "@/lib/finance/accounts";
import { createAccountAction } from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { AccountMap } from "@/lib/utils/taxonomy";
import { DistributionPieChart } from "@/components/dashboard/MiniCharts";
import { ScrollReveal } from "@/components/ui/scroll-reveal";
import { Wallet } from "lucide-react";

interface AccountSectionProps {
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

const emptySubscribe = () => () => {};

export function AccountSection({ accounts, onSnapshot }: AccountSectionProps) {
  const isClient = useSyncExternalStore(
    emptySubscribe,
    () => true,
```

```
    () => false,
  );

  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [sourceForm, setSourceForm] = useState({
    account_name: "",
    account_type: "checking",
    current_balance: "",
    institution: "",
  });

  const chartData = accounts.map((a) => ({
    name: a.account_name,
    value: Number(a.current_balance),
  }));

  async function handleSourceSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createAccountAction({
        account_name: sourceForm.account_name,
        account_type: sourceForm.account_type,
        current_balance: Number(sourceForm.current_balance),
        institution: sourceForm.institution || undefined,
      });
      setSourceForm({
        account_name: "",
        account_type: "checking",
        current_balance: "",
        institution: "",
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(err instanceof Error ? err.message : "Failed to create source");
    } finally {
      setIsLoading(false);
    }
  }

  if (!isClient) return null;

  return (
    <div className="space-y-12">
      <div className="flex items-center justify-between">
        <div className="flex items-center gap-4">
          <Wallet strokeWidth={1.5} size={20} className="text-truth" />
          <h2 className="font-mono text-xl text-foreground tracking-widest uppercase">
            Accounts
          </h2>
        </div>
        <button
          onClick={() => setIsAdding(!isAdding)}
                  className="font-mono  text-[9px]  tracking-[0.4em]  uppercase  text-muted-foreground/80  hover:text-foreground
transition-colors"
        >
          {isAdding ? "? CANCEL" : "+ APPEND ACCOUNT"}
        </button>
      </div>
      {isAdding && (
```

```
        <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4 duration-500">
          <form onSubmit={handleSourceSubmit} className="space-y-8">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Account Name
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. M-Pesa, Savings"
                  value={sourceForm.account_name}
                  onChange={(e) =>
                    setSourceForm({
                      ...sourceForm,
                      account_name: e.target.value,
                    })
                  }
                                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-sm
focus:outline-none focus:border-white transition-colors uppercase tracking-wider"
                />
              </div>
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Current Liquidity
                </label>
                <input
                  type="number"
                  required
                  placeholder="0.00"
                  value={sourceForm.current_balance}
                  onChange={(e) =>
                    setSourceForm({
                      ...sourceForm,
                      current_balance: e.target.value,
                    })
                  }
                                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-lg
focus:outline-none focus:border-white transition-colors tabular-nums"
                />
              </div>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Account Type
                </label>
                <select
                  value={sourceForm.account_type}
                  onChange={(e) =>
                    setSourceForm({
                      ...sourceForm,
                      account_type: e.target.value,
                    })
                  }
                        className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                >
                  {ACCOUNT_TYPES.map((t) => (
                    <option key={t.value} value={t.value} className="bg-black">
                      {t.label}
                    </option>
                  ))}
                </select>
```

```
              </div>
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Financial Institution
                </label>
                <input
                  type="text"
                  placeholder="Optional"
                  value={sourceForm.institution}
                  onChange={(e) =>
                    setSourceForm({
                      ...sourceForm,
                      institution: e.target.value,
                    })
                  }
                              className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-sm
focus:outline-none focus:border-white transition-colors uppercase tracking-wider"
                />
              </div>
            </div>

            {error && (
              <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
               className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px] hover:bg-white/90
transition-colors disabled:opacity-50"
            >
              {isLoading ? "INITIALIZING..." : "CONFIRM ACCOUNT"}
            </button>
          </form>
        </div>
      )}

      {accounts.length > 0 && (
        <div className="py-6 border-b border-white/5 mb-6">
          <DistributionPieChart data={chartData} />
        </div>
      )}

      <div className="space-y-2">
        {accounts.length === 0 ? (
          <div className="py-12 text-center opacity-20">
            <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
              No accounts active
            </p>
          </div>
        ) : (
          accounts.map((account, index) => (
            <ScrollReveal key={account.id} delay={index * 0.05} distance={10}>
                      <div className="flex items-center justify-between py-6 border-b border-white/5 group hover:bg-white/1
transition-all px-2">
                <div className="space-y-1">
                  <span className="text-[8px] font-mono tracking-widest uppercase text-muted-foreground/60">
                    {AccountMap[account.account_type] || account.account_type}
                  </span>
                                <h3 className="font-mono text-sm uppercase tracking-wider text-white transition-transform
group-hover:translate-x-2">
                    {account.account_name}
                  </h3>
```

```
                </div>
                <div className="text-right">
                  <p className="font-mono text-lg text-white tabular-nums">
                    {formatCurrency(account.current_balance)}
                  </p>
                  {account.institution && (
                    <p className="text-[8px] font-mono text-muted-foreground/50 uppercase tracking-widest mt-1">
                      {account.institution}
                    </p>
                  )}
                </div>
              </div>
            </ScrollReveal>
          ))
        )}
      </div>
    </div>
  );
}
```

FILE: src\app\dashboard\actions.ts

```
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";
import {
  createDeployment,
  updateDeployment,
  deleteDeployment,
} from "@/lib/db/deployments";
import { createAccount, updateAccount, deleteAccount } from "@/lib/db/accounts";
import {
  createLiability,
  updateLiability,
  deleteLiability,
} from "@/lib/db/liabilities";
import {
  createIncomeStreams,
  updateIncomeStream,
  deleteIncomeStream,
} from "@/lib/db/income";
import { createGoal, updateGoal, deleteGoal } from "@/lib/db/goals";
import {
  createStrategicObjective,
  updateStrategicObjective,
  deleteStrategicObjective,
} from "@/lib/db/objectives";
import {
  createOperationalBaseline,
  updateOperationalBaseline,
  deleteOperationalBaseline,
} from "@/lib/db/baseline";
import { updateLiquidity } from "@/lib/db/settings";
import { LiquidityService } from "@/lib/finance/liquidity";
import { getDeletedLedgerRecords } from "@/lib/db/audit";
import { saveInsight } from "@/lib/db/insights";
import { SnapshotService } from "@/lib/services/snapshot";
import type { DashboardSnapshot } from "@/lib/dashboard/types";
import type {
  OperationalBaseline,
  BaselineCadence,
} from "@/lib/analytics/types";
```

```
import type { DeploymentAdvancedContextInput } from "@/lib/finance/deploymentContext";

export interface CreateDeploymentActionInput {
  title: string;
  amount: number;
  category: string;
  advancedContext?: DeploymentAdvancedContextInput;
  accountId?: string;
}

export interface UpdateDeploymentActionInput {
  title: string;
  amount: number;
  category: string;
  accountId?: string;
}

export interface CreateAccountActionInput {
  account_name: string;
  account_type: string;
  current_balance: number;
  institution?: string;
}

export interface CreateLiabilityActionInput {
  liability_name: string;
  liability_type: string;
  outstanding_balance: number;
  interest_rate?: number;
  minimum_payment?: number;
  due_date?: string | null;
  institution?: string;
  is_paid_in_cadences?: boolean;
  cadence?: string | null;
  cadence_day_date?: string | null;
  cadence_amount?: number | null;
}

export interface CreateIncomeActionInput {
  income_name: string;
  income_type: string;
  amount: number;
  cadence: string;
  execution_day?: number | null;
  is_recurring?: boolean;
  source?: string;
  start_date?: string;
  end_date?: string | null;
}

export interface CreateGoalActionInput {
  goal_name: string;
  goal_type: string;
  target_amount: number;
  current_progress?: number;
  target_date?: string | null;
  priority: string;
  status: string;
  notes?: string;
}

export interface CreateObjectiveActionInput {
  objective_name: string;
  objective_type: string;
  target_amount: number;
```

```
  current_amount?: number;
  target_date?: string | null;
  priority_level: string;
  status: string;
  notes?: string;
}

const requireAuth = async () => {
  const supabase = await createClient();
  const {
    data: { user },
    error,
  } = await supabase.auth.getUser();
  if (error || !user) throw new Error("Unauthorized mutation attempt.");
  return { userId: user.id, supabase };
};

export async function getDashboardSnapshotAction() {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) return SnapshotService.unauthenticated();

  const { snapshot } = await SnapshotService.getSnapshot(supabase, user.id);
  return snapshot;
}

export async function checkOnboardingStatusAction() {
  const { userId, supabase } = await requireAuth();

  const [accounts, baseline] = await Promise.all([
    supabase
      .from("accounts")
      .select("id", { count: "exact", head: true })
      .eq("user_id", userId),
    supabase
      .from("operational_baseline")
      .select("id", { count: "exact", head: true })
      .eq("user_id", userId),
  ]);

  return {
    isOnboarded: (accounts.count ?? 0) > 0 || (baseline.count ?? 0) > 0,
  };
}

export async function createDeploymentAction(
  input: CreateDeploymentActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Create the deployment record (The database trigger handles account balance deduction)
    await createDeployment(
      supabase,
      input.title,
      input.amount,
      userId,
      input.category,
      0,
      input.advancedContext,
      input.accountId,
    );
```

```
    // 2. Synchronize global liquidity (The database trigger handles account balance deduction)
    if (input.accountId) {
      await LiquidityService.sync(supabase, userId);
    }

    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to create deployment:", error);
    throw error;
  }
}

export async function updateDeploymentAction(
  id: string,
  input: UpdateDeploymentActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateDeployment(supabase, id, userId, {
      title: input.title,
      amount: input.amount,
      category: input.category,
    });
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to update deployment:", error);
    throw error;
  }
}

export async function deleteDeploymentAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteDeployment(supabase, id, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );
```

```
    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to delete deployment:", error);
    throw error;
  }
}

export async function updateLiquidityAction(amount: number) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateLiquidity(supabase, userId, amount);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to update liquidity:", error);
    throw error;
  }
}

export async function createAccountAction(input: CreateAccountActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createAccount(supabase, userId, input);
    await LiquidityService.sync(supabase, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to create account:", error);
    throw error;
  }
}

export async function updateAccountAction(
  id: string,
  input: Partial<CreateAccountActionInput>,
) {
  const { userId, supabase } = await requireAuth();
```

```
  try {
    await updateAccount(supabase, id, userId, input);
    await LiquidityService.sync(supabase, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to update account:", error);
    throw error;
  }
}

export async function deleteAccountAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteAccount(supabase, id, userId);
    await LiquidityService.sync(supabase, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to delete account:", error);
    throw error;
  }
}

export async function createLiabilityAction(input: CreateLiabilityActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createLiability(supabase, userId, input);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to create liability:", error);
    throw error;
```

```
  }
}

export async function updateLiabilityAction(
  id: string,
  input: Partial<CreateLiabilityActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateLiability(supabase, id, userId, input);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to update liability:", error);
    throw error;
  }
}

export async function deleteLiabilityAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteLiability(supabase, id, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to delete liability:", error);
    throw error;
  }
}

export async function createIncomeAction(inputs: CreateIncomeActionInput[]) {
  const { userId, supabase } = await requireAuth();

  try {
    await createIncomeStreams(supabase, userId, inputs);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
```

```
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to create income streams:", error);
    throw error;
  }
}

export async function updateIncomeAction(
  id: string,
  input: Partial<CreateIncomeActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateIncomeStream(supabase, id, userId, input);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to update income stream:", error);
    throw error;
  }
}

export async function deleteIncomeAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteIncomeStream(supabase, id, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to delete income stream:", error);
    throw error;
  }
}

export async function resolvePendingInflowAction(payload: {
  incomeId: string;
  accountId: string;
  amount: number;
}) {
  const { userId, supabase } = await requireAuth();
```

```
  try {
    // 1. Get current account balance
    const { data: account, error: accError } = await supabase
      .from("accounts")
      .select("current_balance")
      .eq("id", payload.accountId)
      .eq("user_id", userId)
      .single();

    if (accError || !account) throw new Error("Target account not found.");

    // 2. Update the target account balance
    const newBalance = Number(account.current_balance) + payload.amount;
    await updateAccount(supabase, payload.accountId, userId, {
      current_balance: newBalance,
    });

    // 3. Update the income stream last_executed_at
    await updateIncomeStream(supabase, payload.incomeId, userId, {
      last_executed_at: new Date().toISOString(),
    });

    // 4. Synchronize global liquidity
    await LiquidityService.sync(supabase, userId);

    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to resolve pending income:", error);
    throw error;
  }
}

export async function resolvePendingLiabilityAction(payload: {
  liabilityId: string;
  accountId: string;
  amount: number;
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Get current account and liability state
    const [accountRes, liabRes] = await Promise.all([
      supabase
        .from("accounts")
        .select("current_balance")
        .eq("id", payload.accountId)
        .eq("user_id", userId)
        .single(),
      supabase
        .from("liabilities")
        .select("outstanding_balance")
        .eq("id", payload.liabilityId)
        .eq("user_id", userId)
```

```
        .single(),
    ]);

    if (accountRes.error || !accountRes.data)
      throw new Error("Source account not found.");
    if (liabRes.error || !liabRes.data)
      throw new Error("Liability record not found.");

    // 2. Update balances
    const newAccBalance =
      Number(accountRes.data.current_balance) - payload.amount;
    const newLiabBalance = Math.max(
      0,
      Number(liabRes.data.outstanding_balance) - payload.amount,
    );

    await Promise.all([
      updateAccount(supabase, payload.accountId, userId, {
        current_balance: newAccBalance,
      }),
      updateLiability(supabase, payload.liabilityId, userId, {
        outstanding_balance: newLiabBalance,
        last_executed_at: new Date().toISOString(),
      }),
    ]);

    // 3. Synchronize global liquidity
    await LiquidityService.sync(supabase, userId);

    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to resolve pending liability:", error);
    throw error;
  }
}

export async function resolvePendingBaselineAction(payload: {
  baselineId: string;
  accountId: string;
  amount: number;
  title: string;
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Get current account balance
    const { data: account, error: accError } = await supabase
      .from("accounts")
      .select("current_balance")
      .eq("id", payload.accountId)
      .eq("user_id", userId)
      .single();

    if (accError || !account) throw new Error("Source account not found.");
```

```
    // 2. Deduct from account
    const newBalance = Number(account.current_balance) - payload.amount;
    await updateAccount(supabase, payload.accountId, userId, {
      current_balance: newBalance,
    });

    // 3. Log Deployment (Maintenance Expense)
    await createDeployment(
      supabase,
      payload.title,
      payload.amount,
      userId,
      "Maintenance",
      0,
      undefined,
      payload.accountId,
    );

    // 4. Update the baseline last_executed_at
    await updateOperationalBaseline(supabase, payload.baselineId, userId, {
      last_executed_at: new Date().toISOString(),
    });

    // 5. Synchronize global liquidity
    await LiquidityService.sync(supabase, userId);

    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to resolve pending baseline:", error);
    throw error;
  }
}

export async function createGoalAction(input: CreateGoalActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createGoal(supabase, userId, input);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to create goal:", error);
    throw error;
```

```
  }
}

export async function updateGoalAction(
  id: string,
  input: Partial<CreateGoalActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateGoal(supabase, id, userId, input);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to update goal:", error);
    throw error;
  }
}

export async function deleteGoalAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteGoal(supabase, id, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to delete goal:", error);
    throw error;
  }
}

export async function createStrategicObjectiveAction(
  input: CreateObjectiveActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await createStrategicObjective(supabase, userId, input);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );
```

```
    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to create strategic objective:", error);
    throw error;
  }
}

export const createObjectiveAction = createStrategicObjectiveAction;

export async function updateStrategicObjectiveAction(
  id: string,
  input: Partial<CreateObjectiveActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateStrategicObjective(supabase, id, userId, input);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to update strategic objective:", error);
    throw error;
  }
}

export const updateObjectiveAction = updateStrategicObjectiveAction;

export async function deleteStrategicObjectiveAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteStrategicObjective(supabase, id, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to delete strategic objective:", error);
    throw error;
  }
}
```

```
export const deleteObjectiveAction = deleteStrategicObjectiveAction;

export async function createOperationalBaselineAction(
  data: Omit<
    OperationalBaseline,
    "id" | "user_id" | "created_at" | "updated_at"
  >,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await createOperationalBaseline(supabase, userId, data);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to create operational baseline:", error);
    throw error;
  }
}

export async function updateOperationalBaselineAction(
  id: string,
  updates: Partial<
    Omit<OperationalBaseline, "id" | "user_id" | "created_at" | "updated_at">
  >,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateOperationalBaseline(supabase, id, userId, updates);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to update operational baseline:", error);
    throw error;
  }
}

export async function deleteOperationalBaselineAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteOperationalBaseline(supabase, id, userId);
    revalidatePath("/dashboard");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
```

```
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("Failed to delete operational baseline:", error);
    throw error;
  }
}

export async function submitDayZeroOnboardingAction(payload: {
  accounts: {
    account_name: string;
    account_type: string;
    current_balance: string;
    institution?: string;
  }[];
  incomes: {
    income_name: string;
    income_type: string;
    amount: string;
    cadence: string;
    execution_day: string;
    is_recurring: boolean;
    source?: string;
  }[];
  liabilities: {
    liability_name: string;
    liability_type: string;
    outstanding_balance: string;
    interest_rate: string;
    institution?: string;
    is_paid_in_cadences: boolean;
    cadence: string;
    cadence_day_date: string;
    cadence_amount: string;
  }[];
  baselines: {
    title: string;
    amount: string;
    cadence: string;
    execution_day: string;
    is_recurring: boolean;
    category: string;
  }[];
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Insert Accounts
    for (const acc of payload.accounts) {
      await createAccount(supabase, userId, {
        ...acc,
        current_balance: Number(acc.current_balance),
      });
    }

    // 2. Insert Incomes (if any)
    if (payload.incomes.length > 0) {
```

```
  await createIncomeStreams(
    supabase,
    userId,
    payload.incomes.map((inc) => ({
      income_name: inc.income_name,
      income_type: inc.income_type,
      amount: Number(inc.amount),
      cadence: inc.is_recurring ? inc.cadence : "irregular",
      execution_day:
        inc.is_recurring &&
        (inc.cadence === "monthly" ||
          inc.cadence === "weekly" ||
          inc.cadence === "biweekly")
          ? Number(inc.execution_day)
          : null,
      is_recurring: inc.is_recurring,
      source: inc.source || undefined,
    })),
  );
}

// 3. Insert Liabilities (if any)
for (const liab of payload.liabilities) {
  const balance = Number(liab.outstanding_balance);
  if (balance > 0) {
    await createLiability(supabase, userId, {
      liability_name: liab.liability_name,
      liability_type: liab.liability_type,
      outstanding_balance: balance,
      interest_rate: Number(liab.interest_rate) || 0,
      institution: liab.institution || undefined,
      is_paid_in_cadences: liab.is_paid_in_cadences,
      cadence: liab.is_paid_in_cadences ? liab.cadence : null,
      cadence_day_date: liab.is_paid_in_cadences
        ? String(liab.cadence_day_date)
        : null,
      cadence_amount: liab.is_paid_in_cadences
        ? Number(liab.cadence_amount)
        : null,
    });
  }
}

// 4. Insert Operational Baselines
for (const item of payload.baselines) {
  await createOperationalBaseline(supabase, userId, {
    title: item.title || "Core Survival Baseline",
    amount: Number(item.amount),
    category: item.category || "Maintenance",
    cadence: (item.is_recurring
      ? item.cadence
      : "monthly") as BaselineCadence,
    execution_day:
      item.is_recurring &&
      (item.cadence === "monthly" ||
        item.cadence === "weekly" ||
        item.cadence === "biweekly")
        ? Number(item.execution_day)
        : null,
    is_recurring: item.is_recurring,
    baseline_type: "expense",
    is_active: true,
  });
}
```

```
    // 5. Synchronize initial liquidity pool with account balances
    await LiquidityService.sync(supabase, userId);

    revalidatePath("/dashboard", "layout");
    const { snapshot, volatileState } = await SnapshotService.getSnapshot(
      supabase,
      userId,
      { forceInsightEvaluation: true },
    );

    if (volatileState.requiresPersistence && snapshot.kairosInsight) {
      await saveInsight(supabase, snapshot.kairosInsight, userId);
    }

    return snapshot;
  } catch (error) {
    console.error("[Onboarding] Submission failed:", error);
    throw error;
  }
}

export const submitDayZeroBaselineAction = submitDayZeroOnboardingAction;

export async function fetchHistoricalAuditAction() {
  const { userId, supabase } = await requireAuth();
  return getDeletedLedgerRecords(supabase, userId);
}

export interface AuditLog {
  id: string;
  rule_id: string;
  severity: string;
  evaluation_time_ms: number;
  created_at: string;
}

export interface TelemetrySummary {
  logs: AuditLog[];
  averageLatency: number;
  matchRate: number;
  totalCycles: number;
  ruleHits: Record<string, number>;
}

export async function fetchTelemetryLogsAction(): Promise<TelemetrySummary> {
  const { userId, supabase } = await requireAuth();

  const { data, error } = await supabase
    .from("kairos_insights")
    .select("*")
    .eq("user_id", userId)
    .order("created_at", { ascending: false })
    .limit(50);

  if (error) {
    console.error("[Telemetry] Fetch failed:", error.message);
    return {
      logs: [],
      averageLatency: 0,
      matchRate: 0,
      totalCycles: 0,
      ruleHits: {},
    };
  }
```

```
  const logs = (data || []) as AuditLog[];
  const totalLogs = logs.length;

  if (totalLogs === 0) {
    return {
      logs: [],
      averageLatency: 0,
      matchRate: 0,
      totalCycles: 0,
      ruleHits: {},
    };
  }

  const totalLatency = logs.reduce(
    (sum, log) => sum + (log.evaluation_time_ms || 0),
    0,
  );
  const averageLatency = totalLatency / totalLogs;

  const matches = logs.filter((log) => log.severity !== "none").length;
  const matchRate = (matches / totalLogs) * 100;

  const ruleHits: Record<string, number> = {};
  logs.forEach((log) => {
    if (log.severity && log.severity !== "none") {
      ruleHits[log.rule_id] = (ruleHits[log.rule_id] || 0) + 1;
    }
  });

  return {
    logs,
    averageLatency: Math.round(averageLatency * 100) / 100,
    matchRate: Math.round(matchRate * 10) / 10,
    totalCycles: totalLogs,
    ruleHits,
  };
}
```

FILE: src\app\dashboard\BaselineSection.tsx

```
"use client";

import { useState } from "react";
import { TAXONOMY_CATEGORIES } from "@/lib/finance/taxonomy";
import { createOperationalBaselineAction } from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { OperationalBaseline, BaselineCadence } from "@/lib/analytics/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { DistributionPieChart } from "@/components/dashboard/MiniCharts";
import { ScrollReveal } from "@/components/ui/scroll-reveal";

interface BaselineSectionProps {
  baseline: OperationalBaseline[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

const CADENCES = [
  { value: "daily", label: "Daily" },
  { value: "weekly", label: "Weekly" },
  { value: "biweekly", label: "Bi-weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "quarterly", label: "Quarterly" },
  { value: "yearly", label: "Yearly" },
```

```
];

export function BaselineSection({
  baseline,
  onSnapshot,
}: BaselineSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const chartData = baseline.map((b) => ({
    name: b.title,
    value: Number(b.amount),
  }));

  const [form, setForm] = useState({
    title: "",
    amount: "",
    category: "Maintenance",
    cadence: "monthly" as BaselineCadence,
    execution_day: 1,
    is_recurring: true,
    baseline_type: "expense" as "expense" | "allocation",
    notes: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createOperationalBaselineAction({
        title: form.title,
        amount: Number(form.amount),
        category: form.category,
          cadence: form.is_recurring ? form.cadence : "monthly", // Default or special irregular? Plan says use is_recurring
logic.
        execution_day:
          form.is_recurring &&
          (form.cadence === "monthly" ||
            form.cadence === "weekly" ||
            form.cadence === "biweekly")
            ? Number(form.execution_day)
            : null,
        is_recurring: form.is_recurring,
        baseline_type: form.baseline_type,
        is_active: true,
        notes: form.notes || undefined,
      });
      setForm({
        title: "",
        amount: "",
        category: "Maintenance",
        cadence: "monthly",
        execution_day: 1,
        is_recurring: true,
        baseline_type: "expense",
        notes: "",
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error ? err.message : "Failed to create baseline entry",
      );
```

```
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-12">
      <div className="flex items-center justify-between">
        <h2 className="font-mono text-xl text-white tracking-widest uppercase">
          Operational Maintenance
        </h2>
        <button
          onClick={() => setIsAdding(!isAdding)}
                    className="font-mono  text-[9px]  tracking-[0.4em]  uppercase  text-muted-foreground/80  hover:text-white
transition-colors"
        >
          {isAdding ? "? CANCEL" : "+ APPEND RECURRING"}
        </button>
      </div>

      {isAdding && (
        <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4 duration-500">
          <form onSubmit={handleSubmit} className="space-y-8">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Baseline Name
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. Rent, Netflix"
                  value={form.title}
                  onChange={(e) => setForm({ ...form, title: e.target.value })}
                              className="w-full  bg-transparent  border-b  border-white/10  py-3  text-white  font-mono  text-sm
focus:outline-none focus:border-white transition-colors uppercase tracking-wider"
                />
              </div>
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Amount
                </label>
                <input
                  type="number"
                  required
                  placeholder="0.00"
                  value={form.amount}
                  onChange={(e) => setForm({ ...form, amount: e.target.value })}
                              className="w-full  bg-transparent  border-b  border-white/10  py-3  text-white  font-mono  text-lg
focus:outline-none focus:border-white transition-colors tabular-nums"
                />
              </div>
            </div>

            <div className="space-y-6">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Category
                </label>
                <select
                  value={form.category}
                  onChange={(e) =>
                    setForm({ ...form, category: e.target.value })
                  }
                          className="w-full  bg-transparent  border-b  border-white/10  py-3  text-white/60  font-mono  text-[10px]
```

```
tracking-widest uppercase focus:outline-none"
                >
                  {TAXONOMY_CATEGORIES.map((cat) => (
                    <option
                      key={cat.value}
                      value={cat.value}
                      className="bg-black"
                    >
                      {cat.label}
                    </option>
                  ))}
                </select>
              </div>

              <div className="space-y-4">
                <label className="flex items-center space-x-3 cursor-pointer group">
                  <input
                    type="checkbox"
                    checked={form.is_recurring}
                    onChange={(e) =>
                      setForm({ ...form, is_recurring: e.target.checked })
                    }
                                 className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white transition-colors
cursor-pointer"
                  />
                                   <span  className="text-[10px]  font-mono  text-muted-foreground/80  uppercase  tracking-widest
group-hover:text-white/60 transition-colors">
                    Recurring Expense
                  </span>
                </label>

                {form.is_recurring && (
                  <div className="space-y-8 pt-4 animate-in fade-in slide-in-from-top-2 border-l border-white/5 pl-6 ml-2">
                    <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
                      <div className="space-y-2">
                        <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                          Frequency
                        </label>
                        <select
                          value={form.cadence}
                          onChange={(e) =>
                            setForm({
                              ...form,
                              cadence: e.target.value as BaselineCadence,
                            })
                          }
                            className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                        >
                          {CADENCES.map((c) => (
                            <option
                              key={c.value}
                              value={c.value}
                              className="bg-black"
                            >
                              {c.label}
                            </option>
                          ))}
                        </select>
                      </div>

                      {form.cadence === "daily" && (
                        <div className="flex items-center pt-4">
                                      <p  className="text-[10px]  font-mono  text-muted-foreground/80  uppercase  tracking-widest
leading-relaxed">
```

```
                <span className="text-white/60">Note:</span> You
                will be prompted everyday to verify this baseline.
              </p>
            </div>
          )}

          {(form.cadence === "monthly" ||
            form.cadence === "weekly" ||
            form.cadence === "biweekly") && (
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                {form.cadence === "monthly"
                  ? "Day of Month (1-31)"
                  : "Day of Week"}
              </label>
              {form.cadence === "monthly" ? (
                <input
                  type="number"
                  min="1"
                  max="31"
                  required
                  value={form.execution_day || ""}
                  onChange={(e) =>
                    setForm({
                      ...form,
                      execution_day: Number(e.target.value),
                    })
                  }
                                  className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
                />
              ) : (
                <select
                  value={form.execution_day || 1}
                  onChange={(e) =>
                    setForm({
                      ...form,
                      execution_day: Number(e.target.value),
                    })
                  }
                            className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
                >
                  {[
                    { label: "Monday", value: 1 },
                    { label: "Tuesday", value: 2 },
                    { label: "Wednesday", value: 3 },
                    { label: "Thursday", value: 4 },
                    { label: "Friday", value: 5 },
                    { label: "Saturday", value: 6 },
                    { label: "Sunday", value: 7 },
                  ].map((day) => (
                    <option
                      key={day.value}
                      value={day.value}
                      className="bg-black"
                    >
                      {day.label}
                    </option>
                  ))}
                </select>
              )}
            </div>
          )}
        </div>
```

```
                </div>
              )}
            </div>
          </div>

          {error && (
            <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
              {error}
            </p>
          )}

          <button
            type="submit"
            disabled={isLoading}
             className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px] hover:bg-white/90
transition-colors disabled:opacity-50"
          >
            {isLoading ? "SAVING..." : "CONFIRM RECURRING"}
          </button>
        </form>
      </div>
    )}

    {baseline.length > 0 && (
      <div className="py-6 border-b border-white/5 mb-6">
        <DistributionPieChart data={chartData} />
      </div>
    )}

    <div className="space-y-2">
      {baseline.length === 0 ? (
        <div className="py-12 text-center opacity-20">
          <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
            No recurring flows active
          </p>
        </div>
      ) : (
        baseline.map((item, index) => (
          <ScrollReveal key={item.id} delay={item.execution_day ? (index * 0.05) : index * 0.05} distance={10}>
                    <div  className="flex  items-center  justify-between  py-6  border-b  border-white/5  group  hover:bg-white/1
transition-all px-2">
              <div className="space-y-1">
                <div className="flex items-center gap-3">
                  <span className="text-[8px] font-mono tracking-widest uppercase text-muted-foreground/60">
                    {item.cadence}
                  </span>
                  {item.execution_day && (
                    <span className="text-[8px] font-mono tracking-widest uppercase text-muted-foreground/50">
                      ? Day {item.execution_day}
                    </span>
                  )}
                </div>
                                <h3  className="font-mono  text-sm  uppercase  tracking-wider  text-white  transition-transform
group-hover:translate-x-2">
                  {item.title}
                </h3>
              </div>
              <div className="text-right">
                <p className="font-mono text-lg text-white tabular-nums">
                  {formatCurrency(item.amount)}
                </p>
              </div>
            </div>
          </ScrollReveal>
        ))
```

```
        )}
      </div>
    </div>
  );
}
```

FILE: src\app\dashboard\DashboardClient.tsx

```
"use client";

import { useState, useCallback } from "react";
import { createDeploymentAction } from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { AccountSection } from "./AccountSection";
import { LiabilitySection } from "./LiabilitySection";
import { IncomeSection } from "./IncomeSection";
import { GoalSection } from "./GoalSection";
import { StrategicObjectiveSection } from "./StrategicObjectiveSection";
import { BaselineSection } from "./BaselineSection";
import { motion } from "framer-motion";
import { PendingInflows } from "./PendingInflows";
import { PendingLiabilities } from "./PendingLiabilities";
import { PendingBaselines } from "./PendingBaselines";
import { HistoricalAudit } from "./HistoricalAudit";
import { KairosNarrative } from "@/components/dashboard/KairosNarrative";
import {
  MiniSparkline,
  MiniBarChart,
  MiniDonut,
} from "@/components/dashboard/MiniCharts";
import { AnimatedNumber } from "@/components/ui/animated-number";
import { TelemetryDashboard } from "@/components/dashboard/TelemetryDashboard";
import { NewEntryForm } from "./NewEntryForm";
import {
  ShieldCheck,
  Waves,
  Flame,
  Timer,
  Cpu,
  Brain,
  Zap,
  Hexagon,
  Activity,
  Database,
  RefreshCw
} from "lucide-react";
import type {
  AnalyticsSummary,
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
} from "@/lib/analytics/types";
import type { KairosInsight } from "@/lib/ai/kairos";
import { getDashboardSnapshotAction } from "./actions";

interface LedgerState {
  deployments: Deployment[];
  accounts: Account[];
  liabilities: Liability[];
```

```
  incomeStreams: IncomeStream[];
  goals: FinancialGoal[];
  objectives: StrategicObjective[];
  baseline: OperationalBaseline[];
  analytics: AnalyticsSummary | null;
}

interface DashboardClientProps {
  initialSnapshot: DashboardSnapshot;
}

export function DashboardClient({ initialSnapshot }: DashboardClientProps) {
  const [ledger, setLedger] = useState<LedgerState>({
    deployments: initialSnapshot.deployments,
    accounts: initialSnapshot.accounts,
    liabilities: initialSnapshot.liabilities,
    incomeStreams: initialSnapshot.incomeStreams,
    goals: initialSnapshot.goals,
    objectives: initialSnapshot.objectives,
    baseline: initialSnapshot.baseline,
    analytics: initialSnapshot.analytics,
  });

  const [kairosInsight, setKairosInsight] = useState<KairosInsight | null>(
    initialSnapshot.kairosInsight,
  );
  const [isExecuting, setIsExecuting] = useState(false);

  const applyDashboardSnapshot = useCallback((snapshot: DashboardSnapshot) => {
    setLedger({
      deployments: snapshot.deployments,
      accounts: snapshot.accounts,
      liabilities: snapshot.liabilities,
      incomeStreams: snapshot.incomeStreams,
      goals: snapshot.goals,
      objectives: snapshot.objectives,
      baseline: snapshot.baseline,
      analytics: snapshot.analytics,
    });
    setKairosInsight(snapshot.kairosInsight);
  }, []);

  const fetchDashboardData = useCallback(async () => {
    try {
      const snapshot = await getDashboardSnapshotAction();
      applyDashboardSnapshot(snapshot);
    } catch (error) {
      console.error("Failed to fetch dashboard data:", error);
    }
  }, [applyDashboardSnapshot]);

  // 1. HUD / MACRO KPI ROW
  const hudMetrics = [
    {
      label: "Sovereign Wealth",
      icon: ShieldCheck,
      value: ledger.analytics?.totalAssets || 0,
      prefix: "KES ",
      desc: "Total Capitalized Architecture",
      chartType: "sparkline",
      chartColor: "var(--truth)",
      chartData: [100, 110, 105, 120, 115, 130, 140],
    },
    {
      label: "Liquid Capital",
```

```
    icon: Waves,
    value: ledger.analytics?.liquidity || 0,
    prefix: "KES ",
    desc: "Immediate Deployment Capacity",
    chartType: "sparkline",
    chartColor: "var(--truth)",
    chartData: [50, 45, 60, 55, 70, 65, 80],
  },
  {
    label: "Structural Burn",
    icon: Flame,
    value: ledger.analytics?.totalStructuralMonthlyBurn || 0,
    prefix: "KES ",
    suffix: " /mo",
    desc: "Operational Maintenance Cost",
    chartType: "bar",
    chartColor: "var(--leakage)",
    chartData: [20, 22, 18, 25, 20, 24, 21],
  },
  {
    label: "Operational Runway",
    icon: Timer,
    value: ledger.analytics?.runwayDays || 0,
    suffix: " DAYS",
    desc: "Time to Critical Depletion",
    chartType: "donut",
    chartColor: "var(--opportunity)",
    chartData: [],
  },
];

const containerVariants = {
  hidden: { opacity: 0 },
  show: {
    opacity: 1,
    transition: { staggerChildren: 0.1, delayChildren: 0.1 },
  },
};

const itemVariants = {
  hidden: { opacity: 0, y: 10 },
  show: {
    opacity: 1,
    y: 0,
    transition: { duration: 0.4, ease: "easeOut" as const },
  },
};

return (
  <motion.div
    variants={containerVariants}
    initial="hidden"
    animate="show"
    className="max-w-7xl mx-auto p-4 md:p-8 pb-32 space-y-12 md:space-y-16 font-mono"
  >
    {/* HEADER SECTION - CLINICAL */}
    <motion.header
      variants={itemVariants}
      className="flex flex-col md:flex-row md:items-end justify-between gap-6 border-b border-white/10 pb-8"
    >
      <div className="space-y-1">
        <p className="text-[10px] tracking-[0.4em] text-truth uppercase font-bold">
          AXIOM // STRATEGIC COMMAND
        </p>
        <h1 className="text-3xl md:text-5xl font-bold text-white uppercase tracking-tighter">
```

```
            Architect <span className="text-zinc-500">Session</span>
          </h1>
        </div>
        <div className="flex gap-4">
          <button
            onClick={fetchDashboardData}
             className="group px-6 py-2 border border-white/10 bg-zinc-900/50 rounded-sm text-[9px] tracking-widest uppercase
hover:bg-white hover:text-black transition-all flex items-center justify-center gap-3 text-zinc-400"
          >
            <RefreshCw size={12} className="group-hover:rotate-180 transition-transform duration-500" />
            Sync Intel
          </button>
        </div>
      </motion.header>

      {/* ZONE 1: THE HUD (Macro KPIs) */}
      <motion.section
        id="overview"
        variants={itemVariants}
        className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4 md:gap-6"
      >
        {hudMetrics.map((metric) => (
          <div
            key={metric.label}
                  className="p-6 bg-[#0a0a0a] border border-white/10 rounded-sm group hover:border-truth/40 transition-all
duration-300 flex flex-col justify-between overflow-hidden relative"
          >
            <div className="relative z-10">
              <div className="flex justify-between items-start">
                        <p className="text-[9px] tracking-[0.2em] text-zinc-500 uppercase font-bold group-hover:text-zinc-300
transition-colors">
                  {metric.label}
                </p>
                <metric.icon strokeWidth={1.5} size={16} className="text-zinc-600 group-hover:text-truth transition-colors" />
              </div>
              <div className="space-y-1 mt-6">
                <div className="flex items-center justify-between">
                          <h3 className="text-[clamp(1.25rem,5vw,2.25rem)] font-bold text-white tabular-nums tracking-tighter
leading-none">
                    <AnimatedNumber
                      value={metric.value}
                      prefix={metric.prefix}
                      suffix={metric.suffix}
                      compact={true}
                    />
                  </h3>
                </div>
                                <p  className="text-[8px]  tracking-[0.1em]  text-zinc-700  uppercase  group-hover:text-zinc-500
transition-colors">
                  {metric.desc}
                </p>
              </div>
            </div>

            {/* Chart Area */}
            {metric.chartType === "sparkline" && (
              <div className="mt-6 -mx-2 opacity-40 group-hover:opacity-100 transition-opacity">
                <MiniSparkline
                  data={metric.chartData}
                  color={metric.chartColor}
                />
              </div>
            )}
            {metric.chartType === "bar" && (
              <div className="mt-6 -mx-2 opacity-40 group-hover:opacity-100 transition-opacity">
```

```
          <MiniBarChart
            data={metric.chartData}
            color={metric.chartColor}
          />
        </div>
      )}
      {metric.chartType === "donut" && (
        <div className="mt-6 w-full -mx-2 opacity-40 group-hover:opacity-100 transition-opacity">
          <MiniDonut
            value={metric.value}
            max={365}
            color={metric.chartColor}
          />
        </div>
      )}
    </div>
  ))}
</motion.section>

{/* ZONE 2: ARCHITECTURE DEPLOYMENT */}
<motion.section id="deploy" variants={itemVariants} className="w-full">
  <div className="p-8 md:p-12 bg-[#0a0a0a] border border-white/10 rounded-sm">
    <div className="flex flex-col md:flex-row gap-8 md:gap-16 items-start">
      <div className="space-y-4 md:w-1/3">
        <div className="flex w-10 h-10 bg-truth/10 items-center justify-center border border-truth/20">
          <Cpu strokeWidth={1.5} size={20} className="text-truth" />
        </div>
        <h2 className="text-2xl font-bold text-white uppercase tracking-tight">
          Capital Deployment
        </h2>
        <p className="text-[10px] tracking-tight text-zinc-500 uppercase leading-relaxed">
          Execute resource allocation. Log deployments and dictate the flow of
          capital across the architecture.
        </p>
      </div>
      <div className="md:w-2/3 w-full bg-black/40 p-6 border border-white/5">
        <NewEntryForm
          accounts={ledger.accounts}
          liquidity={ledger.analytics?.liquidity || 0}
          isActionLoading={isExecuting}
          onSubmit={async (data) => {
            if (isExecuting) return;
            setIsExecuting(true);
            try {
              const snapshot = await createDeploymentAction({
                title: data.title,
                amount: data.amount,
                category: data.category,
                accountId: data.accountId,
              });
              applyDashboardSnapshot(snapshot);
            } finally {
              setIsExecuting(false);
            }
          }}
        />
      </div>
    </div>
  </div>
</motion.section>

{/* ZONE 3: KAIROS INTELLIGENCE */}
<motion.section
  id="intelligence"
  variants={itemVariants}
```

```
  className="space-y-6"
>
  <div className="flex items-center gap-4 md:gap-6">
    <div className="flex items-center gap-3">
      <span className="text-zinc-700 text-lg font-bold">01.</span>
      <Brain strokeWidth={1.5} size={20} className="text-truth" />
    </div>
    <h2 className="text-xl font-bold text-white uppercase tracking-widest">
      Kairos Intel
    </h2>
    <div className="flex-1 h-px bg-white/10" />
  </div>
  <KairosNarrative insight={kairosInsight} />
</motion.section>

{/* ZONE 4: THE LEDGER */}
<motion.div
  id="ledger"
  variants={itemVariants}
  className="space-y-12 md:space-y-24"
>
  {/* CAPITAL ACCOUNTS - NOW ON TOP */}
  <section className="space-y-6">
    <div className="flex items-center gap-4 md:gap-6">
      <div className="flex items-center gap-3">
        <span className="text-zinc-700 text-lg font-bold">02.</span>
        <Database strokeWidth={1.5} size={20} className="text-truth" />
      </div>
      <h2 className="text-xl font-bold text-white uppercase tracking-widest">
        Capital Accounts
      </h2>
      <div className="flex-1 h-px bg-white/10" />
    </div>
    <div className="p-6 bg-[#0a0a0a] border border-white/10 rounded-sm">
      <AccountSection
        accounts={ledger.accounts}
        onSnapshot={applyDashboardSnapshot}
      />
    </div>
  </section>

  {/* TACTICAL & STRATEGIC GRID */}
  <div className="grid grid-cols-1 lg:grid-cols-[1.5fr_1fr] gap-12">
    {/* LEFT COLUMN: TACTICAL ENGINE */}
    <section className="space-y-12">
      {/* PENDING ACTIONS */}
      <div className="space-y-6">
        <div className="flex items-center gap-4 md:gap-6">
          <div className="flex items-center gap-3">
            <span className="text-zinc-700 text-lg font-bold">03.</span>
            <Zap strokeWidth={1.5} size={20} className="text-warning" />
          </div>
          <h2 className="text-xl font-bold text-white uppercase tracking-widest">
            Tactical Units
          </h2>
          <div className="flex-1 h-px bg-white/10" />
        </div>
        <div className="space-y-3">
          <PendingInflows
            incomeStreams={ledger.incomeStreams}
            accounts={ledger.accounts}
            onSnapshot={applyDashboardSnapshot}
          />
          <PendingLiabilities
            liabilities={ledger.liabilities}
```

```
        accounts={ledger.accounts}
        onSnapshot={applyDashboardSnapshot}
      />
      <PendingBaselines
        baseline={ledger.baseline}
        accounts={ledger.accounts}
        onSnapshot={applyDashboardSnapshot}
      />
    </div>
  </div>

  <div className="space-y-6">
    <div className="p-6 bg-[#0a0a0a] border border-white/10 rounded-sm">
      <IncomeSection
        incomeStreams={ledger.incomeStreams}
        onSnapshot={applyDashboardSnapshot}
      />
    </div>
    <div className="p-6 bg-[#0a0a0a] border border-white/10 rounded-sm">
      <BaselineSection
        baseline={ledger.baseline}
        onSnapshot={applyDashboardSnapshot}
      />
    </div>
  </div>

  <div className="p-6 bg-[#0a0a0a] border border-white/10 rounded-sm">
    <LiabilitySection
      liabilities={ledger.liabilities}
      onSnapshot={applyDashboardSnapshot}
    />
  </div>
</section>

{/* RIGHT COLUMN: STRATEGIC ENGINE */}
<section className="space-y-12">
  <div className="space-y-6">
    <div className="flex items-center gap-4 md:gap-6">
      <div className="flex items-center gap-3">
        <span className="text-zinc-700 text-lg font-bold">04.</span>
        <Hexagon strokeWidth={1.5} size={20} className="text-truth" />
      </div>
      <h2 className="text-xl font-bold text-white uppercase tracking-widest">
        Strategic Alignment
      </h2>
      <div className="flex-1 h-px bg-white/10" />
    </div>
    <div className="p-6 bg-[#0a0a0a] border border-white/10 rounded-sm">
      <StrategicObjectiveSection
        objectives={ledger.objectives}
        onSnapshot={applyDashboardSnapshot}
      />
    </div>
  </div>

  <div className="p-6 bg-[#0a0a0a] border border-white/10 rounded-sm">
    <GoalSection
      goals={ledger.goals}
      onSnapshot={applyDashboardSnapshot}
    />
  </div>

  <div className="p-6 bg-[#0a0a0a] border border-white/10 rounded-sm">
    <HistoricalAudit deployments={ledger.deployments} />
  </div>
```

```
            {/* OBSERVABILITY */}
            <section className="opacity-40 hover:opacity-100 transition-opacity duration-500 border-t border-white/5 pt-12">
              <div className="flex items-center gap-4 md:gap-6 mb-8">
                <div className="flex items-center gap-3">
                  <span className="text-zinc-700 text-lg font-bold">05.</span>
                  <Activity strokeWidth={1.5} size={20} className="text-truth" />
                </div>
                <h2 className="text-xl font-bold text-white uppercase tracking-widest">
                  Telemetry
                </h2>
                <div className="flex-1 h-px bg-white/10" />
              </div>
              <TelemetryDashboard />
            </section>
          </section>
        </div>
      </motion.div>
    </motion.div>
  );
}
```

FILE: src\app\dashboard\error.tsx

```
"use client";

import { useEffect } from "react";

export default function DashboardError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  useEffect(() => {
    // Log the error to an error reporting service
    console.error("Dashboard Route Error:", error);
  }, [error]);

  return (
    <main className="min-h-[80vh] flex items-center justify-center p-6">
      <div className="w-full max-w-md bg-background border-2 border-red-500/20 rounded-[2.5rem] p-10 shadow-2xl text-center">
        <div className="w-20 h-20 bg-red-500/10 rounded-3xl flex items-center justify-center mx-auto mb-6">
          <svg
            width="40"
            height="40"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            strokeWidth="3"
            className="text-red-500"
          >
            <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z" />
            <line x1="12" y1="9" x2="12" y2="13" />
            <line x1="12" y1="17" x2="12.01" y2="17" />
          </svg>
        </div>

        <h1 className="text-3xl font-black tracking-tighter uppercase mb-4 text-foreground">
          System Interrupted
        </h1>
```

```
        <p className="text-foreground/60 text-sm font-bold uppercase tracking-widest mb-8 leading-relaxed">
          The intelligence engine encountered an unhandled exception.
        </p>

        <div className="bg-foreground/5 rounded-2xl p-4 mb-8 text-left">
          <p className="text-[10px] font-black text-foreground/40 uppercase tracking-widest mb-2">
            Error Diagnostic
          </p>
          <p className="text-xs font-mono text-red-400 break-all leading-tight">
            {error.message || "Unknown cryptographic failure"}
          </p>
        </div>

        <button
          onClick={() => reset()}
               className="w-full bg-foreground text-background px-4 py-5 rounded-2xl font-black text-lg hover:scale-[1.02]
active:scale-[0.98] transition-all shadow-xl shadow-foreground/10 uppercase"
        >
          Attempt Recovery
        </button>
      </div>
    </main>
  );
}
```

FILE: src\app\dashboard\GoalSection.tsx

```
"use client";

import { useState } from "react";
import {
  calculateGoalProgressPercentage,
  type FinancialGoal,
  GOAL_TYPES,
  GOAL_PRIORITIES,
  GOAL_STATUSES,
} from "@/lib/finance/goals";
import { createGoalAction } from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { ScrollReveal } from "@/components/ui/scroll-reveal";
import { motion } from "framer-motion";

interface GoalSectionProps {
  goals: FinancialGoal[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function GoalSection({ goals, onSnapshot }: GoalSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    goal_name: "",
    goal_type: "emergency_fund",
    target_amount: "",
    current_progress: "",
    priority: "medium",
    status: "active",
    target_date: "",
    notes: "",
  });
```

```
  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createGoalAction({
        goal_name: form.goal_name,
        goal_type: form.goal_type,
        target_amount: Number(form.target_amount),
        current_progress: form.current_progress
          ? Number(form.current_progress)
          : 0,
        priority: form.priority,
        status: form.status,
        target_date: form.target_date || null,
        notes: form.notes || undefined,
      });
      setForm({
        goal_name: "",
        goal_type: "emergency_fund",
        target_amount: "",
        current_progress: "",
        priority: "medium",
        status: "active",
        target_date: "",
        notes: "",
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error ? err.message : "Failed to record financial goal",
      );
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-12">
      <div className="flex items-center justify-between">
        <h2 className="font-mono text-xl text-white tracking-widest uppercase">
          Wealth Milestones
        </h2>
        <button
          onClick={() => setIsAdding(!isAdding)}
                    className="font-mono  text-[9px]  tracking-[0.4em]  uppercase  text-muted-foreground/80  hover:text-white
transition-colors"
        >
          {isAdding ? "? CANCEL" : "+ APPEND MILESTONE"}
        </button>
      </div>

      {isAdding && (
        <div className="bg-white2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4 duration-500">
          <form onSubmit={handleSubmit} className="space-y-8">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Milestone Name
                </label>
                <input
                  type="text"
                  required
```

```
                placeholder="e.g. Dream House"
                value={form.goal_name}
                onChange={(e) =>
                  setForm({ ...form, goal_name: e.target.value })
                }
                            className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-sm
focus:outline-none focus:border-white transition-colors uppercase tracking-wider"
              />
            </div>
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                Milestone Type
              </label>
              <select
                value={form.goal_type}
                onChange={(e) =>
                  setForm({ ...form, goal_type: e.target.value })
                }
                       className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
              >
                {GOAL_TYPES.map((type) => (
                  <option
                    key={type.value}
                    value={type.value}
                    className="bg-black"
                  >
                    {type.label}
                  </option>
                ))}
              </select>
            </div>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                Priority
              </label>
              <select
                value={form.priority}
                onChange={(e) =>
                  setForm({ ...form, priority: e.target.value })
                }
                       className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
              >
                {GOAL_PRIORITIES.map((p) => (
                  <option key={p.value} value={p.value} className="bg-black">
                    {p.label}
                  </option>
                ))}
              </select>
            </div>
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                Status
              </label>
              <select
                value={form.status}
                onChange={(e) => setForm({ ...form, status: e.target.value })}
                       className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
              >
                {GOAL_STATUSES.map((s) => (
```

```
                <option key={s.value} value={s.value} className="bg-black">
                  {s.label}
                </option>
              ))}
            </select>
          </div>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
          <div className="space-y-2">
            <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
              Target Amount
            </label>
            <input
              type="number"
              required
              placeholder="0.00"
              value={form.target_amount}
              onChange={(e) =>
                setForm({ ...form, target_amount: e.target.value })
              }
                          className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-lg
focus:outline-none focus:border-white transition-colors tabular-nums"
              />
          </div>
          <div className="space-y-2">
            <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
              Current Progress
            </label>
            <input
              type="number"
              placeholder="0.00"
              value={form.current_progress}
              onChange={(e) =>
                setForm({ ...form, current_progress: e.target.value })
              }
                          className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-lg
focus:outline-none focus:border-white transition-colors tabular-nums"
              />
          </div>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
          <div className="space-y-2">
            <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
              Target Date
            </label>
            <input
              type="date"
              value={form.target_date}
              onChange={(e) =>
                setForm({ ...form, target_date: e.target.value })
              }
                      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
              />
          </div>
          <div className="space-y-2">
            <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
              Milestone Notes
            </label>
            <input
              type="text"
              placeholder="Add context..."
              value={form.notes}
```

```
                onChange={(e) => setForm({ ...form, notes: e.target.value })}
                            className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-sm
focus:outline-none focus:border-white transition-colors uppercase tracking-wider"
                />
              </div>
            </div>

            {error && (
              <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
               className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px] hover:bg-white/90
transition-colors disabled:opacity-50"
            >
              {isLoading ? "SAVING..." : "CONFIRM MILESTONE"}
            </button>
          </form>
        </div>
      )}

      <div className="space-y-8">
        {goals.length === 0 ? (
          <div className="py-12 text-center opacity-20">
            <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
              No milestones active
            </p>
          </div>
        ) : (
          goals.map((goal, index) => {
            const progress = calculateGoalProgressPercentage(goal);
            return (
              <ScrollReveal key={goal.id} delay={index * 0.1}>
                <div className="space-y-4 group cursor-default pb-8 border-b border-white/5 last:border-0">
                  <div className="flex items-center justify-between mb-2">
                    <div className="space-y-1">
                      <span
                        className={`text-[8px] font-mono tracking-widest uppercase ${
                          goal.priority === "critical"
                            ? "text-red-500"
                            : "text-muted-foreground/60"
                        }`}
                      >
                        {goal.priority}
                      </span>
                                  <h3 className="font-mono text-lg uppercase tracking-wider text-white transition-transform
group-hover:translate-x-2">
                        {goal.goal_name}
                      </h3>
                    </div>
                    <div className="text-right">
                      <p className="font-mono text-2xl text-white tabular-nums">
                        {Math.round(progress)}%
                      </p>
                    </div>
                  </div>

                  <div className="h-px w-full bg-white/5 relative overflow-hidden">
                    <motion.div
                      initial={{ width: 0 }}
                      whileInView={{ width: `${Math.min(100, progress)}%` }}
```

```
                    viewport={{ once: true }}
                    transition={{
                      duration: 1.5,
                      delay: 0.5 + index * 0.1,
                      ease: "easeOut",
                    }}
                    className="absolute inset-y-0 left-0 bg-white/60"
                  />
                </div>

                <div className="flex justify-between items-end pt-2 opacity-30 group-hover:opacity-60 transition-opacity">
                  <p className="text-[9px] font-mono uppercase tracking-widest">
                    {formatCurrency(goal.current_progress)}
                  </p>
                  <p className="text-[9px] font-mono uppercase tracking-widest">
                    Target: {formatCurrency(goal.target_amount)}
                  </p>
                </div>
              </div>
            </ScrollReveal>
          );
        })
      )}
    </div>
  </div>
);
}
```

FILE: src\app\dashboard\HistoricalAudit.tsx

```
"use client";

import React from "react";
import { Deployment } from "@/lib/analytics/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { DeploymentMap } from "@/lib/utils/taxonomy";
import { Radar } from "lucide-react";

interface HistoricalAuditProps {
  deployments: Deployment[];
}

export function HistoricalAudit({ deployments }: HistoricalAuditProps) {
  if (!deployments || deployments.length === 0) {
    return (
      <div className="flex flex-col items-center justify-center py-20 px-8 text-center space-y-6 relative overflow-hidden">
        {/* Radar Ping Illustration */}
        <div className="relative flex items-center justify-center w-32 h-32 mb-4">
          <div className="absolute inset-0 rounded-full border border-truth/20" />
          <div className="absolute inset-4 rounded-full border border-truth/30" />
          <div className="absolute inset-8 rounded-full border border-truth/40" />
           <div className="absolute inset-0 rounded-full border-t border-truth animate-spin" style={{ animationDuration: '3s'
}} />
          <Radar size={32} className="text-truth relative z-10 opacity-80" />
          {/* Scanning Sweep */}
           <div className="absolute top-1/2 left-1/2 w-16 h-16 origin-top-left animate-spin" style={{ animationDuration: '3s'
}}>
            <div className="w-full h-full bg-gradient-to-br from-truth/20 to-transparent transform -skew-x-12" />
          </div>
        </div>

        <div className="space-y-2 relative z-10">
          <h3 className="font-mono text-sm tracking-[0.2em] text-truth uppercase font-bold">
```

```
            Audit Ledger Clear
          </h3>
          <p className="font-mono text-[10px] tracking-widest text-zinc-500 uppercase max-w-xs leading-relaxed">
            No historical deployments detected. Awaiting initial capital allocation to establish forensic baseline.
          </p>
        </div>
      </div>
    );
  }

  // Sort by date descending
  const sortedDeployments = [...deployments].sort(
    (a, b) =>
      new Date(b.created_at).getTime() - new Date(a.created_at).getTime(),
  );

  return (
    <div className="space-y-12">
      <div className="flex items-center gap-6">
        <h3 className="font-mono text-xl text-white tracking-widest uppercase">
          Historical Audit
        </h3>
        <div className="flex-1 h-px bg-white/5" />
      </div>

      <div className="space-y-4 max-h-[600px] overflow-y-auto scrollbar-hide pr-2">
        {sortedDeployments.map((record) => (
          <div
            key={record.id}
              className="group relative bg-white/[0.02] border border-white/5 rounded-sm p-6 flex items-center justify-between
transition-all hover:bg-white/[0.04] hover:border-white/10"
          >
            <div className="flex items-center gap-6">
                    <div  className="w-12  h-12  bg-white/5  rounded-sm  flex  items-center  justify-center  group-hover:scale-110
transition-transform">
                <svg
                  width="20"
                  height="20"
                  viewBox="0 0 24 24"
                  fill="none"
                  stroke="white"
                  strokeWidth="1.5"
                  className="opacity-40 group-hover:opacity-80 transition-opacity"
                >
                  <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
                </svg>
              </div>
              <div className="space-y-1">
                        <h4  className="font-mono  text-sm  uppercase  tracking-wider  text-white/80  group-hover:text-white
transition-colors">
                  {record.title}
                </h4>
                <div className="flex items-center gap-3">
                          <span  className="text-[9px]  font-mono  bg-white/5  px-2  py-0.5  rounded-sm  text-white/40  uppercase
tracking-widest">
                    {DeploymentMap[
                      record.category as keyof typeof DeploymentMap
                    ] || record.category}
                  </span>
                  <span className="text-[8px] font-mono text-white/20 uppercase tracking-widest">
                    {new Date(record.created_at).toLocaleDateString(undefined, {
                      month: "short",
                      day: "numeric",
                    })}
                  </span>
```

```
              </div>
            </div>
          </div>
          <div className="text-right">
            <p className="font-mono text-lg text-white tabular-nums">
              {formatCurrency(record.amount)}
            </p>
            <p className="text-[8px] font-mono text-white/10 uppercase tracking-[0.3em] mt-1">
              Verified Asset
            </p>
          </div>
        </div>
      ))}
    </div>

    <p className="text-center text-[8px] font-mono text-white/10 uppercase tracking-[0.5em]">
      Forensic layer immutable // Protocol v1.0
    </p>
  </div>
 );
}
```

FILE: src\app\dashboard\IncomeSection.tsx

```
"use client";

import { useState } from "react";
import {
  INCOME_TYPES,
  CADENCES,
  type IncomeStream,
  type IncomeType,
  type Cadence,
} from "@/lib/finance/income";
import { createIncomeAction } from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { IncomeMap } from "@/lib/utils/taxonomy";
import { DistributionPieChart } from "@/components/dashboard/MiniCharts";
import { ScrollReveal } from "@/components/ui/scroll-reveal";

interface IncomeSectionProps {
  incomeStreams: IncomeStream[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

interface FormEntry {
  income_name: string;
  income_type: IncomeType;
  amount: string;
  cadence: Cadence;
  execution_day: number;
  is_recurring: boolean;
  source: string;
}

export function IncomeSection({
  incomeStreams,
  onSnapshot,
}: IncomeSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
```

```
const chartData = incomeStreams.map((s) => ({
  name: s.income_name,
  value: Number(s.amount),
}));

const [entries, setEntries] = useState<FormEntry[]>([
  {
    income_name: "",
    income_type: "salary",
    amount: "",
    cadence: "monthly",
    execution_day: 1,
    is_recurring: true,
    source: "",
  },
]);

const addEntry = () => {
  setEntries([
    ...entries,
    {
      income_name: "",
      income_type: "salary",
      amount: "",
      cadence: "monthly",
      execution_day: 1,
      is_recurring: true,
      source: "",
    },
  ]);
};

const updateEntry = <K extends keyof FormEntry>(
  index: number,
  field: K,
  value: FormEntry[K],
) => {
  const newEntries = [...entries];
  newEntries[index][field] = value;
  setEntries(newEntries);
};

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  setIsLoading(true);
  setError(null);
  try {
    const snapshot = await createIncomeAction(
      entries.map((entry) => ({
        ...entry,
        amount: Number(entry.amount),
        cadence: entry.is_recurring ? entry.cadence : "irregular",
        execution_day:
          entry.is_recurring &&
          (entry.cadence === "monthly" ||
            entry.cadence === "weekly" ||
            entry.cadence === "biweekly")
            ? Number(entry.execution_day)
            : null,
        source: entry.source || undefined,
      })),
    );
    setEntries([
      {
```

```
          income_name: "",
          income_type: "salary",
          amount: "",
          cadence: "monthly",
          execution_day: 1,
          is_recurring: true,
          source: "",
        },
      ]);
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error ? err.message : "Failed to record income streams",
      );
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-12">
      <div className="flex items-center justify-between">
        <h2 className="font-mono text-xl text-white tracking-widest uppercase">
          Income Velocity
        </h2>
        <button
          onClick={() => setIsAdding(!isAdding)}
                    className="font-mono  text-[9px]  tracking-[0.4em]  uppercase  text-muted-foreground/80  hover:text-white
transition-colors"
        >
          {isAdding ? "? CANCEL" : "+ APPEND INCOME"}
        </button>
      </div>

      {isAdding && (
        <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4 duration-500">
          <form onSubmit={handleSubmit} className="space-y-12">
            {entries.map((entry, index) => (
              <div key={index} className="space-y-8 relative">
                {index > 0 && <div className="border-t border-white/5 pt-8" />}

                <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
                  <div className="space-y-2">
                    <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                      Source Name
                    </label>
                    <input
                      type="text"
                      required
                      placeholder="e.g. Salary"
                      value={entry.income_name}
                      onChange={(e) =>
                        updateEntry(index, "income_name", e.target.value)
                      }
                                  className="w-full  bg-transparent  border-b  border-white/10  py-3  text-white  font-mono  text-sm
focus:outline-none focus:border-white transition-colors uppercase tracking-wider"
                    />
                  </div>
                  <div className="space-y-2">
                    <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                      Origin / Payer
                    </label>
                    <input
                      type="text"
```

```
                    placeholder="e.g. Acme Corp"
                    value={entry.source}
                    onChange={(e) =>
                      updateEntry(index, "source", e.target.value)
                    }
                              className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-sm
focus:outline-none focus:border-white transition-colors uppercase tracking-wider"
                  />
                </div>
              </div>

              <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
                <div className="space-y-2">
                  <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                    Type
                  </label>
                  <select
                    value={entry.income_type}
                    onChange={(e) =>
                      updateEntry(
                        index,
                        "income_type",
                        e.target.value as IncomeType,
                      )
                    }
                         className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                  >
                    {INCOME_TYPES.map((t) => (
                      <option
                        key={t.value}
                        value={t.value}
                        className="bg-black"
                      >
                        {t.label}
                      </option>
                    ))}
                  </select>
                </div>
                <div className="space-y-2">
                  <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                    Amount
                  </label>
                  <input
                    type="number"
                    required
                    placeholder="0.00"
                    value={entry.amount}
                    onChange={(e) =>
                      updateEntry(index, "amount", e.target.value)
                    }
                              className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-lg
focus:outline-none focus:border-white transition-colors tabular-nums"
                  />
                </div>
              </div>

              <div className="space-y-4">
                <label className="flex items-center space-x-3 cursor-pointer group">
                  <input
                    type="checkbox"
                    checked={entry.is_recurring}
                    onChange={(e) =>
                      updateEntry(index, "is_recurring", e.target.checked)
                    }
```

```
                            className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white transition-colors
cursor-pointer"
                    />
                              <span className="text-[10px] font-mono text-muted-foreground/80 uppercase tracking-widest
group-hover:text-white/60 transition-colors">
                      Recurring Inflow
                    </span>
                  </label>

                  {entry.is_recurring && (
                    <div className="space-y-8 pt-4 animate-in fade-in slide-in-from-top-2 border-l border-white/5 pl-6 ml-2">
                      <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
                        <div className="space-y-2">
                          <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                            Frequency
                          </label>
                          <select
                            value={entry.cadence}
                            onChange={(e) =>
                              updateEntry(
                                index,
                                "cadence",
                                e.target.value as Cadence,
                              )
                            }
                             className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                          >
                            {CADENCES.map((c) => (
                              <option
                                key={c.value}
                                value={c.value}
                                className="bg-black"
                              >
                                {c.label}
                              </option>
                            ))}
                          </select>
                        </div>
                        {(entry.cadence === "monthly" ||
                          entry.cadence === "weekly" ||
                          entry.cadence === "biweekly") && (
                          <div className="space-y-2">
                            <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                              {entry.cadence === "monthly"
                                ? "Day of Month (1-31)"
                                : "Day of Week"}
                            </label>
                            {entry.cadence === "monthly" ? (
                              <input
                                type="number"
                                min="1"
                                max="31"
                                required
                                value={entry.execution_day || ""}
                                onChange={(e) =>
                                  updateEntry(
                                    index,
                                    "execution_day",
                                    Number(e.target.value),
                                  )
                                }
                                   className="w-full bg-transparent border-b border-white/10 py-3 text-white font-mono text-sm
focus:outline-none focus:border-white transition-colors tabular-nums"
                              />
```

```
                          ) : (
                            <select
                              value={entry.execution_day || 1}
                              onChange={(e) =>
                                updateEntry(
                                  index,
                                  "execution_day",
                                  Number(e.target.value),
                                )
                              }
                                      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
                            >
                              {[
                                { label: "Monday", value: 1 },
                                { label: "Tuesday", value: 2 },
                                { label: "Wednesday", value: 3 },
                                { label: "Thursday", value: 4 },
                                { label: "Friday", value: 5 },
                                { label: "Saturday", value: 6 },
                                { label: "Sunday", value: 7 },
                              ].map((day) => (
                                <option
                                  key={day.value}
                                  value={day.value}
                                  className="bg-black"
                                >
                                  {day.label}
                                </option>
                              ))}
                            </select>
                          )}
                        </div>
                      )}
                    </div>
                  </div>
                )}
              </div>
            </div>
          ))}

          <div className="space-y-6">
            <button
              type="button"
              onClick={addEntry}
                       className="text-[9px] font-mono tracking-widest uppercase text-muted-foreground/60 hover:text-white
transition-colors"
            >
              + APPEND ANOTHER SOURCE
            </button>

            {error && (
              <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
                          className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
            >
              {isLoading ? "SAVING..." : "CONFIRM INCOMES"}
            </button>
```

```
            </div>
          </form>
        </div>
      )}

      {incomeStreams.length > 0 && (
        <div className="py-6 border-b border-white/5 mb-6">
          <DistributionPieChart data={chartData} />
        </div>
      )}

      <div className="space-y-2">
        {incomeStreams.length === 0 ? (
          <div className="py-12 text-center opacity-20">
            <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
              No income streams active
            </p>
          </div>
        ) : (
          incomeStreams.map((stream, index) => (
            <ScrollReveal key={stream.id} delay={index * 0.05} distance={10}>
                      <div  className="flex  items-center  justify-between  py-6  border-b  border-white/5  group  hover:bg-white/1
transition-all px-2">
                <div className="space-y-1">
                  <span className="text-[8px] font-mono tracking-widest uppercase text-muted-foreground/60">
                    {IncomeMap[stream.income_type] || stream.income_type}
                    {stream.source && ` ? ${stream.source}`}
                  </span>
                                <h3  className="font-mono  text-sm  uppercase  tracking-wider  text-white  transition-transform
group-hover:translate-x-2">
                    {stream.income_name}
                  </h3>
                </div>
                <div className="text-right">
                  <p className="font-mono text-lg text-white tabular-nums">
                    {formatCurrency(stream.amount)}
                    <span className="text-[10px] ml-2 text-muted-foreground/60 font-mono tracking-widest uppercase">
                      / {stream.cadence}
                    </span>
                  </p>
                  {stream.execution_day && (
                    <p className="text-[8px] font-mono text-muted-foreground/50 uppercase tracking-widest mt-1">
                      Executes on day {stream.execution_day}
                    </p>
                  )}
                </div>
              </div>
            </ScrollReveal>
          ))
        )}
      </div>
    </div>
  );
}
```

FILE: src\app\dashboard\layout.tsx

```
"use client";

import { supabase } from "@/lib/supabase";
import { useEffect, useState } from "react";
import { User } from "@supabase/supabase-js";
import Link from "next/link";
```

```
import { SmoothScrollProvider } from "@/components/ui/smooth-scroll-provider";
import { checkOnboardingStatusAction } from "./actions";
import { BrandMark } from "@/components/ui/brand-mark";
import DayZeroOnboarding from "@/components/dashboard/DayZeroOnboarding";
import DashboardLoading from "./loading";

import {
  LayoutDashboard,
  Cpu,
  Brain,
  Database,
  LogOut,
  Activity
} from "lucide-react";

export const dynamic = "force-dynamic";

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const [user, setUser] = useState<User | null>(null);
  const [isProfileOpen, setIsProfileOpen] = useState(false);
  const [activeSection, setActiveSection] = useState("overview");
  const [isOnboarded, setIsOnboarded] = useState<boolean | null>(null);

  useEffect(() => {
    async function init() {
      const {
        data: { user },
      } = await supabase.auth.getUser();
      setUser(user);

      try {
        const { isOnboarded } = await checkOnboardingStatusAction();
        setIsOnboarded(isOnboarded);
      } catch {
        setIsOnboarded(false);
      }
    }
    init();

    const handleScroll = () => {
      const sections = ["overview", "deploy", "intelligence", "ledger"];
      const scrollPosition = window.scrollY + 200;

      for (const section of sections) {
        const element = document.getElementById(section);
        if (element) {
          const { offsetTop, offsetHeight } = element;
          if (
            scrollPosition >= offsetTop &&
            scrollPosition < offsetTop + offsetHeight
          ) {
            setActiveSection(section);
            break;
          }
        }
      }
    };

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);
```

```
  async function handleLogout() {
    await supabase.auth.signOut();
    window.location.href = "/";
  }

  const navItems = [
    { id: "overview", label: "Overview", icon: LayoutDashboard },
    { id: "deploy", label: "Architecture", icon: Cpu },
    { id: "intelligence", label: "Intelligence", icon: Brain },
    { id: "ledger", label: "Ledger", icon: Database },
  ];

  const scrollToSection = (id: string) => {
    const element = document.getElementById(id);
    if (element) {
      const offset = 100;
      const bodyRect = document.body.getBoundingClientRect().top;
      const elementRect = element.getBoundingClientRect().top;
      const elementPosition = elementRect - bodyRect;
      const offsetPosition = elementPosition - offset;

      window.scrollTo({
        top: offsetPosition,
        behavior: "smooth",
      });
    }
  };

  const userAvatar = user?.user_metadata?.avatar_url;

  return (
           <div   className="min-h-screen   bg-black   text-white   flex   flex-col   md:flex-row   relative   selection:bg-white
selection:text-black">
      {/* Background Atmosphere */}
      <div className="fixed inset-0 z-0">
        <div className="absolute top-0 right-0 w-[40%] h-[40%] bg-white/[0.02] rounded-full blur-[120px]" />
        <div className="absolute bottom-0 left-0 w-[40%] h-[40%] bg-white/[0.02] rounded-full blur-[120px]" />
      </div>

      {isOnboarded === true && (
        <>
          {/* SIDEBAR */}
            <aside className="hidden md:flex flex-col fixed left-0 top-0 bottom-0 w-80 bg-black/40 backdrop-blur-3xl border-r
border-white/5 z-50 p-12 animate-in fade-in slide-in-from-left-4 duration-1000 overflow-hidden">
             {/* Ambient Sidebar Glow */}
                                                      <div     className="absolute     top-0     left-0     w-full     h-full
bg-[radial-gradient(ellipse_at_top_left,rgba(255,255,255,0.03),transparent_70%)] pointer-events-none" />

             <div className="relative z-10 flex flex-col h-full justify-between">
               <div className="space-y-24">
                 {/* Logo */}
                 <Link href="/" className="group mb-8 block">
                   <div className="flex items-center gap-4">
                     <BrandMark className="w-10 h-10 group-hover:scale-105 transition-transform duration-700" />
                           <span className="font-mono text-xl tracking-[0.4em] uppercase text-white group-hover:text-white/80
transition-colors">
                       AXIOM
                     </span>
                   </div>
                 </Link>

                 {/* Navigation */}
                 <nav className="space-y-6">
                   {navItems.map((item) => (
```

```
                  <button
                    key={item.id}
                    onClick={() => scrollToSection(item.id)}
                    className={`w-full group text-left transition-all duration-500 py-3 ${
                      activeSection === item.id
                        ? "opacity-100"
                        : "opacity-30 hover:opacity-100"
                    }`}
                  >
                    <div className="flex items-center gap-6 relative">
                      {/* Active Line Indicator */}
                      <div
                          className={`absolute -left-6 top-1/2 -translate-y-1/2 w-1 h-0 bg-white transition-all duration-500
${
                          activeSection === item.id
                            ? "h-full"
                            : "group-hover:h-1/2 opacity-30"
                        }`}
                      />
                      <item.icon
                        size={18}
                        strokeWidth={activeSection === item.id ? 2.5 : 1.5}
                        className={`transition-all duration-500 ${
                          activeSection === item.id ? "text-truth scale-110" : "text-white/40 group-hover:text-white/60"
                        }`}
                      />
                      <span
                        className={`font-mono text-base tracking-widest transition-all duration-500 ${
                          activeSection === item.id
                            ? "translate-x-2 text-white font-bold"
                            : "group-hover:translate-x-2 text-white/80"
                        }`}
                      >
                        {item.label}
                      </span>
                    </div>
                  </button>
                ))}
              </nav>
            </div>

            {/* User Profile */}
            <div className="pt-12 border-t border-white/5 relative">
              <button
                onClick={() => setIsProfileOpen(!isProfileOpen)}
                className="flex items-center gap-4 w-full group"
              >
                        <div  className="w-10  h-10  border  border-white/10  bg-zinc-900  overflow-hidden  flex  items-center
justify-center      font-mono      text-xs      text-white      group-hover:border-white      transition-all      duration-500
shadow-[0_0_15px_rgba(255,255,255,0.05)]">
                  {userAvatar ? (
                    <img src={userAvatar} alt="Profile" className="w-full h-full object-cover" />
                  ) : (
                    user?.email?.[0].toUpperCase()
                  )}
                </div>
                <div className="flex flex-col text-left">
                          <span  className="text-[10px]  font-mono  tracking-widest  text-white/80  uppercase  truncate  max-w-30
group-hover:text-white transition-colors">
                    {user?.email?.split("@")[0]}
                  </span>
                  <span className="text-[8px] font-mono tracking-widest text-zinc-500 uppercase flex items-center gap-1.5">
                    <Activity size={8} className="text-truth" />
                    Active Session
                  </span>
```

```
                </div>
              </button>

              {isProfileOpen && (
                  <div className="absolute bottom-full left-0 mb-4 w-full bg-[#0a0a0a] border border-white/10 p-1 shadow-2xl
animate-in fade-in slide-in-from-bottom-2 duration-300">
                  <button
                    onClick={handleLogout}
                            className="w-full text-left font-mono text-[9px] tracking-widest text-red-500 hover:text-white
hover:bg-red-500 transition-all uppercase p-4 flex items-center justify-between group/logout"
                  >
                    Terminate Session
                    <LogOut size={12} className="group-hover/logout:translate-x-1 transition-transform" />
                  </button>
                </div>
              )}
            </div>
          </div>
        </aside>

        {/* MOBILE HEADER */}
         <header className="md:hidden sticky top-0 bg-black/80 backdrop-blur-xl border-b border-white/5 z-50 px-6 py-6 flex
justify-between items-center">
          <div className="flex items-center gap-3">
            <BrandMark className="w-8 h-8" />
            <span className="font-mono text-[10px] tracking-widest uppercase text-white/40">
              Axiom // Portal
            </span>
          </div>
          <button onClick={() => setIsProfileOpen(!isProfileOpen)}>
                <div className="w-8 h-8 border border-white/10 overflow-hidden flex items-center justify-center font-mono
text-[10px]">
              {userAvatar ? (
                <img src={userAvatar} alt="Profile" className="w-full h-full object-cover" />
              ) : (
                user?.email?.[0].toUpperCase()
              )}
            </div>
          </button>
        </header>
      </>
    )}

    {/* MAIN CONTENT AREA */}
    <main
         className={`flex-1 min-h-screen relative z-10 transition-all duration-1000 ${isOnboarded === true ? "md:ml-80" :
"md:ml-0"}`}
    >
      <SmoothScrollProvider>
        <div
             className={`max-w-7xl mx-auto px-4 md:px-12 py-8 md:py-24 ${isOnboarded === false ? "h-full flex items-center
justify-center" : ""}`}
        >
          {isOnboarded === null ? (
            <DashboardLoading />
          ) : isOnboarded === false ? (
            <DayZeroOnboarding onComplete={() => setIsOnboarded(true)} />
          ) : (
            children
          )}
        </div>
      </SmoothScrollProvider>
    </main>

    {/* MOBILE BOTTOM NAV */}
```

```
      {isOnboarded === true && (
        <nav className="md:hidden fixed bottom-0 left-0 right-0 bg-black/80 backdrop-blur-xl border-t border-white/5 z-50 px-6
py-4 flex justify-between items-center">
          {navItems.map((item) => (
            <button
              key={item.id}
              onClick={() => scrollToSection(item.id)}
              className={`flex flex-col items-center gap-1 font-mono text-[7px] tracking-widest uppercase transition-all ${
                activeSection === item.id ? "text-white" : "text-white/20"
              }`}
            >
              <item.icon size={14} />
              {item.label}
            </button>
          ))}
        </nav>
      )}
    </div>
  );
}
```

FILE: src\app\dashboard\LiabilitySection.tsx

```
"use client";

import { useState } from "react";
import { LIABILITY_TYPES, type Liability } from "@/lib/finance/liabilities";
import { createLiabilityAction } from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { LiabilityMap } from "@/lib/utils/taxonomy";
import { DistributionPieChart } from "@/components/dashboard/MiniCharts";
import { ScrollReveal } from "@/components/ui/scroll-reveal";

interface LiabilitySectionProps {
  liabilities: Liability[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function LiabilitySection({
  liabilities,
  onSnapshot,
}: LiabilitySectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const chartData = liabilities.map((l) => ({
    name: l.liability_name,
    value: Number(l.outstanding_balance),
  }));

  const [form, setForm] = useState({
    liability_name: "",
    liability_type: "credit_card",
    outstanding_balance: "",
    interest_rate: "",
    is_paid_in_cadences: false,
    cadence: "monthly",
    cadence_day_date: "",
    cadence_amount: "",
    institution: "",
  });
```

```
  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createLiabilityAction({
        liability_name: form.liability_name,
        liability_type: form.liability_type,
        outstanding_balance: Number(form.outstanding_balance),
        interest_rate: form.interest_rate ? Number(form.interest_rate) : 0,
        is_paid_in_cadences: form.is_paid_in_cadences,
        cadence: form.is_paid_in_cadences ? form.cadence : null,
        cadence_day_date: form.is_paid_in_cadences
          ? form.cadence_day_date
          : null,
        cadence_amount: form.is_paid_in_cadences
          ? Number(form.cadence_amount)
          : null,
        institution: form.institution || undefined,
      });
      setForm({
        liability_name: "",
        liability_type: "credit_card",
        outstanding_balance: "",
        interest_rate: "",
        is_paid_in_cadences: false,
        cadence: "monthly",
        cadence_day_date: "",
        cadence_amount: "",
        institution: "",
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error ? err.message : "Failed to record obligation",
      );
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-12">
      <div className="flex items-center justify-between">
        <h2 className="font-mono text-xl text-white tracking-widest uppercase">
          Financial Commitments
        </h2>
        <button
          onClick={() => setIsAdding(!isAdding)}
                    className="font-mono  text-[9px]  tracking-[0.4em]  uppercase  text-muted-foreground/80  hover:text-white
transition-colors"
        >
          {isAdding ? "? CANCEL" : "+ APPEND COMMITMENT"}
        </button>
      </div>

      {isAdding && (
        <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4 duration-500">
          <form onSubmit={handleSubmit} className="space-y-8">
            <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Commitment Name
```

```
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. Equity Bank Loan"
                  value={form.liability_name}
                  onChange={(e) =>
                    setForm({ ...form, liability_name: e.target.value })
                  }
                      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
                />
              </div>
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Financial Institution
                </label>
                <input
                  type="text"
                  placeholder="Optional"
                  value={form.institution}
                  onChange={(e) =>
                    setForm({ ...form, institution: e.target.value })
                  }
                      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
                />
              </div>
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Outstanding Balance
                </label>
                <input
                  type="number"
                  required
                  placeholder="0.00"
                  value={form.outstanding_balance}
                  onChange={(e) =>
                    setForm({ ...form, outstanding_balance: e.target.value })
                  }
                      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
                />
              </div>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Type
                </label>
                <select
                  value={form.liability_type}
                  onChange={(e) =>
                    setForm({ ...form, liability_type: e.target.value })
                  }
                        className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                >
                  {LIABILITY_TYPES.map((t) => (
                    <option key={t.value} value={t.value} className="bg-black">
                      {t.label}
                    </option>
                  ))}
                </select>
```

```
              </div>
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Interest (% per cadence)
                </label>
                <input
                  type="number"
                  step="0.01"
                  placeholder="0.00"
                  value={form.interest_rate}
                  onChange={(e) =>
                    setForm({ ...form, interest_rate: e.target.value })
                  }
                      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
                />
              </div>
            </div>

            <div className="space-y-4">
              <label className="flex items-center space-x-3 cursor-pointer group">
                <input
                  type="checkbox"
                  checked={form.is_paid_in_cadences}
                  onChange={(e) =>
                    setForm({ ...form, is_paid_in_cadences: e.target.checked })
                  }
                  className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white transition-colors cursor-pointer"
                />
                                <span  className="text-[10px]  font-mono  text-muted-foreground/80  uppercase  tracking-widest
group-hover:text-white/60 transition-colors">
                  Paid in Cadences
                </span>
              </label>

              {form.is_paid_in_cadences && (
                <div className="space-y-8 pt-4 animate-in fade-in slide-in-from-top-2">
                  <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
                    <div className="space-y-2">
                      <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                        Cadence
                      </label>
                      <select
                        value={form.cadence}
                        onChange={(e) =>
                          setForm({ ...form, cadence: e.target.value })
                        }
                            className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                      >
                        <option value="weekly" className="bg-black">
                          Weekly
                        </option>
                        <option value="monthly" className="bg-black">
                          Monthly
                        </option>
                      </select>
                    </div>
                    <div className="space-y-2">
                      <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                        {form.cadence === "monthly"
                          ? "Day of Month (1-31)"
                          : "Day of Week"}
                      </label>
                      {form.cadence === "monthly" ? (
```

```
                      <input
                        type="number"
                        min="1"
                        max="31"
                        placeholder="e.g. 15"
                        value={form.cadence_day_date}
                        onChange={(e) =>
                          setForm({
                            ...form,
                            cadence_day_date: e.target.value,
                          })
                        }
                                    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
                      />
                    ) : (
                      <select
                        value={form.cadence_day_date}
                        onChange={(e) =>
                          setForm({
                            ...form,
                            cadence_day_date: e.target.value,
                          })
                        }
                          className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                      >
                        <option value="" className="bg-black">
                          Select Day
                        </option>
                        {[
                          "Monday",
                          "Tuesday",
                          "Wednesday",
                          "Thursday",
                          "Friday",
                          "Saturday",
                          "Sunday",
                        ].map((day) => (
                          <option key={day} value={day} className="bg-black">
                            {day}
                          </option>
                        ))}
                      </select>
                    )}
                  </div>
                </div>

                <div className="space-y-2">
                  <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                    Payment Amount per Cadence
                  </label>
                  <input
                    type="number"
                    required={form.is_paid_in_cadences}
                    placeholder="0.00"
                    value={form.cadence_amount}
                    onChange={(e) =>
                      setForm({ ...form, cadence_amount: e.target.value })
                    }
                     className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
                    />
                  </div>
                </div>
```

```
            )}
          </div>

          {error && (
            <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
              {error}
            </p>
          )}

          <button
            type="submit"
            disabled={isLoading}
             className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px] hover:bg-white/90
transition-colors disabled:opacity-50"
          >
            {isLoading ? "INITIALIZING..." : "CONFIRM COMMITMENT"}
          </button>
        </form>
      </div>
    )}

    {liabilities.length > 0 && (
      <div className="py-6 border-b border-white/5 mb-6">
        <DistributionPieChart data={chartData} />
      </div>
    )}

    <div className="space-y-2">
      {liabilities.length === 0 ? (
        <div className="py-12 text-center opacity-20">
          <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
            No active commitments
          </p>
        </div>
      ) : (
        liabilities.map((liability, index) => (
          <ScrollReveal key={liability.id} delay={index * 0.05} distance={10}>
                    <div  className="flex  items-center  justify-between  py-6  border-b  border-white/5  group  hover:bg-white/1
transition-all px-2">
              <div className="space-y-1">
                <span className="text-[8px] font-mono tracking-widest uppercase text-muted-foreground/60">
                  {LiabilityMap[liability.liability_type] ||
                    liability.liability_type}
                  {liability.is_paid_in_cadences && (
                    <span className="ml-2 text-muted-foreground/50">
                      ? {liability.cadence} ({liability.cadence_day_date})
                    </span>
                  )}
                </span>
                <h3 className="font-mono text-lg text-white transition-transform group-hover:translate-x-2">
                  {liability.liability_name}
                </h3>
              </div>
              <div className="text-right">
                <p className="font-mono text-lg text-white tabular-nums">
                  {formatCurrency(liability.outstanding_balance)}
                </p>
                {liability.interest_rate > 0 && (
                  <p className="text-[8px] font-mono text-red-500/40 uppercase tracking-widest mt-1">
                    {liability.interest_rate}% Per Cadence
                  </p>
                )}
              </div>
            </div>
          </ScrollReveal>
```

```
          ))
        )}
      </div>
    </div>
  );
}
```

FILE: src\app\dashboard\loading.tsx

```
export default function DashboardLoading() {
  return (
    <div className="fixed inset-0 bg-black z-100 flex items-center justify-center px-8 md:px-12 animate-pulse">
      <div className="max-w-7xl w-full h-150 grid lg:grid-cols-[1.1fr_1.4fr] gap-16 md:gap-24 items-stretch">
        {/* Left Side Skeleton - Roman Numeral & Text */}
        <div className="flex flex-col justify-center h-full border-r border-white/5 pr-12 space-y-6">
          <div className="h-32 w-48 bg-white/5 rounded-2xl" />{" "}
          {/* Roman Numeral */}
          <div className="h-16 w-64 bg-white/10 rounded-xl" /> {/* Title */}
          <div className="space-y-3">
            <div className="h-4 w-full bg-white/5 rounded-full" />
            <div className="h-4 w-5/6 bg-white/5 rounded-full" />
            <div className="h-4 w-4/6 bg-white/5 rounded-full" />
          </div>
        </div>

        {/* Right Side Skeleton - Luxury Card */}
        <div className="h-full bg-white/2 border border-white/10 rounded-3xl p-10 md:p-14 flex flex-col justify-between">
          <div className="space-y-12">
            <div className="h-8 w-48 bg-white/5 rounded-lg" />{" "}
            {/* Form Header */}
            <div className="space-y-10">
              <div className="grid grid-cols-2 gap-8">
                <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
                <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
              </div>
              <div className="grid grid-cols-2 gap-8">
                <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
                <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
              </div>
            </div>
          </div>

          <div className="flex justify-center pt-12">
            <div className="h-14 w-64 bg-white/5 rounded-full border border-white/10" />
          </div>
        </div>
      </div>
    </div>
  );
}
```

FILE: src\app\dashboard\NewEntryForm.tsx

```
"use client";

import { useState, useEffect } from "react";
import { TAXONOMY_CATEGORIES } from "@/lib/finance/taxonomy";
import { DeploymentMap } from "@/lib/utils/taxonomy";
import { formatCurrency } from "@/lib/utils/formatters";
import { Account } from "@/lib/analytics/types";
import { motion, AnimatePresence } from "framer-motion";
```

```
import { Check, Loader2 } from "lucide-react";

interface NewEntryFormProps {
  accounts: Account[];
  liquidity: number;
  isActionLoading: boolean;
  onSubmit: (data: {
    title: string;
    amount: number;
    category: string;
    accountId?: string;
  }) => Promise<void>;
  onCancel?: () => void;
}

export function NewEntryForm({
  accounts,
  liquidity,
  isActionLoading,
  onSubmit,
  onCancel,
}: NewEntryFormProps) {
  const [title, setTitle] = useState("");
  const [amount, setAmount] = useState("");
  const [category, setCategory] = useState("Unclassified");
  const [accountId, setAccountId] = useState("");
  const [error, setError] = useState<string | null>(null);

  // Execution Animation State
  const [executionState, setExecutionState] = useState<"idle" | "loading" | "success">("idle");

  useEffect(() => {
    if (isActionLoading) {
      setExecutionState("loading");
    } else if (executionState === "loading" && !error) {
      setExecutionState("success");
      if (typeof navigator !== "undefined" && navigator.vibrate) {
        navigator.vibrate([30, 50, 30]); // Heavy tactile confirmation
      }
    } else if (executionState === "success") {
      const timer = setTimeout(() => setExecutionState("idle"), 2000);
      return () => clearTimeout(timer);
    } else if (error) {
      setExecutionState("idle");
    }
  }, [isActionLoading, error, executionState]);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);

    if (category === "Unclassified") {
      setError("Strategic intent required.");
      return;
    }

    try {
      await onSubmit({
        title,
        amount: Number(amount),
        category,
        accountId: accountId || undefined,
      });
      setTitle("");
      setAmount("");
```

```
      setCategory("Unclassified");
      setAccountId("");
    } catch (err: unknown) {
      setError(err instanceof Error ? err.message : "Execution failed");
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-8 md:space-y-12">
      <div className="space-y-8 md:space-y-12">
        <div className="space-y-4">
          <label className="text-[10px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em] ml-1">
            Intent
          </label>
          <input
            type="text"
            disabled={isActionLoading}
            placeholder="Strategic objective?"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
                  className="w-full bg-transparent border-b border-white/10 py-6 font-mono text-xl md:text-3xl text-white
placeholder:text-muted-foreground/40 focus:outline-none focus:border-white transition-all"
            required
          />
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-12 md:gap-16">
          <div className="space-y-12">
            <div className="space-y-4">
              <div className="flex justify-between items-center">
                <label className="text-[10px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em] ml-1">
                  Amount (KES)
                </label>
                <span className="text-[8px] font-mono text-zinc-500 uppercase tracking-widest">
                  Available: {formatCurrency(liquidity)}
                </span>
              </div>
              <input
                type="number"
                disabled={isActionLoading}
                placeholder="0.00"
                value={amount}
                onChange={(e) => setAmount(e.target.value)}
                            className="w-full bg-transparent border-b border-white/10 py-4 font-mono text-2xl text-white
placeholder:text-muted-foreground/60 focus:outline-none focus:border-white transition-all tabular-nums"
                required
              />
            </div>
            <div className="space-y-4">
              <label className="text-[10px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em] ml-1">
                Funding Source
              </label>
              <select
                disabled={isActionLoading}
                value={accountId}
                onChange={(e) => setAccountId(e.target.value)}
                 className="w-full bg-transparent border-b border-white/10 py-3 font-mono text-[10px] tracking-widest uppercase
text-white/60 focus:outline-none"
              >
                <option value="" className="bg-black">
                  External / Manual
                </option>
                {accounts.map((acc) => (
                  <option key={acc.id} value={acc.id} className="bg-black">
                    {acc.account_name} ({formatCurrency(acc.current_balance)})
```

```
                </option>
              ))}
            </select>
          </div>
        </div>

        <div className="space-y-6">
          <label className="text-[10px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em] ml-1">
            Strategic Classification
          </label>
          <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-2">
            {TAXONOMY_CATEGORIES.map((cat) => (
              <button
                key={cat.value}
                type="button"
                onClick={() => {
                  // Simulated Web Haptics (Vibration on supported mobile, visual flash otherwise)
                  if (typeof navigator !== "undefined" && navigator.vibrate) {
                    navigator.vibrate(10);
                  }
                  setCategory(cat.value);
                }}
                className={`p-3 border transition-all text-left flex flex-col justify-center items-center active:scale-95 ${
                  category === cat.value
                    ? "border-truth bg-truth/10 text-truth shadow-[inset_0_0_10px_rgba(59,130,246,0.2)]"
                    : "border-white/5 bg-white/2 text-zinc-500 hover:border-white/20 hover:text-white"
                }`}
              >
                <span className="block text-[8px] font-bold tracking-[0.2em] uppercase truncate">
                  {DeploymentMap[cat.value] || cat.label}
                </span>
              </button>
            ))}
          </div>

          {/* Dynamic Definition Display */}
          {category !== "Unclassified" && (
                      <div  className="p-4  bg-zinc-900/50  border-l-2  border-truth  animate-in  fade-in  slide-in-from-left-2
duration-300">
              <p className="text-[9px] text-zinc-400 uppercase tracking-widest font-bold mb-1">
                Intent Definition // {category.replace("_", " ")}
              </p>
              <p className="text-[10px] text-zinc-300 leading-relaxed italic">
                &amp;quot;{TAXONOMY_CATEGORIES.find(c => c.value === category)?.definition}&amp;quot;
              </p>
            </div>
          )}
        </div>
      </div>
    </div>

    {error && (
      <div className="p-4 border border-white/10 font-mono text-[9px] text-white/60 uppercase tracking-widest">
        ERROR: {error}
      </div>
    )}

    <div className="pt-8 flex gap-4">
      {onCancel && (
        <button
          type="button"
          onClick={onCancel}
          disabled={isActionLoading || executionState !== "idle"}
              className="flex-1 border border-white/5 text-muted-foreground/80 py-6 font-medium tracking-[0.3em] uppercase
text-xs hover:bg-white/5 transition-all disabled:opacity-50"
```

```
          >
            Cancel
          </button>
        )}
        <button
          type="submit"
          disabled={isActionLoading || executionState !== "idle"}
             className={`flex-2 py-6 font-bold tracking-[0.3em] uppercase text-[10px] transition-all relative overflow-hidden
flex items-center justify-center min-w-[200px] ${
            executionState === "success"
              ? "bg-truth text-white border-none"
                      :  "bg-white  text-black  hover:bg-zinc-200  active:scale-[0.98]  shadow-[0_0_15px_rgba(255,255,255,0.05)]
disabled:opacity-50"
          }`}
        >
          <AnimatePresence mode="wait">
            {executionState === "idle" && (
              <motion.span
                key="idle"
                initial={{ opacity: 0, y: 10 }}
                animate={{ opacity: 1, y: 0 }}
                exit={{ opacity: 0, y: -10 }}
                transition={{ duration: 0.2 }}
              >
                AUTHORIZE DEPLOYMENT
              </motion.span>
            )}
            {executionState === "loading" && (
              <motion.span
                key="loading"
                initial={{ opacity: 0, scale: 0.8 }}
                animate={{ opacity: 1, scale: 1 }}
                exit={{ opacity: 0, scale: 0.8 }}
                className="flex items-center gap-3"
              >
                <Loader2 size={16} className="animate-spin text-black" />
                EXECUTING...
              </motion.span>
            )}
            {executionState === "success" && (
              <motion.span
                key="success"
                initial={{ opacity: 0, scale: 0.8 }}
                animate={{ opacity: 1, scale: 1 }}
                exit={{ opacity: 0, scale: 1.2 }}
                className="flex items-center gap-3"
              >
                <Check strokeWidth={3} size={16} />
                CONFIRMED
              </motion.span>
            )}
          </AnimatePresence>
        </button>
      </div>
    </form>
  );
}
```

FILE: src\app\dashboard\page.tsx

```
import { getDashboardSnapshotAction } from "./actions";
import { DashboardClient } from "./DashboardClient";
```

```
/**
 * DASHBOARD SERVER COMPONENT (RSC)
 * Adheres to 'Axiom Truth Enforcement' by fetching authoritative data on the server.
 */
export default async function DashboardPage() {
  // Fetch authoritative snapshot directly on the server
  const snapshot = await getDashboardSnapshotAction();

  return <DashboardClient initialSnapshot={snapshot} />;
}
```

FILE: src\app\dashboard\PendingBaselines.tsx

```
"use client";

import { useState } from "react";
import type { OperationalBaseline, Account } from "@/lib/analytics/types";
import { resolvePendingBaselineAction } from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { TacticalEmptyState } from "./PendingLiabilities";

interface PendingBaselinesProps {
  baseline: OperationalBaseline[];
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function PendingBaselines({
  baseline,
  accounts,
  onSnapshot,
}: PendingBaselinesProps) {
  const [loadingId, setLoadingId] = useState<string | null>(null);
  const [selectedAccounts, setSelectedAccounts] = useState<
    Record<string, string>
  >({});

  const today = new Date();
  const currentDayOfWeek = today.getDay() === 0 ? 7 : today.getDay();
  const currentDayOfMonth = today.getDate();

  const pendingBaselines = baseline.filter((b) => {
    if (!b.is_recurring) return false;

    // Check if already executed today
    if (b.last_executed_at) {
      const lastExecuted = new Date(b.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&
        lastExecuted.getFullYear() === today.getFullYear()
      ) {
        return false;
      }
    }

    // Daily baselines are always pending if not executed today
    if (b.cadence === "daily") return true;

    if (b.execution_day) {
      if (b.cadence === "weekly") {
        return currentDayOfWeek === b.execution_day;
```

```
      }
      if (b.cadence === "monthly" || b.cadence === "biweekly") {
        return currentDayOfMonth === b.execution_day;
      }
    }

    return false;
  });

  if (pendingBaselines.length === 0) {
       return <TacticalEmptyState title="Maintenance Clear" message="No pending operational burn or maintenance baseline
requirements detected for the current cycle." />;
  }

  async function handleVerify(item: OperationalBaseline) {
    const accountId = selectedAccounts[item.id];
    if (!accountId) {
      alert("Please select a source account.");
      return;
    }

    setLoadingId(item.id);
    try {
      const snapshot = await resolvePendingBaselineAction({
        baselineId: item.id,
        accountId,
        amount: item.amount,
        title: item.title,
      });
      onSnapshot(snapshot);
    } catch (err: unknown) {
      alert(err instanceof Error ? err.message : "Verification failed");
    } finally {
      setLoadingId(null);
    }
  }

  return (
    <div className="space-y-6 animate-in fade-in duration-700">
      <div className="flex items-center gap-4">
        <div className="w-2 h-2 rounded-full bg-red-500 animate-pulse"></div>
        <h2 className="text-[10px] font-mono text-white/60 uppercase tracking-[0.4em]">
          Baseline Verification Required
        </h2>
      </div>

      {pendingBaselines.map((item) => (
        <div
          key={item.id}
                 className="bg-red-500/10 border border-red-500/20 p-8 md:p-12 flex flex-col md:flex-row md:items-center
justify-between gap-8 text-white"
        >
          <div className="space-y-2">
            <p className="text-[10px] font-mono tracking-widest uppercase text-red-500/60">
              Structural Maintenance Prompt ({item.cadence})
            </p>
            <h3 className="font-mono text-3xl tabular-nums">
              {formatCurrency(item.amount)}
            </h3>
            <p className="text-xs font-light tracking-wide opacity-80">
              {item.title}
            </p>
          </div>

          <div className="flex flex-col sm:flex-row items-stretch sm:items-center gap-4">
```

```
            <select
              disabled={loadingId === item.id}
              value={selectedAccounts[item.id] || ""}
              onChange={(e) =>
                setSelectedAccounts({
                  ...selectedAccounts,
                  [item.id]: e.target.value,
                })
              }
                    className="bg-transparent border-b border-white/40 py-2 font-mono text-[10px] tracking-widest uppercase
focus:outline-none focus:border-white transition-colors min-w-50"
            >
              <option value="" className="text-black">
                Source Account
              </option>
              {accounts.map((acc) => (
                <option key={acc.id} value={acc.id} className="text-black">
                  {acc.account_name}
                </option>
              ))}
            </select>

            <button
              disabled={loadingId === item.id}
              onClick={() => handleVerify(item)}
                 className="bg-white text-black px-10 py-4 font-medium tracking-widest uppercase text-[10px] hover:bg-white/90
transition-all disabled:opacity-50"
            >
              {loadingId === item.id ? "VERIFYING..." : "CONFIRM SETTLEMENT"}
            </button>
          </div>
        </div>
      ))}
    </div>
  );
}
```

FILE: src\app\dashboard\PendingInflows.tsx

```
"use client";

import { useState } from "react";
import type { IncomeStream, Account } from "@/lib/analytics/types";
import { resolvePendingInflowAction } from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { IncomeMap } from "@/lib/utils/taxonomy";
import { TacticalEmptyState } from "./PendingLiabilities";

interface PendingInflowsProps {
  incomeStreams: IncomeStream[];
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function PendingInflows({
  incomeStreams,
  accounts,
  onSnapshot,
}: PendingInflowsProps) {
  const [loadingId, setLoadingId] = useState<string | null>(null);
  const [selectedAccounts, setSelectedAccounts] = useState<
    Record<string, string>
```

```
  >({});

  const today = new Date();
  const currentDayOfWeek = today.getDay();
  const currentDayOfMonth = today.getDate();
  const mappedDayOfWeek = currentDayOfWeek === 0 ? 7 : currentDayOfWeek;

  const pendingInflows = incomeStreams.filter((stream) => {
    if (!stream.is_recurring || !stream.execution_day) return false;

    if (stream.last_executed_at) {
      const lastExecuted = new Date(stream.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&
        lastExecuted.getFullYear() === today.getFullYear()
      ) {
        return false;
      }
    }

    if (stream.cadence === "weekly") {
      return mappedDayOfWeek === stream.execution_day;
    }
    if (stream.cadence === "monthly" || stream.cadence === "biweekly") {
      return currentDayOfMonth === stream.execution_day;
    }

    return false;
  });

  if (pendingInflows.length === 0) {
      return <TacticalEmptyState title="Inbound Clear" message="No pending yield or inbound capital flows detected for the
current operational cycle." />;
  }

  async function handleVerify(stream: IncomeStream) {
    const accountId = selectedAccounts[stream.id];
    if (!accountId) {
      alert("Please select a destination account.");
      return;
    }

    setLoadingId(stream.id);
    try {
      const snapshot = await resolvePendingInflowAction({
        incomeId: stream.id,
        accountId,
        amount: stream.amount,
      });
      onSnapshot(snapshot);
    } catch (err: unknown) {
      alert(err instanceof Error ? err.message : "Verification failed");
    } finally {
      setLoadingId(null);
    }
  }

  return (
    <div className="space-y-6 animate-in fade-in duration-700">
      <div className="flex items-center gap-4">
        <div className="w-2 h-2 rounded-full bg-white animate-pulse"></div>
        <h2 className="text-[10px] font-mono text-white/60 uppercase tracking-[0.4em]">
          Verification Required
        </h2>
```

```
      </div>

      {pendingInflows.map((stream) => (
        <div
          key={stream.id}
          className="bg-white p-8 md:p-12 flex flex-col md:flex-row md:items-center justify-between gap-8 text-black"
        >
          <div className="space-y-2">
            <p className="text-[10px] font-mono tracking-widest uppercase opacity-40">
              Expected Income Detected
            </p>
            <h3 className="font-mono text-3xl tabular-nums">
              {formatCurrency(stream.amount)}
            </h3>
            <p className="text-xs font-light tracking-wide opacity-60">
              {stream.income_name} (
              {IncomeMap[stream.income_type] || stream.income_type})
            </p>
          </div>

          <div className="flex flex-col sm:flex-row items-stretch sm:items-center gap-4">
            <select
              disabled={loadingId === stream.id}
              value={selectedAccounts[stream.id] || ""}
              onChange={(e) =>
                setSelectedAccounts({
                  ...selectedAccounts,
                  [stream.id]: e.target.value,
                })
              }
                    className="bg-transparent border-b border-black/20 py-2 font-mono text-[10px] tracking-widest uppercase
focus:outline-none focus:border-black transition-colors min-w-50"
            >
              <option value="">Destination Account</option>
              {accounts.map((acc) => (
                <option key={acc.id} value={acc.id}>
                  {acc.account_name}
                </option>
              ))}
            </select>

            <button
              disabled={loadingId === stream.id}
              onClick={() => handleVerify(stream)}
                 className="bg-black text-white px-10 py-4 font-medium tracking-widest uppercase text-[10px] hover:bg-black/90
transition-all disabled:opacity-50"
            >
              {loadingId === stream.id ? "VERIFYING..." : "CONFIRM RECEIPT"}
            </button>
          </div>
        </div>
      ))}
    </div>
  );
}
```

FILE: src\app\dashboard\PendingLiabilities.tsx

```
"use client";

import { useState } from "react";
import type { Liability, Account } from "@/lib/analytics/types";
import { resolvePendingLiabilityAction } from "./actions";
```

```
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { LiabilityMap } from "@/lib/utils/taxonomy";

interface PendingLiabilitiesProps {
  liabilities: Liability[];
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

// Shared Empty State Component
import { Radar } from "lucide-react";

export function TacticalEmptyState({ title, message }: { title: string, message: string }) {
  return (
      <div  className="flex  flex-col  items-center  justify-center  py-16  px-6  text-center  space-y-6  relative  overflow-hidden
bg-[#0a0a0a] border border-white/5 rounded-sm">
      <div className="relative flex items-center justify-center w-24 h-24 mb-2">
        <div className="absolute inset-0 rounded-full border border-truth/10" />
        <div className="absolute inset-4 rounded-full border border-truth/20" />
         <div className="absolute inset-0 rounded-full border-t border-truth/60 animate-spin" style={{ animationDuration: '4s'
}} />
        <Radar size={24} className="text-truth/60 relative z-10" />
      </div>
      <div className="space-y-2 relative z-10">
        <h3 className="font-mono text-xs tracking-[0.2em] text-truth/80 uppercase font-bold">
          {title}
        </h3>
        <p className="font-mono text-[9px] tracking-widest text-zinc-500 uppercase max-w-[250px] leading-relaxed">
          {message}
        </p>
      </div>
    </div>
  );
}

export function PendingLiabilities({
  liabilities,
  accounts,
  onSnapshot,
}: PendingLiabilitiesProps) {
  const [loadingId, setLoadingId] = useState<string | null>(null);
  const [selectedAccounts, setSelectedAccounts] = useState<
    Record<string, string>
  >({});

  const today = new Date();
  const currentDayOfWeek = today
    .toLocaleString("en-US", { weekday: "long" })
    .toLowerCase();
  const currentDayOfMonth = today.getDate();

  const pendingLiabilities = liabilities.filter((liab) => {
    if (!liab.is_paid_in_cadences || !liab.cadence_day_date) return false;

    // Check if already executed today
    if (liab.last_executed_at) {
      const lastExecuted = new Date(liab.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&
        lastExecuted.getFullYear() === today.getFullYear()
      ) {
        return false;
      }
```

```
    }

    if (liab.cadence === "weekly") {
      return currentDayOfWeek === liab.cadence_day_date.toLowerCase();
    }
    if (liab.cadence === "monthly") {
      return currentDayOfMonth === parseInt(liab.cadence_day_date, 10);
    }

    return false;
  });

  if (pendingLiabilities.length === 0) {
    return <TacticalEmptyState title="Obligations Clear" message="No pending liabilities or scheduled debt executions detected
for the current operational cycle." />;
  }

  async function handleVerify(liab: Liability) {
    const accountId = selectedAccounts[liab.id];
    if (!accountId) {
      alert("Please select a source account.");
      return;
    }

    setLoadingId(liab.id);
    try {
      const snapshot = await resolvePendingLiabilityAction({
        liabilityId: liab.id,
        accountId,
        amount: liab.cadence_amount || 0,
      });
      onSnapshot(snapshot);
    } catch (err: unknown) {
      alert(err instanceof Error ? err.message : "Verification failed");
    } finally {
      setLoadingId(null);
    }
  }

  return (
    <div className="space-y-6 animate-in fade-in duration-700">
      <div className="flex items-center gap-4">
        <div className="w-2 h-2 rounded-full bg-red-500 animate-pulse"></div>
        <h2 className="text-[10px] font-mono text-white/60 uppercase tracking-[0.4em]">
          Payment Verification Required
        </h2>
      </div>

      {pendingLiabilities.map((liab) => (
        <div
          key={liab.id}
          className="bg-red-500 p-8 md:p-12 flex flex-col md:flex-row md:items-center justify-between gap-8 text-white"
        >
          <div className="space-y-2">
            <p className="text-[10px] font-mono tracking-widest uppercase opacity-60">
              Scheduled Obligation Detected
            </p>
            <h3 className="font-mono text-3xl tabular-nums">
              {formatCurrency(liab.cadence_amount || 0)}
            </h3>
            <p className="text-xs font-light tracking-wide opacity-80">
              {liab.liability_name} (
              {LiabilityMap[liab.liability_type] || liab.liability_type})
            </p>
          </div>
```

```
          <div className="flex flex-col sm:flex-row items-stretch sm:items-center gap-4">
            <select
              disabled={loadingId === liab.id}
              value={selectedAccounts[liab.id] || ""}
              onChange={(e) =>
                setSelectedAccounts({
                  ...selectedAccounts,
                  [liab.id]: e.target.value,
                })
              }
                    className="bg-transparent border-b border-white/40 py-2 font-mono text-[10px] tracking-widest uppercase
focus:outline-none focus:border-white transition-colors min-w-50"
            >
              <option value="" className="text-black">
                Source Account
              </option>
              {accounts.map((acc) => (
                <option key={acc.id} value={acc.id} className="text-black">
                  {acc.account_name}
                </option>
              ))}
            </select>

            <button
              disabled={loadingId === liab.id}
              onClick={() => handleVerify(liab)}
               className="bg-white text-red-500 px-10 py-4 font-medium tracking-widest uppercase text-[10px] hover:bg-white/90
transition-all disabled:opacity-50"
            >
              {loadingId === liab.id ? "VERIFYING..." : "CONFIRM SETTLEMENT"}
            </button>
          </div>
        </div>
      ))}
    </div>
  );
}
```

FILE: src\app\dashboard\RunwayCard.tsx

```
"use client";

import React from "react";

interface RunwayCardProps {
  runwayDays: number | null;
  netWorth?: number;
}

export function RunwayCard({ runwayDays, netWorth = 0 }: RunwayCardProps) {
  // 1. Logic for determining visual state
  const isInfiniteRunway = runwayDays === null;
  const isCritical = !isInfiniteRunway && runwayDays! < 30;
  const isInsolvent = netWorth < 0;

  // 2. Formatting
  const displayValue = isInfiniteRunway
    ? isInsolvent
      ? "INSOLVENT STABLE"
      : "SYSTEM STABLE"
    : `${(runwayDays! / 30).toFixed(1)} MONTHS`;
```

```
  // 3. Conditional Styles
   const containerClasses = `bg-foreground/5 border rounded-2xl p-6 md:p-8 flex flex-col justify-between transition-all
duration-500 ${
    isInfiniteRunway
      ? isInsolvent
        ? "border-rose-500/20"
        : "border-emerald-500/10"
      : isCritical
        ? "border-rose-500/30 bg-rose-500/5 shadow-inner"
        : "border-foreground/10"
  }`;

  const labelClasses = `text-[11px] font-black uppercase tracking-[0.2em] mb-4 ${
    isInfiniteRunway
      ? isInsolvent
        ? "text-rose-500/50"
        : "text-emerald-500/50"
      : isCritical
        ? "text-rose-500/60"
        : "text-foreground/40"
  }`;

  const valueClasses = `text-4xl font-black tabular-nums transition-colors duration-500 ${
    isInfiniteRunway
      ? isInsolvent
        ? "text-rose-500/80 font-mono tracking-tighter"
        : "text-emerald-500/80 font-mono tracking-tighter"
      : isCritical
        ? "text-rose-600"
        : "text-foreground"
  }`;

  const description = isInfiniteRunway
    ? isInsolvent
      ? "Burn is absorbed by incomes, but total commitments exceed assets."
      : "Net inbound capital completely absorbs current structural burn."
    : isCritical
      ? "A recalibration of The Foundation is recommended."
      : "Duration of sustained operation based on current structural burn and liquidity.";

  return (
    <div className={containerClasses}>
      <div className="flex items-center justify-between mb-4">
        <span className={labelClasses}>
          {isInfiniteRunway
            ? isInsolvent
              ? "Insolvent Stability"
              : "Infinite Stability"
            : "Operating Horizon"}
        </span>
        {isInfiniteRunway && !isInsolvent && (
          <div className="flex items-center gap-1.5">
            <span className="w-1.5 h-1.5 bg-emerald-500/40 rounded-full animate-pulse"></span>
            <span className="text-[8px] font-black text-emerald-500/40 uppercase tracking-widest">
              Active Protection
            </span>
          </div>
        )}
        {isInfiniteRunway && isInsolvent && (
          <div className="flex items-center gap-1.5">
            <span className="w-1.5 h-1.5 bg-rose-500/40 rounded-full animate-pulse"></span>
            <span className="text-[8px] font-black text-rose-500/40 uppercase tracking-widest">
              Solvency Risk
            </span>
          </div>
```

```
        )}
      </div>

      <div className="flex flex-col">
        <span className={valueClasses}>{displayValue}</span>
        <p
          className={`text-[10px] font-bold uppercase tracking-widest mt-4 ${
            isInfiniteRunway ? "text-emerald-600/40" : "text-foreground/40"
          }`}
        >
          {description}
        </p>
      </div>
    </div>
  );
}
```

FILE: src\app\dashboard\StrategicObjectiveSection.tsx

```
"use client";

import { useState } from "react";
import {
  calculateObjectiveFundingRatio,
  type StrategicObjective,
  OBJECTIVE_TYPES,
  OBJECTIVE_PRIORITIES,
  OBJECTIVE_STATUSES,
} from "@/lib/finance/objectives";
import {
  createStrategicObjectiveAction,
  updateStrategicObjectiveAction,
} from "./actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { ScrollReveal } from "@/components/ui/scroll-reveal";
import { motion } from "framer-motion";

interface StrategicObjectiveSectionProps {
  objectives: StrategicObjective[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function StrategicObjectiveSection({
  objectives,
  onSnapshot,
}: StrategicObjectiveSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    objective_name: "",
    objective_type: "emergency_reserve",
    target_amount: "",
    current_amount: "",
    priority_level: "moderate",
    status: "active",
    target_date: "",
    notes: "",
  });
```

```
  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      let snapshot;
      if (editingId) {
        snapshot = await updateStrategicObjectiveAction(editingId, {
          objective_name: form.objective_name,
          objective_type: form.objective_type,
          target_amount: Number(form.target_amount),
          current_amount: form.current_amount ? Number(form.current_amount) : 0,
          priority_level: form.priority_level,
          status: form.status,
          target_date: form.target_date || null,
          notes: form.notes || undefined,
        });
      } else {
        snapshot = await createStrategicObjectiveAction({
          objective_name: form.objective_name,
          objective_type: form.objective_type,
          target_amount: Number(form.target_amount),
          current_amount: form.current_amount ? Number(form.current_amount) : 0,
          priority_level: form.priority_level,
          status: form.status,
          target_date: form.target_date || null,
          notes: form.notes || undefined,
        });
      }
      setForm({
        objective_name: "",
        objective_type: "emergency_reserve",
        target_amount: "",
        current_amount: "",
        priority_level: "moderate",
        status: "active",
        target_date: "",
        notes: "",
      });
      setIsAdding(false);
      setEditingId(null);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error
          ? err.message
          : "Failed to establish strategic intent",
      );
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-12">
      <div className="flex items-center justify-between">
        <h2 className="font-mono text-xl text-white tracking-widest uppercase">
          Strategic Intent
        </h2>
        <button
          onClick={() => setIsAdding(!isAdding)}
                    className="font-mono  text-[9px]  tracking-[0.4em]  uppercase  text-muted-foreground/80  hover:text-white
transition-colors"
        >
          {isAdding ? "? CANCEL" : "+ APPEND OBJECTIVE"}
```

```
        </button>
      </div>

      {isAdding && (
        <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4 duration-500">
          <form onSubmit={handleSubmit} className="space-y-8">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Objective Name
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. Emergency Fund"
                  value={form.objective_name}
                  onChange={(e) =>
                    setForm({ ...form, objective_name: e.target.value })
                  }
                      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
                />
              </div>
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Objective Type
                </label>
                <select
                  value={form.objective_type}
                  onChange={(e) =>
                    setForm({ ...form, objective_type: e.target.value })
                  }
                         className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                >
                  {OBJECTIVE_TYPES.map((type) => (
                    <option
                      key={type.value}
                      value={type.value}
                      className="bg-black"
                    >
                      {type.label}
                    </option>
                  ))}
                </select>
              </div>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Priority Level
                </label>
                <select
                  value={form.priority_level}
                  onChange={(e) =>
                    setForm({ ...form, priority_level: e.target.value })
                  }
                         className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                >
                  {OBJECTIVE_PRIORITIES.map((p) => (
                    <option key={p.value} value={p.value} className="bg-black">
                      {p.label}
                    </option>
```

```
                ))}
              </select>
            </div>
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                Status
              </label>
              <select
                value={form.status}
                onChange={(e) => setForm({ ...form, status: e.target.value })}
                       className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
              >
                {OBJECTIVE_STATUSES.map((s) => (
                  <option key={s.value} value={s.value} className="bg-black">
                    {s.label}
                  </option>
                ))}
              </select>
            </div>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                Target Amount
              </label>
              <input
                type="number"
                required
                placeholder="0.00"
                value={form.target_amount}
                onChange={(e) =>
                  setForm({ ...form, target_amount: e.target.value })
                }
                    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
              />
            </div>
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                Current Position
              </label>
              <input
                type="number"
                placeholder="0.00"
                value={form.current_amount}
                onChange={(e) =>
                  setForm({ ...form, current_amount: e.target.value })
                }
                    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
              />
            </div>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                Target Date
              </label>
              <input
                type="date"
                value={form.target_date}
                onChange={(e) =>
```

```
                  setForm({ ...form, target_date: e.target.value })
                }
                       className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono text-[10px]
tracking-widest uppercase focus:outline-none"
                />
              </div>
              <div className="space-y-2">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-widest">
                  Strategic Notes
                </label>
                <input
                  type="text"
                  placeholder="Add context..."
                  value={form.notes}
                  onChange={(e) => setForm({ ...form, notes: e.target.value })}
                      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light focus:outline-none
focus:border-white transition-colors"
                />
              </div>
            </div>

            {error && (
              <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
               className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px] hover:bg-white/90
transition-colors disabled:opacity-50"
            >
              {isLoading ? "ESTABLISHING..." : "SET STRATEGIC INTENT"}
            </button>
          </form>
        </div>
      )}

      <div className="space-y-8">
        {objectives.length === 0 ? (
          <div className="py-12 text-center opacity-20">
            <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
              No objectives active
            </p>
          </div>
        ) : (
          objectives.map((obj, index) => {
            const ratio = calculateObjectiveFundingRatio(obj);
            return (
              <ScrollReveal key={obj.id} delay={index * 0.1}>
                <div className="space-y-4 group cursor-default pb-8 border-b border-white/5 last:border-0">
                  <div className="flex items-center justify-between mb-2">
                    <div className="space-y-1">
                      <span
                        className={`text-[8px] font-mono tracking-widest uppercase ${
                          obj.priority_level === "critical"
                            ? "text-red-500"
                            : "text-muted-foreground/60"
                        }`}
                      >
                        {obj.priority_level}
                      </span>
                      <h3 className="font-mono text-xl text-white transition-transform group-hover:translate-x-2">
                        {obj.objective_name}
```

```
                    </h3>
                  </div>
                  <div className="text-right">
                    <p className="font-mono text-2xl text-white tabular-nums">
                      {Math.round(ratio)}%
                    </p>
                  </div>
                </div>

                <div className="h-px w-full bg-white/5 relative overflow-hidden">
                  <motion.div
                    initial={{ width: 0 }}
                    whileInView={{ width: `${Math.min(100, ratio)}%` }}
                    viewport={{ once: true }}
                    transition={{
                      duration: 1.5,
                      delay: 0.5 + index * 0.1,
                      ease: "easeOut",
                    }}
                    className="absolute inset-y-0 left-0 bg-white/60"
                  />
                </div>

                <div className="flex justify-between items-end pt-2 opacity-30 group-hover:opacity-60 transition-opacity">
                  <p className="text-[9px] font-mono uppercase tracking-widest">
                    {formatCurrency(obj.current_amount)}
                  </p>
                  <p className="text-[9px] font-mono uppercase tracking-widest">
                    Target: {formatCurrency(obj.target_amount)}
                  </p>
                </div>
              </div>
            </ScrollReveal>
          );
        })
      )}
    </div>
  </div>
);
}
```

FILE: src\app\signup\page.tsx

```
"use client";

import { useEffect } from "react";
import { useRouter } from "next/navigation";
import { Auth } from "@supabase/auth-ui-react";
import { ThemeSupa } from "@supabase/auth-ui-shared";
import { supabase } from "@/lib/supabase";
import Link from "next/link";
import { BrandMark } from "@/components/ui/brand-mark";
import { ScrollReveal } from "@/components/ui/scroll-reveal";

export default function SignUp() {
  const router = useRouter();

  useEffect(() => {
    const {
      data: { subscription },
    } = supabase.auth.onAuthStateChange((event, session) => {
      if (session) {
        router.push("/dashboard");
```

```
      }
    });

    return () => {
      subscription.unsubscribe();
    };
  }, [router]);

  return (
     <main className="min-h-screen flex flex-col md:flex-row bg-black text-white relative overflow-hidden selection:bg-white selection:text-black">
      {/* Left Side: Editorial Canvas - Tactical Intelligence */}
         <div className="hidden md:flex md:w-1/2 bg-[#020202] relative items-center justify-center border-r border-white/5 overflow-hidden">
        {/* The Tactical Schematic Image */}
        <div
          className="absolute inset-0 z-0 opacity-40 mix-blend-screen scale-110 grayscale"
          style={{
            backgroundImage: "url('/images/tactical-schematic.png')",
            backgroundSize: "cover",
            backgroundPosition: "center",
          }}
        />

        {/* Depth & Focus Overlays */}
        <div className="absolute inset-0 z-0">
           <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[90%] h-[90%] bg-white/1 rounded-full blur-[140px]" />
          <div className="absolute inset-0 bg-linear-to-t from-black via-transparent to-black opacity-60" />
        </div>

        <div className="relative z-10 max-w-lg px-12 text-center">
          <ScrollReveal direction="up">
            <h2 className="font-mono uppercase tracking-[0.2em] text-3xl lg:text-5xl text-white mb-8 leading-tight font-bold">
              Strategic Wealth <br />
              <span className="text-white/80">
                Architecture.
              </span>
            </h2>
          </ScrollReveal>

          <ScrollReveal direction="up" delay={0.2}>
            <div className="h-px w-12 bg-white/40 mx-auto mb-8" />
            <p className="text-white/40 font-mono text-[9px] tracking-[0.6em] uppercase">
              Axiom Strategic Protocol // v1.0
            </p>
          </ScrollReveal>
        </div>
      </div>

      {/* Right Side: Authentication Portal */}
      <div className="flex-1 flex items-center justify-center p-8 md:p-12 relative z-10 bg-black">
        <div className="w-full max-w-md">
          {/* Brand & Header */}
          <div className="flex flex-col items-center mb-16">
            <ScrollReveal direction="down" delay={0.3}>
              <Link href="/" className="mb-12 block">
                <BrandMark className="w-14 h-14" />
              </Link>
            </ScrollReveal>

            <ScrollReveal direction="up" delay={0.4}>
              <h1 className="font-mono text-xl text-white text-center mb-3 tracking-[0.4em] uppercase font-bold">
                Initialize Access
              </h1>
```

```
            <p className="text-white/30 text-[10px] font-mono text-center tracking-[0.3em] uppercase">
              Enter Sovereign Credentials
            </p>
          </ScrollReveal>
        </div>

        <ScrollReveal direction="up" delay={0.5}>
               <div className="bg-white/[0.02] border border-white/10 p-10 md:p-14 rounded-none backdrop-blur-sm relative overflow-hidden group">
            {/* Subtle Decorative Glow */}
                       <div className="absolute -top-24 -right-24 w-48 h-48 bg-white/5 blur-3xl rounded-full opacity-0 group-hover:opacity-100 transition-opacity duration-1000" />

            <Auth
              supabaseClient={supabase}
              view="sign_up"
              appearance={{
                theme: ThemeSupa,
                variables: {
                  default: {
                    colors: {
                      brand: "white",
                      brandAccent: "white",
                      brandButtonText: "black",
                      defaultButtonBackground: "transparent",
                      defaultButtonBackgroundHover: "rgba(255,255,255,0.05)",
                      defaultButtonText: "white",
                      defaultButtonBorder: "rgba(255,255,255,0.1)",
                      inputText: "white",
                      inputBackground: "transparent",
                      inputBorder: "rgba(255,255,255,0.1)",
                      inputPlaceholder: "rgba(255,255,255,0.2)",
                      inputLabelText: "rgba(255,255,255,0.4)",
                    },
                    radii: {
                      buttonBorderRadius: "0px",
                      inputBorderRadius: "0px",
                    },
                    fonts: {
                       bodyFontFamily: `-apple-system, BlinkMacSystemFont, "SF Pro Display", "SF Pro Text", "Helvetica Neue", Helvetica, Arial, sans-serif`,
                         buttonFontFamily: `var(--font-geist-mono), ui-monospace, SFMono-Regular, Roboto Mono, Menlo, Monaco, Consolas, Liberation Mono, Courier New, monospace`,
                           inputFontFamily: `-apple-system, BlinkMacSystemFont, "SF Pro Display", "SF Pro Text", "Helvetica Neue", Helvetica, Arial, sans-serif`,
                          labelFontFamily: `var(--font-geist-mono), ui-monospace, SFMono-Regular, Roboto Mono, Menlo, Monaco, Consolas, Liberation Mono, Courier New, monospace`,
                    },
                  },
                },
                className: {
                  button:
                                "font-mono uppercase tracking-widest text-[10px] py-4 transition-all hover:scale-[1.02] active:scale-[0.98]",
                  label:
                    "font-mono uppercase tracking-widest text-[8px] mb-3 ml-1",
                  input:
                    "border-white/10 focus:border-white/40 transition-colors bg-white/5 py-3 px-4",
                },
              }}
              localization={{
                variables: {
                  sign_up: {
                    email_label: "EMAIL ADDRESS",
                    password_label: "SECURE ACCESS KEY",
```

```
                    button_label: "INITIALIZE PROTOCOL",
                    link_text: "AUTHORIZED? ACCESS PORTAL",
                  },
                  sign_in: {
                    email_label: "EMAIL ADDRESS",
                    password_label: "SECURE ACCESS KEY",
                    button_label: "AUTHORIZE SESSION",
                    link_text: "NO ACCESS? INITIALIZE LEDGER",
                  },
                },
              }}
              providers={["google"]}
              redirectTo={`${typeof window !== "undefined" ? window.location.origin : ""}/auth/callback`}
            />

            <div className="mt-12 text-center pt-10 border-t border-white/5">
              <Link
                href="/"
                className="text-[9px] font-mono text-white/20 uppercase tracking-[0.4em] hover:text-white transition-colors"
              >
                ? Return to Authorization
              </Link>
            </div>
          </div>
        </ScrollReveal>

        <ScrollReveal direction="up" delay={0.7}>
          <p className="mt-12 text-white/10 text-[8px] font-mono text-center uppercase tracking-[0.5em] leading-relaxed">
            All sessions are encrypted. Authorization granted by <br />
            Axiom Strategic Wealth Intelligence.
          </p>
        </ScrollReveal>
      </div>
    </div>
  </main>
 );
}
```

FILE: src\components\dashboard\DayZeroOnboarding.tsx

```
"use client";

import { useState } from "react";
import { submitDayZeroOnboardingAction } from "@/app/dashboard/actions";
import { type DashboardSnapshot } from "@/lib/dashboard/types";
import { motion, AnimatePresence } from "framer-motion";
import { Wallet, TrendingUp, Flame, ShieldAlert, Zap } from "lucide-react";

import {
  OnboardingHeader,
  OnboardingSidebar,
} from "./onboarding/OnboardingLayout";
import {
  AccountsStep,
  type OnboardingAccount,
} from "./onboarding/AccountsStep";
import {
  IncomeVelocityStep,
  type OnboardingIncome,
} from "./onboarding/IncomeVelocityStep";
import {
  ExpensesStep,
  type OnboardingBaseline,
```

```
} from "./onboarding/ExpensesStep";
import {
  LiabilitiesStep,
  type OnboardingLiability,
} from "./onboarding/LiabilitiesStep";

interface DayZeroOnboardingProps {
  onComplete: (snapshot: DashboardSnapshot) => void;
}

const STEPS = [
  {
    roman: "I",
    icon: Wallet,
    color: "var(--truth)",
    title: "Accounts",
    desc: "Map initial capital distribution. List primary liquidity vehicles for architecture baseline.",
  },
  {
    roman: "II",
    icon: TrendingUp,
    color: "var(--opportunity)",
    title: "Yield Velocity",
    desc: "Define inbound cash flow parameters. Calculate capital replenishment speed.",
  },
  {
    roman: "III",
    icon: Flame,
    color: "var(--leakage)",
    title: "Baseline Burn",
    desc: "Quantify structural maintenance costs. Identify high-leakage vectors.",
  },
  {
    roman: "IV",
    icon: ShieldAlert,
    color: "var(--warning)",
    title: "Solvency Risks",
    desc: "Map current liabilities. Establish debt-to-capital alignment ratio.",
  },
];

export default function DayZeroOnboarding({
  onComplete,
}: DayZeroOnboardingProps) {
  const [step, setStep] = useState(1);
  const [direction, setDirection] = useState(1);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  // State Management
  const [accounts, setAccounts] = useState<OnboardingAccount[]>([
    {
      account_name: "",
      account_type: "mobile_money",
      current_balance: "",
      institution: "",
    },
  ]);
  const [incomes, setIncomes] = useState<OnboardingIncome[]>([]);
  const [baselines, setBaselines] = useState<OnboardingBaseline[]>([
    {
      title: "",
      amount: "",
      cadence: "monthly",
      execution_day: "1",
```

```
    is_recurring: true,
    category: "Maintenance",
  },
]);
const [liabilities, setLiabilities] = useState<OnboardingLiability[]>([]);

const isStepValid = () => {
  if (step === 1)
    return (
      accounts.length > 0 &&
      accounts.every((a) => a.account_name.trim() && a.current_balance !== "")
    );
  if (step === 2)
    return (
      incomes.length === 0 ||
      incomes.every((i) => i.income_name.trim() && i.amount !== "")
    );
  if (step === 3)
    return (
      baselines.length > 0 &&
      baselines.every((b) => b.title.trim() && b.amount !== "")
    );
  if (step === 4)
    return (
      liabilities.length === 0 ||
      liabilities.every(
        (l) => l.liability_name.trim() && l.outstanding_balance !== "",
      )
    );
  return false;
};

const handleNext = () => {
  if (step < 4) {
    setDirection(1);
    setStep(step + 1);
    setError(null);
  }
};
const handlePrev = () => {
  if (step > 1) {
    setDirection(-1);
    setStep(step - 1);
    setError(null);
  }
};

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  if (step < 4) {
    handleNext();
    return;
  }

  setIsLoading(true);
  setError(null);

  try {
    const snapshot = await submitDayZeroOnboardingAction({
      accounts,
      incomes,
      liabilities,
      baselines,
    });
    onComplete(snapshot);
```

```
    } catch (err: unknown) {
      setError(
        err instanceof Error
          ? err.message
          : "System Error: Input verification failed.",
      );
    } finally {
      setIsLoading(false);
    }
  }

  const activeStepConfig = STEPS[step - 1];

  return (
      <div className="fixed inset-0 bg-black z-[100] flex flex-col selection:bg-white selection:text-black overflow-hidden
h-screen w-full font-mono text-foreground">
      {/* Clinical Grid Background */}
                                <div      className="absolute      inset-0      z-0      pointer-events-none      opacity-[0.03]
bg-[linear-gradient(to_right,#80808012_1px,transparent_1px),linear-gradient(to_bottom,#80808012_1px,transparent_1px)]
bg-[size:32px_32px]" />

      <OnboardingHeader step={step} totalSteps={STEPS.length} />

       <div className="relative z-10 flex-1 flex items-start md:items-center justify-center px-4 md:px-12 pt-32 md:pt-16 pb-12
overflow-y-auto">
              <div  className="max-w-7xl  w-full  grid  grid-cols-1  lg:grid-cols-[0.9fr_1.6fr]  gap-12  md:gap-24  items-stretch
overflow-visible">
          <OnboardingSidebar
            roman={activeStepConfig.roman}
            icon={activeStepConfig.icon}
            iconColor={activeStepConfig.color}
            title={activeStepConfig.title}
            desc={activeStepConfig.desc}
            direction={direction}
          />

                <div  className="flex  flex-col  border  border-white/10  bg-[#0a0a0a]  rounded-sm  overflow-hidden  min-h-[500px]
md:min-h-0 shadow-2xl">
            <div className="flex-1 p-6 md:p-14 overflow-hidden relative">
              <form
                onSubmit={handleSubmit}
                className="h-full flex flex-col text-foreground"
              >
                <AnimatePresence mode="wait" custom={direction}>
                  <motion.div
                    key={step}
                    custom={direction}
                    initial={{ opacity: 0, x: direction * 10 }}
                    animate={{ opacity: 1, x: 0 }}
                    exit={{ opacity: 0, x: -direction * 10 }}
                    transition={{ duration: 0.3, ease: "easeOut" }}
                    className="flex-1 flex flex-col"
                  >
                    {step === 1 && (
                      <AccountsStep
                        accounts={accounts}
                        onChange={setAccounts}
                      />
                    )}
                    {step === 2 && (
                      <IncomeVelocityStep
                        incomes={incomes}
                        onChange={setIncomes}
                      />
                    )}
```

```
                  {step === 3 && (
                    <ExpensesStep
                      baselines={baselines}
                      onChange={setBaselines}
                    />
                  )}
                  {step === 4 && (
                    <LiabilitiesStep
                      liabilities={liabilities}
                      onChange={setLiabilities}
                    />
                  )}
                </motion.div>
              </AnimatePresence>
            </form>
          </div>

           <div className="h-24 md:h-28 border-t border-white/10 relative z-10 px-6 md:px-16 flex flex-col justify-center
shrink-0 bg-black">
            {error && (
              <div className="absolute top-0 left-0 w-full transform -translate-y-full px-6 md:px-12 pb-4">
                   <div className="bg-red-500/10 border border-red-500/20 rounded-sm p-3 text-red-500 text-[10px] font-bold
tracking-wider flex items-center gap-3">
                  <span className="shrink-0 w-1.5 h-1.5 rounded-full bg-red-500 animate-pulse" />
                  {error}
                </div>
              </div>
            )}
            <div className="w-full flex items-center justify-between">
              <button
                type="button"
                onClick={handlePrev}
                disabled={step === 1}
                   className="text-[10px] font-bold tracking-[0.4em] text-zinc-600 hover:text-white transition-all uppercase
disabled:opacity-0 py-4 px-8 border border-transparent hover:border-white/5"
              >
                ? PREV_PHASE
              </button>
              <div onClick={handleSubmit}>
                <button
                  disabled={isLoading || !isStepValid()}
                              className="px-12 py-4 bg-white text-black font-bold text-[10px] tracking-[0.4em] uppercase
hover:bg-zinc-200 transition-all flex items-center gap-3 disabled:opacity-50 disabled:cursor-not-allowed"
                >
                  {step === 4 ? (
                    <>
                      <Zap size={14} />
                      ARCHIVE_SETUP
                    </>
                  ) : (
                    "NEXT_PHASE ?"
                  )}
                </button>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}
```

Axiom Repository Audit

FILE: src\components\dashboard\KairosNarrative.tsx

```
"use client";

import React from "react";
import { ScrollReveal } from "../ui/scroll-reveal";
import { KairosInsight } from "@/lib/ai/kairos";

interface KairosNarrativeProps {
  insight: KairosInsight | null;
}

export function KairosNarrative({ insight }: KairosNarrativeProps) {
  if (!insight) return null;

  const severityColor =
    {
      critical: "text-leakage",
      warning: "text-warning",
      advisory: "text-truth",
      observation: "text-zinc-500",
    }[insight.severity] || "text-zinc-500";

  const severityBorder =
    {
      critical: "border-leakage/40",
      warning: "border-warning/40",
      advisory: "border-truth/40",
      observation: "border-white/10",
    }[insight.severity] || "border-white/10";

  const severityPulse = insight.severity === "critical" ? "animate-pulse" : "";

  const displaySeverity =
    {
      critical: "CRITICAL",
      warning: "HIGH",
      advisory: "MEDIUM",
      observation: "LOW",
    }[insight.severity] || "LOW";

  return (
    <ScrollReveal direction="up" distance={10} duration={0.6}>
      <div
        className={`p-8 md:p-12 bg-[#0a0a0a] border ${severityBorder} rounded-sm relative overflow-hidden font-mono`}
      >
        {/* Technical Grid Overlay */}
                                        <div     className="absolute     inset-0     opacity-[0.03]     pointer-events-none
bg-[linear-gradient(to_right,#80808012_1px,transparent_1px),linear-gradient(to_bottom,#80808012_1px,transparent_1px)]
bg-[size:24px_24px]" />

        <div className="space-y-8 relative z-10">
          <div className="flex items-center justify-between border-b border-white/10 pb-4">
            <p className="text-[10px] tracking-[0.4em] text-zinc-500 uppercase font-bold">
              [ANALYSIS_REPORT_01]
            </p>
            <div className="flex items-center gap-2">
              <div
                className={`w-1.5 h-1.5 rounded-full ${severityColor.replace("text-", "bg-")} ${severityPulse}`}
              />
              <p
                className={`text-[9px] tracking-widest uppercase font-bold ${severityColor}`}
              >
                Operational Status: {displaySeverity}
```

```
    </p>
  </div>
</div>

<div className="space-y-6">
  <div className="space-y-1">
    <p className="text-[10px] text-zinc-600 uppercase tracking-widest font-bold">
      SECTOR:
    </p>
    <p className="text-sm text-white uppercase tracking-wider font-bold">
      {insight.category?.replace(/_/g, " ") || "STRATEGIC ANALYSIS"}
    </p>
  </div>
  <div className="space-y-2">
    <p className="text-[10px] text-zinc-600 uppercase tracking-widest font-bold">
      OBSERVATION:
    </p>
    <p className="text-lg leading-relaxed text-zinc-200 selection:bg-white selection:text-black">
      {insight.message}
    </p>
  </div>

  {insight.supportingSignals &&
    insight.supportingSignals.length > 0 && (
      <div className="pt-6 border-t border-white/10 space-y-4">
        <p className="text-[10px] text-zinc-600 uppercase tracking-widest font-bold">
          SUPPORTING DATA:
        </p>
        <div className="space-y-3">
          {insight.supportingSignals.map((signal, idx) => (
            <div
              key={idx}
              className="flex gap-4 items-start group/signal"
            >
              <span className="text-[10px] text-zinc-700 mt-1">
                {idx + 1}.
              </span>
              <p className="text-xs text-zinc-400 group-hover/signal:text-white transition-colors leading-relaxed">
                {signal}
              </p>
            </div>
          ))}
        </div>
      </div>
    )}
</div>

<div className="pt-8 border-t border-white/10 flex justify-between items-center opacity-40">
  <div className="flex gap-8">
    <div className="space-y-1">
      <p className="text-[8px] text-zinc-600 uppercase tracking-widest">
        Efficiency_Idx
      </p>
      <p className="text-[10px] text-white">
        {(insight.capitalEfficiency || 0).toFixed(1)}/100
      </p>
    </div>
    <div className="space-y-1">
      <p className="text-[8px] text-zinc-600 uppercase tracking-widest">
        Runway_Window
      </p>
      <p className="text-[10px] text-white uppercase">
        {insight.runway ? `${insight.runway} Days` : "Stable"}
      </p>
    </div>
```

```
            </div>
            <p className="text-[8px] tracking-[0.4em] text-zinc-700 uppercase">
              Kairos Intel Engine v1.2
            </p>
          </div>
        </div>
      </div>
    </ScrollReveal>
  );
}
```

FILE: src\components\dashboard\MiniCharts.tsx

```
"use client";

import React, { useRef } from "react";
import {
  AreaChart,
  Area,
  BarChart,
  Bar,
  PieChart,
  Pie,
  Cell,
  ResponsiveContainer,
} from "recharts";
import { ScrollReveal } from "../ui/scroll-reveal";
import { useInView } from "framer-motion";

// --- Mini Sparkline (Area) ---
export function MiniSparkline({
  data,
  color = "#8b5cf6", // Default purple
}: {
  data: number[];
  color?: string;
}) {
  const chartData = data.map((val, i) => ({ value: val, index: i }));
  // Unique ID for the gradient to prevent collisions
  const gradientId = `gradient-${color.replace(/[^a-zA-Z0-9]/g, "")}`;

  return (
    <div className="h-12 w-full mt-4" style={{ color }}>
      <ResponsiveContainer width="100%" height="100%">
        <AreaChart data={chartData}>
          <defs>
            <linearGradient id={gradientId} x1="0" y1="0" x2="0" y2="1">
              <stop offset="5%" stopColor="currentColor" stopOpacity={0.3} />
              <stop offset="95%" stopColor="currentColor" stopOpacity={0} />
            </linearGradient>
          </defs>
          <Area
            type="monotone"
            dataKey="value"
            stroke="currentColor"
            strokeWidth={2}
            fill={`url(#${gradientId})`}
            isAnimationActive={true}
            animationDuration={1500}
          />
        </AreaChart>
      </ResponsiveContainer>
    </div>
```

```
  );
}

// --- Mini Bar Chart ---
export function MiniBarChart({
  data,
  color = "#ef4444", // Default red
}: {
  data: number[];
  color?: string;
}) {
  const chartData = data.map((val, i) => ({ value: val, index: i }));
  return (
    <div className="h-12 w-full mt-4" style={{ color }}>
      <ResponsiveContainer width="100%" height="100%">
        <BarChart data={chartData} barSize={4}>
          <Bar
            dataKey="value"
            fill="currentColor"
            radius={[2, 2, 2, 2]}
            isAnimationActive={true}
            animationDuration={1500}
          />
        </BarChart>
      </ResponsiveContainer>
    </div>
  );
}

// --- Distribution Pie Chart ---
export function DistributionPieChart({
  data,
  colors = [
    "#8b5cf6",
    "#3b82f6",
    "#10b981",
    "#f59e0b",
    "#f97316",
    "#ef4444",
    "#6366f1",
    "#8b5cf6",
  ],
}: {
  data: { name: string; value: number }[];
  colors?: string[];
}) {
  const containerRef = useRef(null);
  const isInView = useInView(containerRef, { once: true, margin: "-10%" });

  if (!data || data.length === 0) return null;

  return (
    <div ref={containerRef}>
      <ScrollReveal direction="none" duration={1.2}>
        <div className="h-40 w-full relative">
          <ResponsiveContainer width="100%" height="100%">
            <PieChart>
              <Pie
                data={data}
                cx="50%"
                cy="50%"
                innerRadius="60%"
                outerRadius="90%"
                dataKey="value"
                stroke="none"
```

```
                startAngle={90}
                endAngle={isInView ? -270 : 90}
                isAnimationActive={true}
                animationDuration={1500}
                animationEasing="ease-out"
                paddingAngle={2}
                cornerRadius={4}
              >
                {data.map((entry, index) => (
                  <Cell
                    key={`cell-${index}`}
                    fill={colors[index % colors.length]}
                  />
                ))}
              </Pie>
            </PieChart>
          </ResponsiveContainer>
        </div>
      </ScrollReveal>
    </div>
  );
}

export function MiniDonut({
  value,
  max,
  color = "#3b82f6", // Default blue
}: {
  value: number;
  max: number;
  color?: string;
}) {
  const containerRef = useRef(null);
  const isInView = useInView(containerRef, { once: true, margin: "-10%" });

  const safeValue = Math.min(value, max);
  const remainder = Math.max(0, max - safeValue);
  const data = [
    { name: "Value", value: safeValue },
    { name: "Remainder", value: remainder },
  ];

  return (
    <div
      ref={containerRef}
      className="h-12 w-full relative mt-2"
      style={{ color }}
    >
      <ResponsiveContainer width="100%" height="200%">
        <PieChart>
          <Pie
            data={data}
            cx="50%"
            cy="50%"
            startAngle={180}
            endAngle={isInView ? 0 : 180}
            innerRadius="70%"
            outerRadius="100%"
            dataKey="value"
            stroke="none"
            isAnimationActive={true}
            animationDuration={1500}
            animationEasing="ease-out"
            cornerRadius={4}
          >
```

```
            <Cell key="cell-0" fill="currentColor" />
            <Cell key="cell-1" fill="var(--muted)" />
          </Pie>
        </PieChart>
      </ResponsiveContainer>

      <div className="absolute bottom-0 left-0 w-full flex items-end justify-center pointer-events-none pb-1">
        <span className="font-mono text-[9px] text-white/40 tracking-widest uppercase">
          {Math.round((safeValue / max) * 100)}% Capacity
        </span>
      </div>
    </div>
  );
}
```

FILE: src\components\dashboard\TelemetryDashboard.tsx

```
"use client";

import { useEffect, useState, useCallback } from "react";
import {
  fetchTelemetryLogsAction,
  type TelemetrySummary,
} from "@/app/dashboard/actions";

export function TelemetryDashboard() {
  const [telemetry, setTelemetry] = useState<TelemetrySummary | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const loadTelemetry = useCallback(async (isManualRefresh = false) => {
    if (isManualRefresh) setIsLoading(true);
    setError(null);
    try {
      const summary = await fetchTelemetryLogsAction();
      setTelemetry(summary);
    } catch (err) {
      setError("Failed to retrieve forensic logs.");
      console.error(err);
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    const timer = setTimeout(() => loadTelemetry(), 10);
    return () => clearTimeout(timer);
  }, [loadTelemetry]);

  if (isLoading) {
    return (
      <div className="py-12 space-y-12 animate-pulse opacity-20">
        <div className="h-px bg-white/20 w-full" />
        <div className="grid grid-cols-4 gap-8">
          {[1, 2, 3, 4].map((i) => (
            <div key={i} className="h-24 bg-white/10" />
          ))}
        </div>
      </div>
    );
  }

  if (error) {
```

```
    return (
      <div className="py-12 text-center opacity-40">
        <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
          {error}
        </p>
      </div>
    );
  }

  return (
    <div className="space-y-24">
      {/* PERFORMANCE SUMMARY */}
      <div className="grid grid-cols-1 md:grid-cols-4 gap-px bg-border">
        {[
          { label: "Evaluation Cycles", value: telemetry?.totalCycles || 0 },
          {
            label: "Avg. Latency",
            value: `${telemetry?.averageLatency || 0}ms`,
          },
          {
            label: "Intelligence Yield",
            value: `${telemetry?.matchRate || 0}%`,
          },
          {
            label: "Active Heuristics",
            value: Object.keys(telemetry?.ruleHits || {}).length,
          },
        ].map((stat) => (
          <div key={stat.label} className="bg-background p-8 group border-r border-border last:border-r-0">
                        <span  className="text-[8px]  font-mono  text-muted-foreground  uppercase  tracking-[0.3em]  block  mb-4
group-hover:text-foreground transition-colors">
              {stat.label}
            </span>
            <span className="font-mono text-2xl text-foreground tabular-nums font-bold">
              {stat.value}
            </span>
          </div>
        ))}
      </div>

      {/* FORENSIC LOG TABLE */}
      <div className="space-y-8">
        <div className="flex items-center justify-between border-b border-white/20 pb-4">
          <h3 className="font-mono text-xs uppercase tracking-[0.4em] text-muted-foreground">
            Evaluation Audit Trail
          </h3>
          <button
            onClick={() => loadTelemetry(true)}
                      className="text-[8px]  font-mono  text-muted-foreground  hover:text-foreground  uppercase  tracking-widest
transition-colors"
          >
            [ Refresh Logs ]
          </button>
        </div>

        <div className="overflow-x-auto">
          <table className="w-full text-left">
            <thead>
              <tr className="border-b border-border">
                <th className="py-4 text-[9px] font-mono text-muted-foreground uppercase tracking-widest font-normal">
                  Timestamp
                </th>
                <th className="py-4 text-[9px] font-mono text-muted-foreground uppercase tracking-widest font-normal">
                  Heuristic ID
                </th>
```

```
              <th className="py-4 text-[9px] font-mono text-muted-foreground uppercase tracking-widest font-normal">
                Severity
              </th>
                     <th  className="py-4  text-[9px]  font-mono  text-muted-foreground  uppercase  tracking-widest  font-normal
text-right">
                Compute
              </th>
            </tr>
          </thead>
          <tbody className="divide-y divide-border">
            {telemetry?.logs.map((log) => (
              <tr
                key={log.id}
                className="group hover:bg-muted/30 transition-all"
              >
                <td className="py-4 text-[10px] font-mono text-muted-foreground tabular-nums">
                  {new Date(log.created_at).toLocaleTimeString([], {
                    hour12: false,
                    hour: "2-digit",
                    minute: "2-digit",
                    second: "2-digit",
                  })}
                </td>
                <td className="py-4 text-[11px] text-foreground font-mono group-hover:text-white transition-colors">
                  {log.rule_id}
                </td>
                <td className="py-4">
                  <span
                    className={`text-[8px] font-mono uppercase tracking-widest px-2 py-0.5 border ${
                      log.severity === "critical"
                        ? "text-leakage border-leakage bg-leakage/10"
                        : log.severity === "high" ||
                            log.severity === "warning"
                          ? "text-warning border-warning bg-warning/10"
                          : "text-muted-foreground/70 border-muted-foreground/20"
                    }`}
                  >
                    {log.severity}
                  </span>
                </td>
                <td className="py-4 text-[10px] font-mono text-muted-foreground text-right tabular-nums">
                  {log.evaluation_time_ms?.toFixed(2)}ms
                </td>
              </tr>
            ))}
            {(!telemetry || telemetry.logs.length === 0) && (
              <tr>
                <td
                  colSpan={4}
                  className="py-12 text-center text-[10px] font-mono text-muted-foreground/50 uppercase tracking-widest"
                >
                  No forensic data available.
                </td>
              </tr>
            )}
          </tbody>
        </table>
      </div>
    </div>
  </div>
  );
}
```

Axiom Repository Audit

FILE: src\components\dashboard\onboarding\AccountsStep.tsx

```
"use client";

import { ACCOUNT_TYPES } from "@/lib/finance/accounts";

export interface OnboardingAccount {
  account_name: string;
  account_type: string;
  current_balance: string;
  institution: string;
}

interface AccountsStepProps {
  accounts: OnboardingAccount[];
  onChange: (accounts: OnboardingAccount[]) => void;
}

export function AccountsStep({ accounts, onChange }: AccountsStepProps) {
  const addAccount = () => {
    if (accounts.length < 3) {
      onChange([
        ...accounts,
        {
          account_name: "",
          account_type: "checking",
          current_balance: "",
          institution: "",
        },
      ]);
    }
  };

  const removeAccount = (index: number) => {
    onChange(accounts.filter((_, i) => i !== index));
  };

  const updateAccount = (
    index: number,
    updates: Partial<OnboardingAccount>,
  ) => {
    const next = [...accounts];
    next[index] = { ...next[index], ...updates };
    onChange(next);
  };

  return (
    <div className="flex flex-col h-full gap-6">
      <h2 className="font-mono text-xs text-muted-foreground tracking-[0.4em] uppercase">
        Account Structure
      </h2>
      <div className="flex-1 space-y-6 md:space-y-4 overflow-y-auto scrollbar-hide pr-2">
        {accounts.map((acc, idx) => (
          <div
            key={idx}
            className="space-y-4 md:space-y-3 pb-6 md:pb-3 border-b border-white/10 relative shrink-0"
          >
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-6">
              <input
                type="text"
                placeholder="Account Name"
                value={acc.account_name}
                onChange={(e) =>
                  updateAccount(idx, { account_name: e.target.value })
```

```
              }
                        className="bg-transparent border-b border-border py-1 font-mono text-sm uppercase tracking-widest
text-foreground focus:outline-none placeholder:text-white/10"
              />
              <input
                type="text"
                placeholder="Financial Institution"
                value={acc.institution}
                onChange={(e) =>
                  updateAccount(idx, { institution: e.target.value })
                }
                      className="bg-transparent border-b border-border py-1 text-[10px] font-mono uppercase tracking-widest
text-muted-foreground/60 focus:outline-none placeholder:text-white/10"
              />
            </div>
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-8">
              <div className="space-y-1">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                  Account Type
                </label>
                <select
                  value={acc.account_type}
                  onChange={(e) =>
                    updateAccount(idx, { account_type: e.target.value })
                  }
                   className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono tracking-widest uppercase
text-muted-foreground/80 focus:outline-none"
                >
                  {ACCOUNT_TYPES.map((t) => (
                    <option
                      key={t.value}
                      value={t.value}
                      className="bg-[#080808]"
                    >
                      {t.label}
                    </option>
                  ))}
                </select>
              </div>
              <div className="space-y-1">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                  Current Liquidity
                </label>
                <input
                  type="number"
                  placeholder="0.00"
                  value={acc.current_balance}
                  onChange={(e) =>
                    updateAccount(idx, { current_balance: e.target.value })
                  }
                            className="w-full bg-transparent border-b border-border py-1 text-sm font-mono text-foreground
focus:outline-none tabular-nums"
                />
              </div>
            </div>

            {accounts.length > 1 && (
              <button
                type="button"
                onClick={() => removeAccount(idx)}
                    className="absolute top-0 right-0 p-2 text-muted-foreground/50 hover:text-red-400 bg-white/5 rounded-full
hover:bg-white/10 transition-all"
              >
                ?
              </button>
```

```
            )}
          </div>
        ))}
      </div>
      <button
        type="button"
        onClick={addAccount}
                className="text-[9px]  font-mono  tracking-[0.6em]  uppercase  text-muted-foreground/70  hover:text-foreground
transition-all py-3 px-6 border border-border rounded-full self-start active:scale-95"
      >
        + Append Source
      </button>
    </div>
  );
}
```

FILE: src\components\dashboard\onboarding\ExpensesStep.tsx

```
"use client";

import { motion } from "framer-motion";
import { TAXONOMY_CATEGORIES } from "@/lib/finance/taxonomy";

export interface OnboardingBaseline {
  title: string;
  amount: string;
  cadence: string;
  execution_day: string;
  is_recurring: boolean;
  category: string;
}

const BASELINE_CADENCES = [
  { value: "daily", label: "Daily" },
  { value: "weekly", label: "Weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "yearly", label: "Yearly" },
];

interface ExpensesStepProps {
  baselines: OnboardingBaseline[];
  onChange: (baselines: OnboardingBaseline[]) => void;
}

export function ExpensesStep({ baselines, onChange }: ExpensesStepProps) {
  const addBaseline = () => {
    if (baselines.length < 3) {
      onChange([
        ...baselines,
        {
          title: "",
          amount: "",
          cadence: "monthly",
          execution_day: "1",
          is_recurring: true,
          category: "Maintenance",
        },
      ]);
    }
  };

  const removeBaseline = (index: number) => {
    onChange(baselines.filter((_, i) => i !== index));
```

```
  };

  const updateBaseline = (
    index: number,
    updates: Partial<OnboardingBaseline>,
  ) => {
    const next = [...baselines];
    next[index] = { ...next[index], ...updates };
    onChange(next);
  };

  return (
    <div className="flex flex-col h-full gap-6">
      <h2 className="font-mono text-xs text-muted-foreground tracking-[0.4em] uppercase">
        Operational Baselines
      </h2>
      <div className="flex-1 space-y-6 md:space-y-4 overflow-y-auto scrollbar-hide pr-2">
        {baselines.map((base, idx) => (
          <div
            key={idx}
            className="space-y-4 md:space-y-3 pb-6 md:pb-3 border-b border-white/10 relative shrink-0"
          >
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-6">
              <input
                type="text"
                placeholder="Baseline Name"
                value={base.title}
                onChange={(e) => updateBaseline(idx, { title: e.target.value })}
                         className="bg-transparent  border-b  border-border  py-1  font-mono  text-sm  uppercase  tracking-widest
text-foreground focus:outline-none placeholder:text-white/10"
              />
              <div className="space-y-1">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                  Category
                </label>
                <select
                  value={base.category}
                  onChange={(e) =>
                    updateBaseline(idx, { category: e.target.value })
                  }
                       className="bg-transparent border-b border-border py-1 text-[10px] font-mono tracking-widest uppercase
text-muted-foreground/80 focus:outline-none w-full"
                >
                  {TAXONOMY_CATEGORIES.map((cat) => (
                    <option
                      key={cat.value}
                      value={cat.value}
                      className="bg-[#080808]"
                    >
                      {cat.label}
                    </option>
                  ))}
                </select>
              </div>
            </div>
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-12">
              <div className="space-y-1">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                  Amount
                </label>
                <input
                  type="number"
                  placeholder="0.00"
                  value={base.amount}
                  onChange={(e) =>
```

```
                  updateBaseline(idx, { amount: e.target.value })
                }
                           className="w-full bg-transparent border-b border-border py-1 text-sm font-mono text-foreground
focus:outline-none tabular-nums"
                />
              </div>
            </div>

            <div className="pt-2 space-y-3">
              <label className="flex items-center space-x-3 cursor-pointer group">
                <input
                  type="checkbox"
                  checked={base.is_recurring}
                  onChange={(e) =>
                    updateBaseline(idx, { is_recurring: e.target.checked })
                  }
                  className="w-4 h-4 rounded border-border bg-transparent checked:bg-white transition-colors cursor-pointer"
                />
                               <span  className="text-[10px]  font-mono  text-muted-foreground/70  uppercase  tracking-widest
group-hover:text-muted-foreground/90 transition-colors">
                  Recurring Expense
                </span>
              </label>
              {base.is_recurring && (
                <motion.div
                  initial={{ opacity: 0, y: -5 }}
                  animate={{ opacity: 1, y: 0 }}
                  className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-6 pt-2 pl-6 md:pl-8 border-l border-white/5"
                >
                  <div className="space-y-1">
                    <label className="text-[9px] font-mono text-muted-foreground/60 uppercase">
                      Frequency
                    </label>
                    <select
                      value={base.cadence}
                      onChange={(e) =>
                        updateBaseline(idx, { cadence: e.target.value })
                      }
                                       className="w-full  bg-transparent  border-b  border-border  py-1  text-[10px]  font-mono
text-muted-foreground/90 focus:outline-none"
                    >
                      {BASELINE_CADENCES.map((c) => (
                        <option
                          key={c.value}
                          value={c.value}
                          className="bg-black"
                        >
                          {c.label}
                        </option>
                      ))}
                    </select>
                  </div>

                  {base.cadence === "daily" && (
                    <div className="flex items-center pt-2">
                      <p className="text-[8px] font-mono text-muted-foreground/60 uppercase leading-tight">
                        Prompted everyday to verify.
                      </p>
                    </div>
                  )}

                  {(base.cadence === "monthly" ||
                    base.cadence === "weekly" ||
                    base.cadence === "biweekly") && (
                    <div className="space-y-1">
```

```
                    <label className="text-[9px] font-mono text-muted-foreground/60 uppercase">
                      {base.cadence === "monthly" ? "Day" : "Day of Week"}
                    </label>
                    {base.cadence === "monthly" ? (
                      <input
                        type="number"
                        min="1"
                        max="31"
                        placeholder="15"
                        value={base.execution_day}
                        onChange={(e) =>
                          updateBaseline(idx, {
                            execution_day: e.target.value,
                          })
                        }
                                    className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono
text-muted-foreground/90 focus:outline-none"
                      />
                    ) : (
                      <select
                        value={base.execution_day || 1}
                        onChange={(e) =>
                          updateBaseline(idx, {
                            execution_day: e.target.value,
                          })
                        }
                                    className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono
text-muted-foreground/90 focus:outline-none"
                      >
                        {[
                          { label: "Mon", value: 1 },
                          { label: "Tue", value: 2 },
                          { label: "Wed", value: 3 },
                          { label: "Thu", value: 4 },
                          { label: "Fri", value: 5 },
                          { label: "Sat", value: 6 },
                          { label: "Sun", value: 7 },
                        ].map((day) => (
                          <option
                            key={day.value}
                            value={day.value}
                            className="bg-black"
                          >
                            {day.label}
                          </option>
                        ))}
                      </select>
                    )}
                  </div>
                )}
              </motion.div>
            )}
          </div>
          {baselines.length > 1 && (
            <button
              type="button"
              onClick={() => removeBaseline(idx)}
                  className="absolute top-0 right-0 p-2 text-muted-foreground/50 hover:text-red-400 bg-white/5 rounded-full
hover:bg-white/10 transition-all"
            >
              ?
            </button>
          )}
        </div>
      ))}
```

```
      </div>
      <button
        type="button"
        onClick={addBaseline}
                className="text-[9px]  font-mono  tracking-[0.6em]  uppercase  text-muted-foreground/70  hover:text-foreground
transition-all py-3 px-6 border border-border rounded-full self-start active:scale-95"
      >
        + Append Recurring
      </button>
    </div>
  );
}
```

FILE: src\components\dashboard\onboarding\IncomeVelocityStep.tsx

```
"use client";

import { motion } from "framer-motion";
import { INCOME_TYPES, CADENCES } from "@/lib/finance/income";

export interface OnboardingIncome {
  income_name: string;
  income_type: string;
  amount: string;
  cadence: string;
  execution_day: string;
  is_recurring: boolean;
  source: string;
}

interface IncomeVelocityStepProps {
  incomes: OnboardingIncome[];
  onChange: (incomes: OnboardingIncome[]) => void;
}

export function IncomeVelocityStep({
  incomes,
  onChange,
}: IncomeVelocityStepProps) {
  const addIncome = () => {
    if (incomes.length < 3) {
      onChange([
        ...incomes,
        {
          income_name: "",
          income_type: "salary",
          amount: "",
          cadence: "monthly",
          execution_day: "1",
          is_recurring: true,
          source: "",
        },
      ]);
    }
  };

  const removeIncome = (index: number) => {
    onChange(incomes.filter((_, i) => i !== index));
  };

  const updateIncome = (index: number, updates: Partial<OnboardingIncome>) => {
    const next = [...incomes];
    next[index] = { ...next[index], ...updates };
```

```
    onChange(next);
  };

  return (
    <div className="flex flex-col h-full gap-6">
      <h2 className="font-mono text-xs text-muted-foreground tracking-[0.4em] uppercase">
        Income Velocity
      </h2>
      <div className="flex-1 space-y-6 md:space-y-4 overflow-y-auto scrollbar-hide pr-2">
        {incomes.map((inc, idx) => (
          <div
            key={idx}
            className="space-y-4 md:space-y-3 pb-6 md:pb-3 border-b border-white/10 relative shrink-0"
          >
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-6">
              <input
                type="text"
                placeholder="Income Name"
                value={inc.income_name}
                onChange={(e) =>
                  updateIncome(idx, { income_name: e.target.value })
                }
                          className="bg-transparent border-b border-border py-1 font-mono text-sm uppercase tracking-widest
text-foreground focus:outline-none placeholder:text-white/10"
              />
              <input
                type="text"
                placeholder="Origin"
                value={inc.source}
                onChange={(e) => updateIncome(idx, { source: e.target.value })}
                       className="bg-transparent border-b border-border py-1 text-[10px] font-mono uppercase tracking-widest
text-muted-foreground/60 focus:outline-none placeholder:text-white/10"
              />
            </div>
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-6">
              <div className="space-y-1">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                  Type
                </label>
                <select
                  value={inc.income_type}
                  onChange={(e) =>
                    updateIncome(idx, { income_type: e.target.value })
                  }
                   className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono tracking-widest uppercase
text-muted-foreground/80 focus:outline-none"
                >
                  {INCOME_TYPES.map((t) => (
                    <option
                      key={t.value}
                      value={t.value}
                      className="bg-[#080808]"
                    >
                      {t.label}
                    </option>
                  ))}
                </select>
              </div>
              <div className="space-y-1">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                  Amount
                </label>
                <input
                  type="number"
                  placeholder="0.00"
```

```
                value={inc.amount}
                onChange={(e) =>
                  updateIncome(idx, { amount: e.target.value })
                }
                          className="w-full bg-transparent border-b border-border py-1 text-sm font-mono text-foreground
focus:outline-none tabular-nums"
              />
            </div>
          </div>

          <div className="pt-2 space-y-3">
            <label className="flex items-center space-x-3 cursor-pointer group">
              <input
                type="checkbox"
                checked={inc.is_recurring}
                onChange={(e) =>
                  updateIncome(idx, { is_recurring: e.target.checked })
                }
                className="w-4 h-4 rounded border-border bg-transparent checked:bg-white transition-colors cursor-pointer"
              />
                                <span  className="text-[10px]  font-mono  text-muted-foreground/70  uppercase  tracking-widest
group-hover:text-muted-foreground/90 transition-colors">
                Recurring Income
              </span>
            </label>
            {inc.is_recurring && (
              <motion.div
                initial={{ opacity: 0, y: -5 }}
                animate={{ opacity: 1, y: 0 }}
                className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-6 pt-2 pl-6 md:pl-8 border-l border-white/5"
              >
                <div className="space-y-1">
                  <label className="text-[9px] font-mono text-muted-foreground/60 uppercase">
                    Frequency
                  </label>
                  <select
                    value={inc.cadence}
                    onChange={(e) =>
                      updateIncome(idx, { cadence: e.target.value })
                    }
                                      className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono
text-muted-foreground/90 focus:outline-none"
                  >
                    {CADENCES.map((c) => (
                      <option
                        key={c.value}
                        value={c.value}
                        className="bg-black"
                      >
                        {c.label}
                      </option>
                    ))}
                  </select>
                </div>

                {(inc.cadence === "monthly" ||
                  inc.cadence === "weekly" ||
                  inc.cadence === "biweekly") && (
                  <div className="space-y-1">
                    <label className="text-[9px] font-mono text-muted-foreground/60 uppercase">
                      {inc.cadence === "monthly" ? "Day" : "Day of Week"}
                    </label>
                    {inc.cadence === "monthly" ? (
                      <input
                        type="number"
```

```
                        min="1"
                        max="31"
                        placeholder="15"
                        value={inc.execution_day}
                        onChange={(e) =>
                          updateIncome(idx, { execution_day: e.target.value })
                        }
                                    className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono
text-muted-foreground/90 focus:outline-none"
                      />
                    ) : (
                      <select
                        value={inc.execution_day || 1}
                        onChange={(e) =>
                          updateIncome(idx, { execution_day: e.target.value })
                        }
                                    className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono
text-muted-foreground/90 focus:outline-none"
                      >
                        {[
                          { label: "Mon", value: 1 },
                          { label: "Tue", value: 2 },
                          { label: "Wed", value: 3 },
                          { label: "Thu", value: 4 },
                          { label: "Fri", value: 5 },
                          { label: "Sat", value: 6 },
                          { label: "Sun", value: 7 },
                        ].map((day) => (
                          <option
                            key={day.value}
                            value={day.value}
                            className="bg-black"
                          >
                            {day.label}
                          </option>
                        ))}
                      </select>
                    )}
                  </div>
                )}
              </motion.div>
            )}
          </div>
          <button
            type="button"
            onClick={() => removeIncome(idx)}
                 className="absolute top-0 right-0 p-2 text-muted-foreground/50 hover:text-red-400 bg-white/5 rounded-full
hover:bg-white/10 transition-all"
          >
            ?
          </button>
        </div>
      ))}
    </div>
    <button
      type="button"
      onClick={addIncome}
              className="text-[9px] font-mono tracking-[0.6em] uppercase text-muted-foreground/70 hover:text-foreground
transition-all py-3 px-6 border border-border rounded-full self-start active:scale-95"
    >
      + Append Income
    </button>
  </div>
 );
}
```

FILE: src\components\dashboard\onboarding\LiabilitiesStep.tsx

```
"use client";

import { motion } from "framer-motion";
import { LIABILITY_TYPES } from "@/lib/finance/liabilities";

export interface OnboardingLiability {
  liability_name: string;
  liability_type: string;
  outstanding_balance: string;
  interest_rate: string;
  institution: string;
  is_paid_in_cadences: boolean;
  cadence: string;
  cadence_day_date: string;
  cadence_amount: string;
}

interface LiabilitiesStepProps {
  liabilities: OnboardingLiability[];
  onChange: (liabilities: OnboardingLiability[]) => void;
}

export function LiabilitiesStep({
  liabilities,
  onChange,
}: LiabilitiesStepProps) {
  const addLiability = () => {
    if (liabilities.length < 2) {
      onChange([
        ...liabilities,
        {
          liability_name: "",
          liability_type: "credit_card",
          outstanding_balance: "",
          interest_rate: "0",
          institution: "",
          is_paid_in_cadences: false,
          cadence: "monthly",
          cadence_day_date: "1",
          cadence_amount: "",
        },
      ]);
    }
  };

  const removeLiability = (index: number) => {
    onChange(liabilities.filter((_, i) => i !== index));
  };

  const updateLiability = (
    index: number,
    updates: Partial<OnboardingLiability>,
  ) => {
    const next = [...liabilities];
    next[index] = { ...next[index], ...updates };
    onChange(next);
  };

  return (
    <div className="flex flex-col h-full gap-6">
      <h2 className="font-mono text-xs text-muted-foreground tracking-[0.4em] uppercase">
        Financial Commitments
```

```
      </h2>
      <div className="flex-1 space-y-6 md:space-y-4 overflow-y-auto scrollbar-hide pr-2">
        {liabilities.map((liab, idx) => (
          <div
            key={idx}
            className="space-y-4 md:space-y-3 pb-6 md:pb-3 border-b border-white/10 relative shrink-0"
          >
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4 md:gap-6">
              <input
                type="text"
                placeholder="Obligation Name"
                value={liab.liability_name}
                onChange={(e) =>
                  updateLiability(idx, { liability_name: e.target.value })
                }
                          className="bg-transparent border-b border-border py-1 font-mono text-sm uppercase tracking-widest
text-foreground focus:outline-none placeholder:text-white/10"
              />
              <input
                type="text"
                placeholder="Financial Institution"
                value={liab.institution}
                onChange={(e) =>
                  updateLiability(idx, { institution: e.target.value })
                }
                       className="bg-transparent border-b border-border py-1 text-[10px] font-mono uppercase tracking-widest
text-muted-foreground/60 focus:outline-none placeholder:text-white/10"
              />
            </div>
            <div className="grid grid-cols-1 md:grid-cols-3 gap-4 md:gap-6">
              <div className="space-y-1">
                <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                  Type
                </label>
                <select
                  value={liab.liability_type}
                  onChange={(e) =>
                    updateLiability(idx, { liability_type: e.target.value })
                  }
                   className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono tracking-widest uppercase
text-muted-foreground/80 focus:outline-none"
                >
                  {LIABILITY_TYPES.map((t) => (
                    <option
                      key={t.value}
                      value={t.value}
                      className="bg-[#080808]"
                    >
                      {t.label}
                    </option>
                  ))}
                </select>
              </div>
              <div className="grid grid-cols-2 gap-4 md:col-span-2">
                <div className="space-y-1">
                  <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                    Balance
                  </label>
                  <input
                    type="number"
                    placeholder="0"
                    value={liab.outstanding_balance}
                    onChange={(e) =>
                      updateLiability(idx, {
                        outstanding_balance: e.target.value,
```

```
                    })
                  }
                            className="w-full bg-transparent border-b border-border py-1 text-sm font-mono text-foreground
focus:outline-none tabular-nums"
                  />
                </div>
                <div className="space-y-1">
                  <label className="text-[9px] font-mono text-muted-foreground/60 uppercase tracking-[0.4em]">
                    Interest %
                  </label>
                  <input
                    type="number"
                    step="0.1"
                    placeholder="0"
                    value={liab.interest_rate}
                    onChange={(e) =>
                      updateLiability(idx, { interest_rate: e.target.value })
                    }
                            className="w-full bg-transparent border-b border-border py-1 text-sm font-mono text-foreground
focus:outline-none tabular-nums"
                  />
                </div>
              </div>
            </div>

            <div className="pt-2 space-y-3">
              <label className="flex items-center space-x-3 cursor-pointer group">
                <input
                  type="checkbox"
                  checked={liab.is_paid_in_cadences}
                  onChange={(e) =>
                    updateLiability(idx, {
                      is_paid_in_cadences: e.target.checked,
                    })
                  }
                  className="w-4 h-4 rounded border-border bg-transparent checked:bg-white transition-colors cursor-pointer"
                />
                                <span  className="text-[10px]  font-mono  text-muted-foreground/70  uppercase  tracking-widest
group-hover:text-muted-foreground/90 transition-colors">
                  Paid in Cadence
                </span>
              </label>
              {liab.is_paid_in_cadences && (
                <motion.div
                  initial={{ opacity: 0, y: -5 }}
                  animate={{ opacity: 1, y: 0 }}
                  className="grid grid-cols-1 md:grid-cols-3 gap-4 md:gap-6 pt-2 pl-6 md:pl-8 border-l border-white/5"
                >
                  <div className="space-y-1">
                    <label className="text-[9px] font-mono text-muted-foreground/60 uppercase">
                      Cadence
                    </label>
                    <select
                      value={liab.cadence}
                      onChange={(e) =>
                        updateLiability(idx, { cadence: e.target.value })
                      }
                                      className="w-full  bg-transparent  border-b  border-border  py-1  text-[10px]  font-mono
text-muted-foreground/90 focus:outline-none"
                    >
                      <option value="weekly" className="bg-black">
                        Weekly
                      </option>
                      <option value="monthly" className="bg-black">
                        Monthly
```

```
                  </option>
                </select>
              </div>
              <div className="grid grid-cols-2 gap-4 md:col-span-2">
                <div className="space-y-1">
                  <label className="text-[9px] font-mono text-muted-foreground/60 uppercase">
                    {liab.cadence === "weekly" ? "Day" : "Day"}
                  </label>
                  {liab.cadence === "monthly" ? (
                    <input
                      type="number"
                      min="1"
                      max="31"
                      placeholder="15"
                      value={liab.cadence_day_date}
                      onChange={(e) =>
                        updateLiability(idx, {
                          cadence_day_date: e.target.value,
                        })
                      }
                                    className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono
text-muted-foreground/90 focus:outline-none"
                    />
                  ) : (
                    <select
                      value={liab.cadence_day_date || "Monday"}
                      onChange={(e) =>
                        updateLiability(idx, {
                          cadence_day_date: e.target.value,
                        })
                      }
                                    className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono
text-muted-foreground/90 focus:outline-none"
                    >
                      {[
                        "Monday",
                        "Tuesday",
                        "Wednesday",
                        "Thursday",
                        "Friday",
                        "Saturday",
                        "Sunday",
                      ].map((day) => (
                        <option key={day} value={day} className="bg-black">
                          {day.slice(0, 3)}
                        </option>
                      ))}
                    </select>
                  )}
                </div>
                <div className="space-y-1">
                  <label className="text-[9px] font-mono text-muted-foreground/60 uppercase">
                    Payment
                  </label>
                  <input
                    type="number"
                    placeholder="0"
                    value={liab.cadence_amount}
                    onChange={(e) =>
                      updateLiability(idx, {
                        cadence_amount: e.target.value,
                      })
                    }
                                  className="w-full bg-transparent border-b border-border py-1 text-[10px] font-mono
text-muted-foreground/90 focus:outline-none"
```

```
                    />
                  </div>
                </div>
              </motion.div>
            )}
          </div>

          <button
            type="button"
            onClick={() => removeLiability(idx)}
                 className="absolute top-0 right-0 p-2 text-muted-foreground/50 hover:text-red-400 bg-white/5 rounded-full
hover:bg-white/10 transition-all"
          >
            ?
          </button>
        </div>
      ))}
    </div>
    <button
      type="button"
      onClick={addLiability}
             className="text-[11px] font-mono tracking-[0.6em] uppercase text-muted-foreground/70 hover:text-foreground
transition-all py-4 px-8 border border-border rounded-full self-start active:scale-95"
    >
      + Append Obligation
    </button>
  </div>
 );
}
```

FILE: src\components\dashboard\onboarding\OnboardingLayout.tsx

```
"use client";

import { motion } from "framer-motion";
import { BrandMark } from "@/components/ui/brand-mark";
import type { LucideIcon } from "lucide-react";

export function OnboardingHeader({
  step,
  totalSteps,
}: {
  step: number;
  totalSteps: number;
}) {
  const romans = ["I", "II", "III", "IV"];
  return (
      <nav className="absolute top-0 left-0 w-full z-50 px-6 md:px-12 py-6 md:py-10 flex justify-between items-center
mix-blend-difference pointer-events-none">
    <div className="flex items-center gap-4 md:gap-6 group cursor-pointer pointer-events-auto">
      <BrandMark className="w-8 h-8 md:w-10 md:h-10" />
      <div className="flex flex-col">
        <span className="font-mono text-[10px] tracking-[0.6em] uppercase font-bold text-foreground">
          AXIOM
        </span>
        <span className="text-[8px] font-mono tracking-[0.4em] uppercase text-muted-foreground/60">
          Architecture
        </span>
      </div>
    </div>
    <div className="flex gap-6 md:gap-12 items-center pointer-events-auto">
      <div className="flex gap-4 md:gap-8">
        {Array.from({ length: totalSteps }).map((_, idx) => (
```

```
            <div
              key={idx}
              className="flex flex-col items-center gap-2 text-foreground"
            >
              <span
                className={`font-mono text-[9px] tracking-widest transition-colors duration-700 ${
                  idx + 1 === step ? "opacity-100" : "opacity-30"
                }`}
              >
                {romans[idx]}
              </span>
              <motion.div
                animate={{ scaleX: idx + 1 === step ? 1 : 0 }}
                className="h-px w-3 md:w-4 bg-foreground origin-left"
              />
            </div>
          ))}
        </div>
        <span className="hidden md:block font-mono text-[9px] tracking-[0.4em] uppercase text-muted-foreground/50">
          v1.0
        </span>
      </div>
    </nav>
  );
}

export function OnboardingSidebar({
  roman,
  icon: Icon,
  iconColor,
  title,
  desc,
  direction,
}: {
  roman: string;
  icon: LucideIcon;
  iconColor: string;
  title: string;
  desc: string;
  direction: number;
}) {
  return (
     <div className="flex flex-col justify-center h-full md:border-r border-border md:pr-12 text-foreground overflow-hidden
pt-4 md:pt-0">
      <motion.div
        key={title}
        initial={{ opacity: 0, x: direction * 50 }}
        animate={{ opacity: 1, x: 0 }}
        exit={{ opacity: 0, x: -direction * 50 }}
        transition={{ duration: 0.8, ease: [0.215, 0.61, 0.355, 1] }}
        className="space-y-4 md:space-y-6"
      >
        <div className="space-y-4 md:space-y-6">
          <div className="hidden md:flex items-center gap-6 font-mono font-bold text-8xl text-muted-foreground/10 leading-none
select-none">
            {roman}
                              <Icon  strokeWidth={1.5}  size={80}  style={{  color:  iconColor  }}  className="opacity-40
drop-shadow-[0_0_15px_rgba(255,255,255,0.1)]" />
          </div>
          <h1 className="font-mono text-4xl md:text-5xl text-foreground leading-none tracking-[0.2em] uppercase font-bold">
            {title}
          </h1>
        </div>
        <p className="text-muted-foreground/80 text-base md:text-lg font-light leading-relaxed max-w-lg md:max-w-none">
          {desc}
```

```
        </p>
      </motion.div>
    </div>
  );
}
```

FILE: src\components\landing\LandingTerminal.tsx

```
"use client";

import { useState, useEffect, useRef } from "react";

const LOGS = [
  { text: "> INITIALIZING KAIROS ENGINE...", type: "info" },
  { text: "> SYNCING OPERATIONAL BASELINE...", type: "info" },
  { text: "> ANALYZING TRANSACTION TAXONOMY...", type: "info" },
  { text: "> CALCULATING STRUCTURAL RUNWAY...", type: "info" },
  { text: "> [WARNING] INFLOW CONCENTRATION RISK DETECTED.", type: "warning" },
  { text: "> 85% OF YIELD RELIES ON A SINGLE FLOW.", type: "warning" },
  { text: "> STRUCTURAL SOLVENCY: OPTIMAL", type: "success" },
  { text: "> DEPLOYMENT STRATEGY: CALCULATED", type: "info" },
];

export function LandingTerminal() {
  const [visibleLogs, setVisibleLogs] = useState<number[]>([]);
  const [isTyping, setIsTyping] = useState(true);
  const containerRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    let timeoutId: ReturnType<typeof setTimeout>;

    const showNextLog = (index: number) => {
      if (index >= LOGS.length) {
        setIsTyping(false);
        // Reset after a delay to loop the animation
        timeoutId = setTimeout(() => {
          setVisibleLogs([]);
          setIsTyping(true);
          showNextLog(0);
        }, 5000);
        return;
      }

      timeoutId = setTimeout(
        () => {
          setVisibleLogs((prev) => [...prev, index]);
          showNextLog(index + 1);
        },
        index === 0 ? 500 : 1200,
      );
    };

    showNextLog(0);

    return () => clearTimeout(timeoutId);
  }, []);

  return (
    <div className="w-full max-w-2xl mx-auto font-mono text-sm">
      <div className="bg-[#0a0a0a] border border-white/10 rounded-lg overflow-hidden shadow-2xl shadow-white/5">
        {/* Terminal Header */}
        <div className="bg-white/5 px-4 py-2 border-b border-white/10 flex items-center justify-between">
          <div className="flex gap-1.5">
```

```
            <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
            <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
            <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
          </div>
          <div className="text-[10px] uppercase tracking-widest text-white/30">
            Kairos // Audit Terminal
          </div>
        </div>

        {/* Terminal Body */}
        <div
          ref={containerRef}
          className="p-6 h-64 overflow-y-auto space-y-2 scrollbar-hide"
        >
          {visibleLogs.map((logIndex) => {
            const log = LOGS[logIndex];
            return (
              <div
                key={logIndex}
                className={`
                  ${log.type === "warning" ? "text-orange-500" : ""}
                  ${log.type === "success" ? "text-emerald-500" : ""}
                  ${log.type === "info" ? "text-white/70" : ""}
                  animate-in fade-in slide-in-from-left-2 duration-300
                `}
              >
                {log.text}
              </div>
            );
          })}
          {isTyping && (
            <div className="w-2 h-4 bg-white/40 animate-pulse inline-block align-middle ml-1" />
          )}
        </div>
      </div>

      {/* Decorative footer for the terminal */}
      <div className="mt-4 flex justify-between items-center px-4 text-[10px] text-white/20 uppercase tracking-[0.2em]">
        <div>Status: Running Clinical Audit</div>
        <div>Memory: 0.124ms / 256MB</div>
      </div>
    </div>
  );
}
```

FILE: src\components\landing\ProcessFlow.tsx

```
"use client";

import { ScrollReveal } from "@/components/ui/scroll-reveal";
import { Cpu, Database, Brain, Zap } from "lucide-react";

const STEPS = [
  {
    icon: Database,
    title: "Ingestion",
     desc: "Authorize the system to index your capital flows. M-Pesa, MMF, and SACCO logs are ingested into the private
ledger.",
  },
  {
    icon: Cpu,
    title: "Taxonomy",
    desc: "Automated classification of every shilling. Axiom identifies the strategic intent behind every transaction.",
```

```
  },
  {
    icon: Brain,
    title: "Inference",
    desc: "The Kairos Engine simulates survival windows and calculates the strategic cost of behavioral leakage.",
  },
  {
    icon: Zap,
    title: "Deployment",
    desc: "Execute the optimized protocol. Align your liquidity with high-order objectives for maximum yield.",
  },
];

export function ProcessFlow() {
  return (
    <section className="py-40 px-6 bg-black font-mono">
      <div className="max-w-7xl mx-auto space-y-32">
        <ScrollReveal direction="up" distance={20}>
          <div className="space-y-4">
            <span className="text-[10px] tracking-[0.8em] text-truth uppercase font-bold">
              Operational Cycle
            </span>
            <h2 className="text-6xl md:text-9xl font-bold text-white uppercase tracking-tighter">
              The <span className="text-zinc-700">Protocol.</span>
            </h2>
          </div>
        </ScrollReveal>

        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-8">
          {STEPS.map((step, i) => (
            <ScrollReveal
              key={step.title}
              direction="up"
              distance={20}
              delay={i * 0.15}
            >
                  <div className="group space-y-8 p-8 border border-white/5 bg-[#0a0a0a] hover:border-white/20 transition-all
duration-500 h-full">
                <div className="flex items-center justify-between">
                  <div className="p-3 bg-zinc-950 border border-white/5 group-hover:border-truth/30 transition-colors">
                    <step.icon strokeWidth={1.5} size={24} className="text-truth" />
                  </div>
                  <span className="text-sm font-bold text-zinc-800">0{i + 1}</span>
                </div>
                <div className="space-y-4">
                  <h3 className="text-xl font-bold text-white uppercase tracking-tight">
                    {step.title}
                  </h3>
                              <p  className="text-zinc-500  text-xs  leading-relaxed  tracking-tight  group-hover:text-zinc-400
transition-colors">
                    {step.desc}
                  </p>
                </div>
              </div>
            </ScrollReveal>
          ))}
        </div>

        <ScrollReveal direction="up" distance={20} delay={0.6}>
          <div className="p-12 md:p-20 bg-zinc-950 border border-white/5 text-center space-y-12">
                <h3 className="text-3xl md:text-5xl font-bold text-white uppercase tracking-tighter leading-tight max-w-3xl
mx-auto">
              Sovereign wealth is not tracked; <br />
              <span className="text-zinc-600 italic font-serif normal-case font-light">it is architected.</span>
            </h3>
```

```
              <button className="px-12 py-4 border border-white/10 text-white font-bold text-[10px] tracking-widest uppercase
hover:bg-white hover:text-black transition-all">
              Initialize Protocol Access
            </button>
          </div>
        </ScrollReveal>
      </div>
    </section>
  );
}
```

FILE: src\components\landing\RunwaySimulator.tsx

```
"use client";

import { useState, useMemo } from "react";

export function RunwaySimulator() {
  const [liquidity, setLiquidity] = useState(50000);
  const [monthlyBurn, setMonthlyBurn] = useState(30000);

  const runwayResult = useMemo(() => {
    if (monthlyBurn <= 0) return { months: 99, days: 0, isInfinite: true };

    const totalMonths = liquidity / monthlyBurn;
    const months = Math.floor(totalMonths);
    const days = Math.round((totalMonths - months) * 30);

    return { months, days, isInfinite: false };
  }, [liquidity, monthlyBurn]);

  const getVerdict = () => {
    const totalMonths = runwayResult.isInfinite
      ? 99
      : runwayResult.months + runwayResult.days / 30;
    if (totalMonths < 3) {
      return {
        label: "CRITICAL",
        text: "Severe solvency risk. Capital depletion imminent.",
        color: "text-orange-500",
      };
    }
    if (totalMonths >= 3 && totalMonths < 6) {
      return {
        label: "ADVISORY",
        text: "Fragile runway. One systemic shock away from illiquidity.",
        color: "text-white/60",
      };
    }
    return {
      label: "OBSERVATION",
      text: "Runway stable. Capital ready for strategic deployment.",
      color: "text-emerald-500/80",
    };
  };

  const verdict = getVerdict();

  const formatCurrency = (val: number) => {
    return new Intl.NumberFormat("en-KE", {
      style: "currency",
      currency: "KES",
      maximumFractionDigits: 0,
```

```
    }).format(val);
  };

  return (
     <div className="w-full max-w-2xl mx-auto space-y-12 p-8 md:p-12 border border-white/10 bg-[#050505] rounded-2xl relative
overflow-hidden group">
      {/* Background decoration */}
      <div className="absolute top-0 right-0 w-32 h-32 bg-white/2 blur-3xl rounded-full" />

      <div className="space-y-8 relative z-10">
        <div className="flex justify-between items-end">
          <div className="space-y-1">
            <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
              Operational Variable 01
            </div>
            <div className="text-sm font-black uppercase tracking-widest">
              Available Liquidity
            </div>
          </div>
          <div className="font-mono text-2xl font-bold">
            {formatCurrency(liquidity)}
          </div>
        </div>
        <input
          type="range"
          min="0"
          max="1000000"
          step="5000"
          value={liquidity}
          onChange={(e) => setLiquidity(Number(e.target.value))}
          className="w-full h-px bg-white/20 appearance-none cursor-crosshair accent-white"
        />
      </div>

      <div className="space-y-8 relative z-10">
        <div className="flex justify-between items-end">
          <div className="space-y-1">
            <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
              Operational Variable 02
            </div>
            <div className="text-sm font-black uppercase tracking-widest">
              Monthly Burn Rate
            </div>
          </div>
          <div className="font-mono text-2xl font-bold">
            {formatCurrency(monthlyBurn)}
          </div>
        </div>
        <input
          type="range"
          min="1000"
          max="500000"
          step="1000"
          value={monthlyBurn}
          onChange={(e) => setMonthlyBurn(Number(e.target.value))}
          className="w-full h-px bg-white/20 appearance-none cursor-crosshair accent-white"
        />
      </div>

      <div className="pt-8 border-t border-white/5 space-y-6">
        <div className="flex justify-between items-baseline">
          <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
            Calculated Outcome
          </div>
          <div className="text-4xl font-black font-mono tracking-tighter flex items-baseline gap-2">
```

```
            {runwayResult.isInfinite ? (
              "?"
            ) : (
              <>
                <span>{runwayResult.months}</span>
                <span className="text-xs uppercase tracking-widest text-white/40">
                  Months
                </span>
                {runwayResult.days > 0 && (
                  <>
                    <span className="ml-2">{runwayResult.days}</span>
                    <span className="text-xs uppercase tracking-widest text-white/40">
                      Days
                    </span>
                  </>
                )}
              </>
            )}
          </div>
        </div>

        <div
             className={`p-6 border border-white/5 bg-white/2 font-mono text-xs leading-relaxed space-y-2 animate-in fade-in
slide-in-from-top-1 duration-500`}
        >
          <div className="flex items-center gap-2">
            <div
                      className={`w-1.5 h-1.5 rounded-full animate-pulse ${verdict.label === "CRITICAL" ? "bg-orange-500" :
verdict.label === "ADVISORY" ? "bg-white/40" : "bg-emerald-500"}`}
            />
            <span className={`font-black tracking-[0.2em] ${verdict.color}`}>
              {verdict.label}
            </span>
          </div>
          <div className="text-white/60">{verdict.text}</div>
        </div>
      </div>

      <div className="absolute bottom-4 right-8 text-[8px] font-mono uppercase tracking-[0.4em] text-white/10">
        Deterministic Solver v0.4
      </div>
    </div>
  );
}
```

FILE: src\components\landing\SolvencyCalculator.tsx

```
"use client";

import { useState, useMemo } from "react";
import { ScrollReveal } from "@/components/ui/scroll-reveal";

export function SolvencyCalculator() {
  const [balance, setBalance] = useState(500000);
  const [burn, setBurn] = useState(80000);

  const runway = useMemo(() => {
    if (burn === 0) return 99;
    const months = balance / burn;
    return months > 99 ? 99 : Math.floor(months);
  }, [balance, burn]);

  const formattedBalance = new Intl.NumberFormat("en-KE", {
```

```
  style: "currency",
  currency: "KES",
  maximumFractionDigits: 0,
}).format(balance);

const formattedBurn = new Intl.NumberFormat("en-KE", {
  style: "currency",
  currency: "KES",
  maximumFractionDigits: 0,
}).format(burn);

return (
  <section className="py-32 px-6 bg-black font-mono">
    <div className="max-w-7xl mx-auto grid grid-cols-1 lg:grid-cols-2 gap-16 md:gap-32 items-center">
      {/* INPUTS */}
      <ScrollReveal direction="left" distance={20}>
        <div className="space-y-12">
          <div className="space-y-4">
            <span className="text-[10px] tracking-[0.8em] text-truth uppercase font-bold">
              Simulation Engine
            </span>
            <h2 className="text-5xl md:text-7xl font-bold text-white uppercase tracking-tighter leading-none">
              Calculate your <br />
              <span className="text-zinc-600">Survival Window.</span>
            </h2>
          </div>

          <div className="space-y-16">
            <div className="space-y-8">
              <div className="flex justify-between items-end border-b border-white/10 pb-2">
                <span className="text-[10px] tracking-widest text-zinc-500 uppercase font-bold">
                  Liquid Architecture
                </span>
                <span className="text-2xl font-bold text-white tabular-nums">
                  {formattedBalance}
                </span>
              </div>
              <input
                type="range"
                min="50000"
                max="5000000"
                step="50000"
                value={balance}
                onChange={(e) => setBalance(parseInt(e.target.value))}
                className="w-full accent-white"
              />
            </div>

            <div className="space-y-8">
              <div className="flex justify-between items-end border-b border-white/10 pb-2">
                <span className="text-[10px] tracking-widest text-zinc-500 uppercase font-bold">
                  Structural Burn (Monthly)
                </span>
                <span className="text-2xl font-bold text-white tabular-nums">
                  {formattedBurn}
                </span>
              </div>
              <input
                type="range"
                min="10000"
                max="1000000"
                step="10000"
                value={burn}
                onChange={(e) => setBurn(parseInt(e.target.value))}
                className="w-full accent-white"
```

```
              />
            </div>
          </div>
        </div>
      </ScrollReveal>

      {/* RESULT */}
      <ScrollReveal direction="right" distance={20} delay={0.3}>
        <div className="relative p-12 bg-[#0a0a0a] border border-white/10 rounded-sm overflow-hidden">
          {/* Background Data Matrix */}
                                              <div    className="absolute    inset-0    opacity-10    pointer-events-none
bg-[linear-gradient(to_right,#80808012_1px,transparent_1px),linear-gradient(to_bottom,#80808012_1px,transparent_1px)]
bg-[size:24px_24px]" />

          <div className="relative z-10 space-y-10">
            <div className="space-y-1">
              <p className="text-[10px] tracking-[0.5em] text-zinc-500 uppercase font-bold">
                Computed Solvency
              </p>
              <div className="flex items-baseline gap-4">
                <span className="text-[10rem] md:text-[12rem] font-bold text-truth leading-none tracking-tighter">
                  {runway}
                </span>
                <span className="text-3xl font-bold text-zinc-700 uppercase tracking-widest">
                  Months
                </span>
              </div>
            </div>

            <div className="pt-12 border-t border-white/10 space-y-10">
              <p className="text-zinc-500 text-xs leading-relaxed max-w-sm uppercase tracking-tight">
                This projection is based on your current liquid architecture.
                Axiom provides the technical units to optimize these
                variables and extend your survival window indefinitely.
              </p>

                    <button className="w-full py-6 bg-white text-black font-bold text-[10px] tracking-[0.4em] uppercase
hover:bg-zinc-200 transition-all">
                Initialize Baseline Architecture
              </button>

              <p className="text-[9px] text-zinc-800 uppercase tracking-widest text-center">
                * Clinical mathematical truth // engine: kairos
              </p>
            </div>
          </div>
        </div>
      </ScrollReveal>
    </div>
  </section>
 );
}
```

FILE: src\components\landing\StrategicPillars.tsx

```
"use client";

import { ScrollReveal } from "@/components/ui/scroll-reveal";
import { Shield, Target, Zap, Activity } from "lucide-react";

const PILLARS = [
  {
    icon: Shield,
```

```
    title: "Structural Solvency",
     desc: "Deterministic modeling to ensure your liquid architecture remains impenetrable to market volatility and erratic flows.",
    color: "text-truth",
  },
  {
    icon: Target,
    title: "Strategic Alignment",
    desc: "Dynamic mapping of capital deployment to higher-order objectives. Every shilling must serve a tactical purpose.",
    color: "text-truth",
  },
  {
    icon: Zap,
    title: "Velocity Optimization",
       desc: "Calculating the replenishment speed of your capital pool. Optimizing yield cycles for maximum deployment capacity.",
    color: "text-warning",
  },
  {
    icon: Activity,
    title: "Behavioral Auditing",
      desc: "The Kairos Engine continuously monitors transaction taxonomy, identifying leakage and enforcing operational discipline.",
    color: "text-opportunity",
  },
];

export function StrategicPillars() {
  return (
    <section className="py-40 px-6 bg-black font-mono">
      <div className="max-w-7xl mx-auto space-y-32">
        {/* HEADER */}
        <ScrollReveal direction="up" distance={20}>
          <div className="flex flex-col md:flex-row items-end justify-between gap-12">
            <h2 className="text-6xl md:text-9xl font-bold text-white uppercase tracking-tighter leading-none">
              Strategic <br />
              <span className="text-zinc-700">Pillars.</span>
            </h2>
            <div className="max-w-sm space-y-6">
              <p className="text-zinc-500 text-xs tracking-tight uppercase leading-relaxed">
                Axiom is built on four core technical units designed to provide
                absolute financial sovereignty in any ecosystem.
              </p>
              <div className="h-px w-full bg-white/10" />
            </div>
          </div>
        </ScrollReveal>

        {/* GRID */}
        <div className="grid grid-cols-1 md:grid-cols-2 gap-px bg-white/10 border border-white/10">
          {PILLARS.map((pillar, i) => (
            <ScrollReveal
              key={pillar.title}
              direction="up"
              distance={20}
              delay={i * 0.1}
            >
              <div className="group p-12 md:p-20 bg-black hover:bg-zinc-900 transition-all duration-500 space-y-12 h-full">
                <div className="flex items-start justify-between">
                                            <div  className={`p-4  border  border-white/5  bg-zinc-950  transition-colors group-hover:border-${pillar.color.split("-")[1]}/30`}>
                    <pillar.icon strokeWidth={1.5} size={32} className={pillar.color} />
                  </div>
                  <span className="text-2xl font-bold text-zinc-800 group-hover:text-zinc-600 transition-colors">
                    0{i + 1}
```

```
                </span>
              </div>
              <div className="space-y-6">
                <h3 className="text-3xl font-bold text-white uppercase tracking-tight">
                  {pillar.title}
                </h3>
                            <p  className="text-zinc-500  text-sm  leading-relaxed  tracking-tight  group-hover:text-zinc-400
transition-colors">
                  {pillar.desc}
                </p>
              </div>
            </div>
          </ScrollReveal>
        ))}
      </div>
    </div>
  </section>
 );
}
```

FILE: src\components\ui\animated-number.tsx

```
"use client";

import { useEffect, useState } from "react";
import { motion, useSpring, useTransform } from "framer-motion";

interface AnimatedNumberProps {
  value: number;
  prefix?: string;
  suffix?: string;
  className?: string;
  precision?: number;
  compact?: boolean;
}

export function AnimatedNumber({
  value,
  prefix = "",
  suffix = "",
  className = "",
  precision = 0,
  compact = false,
}: AnimatedNumberProps) {
  const spring = useSpring(0, {
    mass: 1,
    stiffness: 50,
    damping: 15,
  });

  const display = useTransform(spring, (current) => {
    if (compact && current >= 10000) {
      return `${prefix}${new Intl.NumberFormat("en-KE", {
        notation: "compact",
        compactDisplay: "short",
        maximumFractionDigits: 1,
      }).format(current)}${suffix}`;
    }

    return `${prefix}${current.toLocaleString(undefined, {
      minimumFractionDigits: precision,
      maximumFractionDigits: precision,
    })}${suffix}`;
```

```
  });

  useEffect(() => {
    spring.set(value);
  }, [value, spring]);

  return <motion.span className={className}>{display}</motion.span>;
}
```

FILE: src\components\ui\brand-mark.tsx

```
"use client";

import { motion } from "framer-motion";

export function BrandMark({ className = "w-10 h-10" }: { className?: string }) {
  return (
    <motion.div
      initial="initial"
      whileHover="hover"
      whileTap="tap"
      className={`${className} relative flex items-center justify-center cursor-pointer`}
    >
      {/* Main Glass Background */}
      <motion.div
        variants={{
          initial: { rotate: 0, scale: 1 },
          hover: {
            rotate: 4,
            scale: 1.08,
            boxShadow: "0 15px 30px rgba(59, 130, 246, 0.2)",
          },
          tap: { scale: 0.95 },
        }}
        transition={{
          type: "spring",
          stiffness: 300,
          damping: 25,
        }}
        className="absolute inset-0 glass-panel rounded-[32%] shadow-lg border border-truth/20"
      />

      {/* Pop-out Frame - Functional Glow */}
      <motion.div
        variants={{
          initial: { rotate: 0, scale: 1, opacity: 0 },
          hover: {
            rotate: 10,
            scale: 1.25,
            opacity: 0.4,
          },
        }}
        transition={{
          type: "spring",
          stiffness: 250,
          damping: 20,
        }}
        className="absolute inset-0 border border-truth rounded-[32%] pointer-events-none blur-[2px]"
      />

      {/* Static Text - San Francisco (SF Pro) */}
      <span
                        className="relative   z-10   text-truth   font-black   text-xl   select-none   pointer-events-none
```

```
drop-shadow-[0_0_10px_rgba(59,130,246,0.5)]"
        style={{
          fontFamily:
                 '-apple-system, BlinkMacSystemFont, "SF Pro Display", "SF Pro Text", "Helvetica Neue", Helvetica, Arial,
sans-serif',
        }}
      >
        A
      </span>
    </motion.div>
  );
}
```

FILE: src\components\ui\scroll-reveal.tsx

```
"use client";

import { motion } from "framer-motion";
import { ReactNode } from "react";

interface ScrollRevealProps {
  children: ReactNode;
  delay?: number;
  direction?: "up" | "down" | "left" | "right" | "none";
  distance?: number;
  duration?: number;
}

export function ScrollReveal({
  children,
  delay = 0,
  direction = "up",
  distance = 40,
  duration = 0.8,
}: ScrollRevealProps) {
  const directions = {
    up: { y: distance },
    down: { y: -distance },
    left: { x: distance },
    right: { x: -distance },
    none: {},
  };

  return (
    <motion.div
      initial={{
        opacity: 0,
        ...directions[direction]
      }}
      whileInView={{
        opacity: 1,
        x: 0,
        y: 0
      }}
      viewport={{ once: true, margin: "-10%" }}
      transition={{
        duration: duration,
        delay: delay,
        ease: [0.215, 0.61, 0.355, 1], // Luxury cubic bezier (easeOutCirc)
      }}
    >
      {children}
    </motion.div>
```

```
  );
}
```

FILE: src\components\ui\smooth-scroll-provider.tsx

```
"use client";

import { ReactLenis } from "lenis/react";
import { ReactNode } from "react";

export function SmoothScrollProvider({ children }: { children: ReactNode }) {
  return (
    <ReactLenis
      root
      options={{
        duration: 1.5,
        easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)),
        orientation: "vertical",
        gestureOrientation: "vertical",
        smoothWheel: true,
        wheelMultiplier: 1,
        touchMultiplier: 2,
        infinite: false,
      }}
    >
      {children}
    </ReactLenis>
  );
}
```

FILE: src\lib\supabase.ts

```
import { createClient } from "./supabase/client";

export const supabase = createClient();
```

FILE: src\lib\ai\kairos.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";
import { generateSystemInsights } from "./kairos";
import { AnalyticsSummary } from "../analytics/types";
import { buildBehavioralContext } from "../context/buildBehavioralContext";
import { evaluateInsights } from "../insights/generateInsights";

const MOCK_METADATA_QUALITY = {
  averageScore: 1,
  lowQualityCount: 0,
  lowQualityRatio: 0,
  confidenceMultiplier: 1,
};

const MOCK_ANALYTICS: AnalyticsSummary = {
  totalDeployed: 1000,
  averageDeployment: 100,
  dailyBurnRate: 10,
  runwayDays: 100,
  categoryBreakdown: { Leakage: 500, Asset: 500 },
  deploymentCount: 10,
```

```
  metadataQuality: MOCK_METADATA_QUALITY,
  totalLiabilities: 0,
  totalAssets: 2000,
  netWorth: 2000,
  liquidity: 1000,
  totalMonthlyIncome: 300,
  recurringIncome: 300,
  irregularIncome: 0,
  incomeConcentration: {},
  maxIncomeConcentrationRatio: 0,
  adjustedDailyBurn: 0,
  totalStrategicTargets: 1000,
  totalCurrentProgress: 500,
  averageGoalProgress: 50,
  criticalGoalCount: 0,
  fundingGap: 500,
  pendingVerificationCount: 0,
  // Strategic Objectives v1
  strategicAlignment: {
    fundingRatios: {},
    alignmentPressure: 0,
    liquiditySufficiency: {
      isSufficient: true,
      message:
        "Current liquidity structure sufficiently supports short-term obligations.",
      shortfall: 0,
    },
    objectiveStarvationSignals: [],
    strategicAllocationSignals: [],
    velocity: {},
  },
  // Operational Baseline v1
  totalStructuralMonthlyBurn: 0,
  totalSystemicMonthlyAllocation: 0,
};

test("Kairos Insights: correctly invokes rule engine and returns primary insight", async () => {
  const context = buildBehavioralContext(
    { currentAnalytics: MOCK_ANALYTICS },
    [100, 200, 300],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.ok(primaryInsight.message);
  assert.ok(primaryInsight.severity);
  assert.ok(primaryInsight.category);
});

test("Kairos Insights: critical runway trigger", async () => {
  const analytics = {
    ...MOCK_ANALYTICS,
    runwayDays: 10,
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100, 200, 300],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.equal(primaryInsight.severity, "critical");
  assert.equal(primaryInsight.category, "solvency_pressure");
  assert.match(primaryInsight.message, /runway has contracted/);
});

test("Kairos Insights: efficiency crisis trigger", async () => {
```

```
  const analytics = {
    ...MOCK_ANALYTICS,
    categoryBreakdown: { Leakage: 1000, Asset: 0 },
    runwayDays: 10, // Force low runway to plummet efficiency score < 40
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100, 2000, 50, 3000],
  );

  // Rule condition: capitalEfficiencyScore < 40
  // buildBehavioralContext for runway < 90 gives -40 penalty.
  // Volatility penalty is vol * 60.

  const { primaryInsight } = await evaluateInsights(context);

  // Solvency pressure (runway) usually has higher priority than efficiency crisis
  // if both are triggered, since runway is status: 'critical'.
  // However, in registry.ts, both have priority: 'high'.
  // evaluateInsights sorts by priority and then maintains order.
  // runway_critical is ID 1, efficiency_crisis is ID 2.

  assert.equal(primaryInsight.severity, "critical");
  assert.match(
    primaryInsight.message,
    /(runway has contracted|efficiency crisis)/,
  );
});

test("Kairos Insights: unclassified data advisory", async () => {
  const analytics = {
    ...MOCK_ANALYTICS,
    categoryBreakdown: { Unknown: 500, Asset: 500 },
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [500, 500],
  );
  const { allInsights } = await evaluateInsights(context);

  const unclassifiedInsight = allInsights.find((i) =>
    i.message.includes("Unclassified"),
  );
  assert.ok(unclassifiedInsight);
  assert.equal(unclassifiedInsight.severity, "advisory");
});

test("Kairos Insights: high discipline pattern", async () => {
  // Healthy state with stable spending
  const context = buildBehavioralContext(
    { currentAnalytics: MOCK_ANALYTICS },
    [100, 100, 100],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.equal(primaryInsight.severity, "observation");
  assert.match(primaryInsight.message, /High operational discipline/);
  assert.equal(primaryInsight.isSilent, true);
});

test("Kairos Insights: silence state default pattern", async () => {
  // State that doesn't trigger High Discipline but falls into Default Pattern
  const analytics = {
    ...MOCK_ANALYTICS,
    deploymentCount: 1, // High Discipline requires count > 1
```

```
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.equal(primaryInsight.severity, "observation");
  assert.match(primaryInsight.message, /No material behavioral shifts/);
  assert.equal(primaryInsight.isSilent, true);
});
```

FILE: src\lib\ai\kairos.ts

```
import { createClient } from "../supabase/server";
import { getDeployments } from "../db/deployments";
import { getAccounts } from "../db/accounts";
import { getLiabilities } from "../db/liabilities";
import { getIncomeStreams } from "../db/income";
import { getGoals } from "../db/goals";
import { getStrategicObjectives } from "../db/objectives";
import { getOperationalBaseline } from "../db/baseline";
import { getUserSettings } from "../db/settings";
import { generateSummary } from "../analytics/engine";
import {
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
} from "../analytics/types";
import { MetadataQualitySummary } from "../finance/metadataQuality";
import { buildBehavioralContext } from "../context/buildBehavioralContext";
import { evaluateInsights } from "../insights/generateInsights";

export type InsightSeverity =
  | "observation"
  | "advisory"
  | "warning"
  | "critical";

export interface KairosInsight {
  type: "warning" | "info" | "pattern" | "opportunity";
  severity: InsightSeverity;
  category:
    | "capital_efficiency"
    | "solvency_pressure"
    | "strategic_alignment"
    | "objective_starvation";
  message: string;
  supportingSignals: string[];
  timestamp: string;
  runway: number | null;
  capitalEfficiency: number;
  isSilent: boolean;
  metadataQuality?: MetadataQualitySummary;
  is_new_signal?: boolean;
}

export interface KairosTelemetry {
  deployments?: Deployment[];
```

```
  liquidity?: number;
  accounts?: Account[];
  liabilities?: Liability[];
  incomeStreams?: IncomeStream[];
  goals?: FinancialGoal[];
  objectives?: StrategicObjective[];
  baseline?: OperationalBaseline[];
  previousInsight?: KairosInsight | null;
}

/**
 * SERVER-SIDE ORCHESTRATOR: generateSystemInsights
 * Safely hydrates the Kairos AI layer with analytical context.
 */
export async function generateSystemInsights(
  telemetry: KairosTelemetry = {},
): Promise<KairosInsight> {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) {
    throw new Error("Authentication required for Kairos hydration.");
  }

  // 1. Resolve Canonical Context (Hydrate missing pieces only)
  const deps =
    (telemetry.deployments ?? (await getDeployments(supabase))) || [];
  const accounts = (telemetry.accounts ?? (await getAccounts(supabase))) || [];
  const liabilities =
    (telemetry.liabilities ?? (await getLiabilities(supabase))) || [];
  const income =
    (telemetry.incomeStreams ?? (await getIncomeStreams(supabase))) || [];
  const goals = (telemetry.goals ?? (await getGoals(supabase))) || [];
  const objectives =
    (telemetry.objectives ?? (await getStrategicObjectives(supabase))) || [];
  const baseline =
    (telemetry.baseline ?? (await getOperationalBaseline(supabase))) || [];

  let liquidity = telemetry.liquidity;
  if (liquidity === undefined) {
    const settings = await getUserSettings(supabase, user.id);
    liquidity = Number(
      (settings as { total_liquidity: number | string }).total_liquidity,
    );
  }

  // 2. Build Deterministic Analytics
  const analytics = generateSummary(
    deps,
    liquidity,
    accounts,
    liabilities,
    income,
    goals,
    objectives,
    baseline,
  );

  // 3. Build Behavioral Context for Insight Engine
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    deps.map((d) => Number(d.amount)),
  );
```

```
  // 4. Evaluate Insights via Rule Registry (V1.1)
  console.log("KAIROS TELEMETRY:", {
    deploymentsCount: deps.length,
    runwayDays: analytics.runwayDays,
    alignmentPressure: analytics.strategicAlignment.alignmentPressure,
    netWorth: analytics.netWorth,
  });

  const { primaryInsight } = await evaluateInsights(context);

  const previousInsight = telemetry.previousInsight;
  const isMessageChanged = primaryInsight.message !== previousInsight?.message;
  const isSeverityChanged =
    primaryInsight.severity !== previousInsight?.severity;
  const isSilentChanged = primaryInsight.isSilent !== previousInsight?.isSilent;

  const isNewSignal = isMessageChanged || isSeverityChanged || isSilentChanged;

  return {
    ...primaryInsight,
    is_new_signal: isNewSignal,
  };
}

/**
 * Bridge support for streamed telemetry from Dashboard Actions.
 */
export async function generateKairosAIInsight(
  telemetry: KairosTelemetry,
): Promise<KairosInsight> {
  return generateSystemInsights(telemetry);
}
```

FILE: src\lib\analytics\engine.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";
import {
  projectRunway,
  calculateBurnRate,
  calculateIncomeMetrics,
  countPendingVerifications,
} from "./engine";
import type { Deployment, IncomeStream, OperationalBaseline } from "./types";

test("projectRunway handles dimensional math correctly", () => {
  // Scenario: 1000 balance, 10 daily burn, 0 income
  // Runway = 1000 / 10 = 100 days
  assert.equal(projectRunway(1000, 10, 0), 100);

  // Scenario: 1000 balance, 10 daily burn, 150 monthly income
  // adjustedDailyBurn = 10 - (150 / 30) = 5
  // Runway = 1000 / 5 = 200 days
  assert.equal(projectRunway(1000, 10, 150), 200);
});

test("projectRunway returns null when state is stable (income >= burn)", () => {
  // Scenario: 10 daily burn, 300 monthly income
  // adjustedDailyBurn = 10 - (300 / 30) = 0
  assert.equal(projectRunway(1000, 10, 300), null);

  // Scenario: 10 daily burn, 450 monthly income
```

```
  // adjustedDailyBurn = 10 - (450 / 30) = -5
  assert.equal(projectRunway(1000, 10, 450), null);
});

test("projectRunway returns 0 when balance is 0 or negative (and burn > income)", () => {
  assert.equal(projectRunway(0, 10, 0), 0);
  assert.equal(projectRunway(-100, 10, 0), 0);
});

test("projectRunway returns null when dailyBurnRate is 0 or negative (unstable state but no burn)", () => {
  assert.equal(projectRunway(1000, 0, 0), null);
  assert.equal(projectRunway(1000, -5, 0), null);
});

test("calculateBurnRate remains deterministic", () => {
  const deployments: Deployment[] = [
    {
      id: "1",
      amount: 300,
      title: "Rent",
      category: "Leakage",
      created_at: new Date().toISOString(),
    },
    {
      id: "2",
      amount: 150,
      title: "Cloud",
      category: "Leverage",
      created_at: new Date().toISOString(),
    },
  ];

  // 450 total / 30 days = 15/day
  assert.equal(calculateBurnRate(deployments, 30), 15);
});

test("calculateMaxIncomeConcentration: 90% dependency", () => {
  const streams: Partial<IncomeStream>[] = [
    {
      income_name: "Source A",
      amount: 900,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source B",
      amount: 100,
      cadence: "monthly",
      income_type: "freelance",
      is_recurring: true,
    },
  ];

  // total = 1000
  // max = 900
  // ratio = 0.9
  const metrics = calculateIncomeMetrics(streams as IncomeStream[]);
  assert.equal(metrics.maxRatio, 0.9);
});

test("calculateMaxIncomeConcentration: balanced split", () => {
  const streams: Partial<IncomeStream>[] = [
    {
      income_name: "Source A",
```

```
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source B",
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source C",
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
  ];

  // total = 300
  // max = 100
  // ratio = 0.333...
  const metrics = calculateIncomeMetrics(streams as IncomeStream[]);
  assert.ok(Math.abs(metrics.maxRatio - 0.3333333333333333) < 0.0001);
});

test("calculateMaxIncomeConcentration: zero income", () => {
  const metrics = calculateIncomeMetrics([]);
  assert.equal(metrics.maxRatio, 0);
});

test("countPendingVerifications includes baseline logic", () => {
  const today = new Date();
  const currentDayOfMonth = today.getDate();
  const currentDayOfWeek = today.getDay() === 0 ? 7 : today.getDay();

  const baseline: Partial<OperationalBaseline>[] = [
    {
      id: "b1",
      title: "Daily Coffee",
      cadence: "daily",
      is_recurring: true,
      last_executed_at: null,
    },
    {
      id: "b2",
      title: "Weekly Server",
      cadence: "weekly",
      is_recurring: true,
      execution_day: currentDayOfWeek,
      last_executed_at: null,
    },
    {
      id: "b3",
      title: "Monthly Rent",
      cadence: "monthly",
      is_recurring: true,
      execution_day: currentDayOfMonth,
      last_executed_at: null,
    },
  ];

  assert.equal(
```

```
    countPendingVerifications([], [], baseline as OperationalBaseline[]),
    3,
  );

  // If already executed today, should be 0
  const executedTodayBaseline = baseline.map((b) => ({
    ...b,
    last_executed_at: today.toISOString(),
  }));
  assert.equal(
    countPendingVerifications(
      [],
      [],
      executedTodayBaseline as OperationalBaseline[],
    ),
    0,
  );
});
```

FILE: src\lib\analytics\engine.ts

```
import {
  Deployment,
  AnalyticsSummary,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  StrategicAlignment,
  OperationalBaseline,
} from "./types";

/**
 * Normalizes cadences into monthly values.
 */
export const normalizeToMonthly = (amount: number, cadence: string): number => {
  switch (cadence) {
    case "daily":
      return amount * 30.4375;
    case "weekly":
      return amount * 4.345;
    case "biweekly":
      return amount * 2.1725;
    case "monthly":
      return amount;
    case "quarterly":
      return amount / 3;
    case "yearly":
      return amount / 12;
    default:
      return amount;
  }
};

/**
 * Calculates monthly structural burn and systemic allocation metrics.
 */
export const calculateBaselineMetrics = (baseline: OperationalBaseline[]) => {
  return baseline.reduce(
    (acc, item) => {
      if (!item.is_active || !item.is_recurring) return acc;
      const monthlyAmount = normalizeToMonthly(item.amount, item.cadence);
```

```
      if (item.baseline_type === "expense") {
        acc.monthlyBurn += monthlyAmount;
      } else {
        acc.monthlyAllocation += monthlyAmount;
      }
      return acc;
    },
    { monthlyBurn: 0, monthlyAllocation: 0 },
  );
};

import { summarizeMetadataQuality } from "../finance/metadataQuality";
import { calculateMonthlyInflow } from "../finance/income";
import { calculateGoalProgressPercentage } from "../finance/goals";
import { calculateObjectiveFundingRatio } from "../finance/objectives";
import { LiquidityService } from "../finance/liquidity";

/**
 * Pure, deterministic engine for financial intelligence.
 * Rule: The database is truth. This engine only interprets.
 */

export const calculateLiabilityMetrics = (liabilities: Liability[]) => {
  return liabilities.reduce(
    (acc, l) => {
      const balance = Number(l.outstanding_balance);
      acc.totalBalance += balance;

      if (l.is_paid_in_cadences && l.interest_rate > 0) {
        // Interest is explicitly "per cadence" as per UI/Requirement
        const periodicInterest = balance * (l.interest_rate / 100);

        let monthlyInterest = 0;
        if (l.cadence === "weekly") {
          monthlyInterest = periodicInterest * (52 / 12);
        } else if (l.cadence === "monthly") {
          monthlyInterest = periodicInterest;
        }
        acc.monthlyInterestAccrual += monthlyInterest;
      }
      return acc;
    },
    { totalBalance: 0, monthlyInterestAccrual: 0 },
  );
};

export const calculateTotal = (deployments: Deployment[]): number => {
  return deployments.reduce((sum, d) => sum + Number(d.amount), 0);
};

export const calculateAccountTotal = (accounts: Account[]): number => {
  return LiquidityService.calculateTotal(accounts);
};

export const calculateLiabilityTotal = (liabilities: Liability[]): number => {
  return calculateLiabilityMetrics(liabilities).totalBalance;
};

export const calculateIncomeMetrics = (streams: IncomeStream[]) => {
  const result = streams.reduce(
    (acc, stream) => {
      const monthly = calculateMonthlyInflow(stream);
      acc.total += monthly;
      if (stream.is_recurring) {
```

```
        acc.recurring += monthly;
      } else {
        acc.irregular += monthly;
      }
      acc.concentration[stream.income_type] =
        (acc.concentration[stream.income_type] || 0) + monthly;

      // Group by origin source (income_name) for concentration risk analysis
      acc.sourceTotals[stream.income_name] =
        (acc.sourceTotals[stream.income_name] || 0) + monthly;

      return acc;
    },
    {
      total: 0,
      recurring: 0,
      irregular: 0,
      concentration: {} as Record<string, number>,
      sourceTotals: {} as Record<string, number>,
    },
  );

  const highestSourceSum = Math.max(0, ...Object.values(result.sourceTotals));
  const maxRatio = result.total > 0 ? highestSourceSum / result.total : 0;

  return {
    total: result.total,
    recurring: result.recurring,
    irregular: result.irregular,
    concentration: result.concentration,
    maxRatio,
  };
};

export const calculateGoalMetrics = (
  goals: FinancialGoal[],
  objectives: StrategicObjective[] = [],
) => {
  const goalStats = goals.reduce(
    (acc, goal) => {
      acc.totalTargets += Number(goal.target_amount);
      acc.totalProgress += Number(goal.current_progress);
      if (goal.priority === "critical" && goal.status === "active") {
        acc.criticalCount += 1;
      }
      acc.progressSum += calculateGoalProgressPercentage(goal);
      return acc;
    },
    {
      totalTargets: 0,
      totalProgress: 0,
      criticalCount: 0,
      progressSum: 0,
    },
  );

  return objectives.reduce((acc, obj) => {
    acc.totalTargets += Number(obj.target_amount);
    acc.totalProgress += Number(obj.current_amount);
    if (obj.priority_level === "critical" && obj.status === "active") {
      acc.criticalCount += 1;
    }
    acc.progressSum += calculateObjectiveFundingRatio(obj);
    return acc;
  }, goalStats);
```

```
};

export const calculateAverage = (deployments: Deployment[]): number => {
  if (deployments.length === 0) return 0;
  return calculateTotal(deployments) / deployments.length;
};

export const countPendingVerifications = (
  income: IncomeStream[],
  liabilities: Liability[],
  baseline: OperationalBaseline[] = [],
): number => {
  const today = new Date();
  const currentDayOfWeek = today.getDay();
  const currentDayOfWeekName = today
    .toLocaleString("en-US", { weekday: "long" })
    .toLowerCase();
  const currentDayOfMonth = today.getDate();
  const mappedDayOfWeek = currentDayOfWeek === 0 ? 7 : currentDayOfWeek;

  const pendingIncomes = income.filter((stream) => {
    if (!stream.is_recurring || !stream.execution_day) return false;

    if (stream.last_executed_at) {
      const lastExecuted = new Date(stream.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&
        lastExecuted.getFullYear() === today.getFullYear()
      ) {
        return false;
      }
    }

    if (stream.cadence === "weekly") {
      return mappedDayOfWeek === stream.execution_day;
    }
    if (stream.cadence === "monthly" || stream.cadence === "biweekly") {
      return currentDayOfMonth === stream.execution_day;
    }

    return false;
  });

  const pendingLiabilities = liabilities.filter((liab) => {
    if (!liab.is_paid_in_cadences || !liab.cadence_day_date) return false;

    if (liab.last_executed_at) {
      const lastExecuted = new Date(liab.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&
        lastExecuted.getFullYear() === today.getFullYear()
      ) {
        return false;
      }
    }

    if (liab.cadence === "weekly") {
      return currentDayOfWeekName === liab.cadence_day_date.toLowerCase();
    }
    if (liab.cadence === "monthly") {
      return currentDayOfMonth === parseInt(liab.cadence_day_date, 10);
    }
```

```
    return false;
  });

  const pendingBaselines = baseline.filter((b) => {
    if (!b.is_recurring) return false;

    if (b.last_executed_at) {
      const lastExecuted = new Date(b.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&
        lastExecuted.getFullYear() === today.getFullYear()
      ) {
        return false;
      }
    }

    if (b.cadence === "daily") return true;

    if (b.execution_day) {
      if (b.cadence === "weekly") {
        return mappedDayOfWeek === b.execution_day;
      }
      if (b.cadence === "monthly" || b.cadence === "biweekly") {
        return currentDayOfMonth === b.execution_day;
      }
    }

    return false;
  });
  return (
    pendingIncomes.length + pendingLiabilities.length + pendingBaselines.length
  );
};

/**
 * Calculates burn rate over a specific window (in days).
 * Defaulting to a 30-day window for normalized metrics.
 */
export const calculateBurnRate = (
  deployments: Deployment[],
  windowDays: number = 30,
): number => {
  if (windowDays <= 0) return 0;
  const total = calculateTotal(deployments);
  return total / windowDays;
};

/**
 * Projects runway based on deterministic liquidity and income offset.
 * Correct Formula: Runway = balance / (dailyBurnRate - (monthlyIncome / 30))
 * Note: Returns days. Returns null if adjusted burn is <= 0 (stable state).
 */
export const projectRunway = (
  balance: number,
  dailyBurnRate: number,
  monthlyIncome: number = 0,
  netWorth: number = 0,
): number | null => {
  const adjustedDailyBurn = dailyBurnRate - monthlyIncome / 30;

  // If net worth is negative, the structural foundation is critical regardless of cash flow
  if (netWorth < 0) return 0;

  if (adjustedDailyBurn <= 0) return null;
```

```
  if (balance <= 0) return 0;

  return balance / adjustedDailyBurn;
};

/**
 * Aggregates deployments into categories.
 */
export const getCategoryBreakdown = (
  deployments: Deployment[],
): Record<string, number> => {
  return deployments.reduce(
    (acc, d) => {
      const cat = d.category || "Unclassified";
      acc[cat] = (acc[cat] || 0) + Number(d.amount);
      return acc;
    },
    {} as Record<string, number>,
  );
};

/**
 * Deterministic Strategic Alignment Logic
 */

export function calculateStrategicAlignment(
  objectives: StrategicObjective[],
  deployments: Deployment[],
  liquidity: number,
  incomeTotal: number,
  burnRate: number,
  liabilities: Liability[],
  baseline: OperationalBaseline[] = [],
): StrategicAlignment {
  const fundingRatios: Record<string, number> = {};
  const velocity: Record<
    string,
    "stable" | "accelerating" | "stagnant" | "regressing"
  > = {};
  const objectiveStarvationSignals: string[] = [];
  const strategicAllocationSignals: string[] = [];
  const now = new Date();

  // 1. Funding Ratios & Starvation
  objectives.forEach((obj) => {
    const ratio = calculateObjectiveFundingRatio(obj);
    fundingRatios[obj.id] = ratio;

    const ageInDays = Math.max(
      1,
      (now.getTime() - new Date(obj.created_at).getTime()) /
        (1000 * 60 * 60 * 24),
    );

    // Starvation detection
    if (obj.status === "active" && ratio < 10 && ageInDays > 90) {
      objectiveStarvationSignals.push(
        `${obj.objective_name} objective has remained below 10% funded for ${Math.floor(ageInDays)} days.`,
      );
    }

    // Velocity (Simplified deterministic approach based on age)
    const dailyRate = obj.current_amount / ageInDays;
    if (dailyRate === 0) {
      velocity[obj.id] = "stagnant";
```

```
  } else if (ratio >= 100) {
    velocity[obj.id] = "stable"; // Completed
  } else {
    velocity[obj.id] = "stable"; // Default for now as we don't have historical delta
  }
});

// 2. Strategic Allocation Awareness
const categories = getCategoryBreakdown(deployments);
const leakage = categories["Leakage"] || 0;
const assets = categories["Asset"] || 0;
const hasActiveAccumulation = objectives.some(
  (o) =>
    o.status === "active" &&
    (o.objective_type === "liquidity" ||
      o.objective_type === "emergency_reserve"),
);

if (hasActiveAccumulation && leakage > assets) {
  strategicAllocationSignals.push(
    "Leakage deployments exceeded Asset deployments during an active liquidity accumulation objective.",
  );
}

// 3. Liquidity Sufficiency
const criticalObjectives = objectives.filter(
  (o) => o.priority_level === "critical" && o.status === "active",
);
const totalCriticalTarget = criticalObjectives.reduce(
  (sum, o) => sum + (o.target_amount - o.current_amount),
  0,
);
const isSufficient = liquidity >= totalCriticalTarget;
const shortfall = Math.max(0, totalCriticalTarget - liquidity);

const liquiditySufficiency = {
  isSufficient,
  message: isSufficient
    ? "Current liquidity structure sufficiently supports short-term obligations."
    : `Emergency reserve target exceeds current liquidity capacity by ${shortfall.toLocaleString()} KSh.`,
  shortfall,
};

// 4. Alignment Pressure (Weighted Scoring 0-100)
let pressureScore = 0;

// Weight: Leakage vs Assets (up to 25 points)
if (leakage > assets) pressureScore += 25;
else if (leakage > 0) pressureScore += 10;

// Weight: Liquidity Sufficiency (up to 30 points)
if (!isSufficient) pressureScore += 30;

// Weight: Starvation (up to 20 points)
if (objectiveStarvationSignals.length > 0) pressureScore += 20;

// Weight: High Liabilities (up to 25 points)
const totalLiabilities = liabilities.reduce(
  (sum, l) => sum + Number(l.outstanding_balance),
  0,
);
if (totalLiabilities > liquidity * 2) pressureScore += 25;

return {
  fundingRatios,
```

```
    velocity,
    alignmentPressure: Math.min(100, pressureScore),
    liquiditySufficiency,
    objectiveStarvationSignals,
    strategicAllocationSignals,
  };
}

/**
 * Main entry point for generating a full analytics context.
 */
export const generateSummary = (
  deployments: Deployment[],
  liquidity: number = 0,
  accounts: Account[] = [],
  liabilities: Liability[] = [],
  incomeStreams: IncomeStream[] = [],
  goals: FinancialGoal[] = [],
  objectives: StrategicObjective[] = [],
  baseline: OperationalBaseline[] = [],
): AnalyticsSummary => {
  const total = calculateTotal(deployments);
  const baselineMetrics = calculateBaselineMetrics(baseline);
  const liabilityMetrics = calculateLiabilityMetrics(liabilities);

  // Total daily burn = (Deployment Burn) + (Baseline Burn / 30) + (Liability Interest Burn / 30)
  const deploymentBurn = calculateBurnRate(deployments);
  const baselineDailyBurn = baselineMetrics.monthlyBurn / 30;
  const liabilityInterestDailyBurn =
    liabilityMetrics.monthlyInterestAccrual / 30;
  const burnRate =
    deploymentBurn + baselineDailyBurn + liabilityInterestDailyBurn;

  const totalAssets = calculateAccountTotal(accounts);
  const totalLiabilities = liabilityMetrics.totalBalance;
  const netWorth = totalAssets - totalLiabilities;

  const income = calculateIncomeMetrics(incomeStreams);
  const goalMetrics = calculateGoalMetrics(goals, objectives);

  const strategicAlignment = calculateStrategicAlignment(
    objectives,
    deployments,
    liquidity,
    income.total,
    burnRate,
    liabilities,
    baseline,
  );

  const totalIntentionsCount = goals.length + objectives.length;

  return {
    totalDeployed: total,
    averageDeployment: calculateAverage(deployments),
    dailyBurnRate: burnRate,
    runwayDays: projectRunway(liquidity, burnRate, income.total, netWorth),
    categoryBreakdown: getCategoryBreakdown(deployments),
    deploymentCount: deployments.length,
    metadataQuality: summarizeMetadataQuality(deployments),
    // Liability System v1
    totalLiabilities,
    totalAssets,
    netWorth,
    liquidity,
```

```
    // Income Engine v1
    totalMonthlyIncome: income.total,
    recurringIncome: income.recurring,
    irregularIncome: income.irregular,
    incomeConcentration: income.concentration,
    maxIncomeConcentrationRatio: income.maxRatio,
    adjustedDailyBurn: Math.max(0, burnRate - income.total / 30),
    // Goal System v1
    totalStrategicTargets: goalMetrics.totalTargets,
    totalCurrentProgress: goalMetrics.totalProgress,
    averageGoalProgress:
      totalIntentionsCount > 0
        ? goalMetrics.progressSum / totalIntentionsCount
        : 0,
    criticalGoalCount: goalMetrics.criticalCount,
    fundingGap: Math.max(
      0,
      goalMetrics.totalTargets - goalMetrics.totalProgress,
    ),
    // Strategic Objectives v1
    strategicAlignment,
    // Operational Baseline v1
    totalStructuralMonthlyBurn: baselineMetrics.monthlyBurn,
    totalSystemicMonthlyAllocation: baselineMetrics.monthlyAllocation,
    pendingVerificationCount: countPendingVerifications(
      incomeStreams,
      liabilities,
      baseline,
    ),
  };
};
```

FILE: src\lib\analytics\index.ts

```
export * from "./types";
export * from "./engine";
```

FILE: src\lib\analytics\strategic.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { calculateStrategicAlignment } from "./engine";
import type { Deployment, StrategicObjective, Liability } from "./types";

test("strategic alignment funding ratios are deterministic", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Emergency Fund",
      objective_type: "emergency_reserve",
      target_amount: 1000,
      current_amount: 500,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];
```

```
  const alignment = calculateStrategicAlignment(objectives, [], 0, 0, 0, []);
  assert.strictEqual(alignment.fundingRatios["obj1"], 50);
});

test("alignment pressure scoring reflects structural conflict", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Liquidity",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 0,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];

  const deployments: Deployment[] = [
    {
      id: "d1",
      amount: 500,
      title: "Bad Spending",
      category: "Leakage",
      created_at: new Date().toISOString(),
    },
  ];

  const liabilities: Liability[] = [
    {
      id: "l1",
      user_id: "u1",
      liability_name: "Debt",
      liability_type: "personal_loan",
      outstanding_balance: 10000,
      created_at: "",
      updated_at: "",
      interest_rate: 0,
      due_date: null,
      currency: "KSh",
      is_paid_in_cadences: false,
      cadence: null,
      cadence_day_date: null,
      cadence_amount: null,
    },
  ];

  // Scenario: High Leakage, No Assets, Insufficient Liquidity, High Liabilities
  const alignment = calculateStrategicAlignment(
    objectives,
    deployments,
    1000,
    0,
    0,
    liabilities,
  );

  // Leakage > Assets (+25)
  // !isSufficient (+30) - totalCriticalTarget is 1000, liquidity is 1000. Wait, 1000 >= 1000 is true.
  // totalLiabilities (10000) > liquidity * 2 (2000) (+25)
  // Score should be at least 50.
```

```
  assert.ok(alignment.alignmentPressure >= 50);
});

test("liquidity sufficiency detects shortfalls", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Reserve",
      objective_type: "emergency_reserve",
      target_amount: 5000,
      current_amount: 1000,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];

  const alignment = calculateStrategicAlignment(objectives, [], 2000, 0, 0, []);

  assert.strictEqual(alignment.liquiditySufficiency.isSufficient, false);
  assert.strictEqual(alignment.liquiditySufficiency.shortfall, 2000); // 4000 remaining target - 2000 liquidity
});

test("objective starvation detection works for old stagnant goals", () => {
  const sixMonthsAgo = new Date();
  sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6);

  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Starved Goal",
      objective_type: "investment",
      target_amount: 10000,
      current_amount: 100, // 1%
      target_date: null,
      priority_level: "high",
      status: "active",
      created_at: sixMonthsAgo.toISOString(),
      updated_at: sixMonthsAgo.toISOString(),
    },
  ];

  const alignment = calculateStrategicAlignment(objectives, [], 0, 0, 0, []);

  assert.ok(alignment.objectiveStarvationSignals.length > 0);
  assert.ok(alignment.objectiveStarvationSignals[0].includes("Starved Goal"));
});

test("strategic allocation awareness detects leakage conflict", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Build Cash",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 0,
      target_date: null,
      priority_level: "high",
      status: "active",
      created_at: new Date().toISOString(),
```

```
      updated_at: new Date().toISOString(),
    },
  ];

  const deployments: Deployment[] = [
    {
      id: "d1",
      amount: 1000,
      title: "Leakage",
      category: "Leakage",
      created_at: new Date().toISOString(),
    },
    {
      id: "d2",
      amount: 500,
      title: "Asset",
      category: "Asset",
      created_at: new Date().toISOString(),
    },
  ];

  const alignment = calculateStrategicAlignment(
    objectives,
    deployments,
    0,
    0,
    0,
    [],
  );

  assert.ok(alignment.strategicAllocationSignals.length > 0);
  assert.ok(
    alignment.strategicAllocationSignals[0].includes(
      "Leakage deployments exceeded Asset deployments",
    ),
  );
});
```

FILE: src\lib\analytics\types.ts

```
import { MetadataQualitySummary } from "../finance/metadataQuality";
import { DeploymentAdvancedContext } from "../finance/deploymentContext";
import { Account } from "../finance/accounts";
import { Liability } from "../finance/liabilities";
import { IncomeStream } from "../finance/income";
import { FinancialGoal } from "../finance/goals";
import { StrategicObjective } from "../finance/objectives";

export type { Account, AccountType } from "../finance/accounts";
export type { Liability, LiabilityType } from "../finance/liabilities";
export type { IncomeStream, IncomeType } from "../finance/income";
export type {
  FinancialGoal,
  GoalType,
  GoalPriority,
  GoalStatus,
} from "../finance/goals";
export type {
  StrategicObjective,
  ObjectiveType,
  ObjectivePriority,
  ObjectiveStatus,
} from "../finance/objectives";
```

```
export type BaselineCadence =
  | "daily"
  | "weekly"
  | "biweekly"
  | "monthly"
  | "quarterly"
  | "yearly";

export type BaselineType = "expense" | "allocation";

export interface OperationalBaseline {
  id: string;
  user_id: string;
  title: string;
  amount: number;
  category: string;
  cadence: BaselineCadence;
  execution_day?: number | null;
  last_executed_at?: string | null;
  is_recurring: boolean;
  baseline_type: BaselineType;
  is_active: boolean;
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export interface Deployment {
  id: string;
  amount: number;
  created_at: string;
  category?: string | null;
  title: string;
  advanced_context?: DeploymentAdvancedContext | null;
  account_id?: string | null;
  deleted_at?: string | null;
}

export interface StrategicAlignment {
  fundingRatios: Record<string, number>; // objective_id -> ratio (0-100)
  alignmentPressure: number; // 0-100 deterministic score
  liquiditySufficiency: {
    isSufficient: boolean;
    message: string;
    shortfall: number;
  };
  objectiveStarvationSignals: string[];
  strategicAllocationSignals: string[];
  velocity: Record<
    string,
    "stable" | "accelerating" | "stagnant" | "regressing"
  >;
}

export interface AnalyticsSummary {
  totalDeployed: number;
  averageDeployment: number;
  dailyBurnRate: number;
  runwayDays: number | null;
  categoryBreakdown: Record<string, number>;
  deploymentCount: number;
  metadataQuality: MetadataQualitySummary;
  // Liability System v1
```

```
  totalLiabilities: number;
  netWorth: number;
  totalAssets: number;
  liquidity: number;
  // Income Engine v1
  totalMonthlyIncome: number;
  recurringIncome: number;
  irregularIncome: number;
  incomeConcentration: Record<string, number>;
  maxIncomeConcentrationRatio: number;
  adjustedDailyBurn: number;
  // Goal System v1
  totalStrategicTargets: number;
  totalCurrentProgress: number;
  averageGoalProgress: number;
  criticalGoalCount: number;
  fundingGap: number;
  // Strategic Objectives v1
  strategicAlignment: StrategicAlignment;
  // Operational Baseline v1
  totalStructuralMonthlyBurn: number;
  totalSystemicMonthlyAllocation: number;
  pendingVerificationCount: number;
}
```

FILE: src\lib\context\behavioralFlags.ts

```
import { TrendState } from "./contextTypes";

/**
 * Identifies discrete behavioral signals for AI interpretation.
 * Objective: High signal, low token usage.
 */

export const generateBehavioralFlags = (params: {
  burnTrend: TrendState;
  volatility: number;
  concentration: number;
  runwayDays: number | null;
  dominantCategory: string;
}): string[] => {
  const flags: string[] = [];

  // 1. Burn Signals
  if (params.burnTrend === "increasing") flags.push("accelerating_burn");
  if (params.burnTrend === "decreasing") flags.push("burn_optimization_detected");

  // 2. Consistency Signals
  if (params.volatility > 0.6) flags.push("erratic_spending_pattern");
  if (params.volatility < 0.2 && params.volatility > 0) flags.push("high_operational_discipline");

  // 3. Allocation Signals
  if (params.concentration > 0.7) flags.push(`heavy_concentration:${params.dominantCategory.toLowerCase()}`);
  if (params.concentration < 0.3) flags.push("fragmented_capital_allocation");

  // 4. Criticality Signals
  if (params.runwayDays !== null && params.runwayDays < 30) flags.push("critical_runway_warning");
  if (params.runwayDays !== null && params.runwayDays > 180) flags.push("extended_operational_security");

  return flags;
};

/**
```

```
 * Analyzes category concentration.
 * Returns a score between 0 (even spread) and 1 (all in one category).
 */
export const calculateConcentrationScore = (distribution: Record<string, number>): number => {
  const values = Object.values(distribution);
  if (values.length === 0) return 0;
  if (values.length === 1) return 1;

  // Use Herfindahl-Hirschman Index (HHI) approach for concentration
  // Sum of squares of shares
  const hhi = values.reduce((sum, val) => sum + Math.pow(val, 2), 0);
  return hhi;
};
```

FILE: src\lib\context\buildBehavioralContext.ts

```
import { BehavioralContext, ContextInput } from "./contextTypes";

import {
  calculateTrendState,
  calculateVolatility,
  inferDiscipline,
} from "./trendAnalysis";
import {
  generateBehavioralFlags,
  calculateConcentrationScore,
} from "./behavioralFlags";
import { isValidCategory } from "../finance/taxonomy";

/**
 * Axiom Behavioral Context Builder
 * Transforms analytics signals into a stable behavioral state for Kairos.
 */
export const buildBehavioralContext = (
  input: ContextInput,
  deploymentAmounts: number[],
): BehavioralContext => {
  const { currentAnalytics, historicalMetrics } = input;

  // 1. Calculate Trends
  const burnTrend = calculateTrendState(
    currentAnalytics.dailyBurnRate,
    historicalMetrics?.previousBurnRate || 0,
  );

  const runwayDelta =
    currentAnalytics.runwayDays !== null &&
    historicalMetrics?.previousRunwayDays !== undefined
      ? currentAnalytics.runwayDays -
        (historicalMetrics.previousRunwayDays || 0)
      : 0;

  // 2. Allocation Analysis
  // 2. Allocation Analysis
  const total = currentAnalytics.totalDeployed || 1;
  const distribution: Record<string, number> = {};
  let dominantCategory = "None";
  let maxAmt = -1;

  let unclassifiedTotal = 0;

  Object.entries(currentAnalytics.categoryBreakdown).forEach(([cat, amt]) => {
    if (!isValidCategory(cat)) {
```

```
    unclassifiedTotal += amt;
  } else {
    distribution[cat] = amt / total;
    if (amt > maxAmt) {
      maxAmt = amt;
      dominantCategory = cat;
    }
  }
});

// Add the merged 'Unclassified' group
if (unclassifiedTotal > 0) {
  distribution["Unclassified"] = unclassifiedTotal / total;
  if (unclassifiedTotal > maxAmt) {
    dominantCategory = "Unclassified";
  }
}

const concentrationScore = calculateConcentrationScore(distribution);
const volatility = calculateVolatility(deploymentAmounts);

// 3. Efficiency Calculation (Heuristic)
// Starts at 100, penalized by volatility, accelerating burn, and low runway.
let efficiency = 100;
efficiency -= volatility * 60; // Much more sensitive to erratic spending
if (burnTrend === "increasing") efficiency -= 15;
if (currentAnalytics.runwayDays && currentAnalytics.runwayDays < 90)
  efficiency -= 40; // Aggressive penalty for low runway

// 4. Status Determination

// 4. Status Determination
let runwayStatus: "healthy" | "concerning" | "critical" = "healthy";
if (currentAnalytics.runwayDays !== null) {
  if (currentAnalytics.runwayDays < 30) runwayStatus = "critical";
  else if (currentAnalytics.runwayDays < 90) runwayStatus = "concerning";
}

return {
  generatedAt: new Date().toISOString(),
  deploymentCount: currentAnalytics.deploymentCount,
  capitalEfficiencyScore: Math.round(Math.max(efficiency, 0)),

  netWorth: currentAnalytics.netWorth,
  liquidity: currentAnalytics.liquidity,
  totalMonthlyIncome: currentAnalytics.totalMonthlyIncome,
  maxIncomeConcentrationRatio: currentAnalytics.maxIncomeConcentrationRatio,
  pendingVerificationCount: currentAnalytics.pendingVerificationCount,

  strategicAlignment: currentAnalytics.strategicAlignment,

  burnRate: {
    current: currentAnalytics.dailyBurnRate,
    trend: burnTrend,
    monthlyProjection: currentAnalytics.dailyBurnRate * 30,
  },

  runway: {
    currentDays: currentAnalytics.runwayDays,
    deltaDays: runwayDelta,
    status: runwayStatus,
  },

  allocation: {
    dominantCategory,
```

```
      categoryDistribution: distribution,
      concentrationScore,
    },

    volatilityScore: volatility,

    metadataQuality: currentAnalytics.metadataQuality,

    disciplineTrend: inferDiscipline(burnTrend, runwayDelta),

    behavioralFlags: generateBehavioralFlags({
      burnTrend,
      volatility,
      concentration: concentrationScore,
      runwayDays: currentAnalytics.runwayDays,
      dominantCategory,
    }),
  };
};

/**
 * Context Compression
 * Strips metadata to optimize for LLM token usage while preserving signal.
 */
export const compressContextForAI = (context: BehavioralContext): string => {
  return JSON.stringify({
    score: context.capitalEfficiencyScore,
    burn: `${context.burnRate.current}/day (${context.burnRate.trend})`,
    runway: `${context.runway.currentDays}d (${context.runway.status})`,
    allocation: `${context.allocation.dominantCategory} (conc: ${context.allocation.concentrationScore.toFixed(2)})`,
    flags: context.behavioralFlags.join(","),
    discipline: context.disciplineTrend,
    metadata: `${context.metadataQuality.lowQualityCount}/${context.deploymentCount} low-quality labels`,
  });
};
```

FILE: src\lib\context\contextTypes.ts

```
import { StrategicAlignment } from "../analytics/types";
import { MetadataQualitySummary } from "../finance/metadataQuality";

export type TrendState = "increasing" | "stable" | "decreasing";
export type DisciplineState = "improving" | "stable" | "declining";

export interface AllocationState {
  dominantCategory: string;
  categoryDistribution: Record<string, number>; // Percentage based (0-1)
  concentrationScore: number; // 0 to 1, where 1 is total concentration in one category
}

export interface BehavioralContext {
  generatedAt: string;
  deploymentCount: number;

  // High-level aggregate metric (0-100)
  capitalEfficiencyScore: number;

  netWorth: number;
  liquidity: number;
  totalMonthlyIncome: number;
  maxIncomeConcentrationRatio: number;
  pendingVerificationCount: number;
```

```
  strategicAlignment: StrategicAlignment;

  burnRate: {
    current: number;
    trend: TrendState;
    monthlyProjection: number;
  };

  runway: {
    currentDays: number | null;
    deltaDays: number; // Change from previous context
    status: "healthy" | "concerning" | "critical";
  };

  allocation: AllocationState;

  // Discrete behavioral markers
  behavioralFlags: string[];

  // Measure of spending consistency
  volatilityScore: number; // 0 to 1, higher is more erratic

  metadataQuality: MetadataQualitySummary;

  disciplineTrend: DisciplineState;
}

/**
 * Input required for the context builder.
 * Represents the current state vs a previous snapshot.
 */
export interface ContextInput {
  currentAnalytics: {
    totalDeployed: number;
    dailyBurnRate: number;
    runwayDays: number | null;
    categoryBreakdown: Record<string, number>;
    deploymentCount: number;
    metadataQuality: MetadataQualitySummary;
    netWorth: number;
    liquidity: number;
    totalMonthlyIncome: number;
    maxIncomeConcentrationRatio: number;
    pendingVerificationCount: number;
    strategicAlignment: StrategicAlignment;
  };
  historicalMetrics?: {
    previousBurnRate: number;
    previousRunwayDays: number | null;
    previousTotalDeployed: number;
  };
}
```

FILE: src\lib\context\trendAnalysis.ts

```
import { TrendState, DisciplineState } from "./contextTypes";

/**
 * Determines the trend direction between two values.
 * Threshold (0.05) prevents noise from triggering trend shifts.
 */
export const calculateTrendState = (
  current: number,
```

```
  previous: number,
  threshold: number = 0.05
): TrendState => {
  if (!previous || previous === 0) return "stable";

  const change = (current - previous) / previous;

  if (change > threshold) return "increasing";
  if (change < -threshold) return "decreasing";
  return "stable";
};

/**
 * Calculates a volatility score (0-1) based on the variance of deployment sizes.
 * High variance = High volatility (erratic spending).
 */
export const calculateVolatility = (amounts: number[]): number => {
  if (amounts.length < 2) return 0;

  const mean = amounts.reduce((a, b) => a + b, 0) / amounts.length;
  const variance = amounts.reduce((a, b) => a + Math.pow(b - mean, 2), 0) / amounts.length;
  const stdDev = Math.sqrt(variance);

  // Normalize volatility relative to the mean.
  // A standard deviation equal to the mean results in a score of 0.5.
  const score = stdDev / (mean + stdDev);
  return Math.min(Math.max(score, 0), 1);
};

/**
 * Infers discipline trend by observing burn rate vs total deployment.
 */
export const inferDiscipline = (
  burnTrend: TrendState,
  runwayDelta: number
): DisciplineState => {
  if (burnTrend === "decreasing" && runwayDelta > 0) return "improving";
  if (burnTrend === "increasing" && runwayDelta < 0) return "declining";
  return "stable";
};
```

FILE: src\lib\dashboard\types.ts

```
import {
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
  AnalyticsSummary,
} from "../analytics/types";
import { KairosInsight } from "../ai/kairos";

export interface DashboardSnapshot {
  authenticated: boolean;
  deployments: Deployment[];
  accounts: Account[];
  liabilities: Liability[];
  incomeStreams: IncomeStream[];
  goals: FinancialGoal[];
  objectives: StrategicObjective[];
```

```
  baseline: OperationalBaseline[];
  analytics: AnalyticsSummary | null;
  liquidity: number;
  kairosInsight: KairosInsight | null;
}
```

FILE: src\lib\db\accounts.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateAccount, type AccountType } from "../finance/accounts";

export async function getAccounts(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("accounts")
    .select("*")
    .is("deleted_at", null)
    .order("account_name", { ascending: true });

  if (error) throw error;
  return data;
}

export async function createAccount(
  supabase: SupabaseClient,
  userId: string,
  data: {
    account_name: string;
    account_type: string;
    current_balance: number;
    institution?: string;
    currency?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateAccount({
    account_name: data.account_name,
    account_type: data.account_type,
    current_balance: data.current_balance,
  });

  const { data: account, error } = await supabase
    .from("accounts")
    .insert({
      user_id: userId,
      account_name: validated.account_name,
      account_type: validated.account_type,
      current_balance: validated.current_balance,
      institution: data.institution,
      currency: data.currency || "KSh",
    })
    .select()
    .single();

  if (error) throw error;
  return account;
}

export async function updateAccount(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    account_name?: string;
```

```
    account_type?: string;
    current_balance?: number;
    institution?: string;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.account_name !== undefined ||
    updates.account_type !== undefined ||
    updates.current_balance !== undefined
  ) {
    // Basic validation for what's provided
    if (
      updates.account_name !== undefined &&
      updates.account_name.trim().length === 0
    ) {
      throw new Error("Account name is required.");
    }
    // Note: We don't use full validateAccount here to allow partial updates
  }

  const { data, error } = await supabase
    .from("accounts")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteAccount(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("accounts")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src\lib\db\audit.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";

export type AuditRecordType = "deployment" | "income" | "liability";

export interface AuditRecord {
  id: string;
  type: AuditRecordType;
  name: string;
  amount: number;
  cadence?: string;
  deleted_at: string;
```

```
  original_created_at: string;
}

/**
 * Retrieves all soft-deleted records from the ledger tables (deployments, income_streams, liabilities).
 * Returns a unified, chronologically sorted timeline of deletions.
 */
export async function getDeletedLedgerRecords(
  supabase: SupabaseClient,
  userId: string,
): Promise<AuditRecord[]> {
  const [deployments, incomeStreams, liabilities] = await Promise.all([
    supabase
      .from("deployments")
      .select("id, title, amount, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
    supabase
      .from("income_streams")
      .select("id, income_name, amount, cadence, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
    supabase
      .from("liabilities")
      .select("id, liability_name, outstanding_balance, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
  ]);

  if (deployments.error) throw deployments.error;
  if (incomeStreams.error) throw incomeStreams.error;
  if (liabilities.error) throw liabilities.error;

  const normalizedDeployments: AuditRecord[] = (deployments.data || []).map(
    (d) => ({
      id: d.id,
      type: "deployment",
      name: d.title,
      amount: d.amount,
      deleted_at: d.deleted_at!,
      original_created_at: d.created_at,
    }),
  );

  const normalizedIncome: AuditRecord[] = (incomeStreams.data || []).map(
    (i) => ({
      id: i.id,
      type: "income",
      name: i.income_name,
      amount: i.amount,
      cadence: i.cadence,
      deleted_at: i.deleted_at!,
      original_created_at: i.created_at,
    }),
  );

  const normalizedLiabilities: AuditRecord[] = (liabilities.data || []).map(
    (l) => ({
      id: l.id,
      type: "liability",
      name: l.liability_name,
      amount: l.outstanding_balance,
      deleted_at: l.deleted_at!,
      original_created_at: l.created_at,
    }),
```

```
  );

  return [
    ...normalizedDeployments,
    ...normalizedIncome,
    ...normalizedLiabilities,
  ].sort(
    (a, b) =>
      new Date(b.deleted_at).getTime() - new Date(a.deleted_at).getTime(),
  );
}
```

FILE: src\lib\db\baseline.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { OperationalBaseline } from "../analytics/types";

export async function getOperationalBaseline(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("operational_baseline")
    .select("*")
    .is("deleted_at", null)
    .order("created_at", { ascending: true });

  if (error) throw error;
  return data as OperationalBaseline[];
}

export async function createOperationalBaseline(
  supabase: SupabaseClient,
  userId: string,
  data: Omit<
    OperationalBaseline,
    "id" | "user_id" | "created_at" | "updated_at" | "deleted_at"
  >,
) {
  const { data: baseline, error } = await supabase
    .from("operational_baseline")
    .insert({
      user_id: userId,
      ...data,
    })
    .select()
    .single();

  if (error) throw error;
  return baseline as OperationalBaseline;
}

export async function updateOperationalBaseline(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: Partial<
    Omit<
      OperationalBaseline,
      "id" | "user_id" | "created_at" | "updated_at" | "deleted_at"
    >
  >,
) {
  const { data: baseline, error } = await supabase
    .from("operational_baseline")
    .update(updates)
```

```
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return baseline as OperationalBaseline;
}

export async function deleteOperationalBaseline(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("operational_baseline")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src\lib\db\deployments.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateDeployment } from "../finance/validateDeployment";
import {
  hasDeploymentAdvancedContext,
  type DeploymentAdvancedContextInput,
} from "../finance/deploymentContext";

function isMissingAdvancedContextColumn(error: { message?: string }) {
  return Boolean(error.message?.toLowerCase().includes("advanced_context"));
}

export async function getDeployments(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("deployments")
    .select("*")
    .is("deleted_at", null)
    .order("created_at", { ascending: false });

  if (error) throw error;
  return data;
}

/**
 * Creates a new deployment record.
 * Guaranteed to pass through the Taxonomy Enforcement Gate.
 */
export async function createDeployment(
  supabase: SupabaseClient,
  title: string,
  amount: number,
  userId: string,
  category: string,
  impactScore: number = 0,
  advancedContext: DeploymentAdvancedContextInput = {},
  accountId?: string,
) {
  // 1. Pass through Taxonomy Enforcement Gate
```

```
  const validated = validateDeployment({
    title,
    amount,
    category,
    impactScore,
    advancedContext,
  });

  // 2. Database Write (Verified Truth)
  const deploymentRecord = {
    title: validated.title,
    amount: validated.amount,
    user_id: userId,
    category: validated.category,
    impact_score: validated.impactScore,
    account_id: accountId || null,
    ...(hasDeploymentAdvancedContext(validated.advancedContext)
      ? { advanced_context: validated.advancedContext }
      : {}),
  };

  const { data, error } = await supabase
    .from("deployments")
    .insert(deploymentRecord);

  if (error) {
    if (
      hasDeploymentAdvancedContext(validated.advancedContext) &&
      isMissingAdvancedContextColumn(error)
    ) {
      throw new Error(
        "Advanced Context schema unavailable. Apply supabase-setup.sql before saving advanced deployment metadata.",
      );
    }

    throw error;
  }
  return data;
}

/**
 * Updates an existing deployment record.
 * Enforces partial taxonomy validation for updated fields.
 */
export async function updateDeployment(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    title?: string;
    amount?: number;
    category?: string;
    impact_score?: number;
    advancedContext?: DeploymentAdvancedContextInput;
    accountId?: string;
  },
) {
  // 1. Enforcement for provided fields
  const dbUpdates: Record<string, unknown> = {};

  if (updates.accountId !== undefined) {
    dbUpdates.account_id = updates.accountId || null;
  }

  const shouldValidate =
```

```
    updates.title !== undefined ||
    updates.amount !== undefined ||
    updates.category !== undefined ||
    updates.advancedContext !== undefined ||
    updates.impact_score !== undefined;

  if (shouldValidate) {
    // We simulate a full input to reuse the Gate logic,
    // but only use the validated fields for the update.
    const validated = validateDeployment({
      title: updates.title ?? "Temporary Title",
      amount: updates.amount ?? 1,
      category: updates.category ?? "Leakage", // Explicit default for gate reuse
      impactScore: updates.impact_score,
      advancedContext: updates.advancedContext,
    });

    if (updates.title !== undefined) dbUpdates.title = validated.title;
    if (updates.amount !== undefined) dbUpdates.amount = validated.amount;
    if (updates.category !== undefined) dbUpdates.category = validated.category;
    if (updates.impact_score !== undefined)
      dbUpdates.impact_score = validated.impactScore;
    if (updates.advancedContext !== undefined)
      dbUpdates.advanced_context = validated.advancedContext;
  }

  // 2. Database Update
  const { data, error } = await supabase
    .from("deployments")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId);

  if (error) {
    if (
      updates.advancedContext !== undefined &&
      isMissingAdvancedContextColumn(error)
    ) {
      throw new Error(
        "Advanced Context schema unavailable. Apply supabase-setup.sql before saving advanced deployment metadata.",
      );
    }

    throw error;
  }
  return data;
}

export async function deleteDeployment(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("deployments")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src\lib\db\goals.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateGoal } from "../finance/goals";

export async function getGoals(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("financial_goals")
    .select("*")
    .is("deleted_at", null)
    .order("priority", { ascending: false });

  if (error) throw error;
  return data;
}

export async function createGoal(
  supabase: SupabaseClient,
  userId: string,
  data: {
    goal_name: string;
    goal_type: string;
    target_amount: number;
    current_progress?: number;
    target_date?: string | null;
    priority: string;
    status: string;
    notes?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateGoal({
    goal_name: data.goal_name,
    goal_type: data.goal_type,
    target_amount: data.target_amount,
    current_progress: data.current_progress,
    priority: data.priority,
    status: data.status,
    target_date: data.target_date,
    notes: data.notes,
  });

  const { data: goal, error } = await supabase
    .from("financial_goals")
    .insert({
      user_id: userId,
      goal_name: validated.goal_name,
      goal_type: validated.goal_type,
      target_amount: validated.target_amount,
      current_progress: validated.current_progress,
      target_date: validated.target_date,
      priority: validated.priority,
      status: validated.status,
      notes: validated.notes,
    })
    .select()
    .single();

  if (error) throw error;
  return goal;
}

export async function updateGoal(
  supabase: SupabaseClient,
```

```
  id: string,
  userId: string,
  updates: {
    goal_name?: string;
    goal_type?: string;
    target_amount?: number;
    current_progress?: number;
    target_date?: string | null;
    priority?: string;
    status?: string;
    notes?: string | null;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.goal_name !== undefined &&
    updates.goal_name.trim().length === 0
  ) {
    throw new Error("Goal name is required.");
  }

  const { data, error } = await supabase
    .from("financial_goals")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteGoal(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("financial_goals")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src\lib\db\income.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateIncomeStream } from "../finance/income";

export async function getIncomeStreams(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("income_streams")
    .select("*")
    .is("deleted_at", null)
    .order("income_name", { ascending: true });

  if (error) throw error;
```

```
  return data;
}

export async function createIncomeStream(
  supabase: SupabaseClient,
  userId: string,
  data: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
    execution_day?: number | null;
    is_recurring?: boolean;
    source?: string;
    currency?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  },
) {
  const validated = validateIncomeStream({
    income_name: data.income_name,
    income_type: data.income_type,
    amount: data.amount,
    cadence: data.cadence,
    execution_day: data.execution_day,
    is_recurring: data.is_recurring,
  });

  const { data: stream, error } = await supabase
    .from("income_streams")
    .insert({
      user_id: userId,
      income_name: validated.income_name,
      income_type: validated.income_type,
      amount: validated.amount,
      cadence: validated.cadence,
      execution_day: validated.execution_day,
      is_recurring: validated.is_recurring,
      source: data.source,
      currency: data.currency || "KSh",
      start_date: data.start_date || new Date().toISOString().split("T")[0],
      end_date: data.end_date,
    })
    .select()
    .single();

  if (error) throw error;
  return stream;
}

export async function createIncomeStreams(
  supabase: SupabaseClient,
  userId: string,
  inputs: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
    execution_day?: number | null;
    is_recurring?: boolean;
    source?: string;
    currency?: string;
    start_date?: string;
    end_date?: string | null;
```

```
    deleted_at?: string | null;
  }[],
) {
  const validatedInputs = inputs.map((data) => {
    const validated = validateIncomeStream({
      income_name: data.income_name,
      income_type: data.income_type,
      amount: data.amount,
      cadence: data.cadence,
      execution_day: data.execution_day,
      is_recurring: data.is_recurring,
    });

    return {
      user_id: userId,
      income_name: validated.income_name,
      income_type: validated.income_type,
      amount: validated.amount,
      cadence: validated.cadence,
      execution_day: validated.execution_day,
      is_recurring: validated.is_recurring,
      source: data.source,
      currency: data.currency || "KSh",
      start_date: data.start_date || new Date().toISOString().split("T")[0],
      end_date: data.end_date,
    };
  });

  const { data: streams, error } = await supabase
    .from("income_streams")
    .insert(validatedInputs)
    .select();

  if (error) throw error;
  return streams;
}

export async function updateIncomeStream(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    income_name?: string;
    income_type?: string;
    amount?: number;
    cadence?: string;
    execution_day?: number | null;
    last_executed_at?: string | null;
    is_recurring?: boolean;
    source?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.income_name !== undefined &&
    updates.income_name.trim().length === 0
  ) {
    throw new Error("Income name is required.");
  }

  const { data, error } = await supabase
```

```
    .from("income_streams")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteIncomeStream(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("income_streams")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src\lib\db\insights.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import type { KairosInsight } from "@/lib/ai/kairos";

export async function saveInsight(
  supabase: SupabaseClient,
  insight: KairosInsight,
  userId: string,
) {
  const { data, error } = await supabase.from("kairos_insights").insert({
    user_id: userId,
    type: insight.type,
    category: insight.category,
    confidence: 1.0, // Deterministic logic has absolute confidence
    message: insight.message,
    metadata: {
      related_ids: [],
      metadata_quality: insight.metadataQuality || null,
      severity: insight.severity,
      supporting_signals: insight.supportingSignals,
      runway: insight.runway,
      capital_efficiency: insight.capitalEfficiency,
      is_silent: insight.isSilent,
      timestamp: insight.timestamp,
      source: "server_orchestrator_v5e",
    },
  });

  if (error) {
    console.error("Error saving insight:", error);
    throw error;
  }
  return data;
}

export async function getInsights(supabase: SupabaseClient, limit: number = 5) {
```

```
  const { data, error } = await supabase
    .from("kairos_insights")
    .select("*")
    .order("created_at", { ascending: false })
    .limit(limit);

  if (error) {
    console.error("Error fetching insights:", error);
    throw error;
  }
  return data;
}
```

FILE: src\lib\db\liabilities.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateLiability } from "../finance/liabilities";

export async function getLiabilities(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("liabilities")
    .select("*")
    .is("deleted_at", null)
    .order("liability_name", { ascending: true });

  if (error) throw error;
  return data;
}

export async function createLiability(
  supabase: SupabaseClient,
  userId: string,
  data: {
    liability_name: string;
    liability_type: string;
    outstanding_balance: number;
    interest_rate?: number;
    is_paid_in_cadences?: boolean;
    cadence?: string | null;
    cadence_day_date?: string | null;
    cadence_amount?: number | null;
    due_date?: string | null;
    institution?: string;
    currency?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateLiability({
    liability_name: data.liability_name,
    liability_type: data.liability_type,
    outstanding_balance: data.outstanding_balance,
    interest_rate: data.interest_rate,
    is_paid_in_cadences: data.is_paid_in_cadences,
    cadence: data.cadence,
    cadence_day_date: data.cadence_day_date,
    cadence_amount: data.cadence_amount,
  });

  const { data: liability, error } = await supabase
    .from("liabilities")
    .insert({
      user_id: userId,
      liability_name: validated.liability_name,
```

```
      liability_type: validated.liability_type,
      outstanding_balance: validated.outstanding_balance,
      interest_rate: validated.interest_rate,
      is_paid_in_cadences: validated.is_paid_in_cadences,
      cadence: validated.cadence,
      cadence_day_date: validated.cadence_day_date,
      cadence_amount: validated.cadence_amount,
      due_date: data.due_date,
      institution: data.institution,
      currency: data.currency || "KSh",
    })
    .select()
    .single();

  if (error) throw error;
  return liability;
}

export async function updateLiability(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    liability_name?: string;
    liability_type?: string;
    outstanding_balance?: number;
    interest_rate?: number;
    is_paid_in_cadences?: boolean;
    cadence?: string | null;
    cadence_day_date?: string | null;
    cadence_amount?: number | null;
    last_executed_at?: string | null;
    due_date?: string | null;
    institution?: string;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.liability_name !== undefined &&
    updates.liability_name.trim().length === 0
  ) {
    throw new Error("Liability name is required.");
  }

  const { data, error } = await supabase
    .from("liabilities")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteLiability(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("liabilities")
```

```
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src\lib\db\objectives.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateObjective } from "../finance/objectives";

export async function getStrategicObjectives(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("strategic_objectives")
    .select("*")
    .is("deleted_at", null)
    .order("priority_level", { ascending: false })
    .order("created_at", { ascending: false });

  if (error) throw error;
  return data;
}

export async function createStrategicObjective(
  supabase: SupabaseClient,
  userId: string,
  data: {
    objective_name: string;
    objective_type: string;
    target_amount: number;
    current_amount?: number;
    target_date?: string | null;
    priority_level: string;
    status: string;
    notes?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateObjective({
    objective_name: data.objective_name,
    objective_type: data.objective_type,
    target_amount: data.target_amount,
    current_amount: data.current_amount,
    priority_level: data.priority_level,
    status: data.status,
    target_date: data.target_date,
    notes: data.notes,
  });

  const { data: objective, error } = await supabase
    .from("strategic_objectives")
    .insert({
      user_id: userId,
      objective_name: validated.objective_name,
      objective_type: validated.objective_type,
      target_amount: validated.target_amount,
      current_amount: validated.current_amount,
      target_date: validated.target_date,
      priority_level: validated.priority_level,
      status: validated.status,
      notes: validated.notes,
```

```
    })
    .select()
    .single();

  if (error) throw error;
  return objective;
}

export async function updateStrategicObjective(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    objective_name?: string;
    objective_type?: string;
    target_amount?: number;
    current_amount?: number;
    target_date?: string | null;
    priority_level?: string;
    status?: string;
    notes?: string | null;
    deleted_at?: string | null;
  },
) {
  // If objective_name is provided, validate it's not empty
  if (
    updates.objective_name !== undefined &&
    updates.objective_name.trim().length === 0
  ) {
    throw new Error("Objective name is required.");
  }

  const { data, error } = await supabase
    .from("strategic_objectives")
    .update(updates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteStrategicObjective(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("strategic_objectives")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src\lib\db\settings.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
```

```
export async function getUserSettings(
  supabase: SupabaseClient,
  userId: string,
) {
  const { data, error } = await supabase
    .from("user_settings")
    .select("*")
    .eq("user_id", userId)
    .maybeSingle();

  if (error) throw error;
  return data;
}

export async function updateLiquidity(
  supabase: SupabaseClient,
  userId: string,
  amount: number,
) {
  const { data, error } = await supabase
    .from("user_settings")
    .upsert({
      user_id: userId,
      total_liquidity: amount,
      updated_at: new Date().toISOString(),
    })
    .select()
    .single();

  if (error) throw error;
  return data;
}
```

FILE: src\lib\db\telemetry.ts

```
import { createClient } from "@/lib/supabase/server";

/**
 * TELEMETRY SERVICE (Phase 1)
 * Records insight evaluation events for observability.
 */
export async function logKairosEvaluation(data: {
  ruleId: string;
  severity: string;
  durationMs: number;
}) {
  try {
    let supabase;
    try {
      supabase = await createClient();
    } catch (_e) {
      // If we're in a test environment or outside a request scope, cookies() will fail.
      // We silenty exit as telemetry is non-critical for these contexts.
      return;
    }

    const {
      data: { user },
    } = await supabase.auth.getUser();
    if (!user) return;

    // Record evaluation event to kairos_insights table.
    // This phase uses the rule registry ID and performance metadata.
```

```
    const { error } = await supabase.from("kairos_insights").insert({
      user_id: user.id,
      rule_id: data.ruleId,
      severity: data.severity,
      evaluation_time_ms: data.durationMs,
    });

    if (error) {
      console.error("[Telemetry] Error recording evaluation:", error.message);
    }
  } catch (err) {
    // Observability must be non-blocking; we swallow errors to protect primary flows.
    console.error("[Telemetry] Critical failure in logging:", err);
  }
}
```

FILE: src\lib\finance\accounts.ts

```
export const ACCOUNT_TYPES = [
  { value: "checking", label: "Checking" },
  { value: "savings", label: "Savings" },
  { value: "mobile_money", label: "Mobile Money" },
  { value: "brokerage", label: "Brokerage" },
  { value: "crypto", label: "Crypto" },
  { value: "cash", label: "Cash" },
] as const;

export type AccountType = (typeof ACCOUNT_TYPES)[number]["value"];

export interface Account {
  id: string;
  user_id: string;
  account_name: string;
  account_type: AccountType;
  current_balance: number;
  currency: string;
  institution?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidAccountType(type: string): type is AccountType {
  return ACCOUNT_TYPES.some((t) => t.value === type);
}

export function validateAccount(data: {
  account_name: string;
  account_type: string;
  current_balance: number;
}) {
  if (!data.account_name || data.account_name.trim().length === 0) {
    throw new Error("Account name is required.");
  }

  if (!isValidAccountType(data.account_type)) {
    throw new Error(`Invalid account type: ${data.account_type}`);
  }

  if (isNaN(data.current_balance)) {
    throw new Error("Initial balance must be a number.");
  }
```

```
  return {
    account_name: data.account_name.trim(),
    account_type: data.account_type as AccountType,
    current_balance: data.current_balance,
  };
}
```

FILE: src\lib\finance\deploymentContext.ts

```
export const EXPECTED_RETURN_HORIZONS = [
  { value: "short-term", label: "Short-term" },
  { value: "medium-term", label: "Medium-term" },
  { value: "long-term", label: "Long-term" },
] as const;

export type ExpectedReturnHorizon =
  (typeof EXPECTED_RETURN_HORIZONS)[number]["value"];

export interface DeploymentAdvancedContext {
  associatedAccount?: string;
  expectedReturnHorizon?: ExpectedReturnHorizon;
  tags?: string[];
}

export interface DeploymentAdvancedContextInput {
  associatedAccount?: string | null;
  expectedReturnHorizon?: string | null;
  tags?: string | string[] | null;
}

const EXPECTED_RETURN_HORIZON_VALUES = new Set(
  EXPECTED_RETURN_HORIZONS.map((horizon) => horizon.value),
);

export function isExpectedReturnHorizon(
  value: string,
): value is ExpectedReturnHorizon {
  return EXPECTED_RETURN_HORIZON_VALUES.has(value as ExpectedReturnHorizon);
}

function normalizeTags(tags: DeploymentAdvancedContextInput["tags"]) {
  const source = Array.isArray(tags) ? tags.join(",") : tags || "";
  const normalizedTags = source
    .split(",")
    .map((tag) => tag.trim().toLowerCase())
    .filter(Boolean);

  return Array.from(new Set(normalizedTags));
}

export function normalizeDeploymentAdvancedContext(
  input: DeploymentAdvancedContextInput | null = {},
): DeploymentAdvancedContext {
  const safeInput = input || {};
  const associatedAccount = (safeInput.associatedAccount || "").trim();
  const expectedReturnHorizon = (
    safeInput.expectedReturnHorizon || ""
  ).trim();
  const tags = normalizeTags(safeInput.tags);

  if (
    expectedReturnHorizon &&
    !isExpectedReturnHorizon(expectedReturnHorizon)
```

```
  ) {
    throw new Error(
      "Context Gate: Unknown expected return horizon. Use short-term, medium-term, or long-term.",
    );
  }

  return {
    ...(associatedAccount ? { associatedAccount } : {}),
    ...(expectedReturnHorizon
      ? { expectedReturnHorizon: expectedReturnHorizon as ExpectedReturnHorizon }
      : {}),
    ...(tags.length > 0 ? { tags } : {}),
  };
}

export function hasDeploymentAdvancedContext(
  context: DeploymentAdvancedContext,
) {
  return Boolean(
    context.associatedAccount ||
      context.expectedReturnHorizon ||
      (context.tags && context.tags.length > 0),
  );
}
```

FILE: src\lib\finance\financeRules.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";

import {
  evaluateMetadataQuality,
  summarizeMetadataQuality,
  weightConfidenceForMetadataQuality,
} from "./metadataQuality";
import {
  normalizeDeploymentAdvancedContext,
} from "./deploymentContext";
import { isValidCategory } from "./taxonomy";
import { validateDeployment } from "./validateDeployment";

test("taxonomy validation accepts only canonical return categories", () => {
  assert.equal(isValidCategory("Asset"), true);
  assert.equal(isValidCategory("Leakage"), true);
  assert.equal(isValidCategory("Unclassified"), false);
  assert.equal(isValidCategory("Unknown"), false);
});

test("metadata quality scoring is deterministic for generic labels", () => {
  assert.deepEqual(evaluateMetadataQuality(" misc ").reasons, [
    "generic_label",
  ]);
  assert.equal(evaluateMetadataQuality("stuff").isLowQuality, true);
  assert.equal(evaluateMetadataQuality("Revenue engine setup").isLowQuality, false);
});

test("metadata quality summary produces stable confidence weighting", () => {
  const summary = summarizeMetadataQuality([
    { title: "misc" },
    { title: "Brokerage allocation" },
  ]);

  assert.equal(summary.lowQualityCount, 1);
```

```
  assert.equal(summary.lowQualityRatio, 0.5);
  assert.equal(summary.confidenceMultiplier, 0.93);
  assert.equal(weightConfidenceForMetadataQuality(0.9, summary), 0.84);
});

test("advanced context normalization preserves optional structured metadata", () => {
  assert.deepEqual(
    normalizeDeploymentAdvancedContext({
      associatedAccount: " Brokerage ",
      expectedReturnHorizon: "long-term",
      tags: "Ops, recurring, ops",
    }),
    {
      associatedAccount: "Brokerage",
      expectedReturnHorizon: "long-term",
      tags: ["ops", "recurring"],
    },
  );
});

test("advanced context rejects unknown return horizons", () => {
  assert.throws(
    () =>
      normalizeDeploymentAdvancedContext({
        expectedReturnHorizon: "eventually",
      }),
    /Unknown expected return horizon/,
  );
});

test("deployment validation remains the write gate", () => {
  assert.throws(
    () =>
      validateDeployment({
        title: "Capital",
        amount: 100,
        category: "Unclassified",
      }),
    /Strategic classification is mandatory/,
  );

  assert.deepEqual(
    validateDeployment({
      title: " Cloud infra ",
      amount: 500,
      category: "Leverage",
      advancedContext: {
        expectedReturnHorizon: "medium-term",
        tags: "ops, infra",
      },
    }).advancedContext,
    {
      expectedReturnHorizon: "medium-term",
      tags: ["ops", "infra"],
    },
  );
});
```

FILE: src\lib\finance\goals.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateGoal, calculateGoalProgressPercentage } from "./goals";
```

```
import { generateSummary } from "../analytics/engine";
import type { FinancialGoal } from "../analytics/types";

test("goal validation enforces structural integrity", () => {
  const data = {
    goal_name: "Emergency Fund",
    goal_type: "emergency_fund",
    target_amount: 1000000,
    current_progress: 100000,
    priority: "critical",
    status: "active",
  };

  const validated = validateGoal(data);
  assert.strictEqual(validated.goal_name, "Emergency Fund");
  assert.strictEqual(validated.goal_type, "emergency_fund");
  assert.strictEqual(validated.target_amount, 1000000);
  assert.strictEqual(validated.current_progress, 100000);
});

test("goal progress calculation is deterministic", () => {
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 500 }), 50);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 1000 }), 100);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 1500 }), 100);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 0, current_progress: 500 }), 0);
});

test("strategic metrics are accurately aggregated", () => {
  const goals: FinancialGoal[] = [
    {
      id: "g1",
      user_id: "u1",
      goal_name: "G1",
      goal_type: "emergency_fund",
      target_amount: 1000,
      current_progress: 200,
      priority: "critical",
      status: "active",
      linked_account_ids: [],
      created_at: "",
      updated_at: "",
      target_date: null,
    },
    {
      id: "g2",
      user_id: "u1",
      goal_name: "G2",
      goal_type: "retirement",
      target_amount: 1000,
      current_progress: 800,
      priority: "medium",
      status: "active",
      linked_account_ids: [],
      created_at: "",
      updated_at: "",
      target_date: null,
    },
  ];

  const summary = generateSummary([], 0, [], [], [], goals);

  assert.strictEqual(summary.totalStrategicTargets, 2000);
  assert.strictEqual(summary.totalCurrentProgress, 1000);
  assert.strictEqual(summary.averageGoalProgress, 50); // (20% + 80%) / 2
  assert.strictEqual(summary.criticalGoalCount, 1);
```

```
  assert.strictEqual(summary.fundingGap, 1000);
});
```

FILE: src\lib\finance\goals.ts

```
export const GOAL_TYPES = [
  { value: "emergency_fund", label: "Emergency Fund" },
  { value: "retirement", label: "Retirement" },
  { value: "home_purchase", label: "Home Purchase" },
  { value: "investment_target", label: "Investment Target" },
  { value: "business_runway", label: "Business Runway" },
  { value: "education", label: "Education" },
  { value: "debt_payoff", label: "Debt Payoff" },
  { value: "wealth_preservation", label: "Wealth Preservation" },
  { value: "other", label: "Other" },
] as const;

export type GoalType = (typeof GOAL_TYPES)[number]["value"];

export const GOAL_PRIORITIES = [
  { value: "critical", label: "Critical" },
  { value: "high", label: "High" },
  { value: "medium", label: "Medium" },
  { value: "low", label: "Low" },
] as const;

export type GoalPriority = (typeof GOAL_PRIORITIES)[number]["value"];

export const GOAL_STATUSES = [
  { value: "active", label: "Active" },
  { value: "paused", label: "Paused" },
  { value: "achieved", label: "Achieved" },
  { value: "archived", label: "Archived" },
] as const;

export type GoalStatus = (typeof GOAL_STATUSES)[number]["value"];

export interface FinancialGoal {
  id: string;
  user_id: string;
  goal_name: string;
  goal_type: GoalType;
  target_amount: number;
  current_progress: number;
  target_date: string | null;
  priority: GoalPriority;
  status: GoalStatus;
  linked_account_ids: string[];
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidGoalType(type: string): type is GoalType {
  return GOAL_TYPES.some((t) => t.value === type);
}

export function isValidPriority(priority: string): priority is GoalPriority {
  return GOAL_PRIORITIES.some((p) => p.value === priority);
}

export function isValidStatus(status: string): status is GoalStatus {
```

```
  return GOAL_STATUSES.some((s) => s.value === status);
}

export function validateGoal(data: {
  goal_name: string;
  goal_type: string;
  target_amount: number;
  current_progress?: number;
  priority: string;
  status: string;
  target_date?: string | null;
  notes?: string | null;
}) {
  if (!data.goal_name || data.goal_name.trim().length === 0) {
    throw new Error("Goal name is required.");
  }

  if (!isValidGoalType(data.goal_type)) {
    throw new Error(`Invalid goal type: ${data.goal_type}`);
  }

  const targetAmount = Number(data.target_amount);
  const currentProgress = Number(data.current_progress || 0);

  if (isNaN(targetAmount) || targetAmount <= 0) {
    throw new Error("Target amount must be a positive number.");
  }

  if (isNaN(currentProgress) || currentProgress < 0) {
    throw new Error("Current progress must be a non-negative number.");
  }

  if (!isValidPriority(data.priority)) {
    throw new Error(`Invalid priority: ${data.priority}`);
  }

  if (!isValidStatus(data.status)) {
    throw new Error(`Invalid status: ${data.status}`);
  }

  // Date Validation
  let normalizedDate: string | null = null;
  if (data.target_date) {
    const d = new Date(data.target_date);
    if (isNaN(d.getTime())) {
      throw new Error("Invalid target date format.");
    }
    normalizedDate = d.toISOString().split("T")[0];
  }

  const finalNotes = data.notes?.trim() || null;

  return {
    goal_name: data.goal_name.trim(),
    goal_type: data.goal_type as GoalType,
    target_amount: targetAmount,
    current_progress: currentProgress,
    priority: data.priority as GoalPriority,
    status: data.status as GoalStatus,
    target_date: normalizedDate,
    notes: finalNotes === "" ? null : finalNotes,
  };
}

/**
```

```
 * Calculates deterministic goal progress.
 */
export function calculateGoalProgressPercentage(
  goal: Pick<FinancialGoal, "current_progress" | "target_amount">,
): number {
  if (goal.target_amount <= 0) return 0;
  return Math.min(
    100,
    Math.max(0, (goal.current_progress / goal.target_amount) * 100),
  );
}
```

FILE: src\lib\finance\income.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateIncomeStream, calculateMonthlyInflow } from "./income";
import { generateSummary } from "../analytics/engine";
import type { Deployment, IncomeStream } from "../analytics/types";

test("income validation enforces structural integrity", () => {
  const data = {
    income_name: "Salary",
    income_type: "salary",
    amount: 100000,
    cadence: "monthly",
  };

  const validated = validateIncomeStream(data);
  assert.strictEqual(validated.income_name, "Salary");
  assert.strictEqual(validated.income_type, "salary");
  assert.strictEqual(validated.amount, 100000);
  assert.strictEqual(validated.cadence, "monthly");
});

test("income normalization is accurate", () => {
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 1000, cadence: "weekly" }),
    1000 * (52 / 12),
  );
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 120000, cadence: "annually" }),
    10000,
  );
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 5000, cadence: "monthly" }),
    5000,
  );
});

test("runway logic incorporates income offset correctly", () => {
  const deployments: Deployment[] = [
    {
      id: "1",
      amount: 3000,
      created_at: new Date().toISOString(),
      title: "Rent",
      category: "Fixed",
    },
  ]; // 100/day burn over 30 days

  const liquidity = 1000;
  const incomeStreams: IncomeStream[] = [
```

```
    {
      id: "s1",
      user_id: "u1",
      income_name: "Salary",
      amount: 1500,
      cadence: "monthly",
      is_recurring: true,
      income_type: "salary",
      currency: "KSh",
      start_date: "",
      created_at: "",
      updated_at: "",
    },
  ]; // 50/day offset

  // Correct Formula: Liquidity / (Daily Burn - (Monthly Income / 30))
  // 1000 / (100 - (1500 / 30)) = 1000 / 50 = 20 days
  const summary = generateSummary(
    deployments,
    liquidity,
    [],
    [],
    incomeStreams,
  );

  assert.strictEqual(summary.dailyBurnRate, 100);
  assert.strictEqual(summary.totalMonthlyIncome, 1500);
  assert.strictEqual(summary.runwayDays, 20);
  assert.strictEqual(summary.adjustedDailyBurn, 50);
});
```

FILE: src\lib\finance\income.ts

```
export const INCOME_TYPES = [
  { value: "salary", label: "Salary" },
  { value: "freelance", label: "Freelance" },
  { value: "business", label: "Business" },
  { value: "dividends", label: "Dividends" },
  { value: "rental", label: "Rental" },
  { value: "contract", label: "Contract" },
  { value: "other", label: "Other" },
] as const;

export type IncomeType = (typeof INCOME_TYPES)[number]["value"];

export const CADENCES = [
  { value: "weekly", label: "Weekly", factor: 52 / 12 },
  { value: "biweekly", label: "Biweekly", factor: 26 / 12 },
  { value: "monthly", label: "Monthly", factor: 1 },
  { value: "quarterly", label: "Quarterly", factor: 1 / 3 },
  { value: "annually", label: "Annually", factor: 1 / 12 },
  { value: "irregular", label: "Irregular", factor: 0 },
] as const;

export type Cadence = (typeof CADENCES)[number]["value"];

export interface IncomeStream {
  id: string;
  user_id: string;
  income_name: string;
  income_type: IncomeType;
  amount: number;
  cadence: Cadence;
```

```
  execution_day?: number | null;
  last_executed_at?: string | null;
  is_recurring: boolean;
  currency: string;
  source?: string | null;
  start_date: string;
  end_date?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidIncomeType(type: string): type is IncomeType {
  return INCOME_TYPES.some((t) => t.value === type);
}

export function isValidCadence(cadence: string): cadence is Cadence {
  return CADENCES.some((c) => c.value === cadence);
}

export function validateIncomeStream(data: {
  income_name: string;
  income_type: string;
  amount: number;
  cadence: string;
  execution_day?: number | null;
  is_recurring?: boolean;
}) {
  if (!data.income_name || data.income_name.trim().length === 0) {
    throw new Error("Income name is required.");
  }

  if (!isValidIncomeType(data.income_type)) {
    throw new Error(`Invalid income type: ${data.income_type}`);
  }

  if (!isValidCadence(data.cadence)) {
    throw new Error(`Invalid cadence: ${data.cadence}`);
  }

  if (isNaN(data.amount) || data.amount < 0) {
    throw new Error("Amount must be a non-negative number.");
  }

  return {
    income_name: data.income_name.trim(),
    income_type: data.income_type as IncomeType,
    amount: data.amount,
    cadence: data.cadence as Cadence,
    execution_day: data.execution_day,
    is_recurring: data.is_recurring ?? true,
  };
}

/**
 * Normalizes an income stream amount to a monthly value.
 */
export function calculateMonthlyInflow(
  stream: Pick<IncomeStream, "amount" | "cadence">,
): number {
  const cadenceConfig = CADENCES.find((c) => c.value === stream.cadence);
  if (!cadenceConfig) return 0;
  return stream.amount * cadenceConfig.factor;
}
```

FILE: src\lib\finance\liabilities.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateLiability } from "./liabilities";
import { generateSummary } from "../analytics/engine";
import type { Account, Liability } from "../analytics/types";

test("liability validation enforces structural integrity", () => {
  const data = {
    liability_name: "Credit Card",
    liability_type: "credit_card",
    outstanding_balance: 50000,
    interest_rate: 2,
    is_paid_in_cadences: true,
    cadence: "monthly",
    cadence_day_date: "15",
    cadence_amount: 5000,
  };

  const validated = validateLiability(data);
  assert.strictEqual(validated.liability_name, "Credit Card");
  assert.strictEqual(validated.liability_type, "credit_card");
  assert.strictEqual(validated.outstanding_balance, 50000);
  assert.strictEqual(validated.is_paid_in_cadences, true);
  assert.strictEqual(validated.cadence, "monthly");
  assert.strictEqual(validated.cadence_day_date, "15");
  assert.strictEqual(validated.cadence_amount, 5000);
});

test("liability validation rejects invalid monthly date", () => {
  assert.throws(() => {
    validateLiability({
      liability_name: "Test",
      liability_type: "credit_card",
      outstanding_balance: 1000,
      is_paid_in_cadences: true,
      cadence: "monthly",
      cadence_day_date: "32",
      cadence_amount: 100,
    });
  }, /Monthly cadence requires a valid date/);
});

test("liability validation rejects invalid weekly day", () => {
  assert.throws(() => {
    validateLiability({
      liability_name: "Test",
      liability_type: "credit_card",
      outstanding_balance: 1000,
      is_paid_in_cadences: true,
      cadence: "weekly",
      cadence_day_date: "Funday",
      cadence_amount: 100,
    });
  }, /Weekly cadence requires a valid day/);
});

test("net worth calculation and interest accrual are deterministic", () => {
  const accounts: Account[] = [
    {
      id: "1",
      user_id: "u1",
      account_name: "Checking",
```

```
      account_type: "checking",
      current_balance: 100000,
      currency: "KSh",
      created_at: "",
      updated_at: "",
      institution: null,
    },
  ];

  const liabilities: Liability[] = [
    {
      id: "l1",
      user_id: "u1",
      liability_name: "Loan",
      liability_type: "personal_loan",
      outstanding_balance: 30000,
      interest_rate: 1, // 1% per month
      is_paid_in_cadences: true,
      cadence: "monthly",
      cadence_day_date: "1",
      cadence_amount: 3000,
      currency: "KSh",
      created_at: "",
      updated_at: "",
      institution: null,
      due_date: null,
    },
  ];

  const summary = generateSummary([], 100000, accounts, liabilities);

  assert.strictEqual(summary.totalAssets, 100000);
  assert.strictEqual(summary.totalLiabilities, 30000);
  assert.strictEqual(summary.netWorth, 70000);

  // Monthly interest = 30000 * 0.01 = 300
  // Daily interest burn = 300 / 30 = 10
  assert.strictEqual(summary.dailyBurnRate, 10);
});
```

FILE: src\lib\finance\liabilities.ts

```
export const LIABILITY_TYPES = [
  { value: "credit_card", label: "Credit Card" },
  { value: "mortgage", label: "Mortgage" },
  { value: "personal_loan", label: "Personal Loan" },
  { value: "student_loan", label: "Student Loan" },
  { value: "business_loan", label: "Business Loan" },
  { value: "line_of_credit", label: "Line of Credit" },
  { value: "other", label: "Other" },
] as const;

export type LiabilityType = (typeof LIABILITY_TYPES)[number]["value"];

export interface Liability {
  id: string;
  user_id: string;
  liability_name: string;
  liability_type: LiabilityType;
  outstanding_balance: number;
  interest_rate: number;
  is_paid_in_cadences: boolean;
  cadence: "weekly" | "monthly" | null;
```

```
  cadence_day_date: string | null;
  cadence_amount: number | null;
  last_executed_at?: string | null;
  due_date: string | null;
  currency: string;
  institution?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidLiabilityType(type: string): type is LiabilityType {
  return LIABILITY_TYPES.some((t) => t.value === type);
}

export function validateLiability(data: {
  liability_name: string;
  liability_type: string;
  outstanding_balance: number;
  interest_rate?: number;
  is_paid_in_cadences?: boolean;
  cadence?: string | null;
  cadence_day_date?: string | null;
  cadence_amount?: number | null;
}) {
  if (!data.liability_name || data.liability_name.trim().length === 0) {
    throw new Error("Liability name is required.");
  }

  if (!isValidLiabilityType(data.liability_type)) {
    throw new Error(`Invalid liability type: ${data.liability_type}`);
  }

  if (isNaN(data.outstanding_balance)) {
    throw new Error("Outstanding balance must be a number.");
  }

  const isPaidInCadences = !!data.is_paid_in_cadences;
  let cadence: "weekly" | "monthly" | null = null;
  let cadenceDayDate: string | null = null;
  let cadenceAmount: number | null = null;

  if (isPaidInCadences) {
    if (data.cadence !== "weekly" && data.cadence !== "monthly") {
      throw new Error("A valid cadence (weekly or monthly) is required.");
    }
    cadence = data.cadence;

    if (!data.cadence_day_date || data.cadence_day_date.trim().length === 0) {
      throw new Error("Cadence day/date is required.");
    }

    if (cadence === "monthly") {
      const dateNum = parseInt(data.cadence_day_date, 10);
      if (isNaN(dateNum) || dateNum < 1 || dateNum > 31) {
        throw new Error("Monthly cadence requires a valid date (1-31).");
      }
    } else if (cadence === "weekly") {
      const validDays = [
        "monday",
        "tuesday",
        "wednesday",
        "thursday",
        "friday",
        "saturday",
```

```
        "sunday",
      ];
      if (!validDays.includes(data.cadence_day_date.toLowerCase())) {
        throw new Error("Weekly cadence requires a valid day of the week.");
      }
    }
    cadenceDayDate = data.cadence_day_date;

    if (
      data.cadence_amount === undefined ||
      data.cadence_amount === null ||
      isNaN(data.cadence_amount)
    ) {
      throw new Error("Cadence payment amount is required.");
    }
    cadenceAmount = data.cadence_amount;
  }

  return {
    liability_name: data.liability_name.trim(),
    liability_type: data.liability_type as LiabilityType,
    outstanding_balance: data.outstanding_balance,
    interest_rate: data.interest_rate || 0,
    is_paid_in_cadences: isPaidInCadences,
    cadence,
    cadence_day_date: cadenceDayDate,
    cadence_amount: cadenceAmount,
  };
}
```

FILE: src\lib\finance\liquidity.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";
import { LiquidityService } from "./liquidity";
import { Account } from "@/lib/analytics/types";

test("LiquidityService.calculateTotal sums account balances correctly", () => {
  const accounts: Partial<Account>[] = [
    { current_balance: 1000 },
    { current_balance: 500 },
    { current_balance: 250.50 },
  ];

  assert.equal(LiquidityService.calculateTotal(accounts as Account[]), 1750.50);
});

test("LiquidityService.calculateTotal handles empty accounts list", () => {
  assert.equal(LiquidityService.calculateTotal([]), 0);
});

test("LiquidityService.calculateTotal handles zero and negative balances", () => {
  const accounts: Partial<Account>[] = [
    { current_balance: 1000 },
    { current_balance: -200 },
    { current_balance: 0 },
  ];

  assert.equal(LiquidityService.calculateTotal(accounts as Account[]), 800);
});

test("LiquidityService.calculateTotal handles missing balances", () => {
  const accounts: Partial<Account>[] = [
```

```
    { current_balance: 1000 },
    { } as Account,
  ];

  assert.equal(LiquidityService.calculateTotal(accounts as Account[]), 1000);
});
```

FILE: src\lib\finance\liquidity.ts

```
import { SupabaseClient } from "@supabase/supabase-js";
import { Account } from "@/lib/analytics/types";
import { getAccounts } from "@/lib/db/accounts";
import { updateLiquidity } from "@/lib/db/settings";

/**
 * Domain service responsible for liquidity calculations and synchronization.
 * Truth: Liquidity is the aggregate of all liquid account balances.
 */
export const LiquidityService = {
  /**
   * Pure function to calculate total liquidity from a list of accounts.
   */
  calculateTotal(accounts: Account[]): number {
    return accounts.reduce((sum, account) => sum + Number(account.current_balance || 0), 0);
  },

  /**
   * Synchronizes the global total_liquidity setting with the current state of accounts.
   * This should be called whenever an account balance changes.
   */
  async sync(supabase: SupabaseClient, userId: string): Promise<number> {
    const accountsData = await getAccounts(supabase);
    const accounts = (accountsData || []) as Account[];

    const totalLiquidity = this.calculateTotal(accounts);

    await updateLiquidity(supabase, userId, totalLiquidity);

    return totalLiquidity;
  }
};
```

FILE: src\lib\finance\metadataQuality.ts

```
export type MetadataQualityLevel = "specific" | "low";

export interface MetadataQualityEvaluation {
  level: MetadataQualityLevel;
  score: number;
  isLowQuality: boolean;
  normalizedLabel: string;
  reasons: string[];
}

export interface MetadataQualitySummary {
  averageScore: number;
  lowQualityCount: number;
  lowQualityRatio: number;
  confidenceMultiplier: number;
}
```

```
type LabelBearingInput = {
  title?: string | null;
};

const LOW_QUALITY_EXACT_LABELS = new Set([
  "misc",
  "stuff",
  "other",
  "random",
  "generic",
  "unclear",
  "unknown",
  "untitled",
  "no title",
  "none",
  "nil",
  "na",
  "n/a",
  "tbd",
  "test",
]);

const EMPTY_LIKE_PATTERNS = [/^-+$/, /^\.+$/, /^\?+$/, /^_+$/];

const roundQualityScore = (value: number) => Math.round(value * 100) / 100;

export function normalizeMetadataLabel(label: string) {
  return label.trim().toLowerCase().replace(/\s+/g, " ");
}

export function evaluateMetadataQuality(
  label: string | null | undefined,
): MetadataQualityEvaluation {
  const normalizedLabel = normalizeMetadataLabel(label || "");
  const semanticLabel = normalizedLabel
    .replace(/[^a-z0-9/?._-]+/g, " ")
    .replace(/\s+/g, " ")
    .trim();
  const comparableLabel = semanticLabel.replace(
    /^[^a-z0-9]+|[^a-z0-9]+$/g,
    "",
  );

  const reasons: string[] = [];

  if (
    !semanticLabel ||
    EMPTY_LIKE_PATTERNS.some((pattern) => pattern.test(semanticLabel))
  ) {
    reasons.push("empty_like_label");
  }

  if (
    LOW_QUALITY_EXACT_LABELS.has(semanticLabel) ||
    LOW_QUALITY_EXACT_LABELS.has(comparableLabel)
  ) {
    reasons.push("generic_label");
  }

  const isLowQuality = reasons.length > 0;

  return {
    level: isLowQuality ? "low" : "specific",
    score: isLowQuality ? 0.35 : 1,
    isLowQuality,
```

```
    normalizedLabel,
    reasons,
  };
}

export function summarizeMetadataQuality(
  deployments: LabelBearingInput[],
): MetadataQualitySummary {
  if (deployments.length === 0) {
    return {
      averageScore: 1,
      lowQualityCount: 0,
      lowQualityRatio: 0,
      confidenceMultiplier: 1,
    };
  }

  const evaluations = deployments.map((deployment) =>
    evaluateMetadataQuality(deployment.title),
  );
  const lowQualityCount = evaluations.filter(
    (evaluation) => evaluation.isLowQuality,
  ).length;
  const averageScore =
    evaluations.reduce((sum, evaluation) => sum + evaluation.score, 0) /
    evaluations.length;
  const lowQualityRatio = lowQualityCount / evaluations.length;

  return {
    averageScore: roundQualityScore(averageScore),
    lowQualityCount,
    lowQualityRatio: roundQualityScore(lowQualityRatio),
    confidenceMultiplier: roundQualityScore(
      Math.max(0.85, 1 - lowQualityRatio * 0.15),
    ),
  };
}

export function weightConfidenceForMetadataQuality(
  confidence: number,
  metadataQuality: MetadataQualitySummary,
) {
  return roundQualityScore(
    Math.min(1, Math.max(0, confidence * metadataQuality.confidenceMultiplier)),
  );
}
```

FILE: src\lib\finance\objectives.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateObjective } from "./objectives";

test("strategic objective validation enforces structural integrity", () => {
  const data = {
    objective_name: "Safety Net",
    objective_type: "emergency_reserve",
    target_amount: 500000,
    current_amount: 50000,
    priority_level: "critical",
    status: "active",
  };
```

```
  const validated = validateObjective(data);
  assert.strictEqual(validated.objective_name, "Safety Net");
  assert.strictEqual(validated.objective_type, "emergency_reserve");
  assert.strictEqual(validated.target_amount, 500000);
  assert.strictEqual(validated.current_amount, 50000);
  assert.strictEqual(validated.priority_level, "critical");
  assert.strictEqual(validated.status, "active");
});

test("objective validation fails on invalid types", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "invalid_type",
      target_amount: 1000,
      priority_level: "high",
      status: "active",
    });
  }, /Invalid objective type/);
});

test("objective validation fails on invalid priority", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "very_high",
      status: "active",
    });
  }, /Invalid priority level/);
});

test("objective validation fails on invalid status", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "high",
      status: "unknown",
    });
  }, /Invalid status/);
});

test("objective validation fails on zero or negative target amount", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 0,
      priority_level: "high",
      status: "active",
    });
  }, /Target amount must be a positive number/);

  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: -1,
      priority_level: "high",
      status: "active",
    });
  }, /Target amount must be a positive number/);
```

```
});

test("objective validation fails on overfunded state", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 1001,
      priority_level: "high",
      status: "active",
    });
  }, /Current amount cannot exceed target amount/);
});

test("objective validation handles date normalization", () => {
  const validated = validateObjective({
    objective_name: "Test",
    objective_type: "liquidity",
    target_amount: 1000,
    priority_level: "high",
    status: "active",
    target_date: "2026-12-31",
  });
  assert.strictEqual(validated.target_date, "2026-12-31");
});

test("objective validation fails on malformed dates", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "high",
      status: "active",
      target_date: "not-a-date",
    });
  }, /Invalid target date format/);
});

test("objective normalization trims whitespace and handles empty notes", () => {
  const validated = validateObjective({
    objective_name: "  Safety Net  ",
    objective_type: "emergency_reserve",
    target_amount: 1000,
    priority_level: "critical",
    status: "active",
    notes: "   Need this for safety   ",
  });
  assert.strictEqual(validated.objective_name, "Safety Net");
  assert.strictEqual(validated.notes, "Need this for safety");

  const validatedEmptyNotes = validateObjective({
    objective_name: "Test",
    objective_type: "liquidity",
    target_amount: 1000,
    priority_level: "high",
    status: "active",
    notes: "    ",
  });
  assert.strictEqual(validatedEmptyNotes.notes, null);
});
```

Axiom Repository Audit

FILE: src\lib\finance\objectives.ts

```
export const OBJECTIVE_TYPES = [
  { value: "emergency_reserve", label: "Emergency Reserve" },
  { value: "liquidity", label: "Liquidity" },
  { value: "debt_elimination", label: "Debt Elimination" },
  { value: "investment", label: "Investment" },
  { value: "retirement", label: "Retirement" },
  { value: "education", label: "Education" },
  { value: "acquisition", label: "Acquisition" },
  { value: "custom", label: "Custom" },
] as const;

export type ObjectiveType = (typeof OBJECTIVE_TYPES)[number]["value"];

export const OBJECTIVE_PRIORITIES = [
  { value: "critical", label: "Critical" },
  { value: "high", label: "High" },
  { value: "moderate", label: "Moderate" },
  { value: "low", label: "Low" },
] as const;

export type ObjectivePriority = (typeof OBJECTIVE_PRIORITIES)[number]["value"];

export const OBJECTIVE_STATUSES = [
  { value: "active", label: "Active" },
  { value: "paused", label: "Paused" },
  { value: "completed", label: "Completed" },
  { value: "abandoned", label: "Abandoned" },
] as const;

export type ObjectiveStatus = (typeof OBJECTIVE_STATUSES)[number]["value"];

export interface StrategicObjective {
  id: string;
  user_id: string;
  objective_name: string;
  objective_type: ObjectiveType;
  target_amount: number;
  current_amount: number;
  target_date: string | null;
  priority_level: ObjectivePriority;
  status: ObjectiveStatus;
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidObjectiveType(type: string): type is ObjectiveType {
  return OBJECTIVE_TYPES.some((t) => t.value === type);
}

export function isValidPriority(
  priority: string,
): priority is ObjectivePriority {
  return OBJECTIVE_PRIORITIES.some((p) => p.value === priority);
}

export function isValidStatus(status: string): status is ObjectiveStatus {
  return OBJECTIVE_STATUSES.some((s) => s.value === status);
}

/**
```

```
 * Validates and normalizes strategic objective data.
 * This is the canonical authority for objective correctness.
 */
/**
 * Calculates deterministic objective progress percentage.
 */
export function calculateObjectiveProgressPercentage(
  objective: Pick<StrategicObjective, "current_amount" | "target_amount">,
): number {
  if (objective.target_amount <= 0) return 0;
  return Math.min(
    100,
    Math.max(0, (objective.current_amount / objective.target_amount) * 100),
  );
}

/**
 * Calculates the funding ratio (alias for progress for semantic clarity in UI).
 */
export function calculateObjectiveFundingRatio(
  objective: Pick<StrategicObjective, "current_amount" | "target_amount">,
): number {
  return calculateObjectiveProgressPercentage(objective);
}

export function validateObjective(data: {
  objective_name: string;
  objective_type: string;
  target_amount: number;
  current_amount?: number;
  priority_level: string;
  status: string;
  target_date?: string | null;
  notes?: string | null;
}) {
  // 1. Required Fields & Basic Types
  if (!data.objective_name || data.objective_name.trim().length === 0) {
    throw new Error("Objective name is required.");
  }

  // 2. Enum Validations
  if (!isValidObjectiveType(data.objective_type)) {
    throw new Error(`Invalid objective type: ${data.objective_type}`);
  }

  if (!isValidPriority(data.priority_level)) {
    throw new Error(`Invalid priority level: ${data.priority_level}`);
  }

  if (!isValidStatus(data.status)) {
    throw new Error(`Invalid status: ${data.status}`);
  }

  // 3. Numeric Validations
  const targetAmount = Number(data.target_amount);
  const currentAmount = Number(data.current_amount || 0);

  if (isNaN(targetAmount) || targetAmount <= 0) {
    throw new Error("Target amount must be a positive number.");
  }

  if (isNaN(currentAmount) || currentAmount < 0) {
    throw new Error("Current amount must be a non-negative number.");
  }
```

```
  if (currentAmount > targetAmount) {
    throw new Error(
      "Current amount cannot exceed target amount (overfunding requires manual strategic adjustment).",
    );
  }

  // 4. Date Validation
  let normalizedDate: string | null = null;
  if (data.target_date) {
    const d = new Date(data.target_date);
    if (isNaN(d.getTime())) {
      throw new Error("Invalid target date format.");
    }
    normalizedDate = d.toISOString().split("T")[0]; // Store as YYYY-MM-DD
  }

  // 5. Normalization
  const normalizedNotes = data.notes?.trim() || null;
  const finalNotes = normalizedNotes === "" ? null : normalizedNotes;

  return {
    objective_name: data.objective_name.trim(),
    objective_type: data.objective_type as ObjectiveType,
    target_amount: targetAmount,
    current_amount: currentAmount,
    priority_level: data.priority_level as ObjectivePriority,
    status: data.status as ObjectiveStatus,
    target_date: normalizedDate,
    notes: finalNotes,
  };
}
```

FILE: src\lib\finance\taxonomy.ts

```
/**
 * AXIOM RETURN-BASED TAXONOMY
 * Centralized definitions for the financial intelligence system.
 */

export const VALID_CATEGORIES = [
  "Asset",
  "Skill",
  "Leverage",
  "Experience",
  "Maintenance",
  "Leakage",
] as const;
export type ValidCategory = (typeof VALID_CATEGORIES)[number];

export interface TaxonomyCategory {
  value: ValidCategory;
  label: string;
  definition: string;
  interpretation: string;
}

export const TAXONOMY_CATEGORIES: TaxonomyCategory[] = [
  {
    value: "Asset",
    label: "Investments",
    definition: "Future value generation (Stocks, Crypto, Property)",
    interpretation:
      "Strategic capital foundation. These deployments are expected to produce tangible appreciation or yield over time.",
```

```
  },
  {
    value: "Skill",
    label: "Growth",
    definition: "Learning and high-income skill development",
    interpretation:
      "Human capital investment. Deployments into skills increase your intrinsic market value and future inflow capacity.",
  },
  {
    value: "Leverage",
    label: "Efficiency",
    definition: "Buying back time or multiplying output",
    interpretation:
      "Operational efficiency. Leverage deployments are designed to buy back time or multiply the results of your effort.",
  },
  {
    value: "Experience",
    label: "Lifestyle",
    definition: "Intentional quality-of-life and memories",
    interpretation:
      "Strategic utility. Non-yielding deployments that provide intentional lived-value and mental sustainability.",
  },
  {
    value: "Maintenance",
    label: "Essentials",
    definition: "Bills, rent, food, and survival core",
    interpretation:
      "Operational core. Survival-layer capital needed to maintain your current system (Rent, utilities, basic nourishment).",
  },
  {
    value: "Leakage",
    label: "Waste",
    definition: "Non-strategic drift or impulse spending",
    interpretation:
       "Systemic inefficiency. Tracked separately to identify recurring capital drift. Excess leakage compresses operational
runway.",
  },
];

export const isValidCategory = (value: string): boolean => {
  return VALID_CATEGORIES.includes(value as ValidCategory);
};

export const formatTaxonomyCategoryList = (): string => {
  return VALID_CATEGORIES.join(", ");
};

export const getTaxonomyDefinition = (value: string) => {
  return TAXONOMY_CATEGORIES.find((c) => c.value === value)?.definition || "";
};

export const getTaxonomyInterpretation = (value: string) => {
  return (
    TAXONOMY_CATEGORIES.find((c) => c.value === value)?.interpretation || ""
  );
};
```

FILE: src\lib\finance\validateDeployment.ts

```
/**
 * TAXONOMY ENFORCEMENT GATE
 * Ensures every deployment written to the ledger follows the strict return-based taxonomy.
 */
```

```
import {
  formatTaxonomyCategoryList,
  isValidCategory,
} from "./taxonomy";
import {
  DeploymentAdvancedContextInput,
  normalizeDeploymentAdvancedContext,
} from "./deploymentContext";
import { evaluateMetadataQuality } from "./metadataQuality";

export { VALID_CATEGORIES, type ValidCategory } from "./taxonomy";

export interface DeploymentInput {
  title: string;
  amount: number;
  category: string;
  impactScore?: number;
  advancedContext?: DeploymentAdvancedContextInput;
}

/**
 * Pre-write Validation Layer
 * Rejects any input that violates taxonomy integrity or data truth.
 */
export function validateDeployment(input: DeploymentInput) {
  const { title, amount, category, impactScore, advancedContext } = input;

  // 1. Structural Integrity
  if (!title || !title.trim()) {
    throw new Error("Taxonomy Gate: Title is required for deployment verification");
  }

  if (amount === undefined || amount === null || isNaN(amount)) {
    throw new Error("Taxonomy Gate: Numeric amount is required");
  }

  if (amount <= 0) {
    throw new Error("Taxonomy Gate: Capital deployment must be a positive value (> 0)");
  }

  // 2. Taxonomy Enforcement
  if (!isValidCategory(category)) {
    // Specifically reject 'Unclassified' and legacy categories
    if (category === "Unclassified") {
      throw new Error("Taxonomy Gate: Strategic classification is mandatory. Entry rejected.");
    }

    throw new Error(`Taxonomy Gate: Unknown category '${category}'. Use ${formatTaxonomyCategoryList()}.`);
  }

  // 3. Normalization
  return {
    title: title.trim(),
    amount: Number(amount),
    category,
    advancedContext: normalizeDeploymentAdvancedContext(advancedContext),
    metadataQuality: evaluateMetadataQuality(title),
    impactScore: Math.min(Math.max(impactScore || 0, 0), 10) // Scale 0-10
  };
}
```

Axiom Repository Audit

FILE: src\lib\insights\generateInsights.ts

```
import { BehavioralContext } from "../context/contextTypes";
import { InsightEngineResult, InsightPriority } from "./types";
import { KairosInsight } from "../ai/kairos";
import { rules } from "./rules/registry";
import { logKairosEvaluation } from "../db/telemetry";

/**
 * Axiom Insight Engine Orchestrator
 * Evaluates behavioral context against the rule registry to generate
 * prioritized strategic insights.
 *
 * V1.2: Added asynchronous telemetry logging for performance audit.
 */
export const evaluateInsights = async (
  context: BehavioralContext,
): Promise<InsightEngineResult> => {
  // 1. Process all rules and capture telemetry
  const evaluationResults = await Promise.all(
    rules.map(async (rule) => {
      const start = Date.now();
      const isMatched = rule.condition(context);

      let insight: KairosInsight | null = null;
      if (isMatched) {
        insight = rule.generate(context);
      }

      const duration = Date.now() - start;

      // Persist forensic telemetry (Non-blocking but awaited for database integrity)
      await logKairosEvaluation({
        ruleId: rule.id,
        severity: insight?.severity || "none",
        durationMs: duration,
      });

      if (isMatched && insight) {
        return {
          priority: rule.priority,
          insight,
        };
      }
      return null;
    }),
  );

  const generatedInsights = evaluationResults.filter(
    (result): result is { priority: InsightPriority; insight: KairosInsight } =>
      result !== null,
  );

  // Axiom Standard: Gracefully handle the "Quiet" state
  if (generatedInsights.length === 0) {
    return {
      primaryInsight: {
        type: "info",
        severity: "observation",
        category: "strategic_alignment",
        message:
          "No material structural deterioration detected since previous evaluation.",
        supportingSignals: [],
        timestamp: new Date().toISOString(),
```

```
        runway: null,
        capitalEfficiency: context.capitalEfficiencyScore ?? 0,
        isSilent: true, // This is the "Quiet System" output
      },
      allInsights: [],
    };
  }

  // 2. Prioritize
  // Sort by priority (High > Medium > Low) and then by confidence
  const sortedInsights = [...generatedInsights].sort((a, b) => {
    const priorityScore: Record<InsightPriority, number> = {
      critical: 4,
      high: 3,
      medium: 2,
      low: 1,
    };

    if (priorityScore[a.priority] !== priorityScore[b.priority]) {
      return priorityScore[b.priority] - priorityScore[a.priority];
    }

    return 0; // Maintain order within same priority
  });

  return {
    primaryInsight: sortedInsights[0].insight,
    allInsights: generatedInsights.map((result) => result.insight),
  };
};
```

FILE: src\lib\insights\types.ts

```
import { BehavioralContext } from "../context/contextTypes";
import { KairosInsight } from "../ai/kairos";

export type InsightPriority = "critical" | "high" | "medium" | "low";

export interface InsightRule {
  id: string;
  priority: InsightPriority;
  condition: (context: BehavioralContext) => boolean;
  generate: (context: BehavioralContext) => KairosInsight;
}

export interface InsightEngineResult {
  primaryInsight: KairosInsight;
  allInsights: KairosInsight[];
}
```

FILE: src\lib\insights\rules\registry.ts

```
import { InsightRule } from "../types";

/**
 * KAIROS RULE REGISTRY (V1.1)
 * Behavioral Presence implementation.
 * Identity: Restrained Operational Analyst.
 */

export const rules: InsightRule[] = [
```

```
  // 0. PENDING VERIFICATION (High Priority Alert)
  {
    id: "pending_verification",
    priority: "critical",
    condition: (ctx) => ctx.pendingVerificationCount > 0,
    generate: (ctx) => ({
      type: "warning",
      severity: "critical",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
        message: `Action Required: ${ctx.pendingVerificationCount} financial verification${ctx.pendingVerificationCount > 1 ?
"s" : ""} detected for today.`,
      supportingSignals: [
        "Axiom has detected scheduled transaction flows that require manual confirmation to ascertain absolute truth.",
        "Verification will update your structural solvency window and capital ledger.",
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },
  // 1. SOLVENCY CRISIS (Critical Severity)
  {
    id: "solvency_crisis",
    priority: "critical",
    condition: (ctx) => ctx.netWorth < 0,
    generate: (ctx) => ({
      type: "warning",
      severity: "critical",
      category: "solvency_pressure",
      timestamp: new Date().toISOString(),
      message:
           "Severe solvency crisis detected. Total credit facilities (e.g., Fuliza, M-Shwari, SACCO Loans) exceed combined
capital assets, resulting in a negative net worth state.",
      supportingSignals: [
        `Net Worth: KSh ${Math.round(ctx.netWorth).toLocaleString()}`,
        `Liquid Reserves (MMF/SACCO): KSh ${Math.round(ctx.liquidity).toLocaleString()}`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 2. STRUCTURAL DEFICIT (Critical Severity)
  {
    id: "structural_deficit",
    priority: "high",
    condition: (ctx) => {
      const monthlyBurn = ctx.burnRate.monthlyProjection;
      const monthlyReplenishment = ctx.totalMonthlyIncome;

      // Trigger if structural burn exceeds income (Structural Deficit)
      return monthlyBurn > monthlyReplenishment && monthlyBurn > 0;
    },
    generate: (ctx) => {
      const deficit = ctx.burnRate.monthlyProjection - ctx.totalMonthlyIncome;
      return {
        type: "warning",
        severity: "critical",
        category: "solvency_pressure",
        timestamp: new Date().toISOString(),
        message:
             "Structural deficit detected. Baseline monthly outflow (including M-Pesa leakage) currently outpaces aggregate
inbound yield (M-Pesa flows / Hustle & Salary).",
```

```
        supportingSignals: [
          `Monthly Deficit: KSh ${Math.round(deficit).toLocaleString()}`,
           `Inbound Yield Coverage: ${Math.round((ctx.totalMonthlyIncome / ctx.burnRate.monthlyProjection) * 100)}% of monthly
burn.`,
          `Survival Window: ${ctx.runway.currentDays ? Math.round(ctx.runway.currentDays) : "Unknown"} days remaining.`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: false,
      };
    },
  },
  // 3. SOLVENCY PRESSURE (Critical/Warning Severity)
  {
    id: "solvency_pressure",
    priority: "high",
    condition: (ctx) =>
      !ctx.strategicAlignment.liquiditySufficiency.isSufficient,
    generate: (ctx) => {
      const { shortfall, message } =
        ctx.strategicAlignment.liquiditySufficiency;

      const isCritical =
        ctx.liquidity === 0 ? shortfall > 0 : shortfall / ctx.liquidity > 0.5;

      return {
        type: "warning",
        severity: isCritical ? "critical" : "warning",
        category: "solvency_pressure",
        timestamp: new Date().toISOString(),
        message:
          "Liquid reserves (MMF, SACCO Deposits) are insufficient to satisfy all critical strategic obligations.",
        supportingSignals: [
          message,
          `Combined Available Liquid Reserves: KSh ${Math.round(ctx.liquidity).toLocaleString()}`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: false,
      };
    },
  },

  // 4. OBJECTIVE STARVATION (Advisory Severity)
  {
    id: "objective_starvation",
    priority: "medium",
    condition: (ctx) =>
      ctx.strategicAlignment.objectiveStarvationSignals.length > 0,
    generate: (ctx) => ({
      type: "info",
      severity: "advisory",
      category: "objective_starvation",
      timestamp: new Date().toISOString(),
      message: `Strategic starvation detected: ${ctx.strategicAlignment.objectiveStarvationSignals[0]}`,
      supportingSignals:
        ctx.strategicAlignment.objectiveStarvationSignals.slice(1, 3),
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 5. STRATEGIC CONFLICT (Warning Severity)
  {
```

```
    id: "strategic_conflict",
    priority: "medium",
    condition: (ctx) =>
      ctx.strategicAlignment.strategicAllocationSignals.length > 0,
    generate: (ctx) => ({
      type: "warning",
      severity: "warning",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
      message: ctx.strategicAlignment.strategicAllocationSignals[0],
      supportingSignals: [
        `Alignment Pressure: ${ctx.strategicAlignment.alignmentPressure}/100`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 6. RUNWAY DEPLETION (Critical Severity)
  {
    id: "runway_critical",
    priority: "critical",
    condition: (ctx) => ctx.runway.status === "critical",
    generate: (ctx) => ({
      type: "warning",
      severity: "critical",
      category: "solvency_pressure",
      timestamp: new Date().toISOString(),
        message: `Operational runway has contracted to ${Math.round(ctx.runway.currentDays || 0)} days. Immediate capital
preservation (minimizing M-Pesa leakage) required.`,
      supportingSignals: [
            `Runway Delta: ${ctx.runway.deltaDays >= 0 ? "+" : ""}${Math.round(ctx.runway.deltaDays)} days since last
evaluation.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 7. LEAKAGE CRISIS (Critical Severity)
  {
    id: "efficiency_crisis",
    priority: "high",
    condition: (ctx) => ctx.capitalEfficiencyScore < 40,
    generate: (ctx) => {
      const isLeakage = ctx.allocation.dominantCategory === "Leakage";
      return {
        type: "warning",
        severity: "critical",
        category: "capital_efficiency",
        timestamp: new Date().toISOString(),
        message: isLeakage
              ? "Capital efficiency crisis detected. Excessive 'Mobile Money (M-Pesa) Leakage' is compromising long-term
sustainability."
            : "Capital efficiency has reached a critical threshold. Current deployment patterns (Chama, MMF, Shamba) are
suboptimal.",
        supportingSignals: [
                    `Efficiency Index: ${ctx.capitalEfficiencyScore}/100 ? Dominant: ${ctx.allocation.dominantCategory}
(${(ctx.allocation.categoryDistribution[ctx.allocation.dominantCategory] * 100).toFixed(1)}%).`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: false,
```

```
      };
    },
  },

  // 8. UNCLASSIFIED DATA (Advisory Severity)
  {
    id: "unclassified_data",
    priority: "medium",
    condition: (ctx) =>
      (ctx.allocation.categoryDistribution["Unclassified"] || 0) > 0.1, // Only alert if > 10%
    generate: (ctx) => ({
      type: "info",
      severity: "advisory",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
      message:
        "Unclassified outflows detected. Reclassify recent M-Pesa flows to restore the accuracy of the efficiency index.",
      supportingSignals: [
          `Metadata Integrity: ${Math.round((ctx.allocation.categoryDistribution["Unclassified"] || 0) * 100)}% of flows lack
strategic classification.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 9. ERRATIC VOLATILITY (Warning Severity)
  {
    id: "erratic_volatility",
    priority: "medium",
    condition: (ctx) => ctx.volatilityScore > 0.4,
    generate: (ctx) => ({
      type: "warning",
      severity: "warning",
      category: "capital_efficiency",
      timestamp: new Date().toISOString(),
      message:
        "High volatility in capital deployment detected. Irregular outbound M-Pesa flows may disrupt runway projections.",
      supportingSignals: [
            `Irregularity Index: HIGH ? Volatility score of ${(ctx.volatilityScore * 100).toFixed(1)}% exceeds variance
threshold.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 10. HIGH DISCIPLINE (Observation Severity)
  {
    id: "high_discipline",
    priority: "medium",
    condition: (ctx) => ctx.volatilityScore < 0.2 && ctx.deploymentCount > 1,
    generate: (ctx) => {
      const isAsset = ctx.allocation.dominantCategory === "Asset";
      return {
        type: "info",
        severity: "observation",
        category: "capital_efficiency",
        timestamp: new Date().toISOString(),
        message: isAsset
          ? "High operational discipline in Capital Deployment (Chama, MMF, Shamba) detected. Consistent deployment provides a
stable foundation for scaling."
            : "High operational discipline detected. Consistent outflow patterns provide a stable foundation for capital
```

```
scaling.",
        supportingSignals: [
              `Stability Metric: ${(100 - ctx.volatilityScore * 100).toFixed(1)}% consistency across ${ctx.deploymentCount}
deployments.`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: true,
      };
    },
  },

  // 11. HEAVY CONCENTRATION (Advisory Severity)
  {
    id: "heavy_concentration",
    priority: "low",
    condition: (ctx) => ctx.allocation.concentrationScore > 0.7,
    generate: (ctx) => ({
      type: "info",
      severity: "advisory",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
         message: `Significant capital concentration detected in ${ctx.allocation.dominantCategory}. Ensure this deployment
(Chama, MMF, Shamba) aligns with strategic intent.`,
      supportingSignals: [
         `Concentration Score: ${ctx.allocation.concentrationScore.toFixed(2)} ? ${ctx.allocation.dominantCategory} represents
${(ctx.allocation.categoryDistribution[ctx.allocation.dominantCategory] * 100).toFixed(1)}% of total deployment.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 12. DEFAULT PATTERN (Legible Silence - Observation)
  {
    id: "default_pattern",
    priority: "low",
    condition: () => true,
    generate: (ctx) => ({
      type: "info",
      severity: "observation",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
      message:
        "No material behavioral shifts detected in M-Pesa flows or capital deployment. Silence is intentional.",
      supportingSignals: [
        "Observing capital behavior: State remains within normal variance parameters.",
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: true,
    }),
  },

  // 13. INCOME CONCENTRATION RISK (Warning Severity)
  {
    id: "income_concentration_risk",
    priority: "medium",
    condition: (ctx) => ctx.maxIncomeConcentrationRatio > 0.8,
    generate: (ctx) => ({
      type: "warning",
      severity: "warning",
      category: "capital_efficiency",
      timestamp: new Date().toISOString(),
```

```
      message:
         "High inbound concentration risk detected. Over 80% of total yield relies on a single primary flow, creating severe
structural fragility.",
      supportingSignals: [
        `Concentration Ratio: ${(ctx.maxIncomeConcentrationRatio * 100).toFixed(1)}% from primary source.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },
];
```

FILE: src\lib\services\snapshot.ts

```
import { SupabaseClient } from "@supabase/supabase-js";
import { getDeployments } from "../db/deployments";
import { getAccounts } from "../db/accounts";
import { getLiabilities } from "../db/liabilities";
import { getIncomeStreams } from "../db/income";
import { getGoals } from "../db/goals";
import { getStrategicObjectives } from "../db/objectives";
import { getOperationalBaseline } from "../db/baseline";
import { getUserSettings } from "../db/settings";
import { getInsights, saveInsight } from "../db/insights";
import { generateSummary } from "../analytics";
import { generateKairosAIInsight, type KairosInsight } from "../ai/kairos";
import { DashboardSnapshot } from "../dashboard/types";
import {
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
} from "../analytics/types";

export interface SnapshotQueryResult {
  snapshot: DashboardSnapshot;
  volatileState: {
    requiresPersistence: boolean;
  };
}

export const SnapshotService = {
  /**
   * Orchestrates the construction of a complete DashboardSnapshot.
   * Gather data, computes analytics, evaluates AI insights.
   * STRICT CQRS: This is a pure query. No database writes allowed.
   */
  async getSnapshot(
    supabase: SupabaseClient,
    userId: string,
    options: { forceInsightEvaluation?: boolean } = {},
  ): Promise<SnapshotQueryResult> {
    console.log("GENERATING SNAPSHOT FOR USER:", userId);

    const [
      deploymentsData,
      accountsData,
      liabilitiesData,
      incomeStreamsData,
```

```
  goalsData,
  objectivesData,
  baselineData,
  settings,
] = await Promise.all([
  getDeployments(supabase),
  getAccounts(supabase),
  getLiabilities(supabase),
  getIncomeStreams(supabase),
  getGoals(supabase),
  getStrategicObjectives(supabase),
  getOperationalBaseline(supabase),
  getUserSettings(supabase, userId),
]);

const deployments = (deploymentsData || []) as Deployment[];
const accounts = (accountsData || []) as Account[];
const liabilities = (liabilitiesData || []) as Liability[];
const incomeStreams = (incomeStreamsData || []) as IncomeStream[];
const goals = (goalsData || []) as FinancialGoal[];
const objectives = (objectivesData || []) as StrategicObjective[];
const baseline = (baselineData || []) as OperationalBaseline[];
const liquidity = Number(settings?.total_liquidity || 0);

const analytics = generateSummary(
  deployments,
  liquidity,
  accounts,
  liabilities,
  incomeStreams,
  goals,
  objectives,
  baseline,
);

const savedInsights = await getInsights(supabase, 1);
const previousInsight =
  savedInsights && savedInsights.length > 0
    ? normalizeSavedInsight(savedInsights[0] as Record<string, unknown>)
    : null;

let kairosInsight: KairosInsight | null = null;
let requiresPersistence = false;

if (!options.forceInsightEvaluation && previousInsight) {
  // Re-use existing signal if no fresh evaluation is requested
  kairosInsight = previousInsight;
} else {
  // Trigger Kairos Strategic Evaluation (Volatile)
  kairosInsight = await generateKairosAIInsight({
    deployments,
    liquidity,
    previousInsight,
    objectives,
    accounts,
    liabilities,
    incomeStreams,
    goals,
    baseline,
  });

  requiresPersistence = Boolean(kairosInsight.is_new_signal);
}

return {
```

```
      snapshot: {
        authenticated: true,
        deployments,
        accounts,
        liabilities,
        incomeStreams,
        goals,
        objectives,
        baseline,
        analytics,
        liquidity,
        kairosInsight,
      },
      volatileState: {
        requiresPersistence,
      },
    };
  },

  /**
   * Produces an empty snapshot for unauthenticated sessions.
   */
  unauthenticated(): DashboardSnapshot {
    return {
      authenticated: false,
      deployments: [],
      accounts: [],
      liabilities: [],
      incomeStreams: [],
      goals: [],
      objectives: [],
      baseline: [],
      analytics: null,
      liquidity: 0,
      kairosInsight: null,
    };
  },
};

/**
 * Normalizes database insight rows into KairosInsight domain objects.
 */
function normalizeSavedInsight(row: Record<string, unknown>): KairosInsight {
  const metadata =
    row.metadata && typeof row.metadata === "object"
      ? (row.metadata as Record<string, unknown>)
      : {};

  return {
    type: row.type as KairosInsight["type"],
    severity: (metadata.severity as KairosInsight["severity"]) || "observation",
    category: row.category as KairosInsight["category"],
    message: String(row.message || ""),
    supportingSignals: Array.isArray(metadata.supporting_signals)
      ? (metadata.supporting_signals as string[])
      : metadata.supporting_signal
        ? [String(metadata.supporting_signal)]
        : [],
    timestamp: String(
      metadata.timestamp || row.created_at || new Date().toISOString(),
    ),
    runway: metadata.runway !== undefined ? Number(metadata.runway) : null,
    capitalEfficiency:
      metadata.capital_efficiency !== undefined
        ? Number(metadata.capital_efficiency)
```

```
        : 100,
    isSilent: Boolean(metadata.is_silent),
    metadataQuality:
      metadata.metadata_quality && typeof metadata.metadata_quality === "object"
        ? (metadata.metadata_quality as KairosInsight["metadataQuality"])
        : undefined,
    is_new_signal: false,
  };
}
```

FILE: src\lib\supabase\client.ts

```
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

FILE: src\lib\supabase\middleware.ts

```
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({
    request,
  });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value),
          );
          supabaseResponse = NextResponse.next({
            request,
          });
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options),
          );
        },
      },
    },
  );

  // refreshing the auth token
  await supabase.auth.getUser();

  return supabaseResponse;
}
```

Axiom Repository Audit

FILE: src\lib\supabase\server.ts

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch (error) {
            // This is expected if calling from a Server Component that is already streaming.
            // We log it only for debugging in dev mode.
            if (process.env.NODE_ENV === "development") {
              console.warn(
                "Supabase SSR: Unable to set cookies from Server Component. Ensure middleware is configured.",
                error,
              );
            }
          }
        },
      },
    },
  );
}
```

FILE: src\lib\utils\formatters.ts

```
/**
 * Formatting utilities for the Axiom presentation layer.
 * Localized for the Kenyan market (KES / en-KE).
 */

export const formatCurrency = (amount: number): string => {
  return new Intl.NumberFormat("en-KE", {
    style: "currency",
    currency: "KES",
    maximumFractionDigits: 0,
  }).format(amount);
};

export const formatNumber = (num: number): string => {
  return new Intl.NumberFormat("en-KE").format(num);
};

export const formatDate = (dateString: string): string => {
  return new Date(dateString).toLocaleDateString("en-KE", {
    year: "numeric",
    month: "short",
    day: "numeric",
  });
```

```
};
```

FILE: src\lib\utils\taxonomy.ts

```
import { AccountType } from "../finance/accounts";
import { LiabilityType } from "../finance/liabilities";
import { IncomeType } from "../finance/income";
import { ValidCategory } from "../finance/taxonomy";

/**
 * TAXONOMY MAPPING DICTIONARY
 * Localizes abstract database enums to Kenyan market realities.
 */

export const AccountMap: Record<AccountType, string> = {
  checking: "M-Pesa / Bank",
  savings: "Savings / MMF",
  mobile_money: "M-Pesa",
  brokerage: "Investment / CDS",
  crypto: "Digital Assets",
  cash: "Cash on Hand",
};

export const LiabilityMap: Record<LiabilityType, string> = {
  credit_card: "Credit Card / Digital Loan",
  mortgage: "Mortgage / Shamba Loan",
  personal_loan: "Fuliza / SACCO Loan",
  student_loan: "HELB / Education Loan",
  business_loan: "Business / SME Loan",
  line_of_credit: "Overdraft facility",
  other: "Other Credit Facilities",
};

export const IncomeMap: Record<IncomeType, string> = {
  salary: "Salary / Hustle",
  freelance: "Freelance / Gig Work",
  business: "Business Yield",
  dividends: "Dividends / Chama Payout",
  rental: "Rental Income",
  contract: "Contract Fee",
  other: "Other Inflows",
};

export const DeploymentMap: Record<ValidCategory, string> = {
  Asset: "Investments",
  Skill: "Growth",
  Leverage: "Efficiency",
  Experience: "Lifestyle",
  Maintenance: "Essentials",
  Leakage: "Waste",
};
```

FILE: scripts\marketing\instagram\generate_carousel.py

```
import os

from PIL import Image, ImageDraw, ImageFont

# Constants for Instagram Carousel (Portrait - Maximum Screen Real Estate)
WIDTH, HEIGHT = 1080, 1350
BG_COLOR = "#050505"  # Pure deep black
```

```
TEXT_COLOR = "#FFFFFF"  # Stark white
SECONDARY_TEXT = "#666666"  # Muted gray for the footer


class CarouselGenerator:
    def __init__(self, output_dir="output"):
        self.output_dir = output_dir
        script_dir = os.path.dirname(os.path.abspath(__file__))
        self.abs_output_dir = os.path.join(script_dir, output_dir)
        os.makedirs(self.abs_output_dir, exist_ok=True)

        # Try to find a system font (Windows specific)
        self.font_path_bold = "C:\\Windows\\Fonts\\arialbd.ttf"
        self.font_path_reg = "C:\\Windows\\Fonts\\arial.ttf"

        if not os.path.exists(self.font_path_bold):
            self.font_path_bold = None
            self.font_path_reg = None

    def get_font(self, size, bold=False):
        path = self.font_path_bold if bold else self.font_path_reg
        try:
            if path:
                return ImageFont.truetype(path, size)
        except Exception:
            pass
        return ImageFont.load_default()

    def wrap_text(self, text, font, max_width):
        lines = []
        words = text.split()
        while words:
            line = ""
            while words and font.getlength(line + words[0]) <= max_width:
                line += words.pop(0) + " "
            lines.append(line.strip())
        return lines

    def create_slide(self, index, text, slide_count):
        img = Image.new("RGB", (WIDTH, HEIGHT), color=BG_COLOR)
        draw = ImageDraw.Draw(img)

        # Fonts - MASSIVELY scaled up for the "Wealth" aesthetic
        main_font = self.get_font(84, bold=True)
        footer_font = self.get_font(28)

        # Text Wrapping (Centered)
        max_text_width = WIDTH - 200  # Leave 100px breathing room on each side
        wrapped_text = self.wrap_text(text, main_font, max_text_width)

        # Calculate total height of the text block to center it vertically
        line_heights = [
            draw.textbbox((0, 0), line, font=main_font)[3] for line in wrapped_text
        ]
        total_text_height = sum(line_heights) + (len(wrapped_text) - 1) * 30

        y_cursor = (HEIGHT - total_text_height) // 2

        # Draw Centered Text
        for line in wrapped_text:
            bbox = draw.textbbox((0, 0), line, font=main_font)
            line_width = bbox[2] - bbox[0]
            x_cursor = (WIDTH - line_width) // 2  # Center horizontally

            draw.text((x_cursor, y_cursor), line, font=main_font, fill=TEXT_COLOR)
```

```
            y_cursor += line_heights[0] + 30

        # Draw Footer / Branding (Centered at bottom)
        footer_text = "? X I O M"
        f_bbox = draw.textbbox((0, 0), footer_text, font=footer_font)
        draw.text(
            ((WIDTH - (f_bbox[2] - f_bbox[0])) // 2, HEIGHT - 120),
            footer_text,
            font=footer_font,
            fill=SECONDARY_TEXT,
        )

        # Page Indicator (Bottom right)
        if index + 1 < slide_count:
            indicator = "Swipe ?"
        else:
            indicator = "Link in bio."

        i_bbox = draw.textbbox((0, 0), indicator, font=footer_font)
        draw.text(
            (WIDTH - 100 - (i_bbox[2] - i_bbox[0]), HEIGHT - 120),
            indicator,
            font=footer_font,
            fill=SECONDARY_TEXT,
        )

        # Save
        filename = f"slide_{index + 1}.png"
        save_path = os.path.join(self.abs_output_dir, filename)
        img.save(save_path)
        print(f"Generated: {save_path}")


def main():
    # The "Wealth" Page Copy Strategy: Short, punchy, aggressive.
    slides = [
        "Stop tracking your expenses. It's keeping you broke.",
        "Tracking is a history lesson. It only tells you where your money went after it's too late.",
        "The wealthy don't track. They allocate.",
        "Every shilling you spend is either an Asset, a Skill, Leverage, or Leakage.",
        "Stop guessing. Establish your Day Zero financial baseline today.",
    ]

    generator = CarouselGenerator()
    for i, text in enumerate(slides):
        generator.create_slide(i, text, len(slides))


if __name__ == "__main__":
    main()
```

FILE: scripts\marketing\instagram\generate_carousel_api.py

```
import os
from io import BytesIO

import requests
from PIL import Image, ImageDraw, ImageEnhance, ImageFont


# Load environment variables (manual simple load)
def load_env():
    env_path = os.path.join(os.path.dirname(__file__), ".env")
```

```
    if os.path.exists(env_path):
        with open(env_path, "r") as f:
            for line in f:
                if "=" in line:
                    key, value = line.strip().split("=", 1)
                    os.environ[key] = value.strip('"').strip("'")


load_env()

# Constants
WIDTH, HEIGHT = 1080, 1080
MARGIN = 80
BG_COLOR = "#0a0a0a"
TEXT_COLOR = "#ededed"
ACCENT_COLOR = "#3b82f6"
SECONDARY_TEXT = "#999999"
UNSPLASH_ACCESS_KEY = os.getenv("UNSPLASH_ACCESS_KEY", "your_key_here")


class CarouselGeneratorAPI:
    def __init__(self, output_dir="output"):
        script_dir = os.path.dirname(os.path.abspath(__file__))
        self.abs_output_dir = os.path.join(script_dir, output_dir)
        os.makedirs(self.abs_output_dir, exist_ok=True)

        # Fonts
        self.font_path_bold = "C:\\Windows\\Fonts\\arialbd.ttf"
        self.font_path_reg = "C:\\Windows\\Fonts\\arial.ttf"

    def get_font(self, size, bold=False):
        path = self.font_path_bold if bold else self.font_path_reg
        try:
            if path and os.path.exists(path):
                return ImageFont.truetype(path, size)
        except Exception:
            pass
        return ImageFont.load_default()

    # 1. NEW: Add the text wrapping logic back
    def wrap_text(self, text, font, max_width):
        lines = []
        words = text.split()
        while words:
            line = ""
            while words and font.getlength(line + words[0]) <= max_width:
                line += words.pop(0) + " "
            lines.append(line.strip())
        return lines

    def fetch_unsplash_background(self, query):
        print(f"Fetching background for: {query}")
        if UNSPLASH_ACCESS_KEY == "your_key_here":
            print("Warning: Unsplash API key not set. Using fallback color.")
            return None

        url = f"https://api.unsplash.com/photos/random?query={query}&orientation=squarish&client_id={UNSPLASH_ACCESS_KEY}"
        try:
            response = requests.get(url, timeout=10)
            data = response.json()
            image_url = data["urls"]["regular"]
            img_response = requests.get(image_url)
            img = Image.open(BytesIO(img_response.content))
            return img.resize((WIDTH, HEIGHT))
        except Exception as e:
```

```
        print(f"Error fetching image: {e}")
        return None

def create_slide(self, index, text, slide_count, bg_image=None):
    img = (
        bg_image.copy()
        if bg_image
        else Image.new("RGB", (WIDTH, HEIGHT), color=BG_COLOR)
    )

    # Darken image for readability
    enhancer = ImageEnhance.Brightness(img)
    img = enhancer.enhance(0.4)

    draw = ImageDraw.Draw(img)

    # Fonts
    title_font = self.get_font(72, bold=True)
    footer_font = self.get_font(24)

    # 2. UPDATED: Handle both manual \n and automatic width wrapping
    # Replace explicit escaped newlines just in case they come through as raw strings
    raw_lines = text.replace("\\n", "\n").split("\n")
    final_lines = []
    max_text_width = WIDTH - (MARGIN * 2)  # Leave 80px breathing room on each side

    for raw_line in raw_lines:
        wrapped = self.wrap_text(raw_line, title_font, max_text_width)
        final_lines.extend(wrapped)

    # Dynamically calculate Y cursor so the whole block stays perfectly centered
    line_spacing = 90
    total_height = len(final_lines) * line_spacing
    y_cursor = (HEIGHT - total_height) // 2

    for line in final_lines:
        w = title_font.getlength(line)
        draw.text(
            ((WIDTH - w) // 2, y_cursor), line, font=title_font, fill=TEXT_COLOR
        )
        y_cursor += line_spacing

    # Branding
    footer_text = "? X I O M"  # Updated to match your high-end brand mark
    draw.text(
        (MARGIN, HEIGHT - MARGIN),
        footer_text,
        font=footer_font,
        fill=SECONDARY_TEXT,
    )

    indicator = f"{index + 1} / {slide_count}"
    draw.text(
        (WIDTH - MARGIN - 60, HEIGHT - MARGIN),
        indicator,
        font=footer_font,
        fill=TEXT_COLOR,
    )

    filename = f"api_slide_{index + 1}.png"
    save_path = os.path.join(self.abs_output_dir, filename)
    img.save(save_path)
    print(f"Generated: {save_path}")
```

```
def main():
    # CONFIGURATION
      THEME = "black command center, holographic financial telemetry, glowing dashboards, dark luxury technology, cinematic shadows"
    SLIDES = [
        "1: Most people do not manage money.\\nThey manage memories.",
        "2: Ask someone where their capital went last month.\\nYou will receive a story.\\nNot data.",
        "3: Without telemetry, patterns remain invisible.\\nInvisible patterns become expensive mistakes.",
        "4: Axiom transforms transactions into behavioral intelligence.\\nEvery deployment becomes observable.",
        "5: Stop guessing.\\nStart measuring.\\nEstablish your Day Zero Baseline.\\nLink in bio.",
    ]

    generator = CarouselGeneratorAPI()

    for i, slide_text in enumerate(SLIDES):
        # Fetch a unique background for every slide
        bg = generator.fetch_unsplash_background(THEME)

        # Remove the leading "X: " prefix if present
        clean_text = slide_text.split(": ", 1)[-1] if ": " in slide_text else slide_text
        generator.create_slide(i, clean_text, len(SLIDES), bg)


if __name__ == "__main__":
    main()
```

FILE: docs\VOICE_AND_TONE.md

```
# Axiom: Voice & Tone Guide

**Core Ethos:** Precise, calm, strategic, trustworthy.
**Objective:** Axiom is not a budgeting app. It is a financial operating system. We do not restrict or scold; we illuminate and project. We replace anxiety with clarity through the delivery of hard figures.

## 1. Brand Voice Principles

*   **Radically Calm:** Money causes anxiety. Our interface must be the antidote. We never use exclamation points. We do not use alarmist colors or language. We state facts plainly.
*    **High-Resolution, Low-Frequency:** We respect the user's attention. We don't flood them with trivial alerts. When we speak, it is because we have synthesized something meaningful. Every word must earn its place.
*    **Strategic, Not Tactical:** We do not tell users to "skip the latte." We talk about capital deployment, systemic structures, and horizon expansion. We treat the user like a CEO managing a treasury.
*    **Deterministic Authority:** We do not guess. If we project a number, it is based on unshakeable math. We use language that conveys structural integrity.
*   **Neutral Truth:** A shortened runway is not a failure; it is simply a state change requiring a strategic adjustment. We strip emotion from financial reality.

## 2. Words We Use

*   **The Foundation:** (The structural minimum required to operate; living expenses).
*   **The Horizon:** (How far the current capital pool extends; runway).
*   **Inflows / Replenishment:** (Income).
*   **Commitments:** (Debt, liabilities, unavoidable future outflows).
*   **Capital Deployment:** (Spending, investing, or allocating money).
*   **Strategic Objective:** (A long-term financial goal).
*   **Hard Figures:** (Confirmed, deterministic data).
*   **Verified / Processing / Staged:** (Transaction statuses).
*   **The Source:** (The origin of truth, connected institutions).
*   **Synthesis / Insight:** (AI analysis).
*   **Align / Recalibrate:** (Adjusting behaviors or plans).

## 3. Words We Avoid
```

*   **Budget / Budgeting:** (Feels restrictive, guilt-inducing, and small).
*   **Expense / Spending:** (Implies loss; we prefer "Deployment" or "Outflows").
*   **Hustle / Grind:** (Hype language; we focus on structure, not burnout).
*   **Danger / Warning / Urgent:** (Fear-based; we use "Advisory," "Attention required," or "Recalibration needed").
*   **Good job! / Congrats!:** (Patronizing; we use "Objective secured" or "Alignment confirmed").
*   **Guess / Estimate / Probably:** (Undermines our deterministic authority).
*   **Oops / Uh oh:** (Too casual for a financial OS).

4. Dashboard Tone Examples

*   **Current State:**
    *   *Avoid:* "Here's how much you spent this month!"
    *   *Axiom:* "Current capital deployment versus historical baseline."
*   **Runway Projection:**
    *   *Avoid:* "You only have 3 months of cash left!"
    *   *Axiom:* "Current horizon: 3.2 months. A recalibration of The Foundation is recommended."
*   **Net Worth:**
    *   *Avoid:* "Your total wealth is looking great."
    *   *Axiom:* "Verified net position."

5. Notification Tone Examples

*   **Large Outflow Detected:**
    *   *Avoid:* "Whoa, big spender! You just dropped $2,000."
    *   *Axiom:* "Significant capital deployment noted. Horizon adjusted by -14 days."
*   **Goal Reached:**
    *   *Avoid:* "Congratulations! You hit your savings goal! ?"
    *   *Axiom:* "Strategic Objective secured. Capital is now staged for the next allocation."
*   **Upcoming Liability:**
    *   *Avoid:* "Don't forget, your credit card bill is due tomorrow!"
    *   *Axiom:* "Scheduled Commitment: $1,450 processing tomorrow."

6. Empty State Examples

*   **No Connected Accounts:**
    *   *Avoid:* "Looks empty here! Connect an account to get started."
    *   *Axiom:* "The Ledger requires data. Connect your institutions to establish The Source."
*   **No Objectives Set:**
    *   *Avoid:* "You haven't set any goals yet. What are you saving for?"
    *   *Axiom:* "No Strategic Objectives defined. Establish a target to align your capital flow."
*   **No Insights Available:**
    *   *Avoid:* "We're still crunching the numbers..."
    *   *Axiom:* "Gathering baseline telemetry. Insights will synthesize once sufficient data is verified."

7. Error State Examples

*   **Sync Failure:**
    *   *Avoid:* "Oops! We couldn't connect to your bank. Please try again."
    *   *Axiom:* "Connection to The Source interrupted. Integrity at 98%. Re-authentication required for Chase."
*   **Insufficient Funds for Allocation:**
    *   *Avoid:* "You don't have enough money to make this transfer!"
    *   *Axiom:* "Deployment exceeds available staging capital. Review inflows before proceeding."
*   **System Error:**
    *   *Avoid:* "Something went wrong on our end. Sorry!"
    *   *Axiom:* "System interruption. Hard Figures may be temporarily out of sync. Service will restore shortly."

8. AI Insight Tone Examples (Kairos Intelligence)

*   **Detecting Lifestyle Creep:**
    *   *Avoid:* "You've been spending way too much on dining out lately."
    *   *Axiom:* "Observation: The Foundation has expanded by 12% over the last quarter, primarily driven by variable outflows. This structural shift reduces your long-term horizon."
*   **Suggesting Reallocation:**
    *   *Avoid:* "You have extra cash! You should put it in your investment account."
    *   *Axiom:* "Synthesis: Inflows have consistently exceeded Foundation requirements for 60 days. Consider deploying

```
surplus capital to your primary investment objective."
*   **Addressing Runway Depletion (Solvency Alert):**
    *   *Avoid:* "Emergency: You're running out of money fast!"
     *    *Axiom:* "Advisory: Current burn rate exceeds replenishment. Without intervention, The Horizon will deplete in 45 days. Immediate recalibration of variable outflows is advised."

## 9. Landing Page Headline Examples

*   *Avoid:* "Take control of your budget today."
*   *Axiom:* "The Math of Wealth."
*   *Avoid:* "The smartest way to track your money."
*   *Axiom:* "Precision wealth management for the disciplined."
*   *Avoid:* "Never worry about your finances again."
*   *Axiom:* "Financial certainty. Hard figures only."
*   *Avoid:* "AI financial advisor in your pocket."
*   *Axiom:* "Calculated intelligence. At the exact right moment."

## 10. Onboarding Copy Examples

*   **Welcome Screen:**
    *   *Avoid:* "Welcome to Axiom! Let's get your budget set up."
    *   *Axiom:* "Welcome to Axiom. Let?s establish your financial foundation."
*   **Connecting Accounts:**
    *   *Avoid:* "Link your bank accounts so we can track your spending."
    *   *Axiom:* "Secure The Source. Connect your institutions to verify your financial reality."
*   **Setting the Baseline:**
    *   *Avoid:* "Tell us what your monthly bills are."
    *   *Axiom:* "Define your Commitments. This establishes your structural burn rate."
*   **Completion:**
    *   *Avoid:* "All done! You're ready to start saving."
    *   *Axiom:* "Telemetry active. Kairos Intelligence is now monitoring your horizon."
```

FILE: supabase\testing-scenarios.sql

```
-- KAIROS STRATEGIC INTERPRETATION LAYER ? COMPREHENSIVE TESTING SUITE (V1.1)
-- IDENTITY: Restrained Operational Intelligence
-- SHARED TESTING UUID: 453007ef-8eba-4f6a-8d86-81445fc0af3d

-- ======================================================================
-- BLOCK 0: TOTAL SYSTEM RESET
-- Use this to restore the system to a Day Zero state.
-- ======================================================================

DELETE FROM public.deployments WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';
DELETE FROM public.strategic_objectives WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';
DELETE FROM public.financial_goals WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';
DELETE FROM public.accounts WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';
DELETE FROM public.liabilities WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';
DELETE FROM public.income_streams WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';
DELETE FROM public.operational_baseline WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';
DELETE FROM public.kairos_insights WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';
DELETE FROM public.user_settings WHERE user_id = '453007ef-8eba-4f6a-8d86-81445fc0af3d';


-- ======================================================================
-- SCENARIO A: THE PRIMITIVE STATE (Day Zero Onboarding)
-- Context: No forensic data. Silence is intentional.
-- Triggers: 'default_pattern' insight.
-- ======================================================================

-- No data required.
```

```
-- ====================================================================
-- SCENARIO B: THE IDEAL STATE (Discipline & Strategic Growth)
-- Context: High liquidity, zero debt, asset-focused deployments, stable burn.
-- Triggers: 'high_discipline', 'default_pattern'.
-- ====================================================================

INSERT INTO public.user_settings (user_id, total_liquidity)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 1500000)
ON CONFLICT (user_id) DO UPDATE SET total_liquidity = 1500000;

INSERT INTO public.accounts (user_id, account_name, account_type, current_balance, institution)
VALUES
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Standard Chartered Current', 'checking', 500000, 'Stanchart'),
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'I&M Wealth Management', 'savings', 1000000, 'I&M Bank');

INSERT INTO public.income_streams (user_id, income_name, income_type, amount, cadence, is_recurring, source)
VALUES
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Principal Consultancy', 'salary', 450000, 'monthly', true, 'Axiom Labs'),
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Equity Dividends', 'dividends', 25000, 'quarterly', true, 'NSE');

INSERT INTO public.operational_baseline (user_id, title, baseline_type, amount, cadence, is_active, category)
VALUES
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Kilimani Apartment', 'expense', 120000, 'monthly', true, 'Maintenance'),
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Z-Fund Allocation', 'allocation', 50000, 'monthly', true, 'Asset');

INSERT INTO public.deployments (user_id, title, amount, category, created_at)
VALUES
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Weekly MMF Allocation', 15000, 'Asset', now() - interval '7 days'),
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Weekly MMF Allocation', 15000, 'Asset', now() - interval '14 days'),
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Weekly MMF Allocation', 15000, 'Asset', now() - interval '21 days');

INSERT INTO public.strategic_objectives (user_id, objective_name, objective_type, target_amount, current_amount,
priority_level, status)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Liquidity Buffer', 'liquidity', 2000000, 1500000, 'moderate', 'active');


-- ====================================================================
-- SCENARIO C: THE FRAGILE STATE (Concentration & Volatility)
-- Context: High inbound concentration risk, erratic spending, high leakage.
-- Triggers: 'income_concentration_risk', 'erratic_volatility', 'heavy_concentration', 'unclassified_data'.
-- ====================================================================

-- First run BLOCK 0.
INSERT INTO public.user_settings (user_id, total_liquidity) VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 50000);

-- 90% of income from one source
INSERT INTO public.income_streams (user_id, income_name, income_type, amount, cadence, is_recurring)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Single Major Hustle', 'freelance', 90000, 'monthly', true);

-- Erratic and unclassified deployments
INSERT INTO public.deployments (user_id, title, amount, category)
VALUES
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Random Purchase A', 45000, 'Leakage'),
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Stuff', 5000, 'Leakage'), -- Changed to Leakage for discipline impact
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Misc', 12000, 'Leakage'), -- Changed to Leakage for discipline impact
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Major Impulse', 85000, 'Leakage');


-- ====================================================================
-- SCENARIO D: THE SURVIVAL STATE (Solvency Crisis & Structural Deficit)
-- Context: Negative net worth, monthly burn > income, runway < 30 days.
-- Triggers: 'solvency_crisis', 'structural_deficit', 'runway_critical'.
-- ====================================================================

-- First run BLOCK 0.
```

```
INSERT INTO public.user_settings (user_id, total_liquidity) VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 10000);

INSERT INTO public.accounts (user_id, account_name, account_type, current_balance)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Equity Bank (Overdrawn)', 'checking', -5000);

INSERT INTO public.liabilities (user_id, liability_name, liability_type, outstanding_balance, interest_rate, is_paid_in_cadences, cadence, cadence_amount)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'High-Interest Sacco Loan', 'personal_loan', 750000, 15.00, true, 'monthly', 45000);

INSERT INTO public.income_streams (user_id, income_name, income_type, amount, cadence, is_recurring)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Part-time Gig', 'freelance', 40000, 'monthly', true);

INSERT INTO public.operational_baseline (user_id, title, baseline_type, amount, cadence, is_active, category)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Basic Survival', 'expense', 65000, 'monthly', true, 'Maintenance');


-- ======================================================================
-- SCENARIO E: THE STRATEGIC DRIFT (Conflict & Starvation)
-- Context: Funding high leakage during accumulation, neglected critical goals.
-- Triggers: 'strategic_conflict', 'objective_starvation', 'solvency_pressure', 'efficiency_crisis'.
-- ======================================================================

-- First run BLOCK 0.
INSERT INTO public.user_settings (user_id, total_liquidity) VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 100000);

-- Neglected objective (created 120 days ago, low progress)
INSERT INTO public.strategic_objectives (user_id, objective_name, objective_type, target_amount, current_amount, priority_level, status, created_at)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Emergency Reserve Alpha', 'emergency_reserve', 500000, 25000, 'critical', 'active', now() - interval '120 days');

-- Strategic Conflict: Accumulation active, but spending on Leakage
INSERT INTO public.strategic_objectives (user_id, objective_name, objective_type, target_amount, current_amount, priority_level, status)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Active Accumulation', 'liquidity', 200000, 100000, 'moderate', 'active');

INSERT INTO public.deployments (user_id, title, amount, category)
VALUES
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Luxury Dinner', 15000, 'Leakage'),
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Impulse Online Shopping', 45000, 'Leakage'),
('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'MMF Deposit', 5000, 'Asset');


-- ======================================================================
-- SCENARIO F: THE OPERATIONAL VERIFICATION (Pending States)
-- Context: Multiple scheduled flows requiring verification today.
-- Triggers: 'pending_verification'.
-- ======================================================================

-- First run BLOCK 0.
INSERT INTO public.user_settings (user_id, total_liquidity) VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 200000);

-- Schedule an income stream for TODAY
INSERT INTO public.income_streams (user_id, income_name, income_type, amount, cadence, is_recurring, execution_day)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Pending Salary Inflow', 'salary', 150000, 'monthly', true, extract(day from now()));

-- Schedule a liability for TODAY
-- Note: Cadence_day_date is text. Use the name for weekly, or number for monthly.
INSERT INTO public.liabilities (user_id, liability_name, liability_type, outstanding_balance, is_paid_in_cadences, cadence, cadence_day_date, cadence_amount)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Loan Repayment Due', 'mortgage', 4500000, true, 'monthly', extract(day from now())::text, 55000);
```

```
-- Schedule a baseline for TODAY
INSERT INTO public.operational_baseline (user_id, title, baseline_type, amount, cadence, is_active, category, execution_day)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Daily Micro-Baseline', 'expense', 500, 'daily', true, 'Maintenance', null);


-- ======================================================================
-- SCENARIO G: FORENSIC INTEGRITY (Soft Deletes & Audit Trail)
-- Context: Testing historical record visibility and telemetry.
-- ======================================================================

-- First run BLOCK 0.
INSERT INTO public.deployments (user_id, title, amount, category, deleted_at)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Mistaken Transaction', 1000000, 'Leakage', now());

INSERT INTO public.kairos_insights (user_id, rule_id, message, severity, evaluation_time_ms)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'audit_test_rule', 'Forensic trace test.', 'observation', 45.5);


-- ======================================================================
-- SCENARIO H: THE WEALTH PRESERVATION STATE (High Liquidity & Reserve)
-- Context: Large asset base in 'Vault' with strategic reserve alignment.
-- Triggers: 'high_discipline', 'default_pattern'.
-- ======================================================================

-- First run BLOCK 0.
INSERT INTO public.user_settings (user_id, total_liquidity) VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 2000000);

INSERT INTO public.accounts (user_id, account_name, account_type, current_balance)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Vault', 'savings', 2000000);

INSERT INTO public.strategic_objectives (user_id, objective_name, objective_type, target_amount, current_amount,
priority_level, status)
VALUES ('453007ef-8eba-4f6a-8d86-81445fc0af3d', 'Base Reserve', 'liquidity', 500000, 450000, 'low', 'active');
```

FILE: supabase\migrations\20260511134054_enforce_deployment_context_constraints.sql

```
ALTER TABLE public.deployments
  ADD COLUMN IF NOT EXISTS advanced_context JSONB DEFAULT '{}'::jsonb;

UPDATE public.deployments
SET advanced_context = '{}'::jsonb
WHERE advanced_context IS NULL;

ALTER TABLE public.deployments
  ALTER COLUMN category DROP DEFAULT,
  ALTER COLUMN advanced_context SET DEFAULT '{}'::jsonb,
  ALTER COLUMN advanced_context SET NOT NULL;

ALTER TABLE public.deployments
  DROP CONSTRAINT IF EXISTS deployments_category_valid_check;

ALTER TABLE public.deployments
  ADD CONSTRAINT deployments_category_valid_check
  CHECK (
    category IS NOT NULL
    AND category IN ('Asset', 'Skill', 'Leverage', 'Experience', 'Leakage')
  )
  NOT VALID;

ALTER TABLE public.deployments
  DROP CONSTRAINT IF EXISTS deployments_advanced_context_shape_check;

ALTER TABLE public.deployments
```

```
  ADD CONSTRAINT deployments_advanced_context_shape_check
  CHECK (
    jsonb_typeof(advanced_context) = 'object'
    AND (
      NOT advanced_context ? 'associatedAccount'
      OR jsonb_typeof(advanced_context -> 'associatedAccount') = 'string'
    )
    AND (
      NOT advanced_context ? 'expectedReturnHorizon'
      OR advanced_context ->> 'expectedReturnHorizon' IN (
        'short-term',
        'medium-term',
        'long-term'
      )
    )
    AND (
      NOT advanced_context ? 'tags'
      OR jsonb_typeof(advanced_context -> 'tags') = 'array'
    )
  )
  NOT VALID;
```

FILE: supabase\migrations\20260512000000_financial_accounts_v1.sql

```
-- AXIOM FINANCIAL ACCOUNTS INFRASTRUCTURE (v1.0)
-- Establishing canonical containers of financial truth.

-- 1. Accounts Table
CREATE TABLE IF NOT EXISTS public.accounts (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    account_name TEXT NOT NULL,
    account_type TEXT NOT NULL,
    current_balance NUMERIC(15, 2) DEFAULT 0.00 NOT NULL,
    currency TEXT DEFAULT 'KSh' NOT NULL,
    institution TEXT,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    CONSTRAINT accounts_account_type_check CHECK (
        account_type IN ('checking', 'savings', 'mobile_money', 'brokerage', 'crypto', 'cash')
    )
);

-- 2. Deployment Linking
-- Add optional account_id to deployments for capital traceability
ALTER TABLE public.deployments ADD COLUMN IF NOT EXISTS account_id UUID REFERENCES public.accounts(id) ON DELETE SET NULL;

-- 3. Security Setup (RLS)
ALTER TABLE public.accounts ENABLE ROW LEVEL SECURITY;

-- Ensure users can only interact with their own accounts
DROP POLICY IF EXISTS "Users can view their own accounts" ON public.accounts;
DROP POLICY IF EXISTS "Users can insert their own accounts" ON public.accounts;
DROP POLICY IF EXISTS "Users can update their own accounts" ON public.accounts;
DROP POLICY IF EXISTS "Users can delete their own accounts" ON public.accounts;

CREATE POLICY "Users can view their own accounts" ON public.accounts FOR SELECT TO authenticated USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own accounts" ON public.accounts FOR INSERT TO authenticated WITH CHECK (auth.uid() =
user_id);
CREATE POLICY "Users can update their own accounts" ON public.accounts FOR UPDATE TO authenticated USING (auth.uid() =
user_id) WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete their own accounts" ON public.accounts FOR DELETE TO authenticated USING (auth.uid() =
user_id);
```

```
-- 4. Grants
GRANT ALL ON public.accounts TO authenticated;
GRANT ALL ON public.accounts TO service_role;

-- 5. Indexes
CREATE INDEX IF NOT EXISTS accounts_user_id_idx ON public.accounts(user_id);
CREATE INDEX IF NOT EXISTS deployments_account_id_idx ON public.deployments(account_id);
```

FILE: supabase\migrations\20260516000000_liability_system_v1.sql

```
-- AXIOM LIABILITY SYSTEM (v1.0)
-- Establishing deterministic awareness of financial obligations.

-- 1. Liabilities Table
CREATE TABLE IF NOT EXISTS public.liabilities (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    liability_name TEXT NOT NULL,
    liability_type TEXT NOT NULL,
    outstanding_balance NUMERIC(15, 2) DEFAULT 0.00 NOT NULL,
    interest_rate NUMERIC(5, 2) DEFAULT 0.00 NOT NULL,
    minimum_payment NUMERIC(15, 2) DEFAULT 0.00 NOT NULL,
    due_date DATE,
    currency TEXT DEFAULT 'KSh' NOT NULL,
    institution TEXT,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    CONSTRAINT liabilities_liability_type_check CHECK (
        liability_type IN (
            'credit_card',
            'mortgage',
            'personal_loan',
            'student_loan',
            'business_loan',
            'line_of_credit',
            'other'
        )
    )
);

-- 2. Security Setup (RLS)
ALTER TABLE public.liabilities ENABLE ROW LEVEL SECURITY;

-- Ensure users can only interact with their own liabilities
DROP POLICY IF EXISTS "Users can view their own liabilities" ON public.liabilities;
DROP POLICY IF EXISTS "Users can insert their own liabilities" ON public.liabilities;
DROP POLICY IF EXISTS "Users can update their own liabilities" ON public.liabilities;
DROP POLICY IF EXISTS "Users can delete their own liabilities" ON public.liabilities;

CREATE POLICY "Users can view their own liabilities" ON public.liabilities FOR SELECT TO authenticated USING (auth.uid() =
user_id);
CREATE POLICY "Users can insert their own liabilities" ON public.liabilities FOR INSERT TO authenticated WITH CHECK
(auth.uid() = user_id);
CREATE POLICY "Users can update their own liabilities" ON public.liabilities FOR UPDATE TO authenticated USING (auth.uid() =
user_id) WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete their own liabilities" ON public.liabilities FOR DELETE TO authenticated USING (auth.uid() =
user_id);

-- 3. Grants
GRANT ALL ON public.liabilities TO authenticated;
GRANT ALL ON public.liabilities TO service_role;
```

```
-- 4. Indexes
CREATE INDEX IF NOT EXISTS liabilities_user_id_idx ON public.liabilities(user_id);
CREATE INDEX IF NOT EXISTS liabilities_due_date_idx ON public.liabilities(due_date);
```

FILE: supabase\migrations\20260517000000_income_engine_v1.sql

```
-- AXIOM INCOME ENGINE (v1.0)
-- Establishing deterministic awareness of financial replenishment.

-- 1. Income Streams Table
CREATE TABLE IF NOT EXISTS public.income_streams (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    income_name TEXT NOT NULL,
    income_type TEXT NOT NULL,
    amount NUMERIC(15, 2) NOT NULL,
    cadence TEXT NOT NULL,
    is_recurring BOOLEAN DEFAULT true NOT NULL,
    currency TEXT DEFAULT 'KSh' NOT NULL,
    source TEXT,
    start_date DATE NOT NULL DEFAULT CURRENT_DATE,
    end_date DATE,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    CONSTRAINT income_streams_income_type_check CHECK (
        income_type IN (
            'salary',
            'freelance',
            'business',
            'dividends',
            'rental',
            'contract',
            'other'
        )
    ),
    CONSTRAINT income_streams_cadence_check CHECK (
        cadence IN (
            'weekly',
            'biweekly',
            'monthly',
            'quarterly',
            'annually',
            'irregular'
        )
    )
);

-- 2. Security Setup (RLS)
ALTER TABLE public.income_streams ENABLE ROW LEVEL SECURITY;

-- Ensure users can only interact with their own income streams
DROP POLICY IF EXISTS "Users can view their own income streams" ON public.income_streams;
DROP POLICY IF EXISTS "Users can insert their own income streams" ON public.income_streams;
DROP POLICY IF EXISTS "Users can update their own income streams" ON public.income_streams;
DROP POLICY IF EXISTS "Users can delete their own income streams" ON public.income_streams;

CREATE POLICY "Users can view their own income streams" ON public.income_streams FOR SELECT TO authenticated USING (auth.uid()
= user_id);
CREATE POLICY "Users can insert their own income streams" ON public.income_streams FOR INSERT TO authenticated WITH CHECK
(auth.uid() = user_id);
CREATE POLICY "Users can update their own income streams" ON public.income_streams FOR UPDATE TO authenticated USING
(auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete their own income streams" ON public.income_streams FOR DELETE TO authenticated USING
```

```
(auth.uid() = user_id);

-- 3. Grants
GRANT ALL ON public.income_streams TO authenticated;
GRANT ALL ON public.income_streams TO service_role;

-- 4. Indexes
CREATE INDEX IF NOT EXISTS income_streams_user_id_idx ON public.income_streams(user_id);
CREATE INDEX IF NOT EXISTS income_streams_start_date_idx ON public.income_streams(start_date);
```

FILE: supabase\migrations\20260518000000_goal_system_v1.sql

```
-- AXIOM GOAL SYSTEM (v1.0)
-- Establishing deterministic awareness of strategic financial direction.

-- 1. Financial Goals Table
CREATE TABLE IF NOT EXISTS public.financial_goals (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    goal_name TEXT NOT NULL,
    goal_type TEXT NOT NULL,
    target_amount NUMERIC(15, 2) NOT NULL,
    current_progress NUMERIC(15, 2) DEFAULT 0.00 NOT NULL,
    target_date DATE,
    priority TEXT DEFAULT 'medium' NOT NULL,
    status TEXT DEFAULT 'active' NOT NULL,
    linked_account_ids UUID[] DEFAULT '{}'::UUID[] NOT NULL,
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,

    CONSTRAINT financial_goals_goal_type_check CHECK (
        goal_type IN (
            'emergency_fund',
            'retirement',
            'home_purchase',
            'investment_target',
            'business_runway',
            'education',
            'debt_payoff',
            'wealth_preservation',
            'other'
        )
    ),
    CONSTRAINT financial_goals_priority_check CHECK (
        priority IN ('critical', 'high', 'medium', 'low')
    ),
    CONSTRAINT financial_goals_status_check CHECK (
        status IN ('active', 'paused', 'achieved', 'archived')
    ),
    CONSTRAINT financial_goals_target_amount_positive CHECK (target_amount > 0)
);

-- 2. Security Setup (RLS)
ALTER TABLE public.financial_goals ENABLE ROW LEVEL SECURITY;

-- Ensure users can only interact with their own goals
DROP POLICY IF EXISTS "Users can view their own goals" ON public.financial_goals;
DROP POLICY IF EXISTS "Users can insert their own goals" ON public.financial_goals;
DROP POLICY IF EXISTS "Users can update their own goals" ON public.financial_goals;
DROP POLICY IF EXISTS "Users can delete their own goals" ON public.financial_goals;

CREATE POLICY "Users can view their own goals" ON public.financial_goals FOR SELECT TO authenticated USING (auth.uid() =
```

```
user_id);
CREATE POLICY "Users can insert their own goals" ON public.financial_goals FOR INSERT TO authenticated WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own goals" ON public.financial_goals FOR UPDATE TO authenticated USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete their own goals" ON public.financial_goals FOR DELETE TO authenticated USING (auth.uid() = user_id);

-- 3. Grants
GRANT ALL ON public.financial_goals TO authenticated;
GRANT ALL ON public.financial_goals TO service_role;

-- 4. Indexes
CREATE INDEX IF NOT EXISTS financial_goals_user_id_idx ON public.financial_goals(user_id);
CREATE INDEX IF NOT EXISTS financial_goals_status_idx ON public.financial_goals(status);
CREATE INDEX IF NOT EXISTS financial_goals_priority_idx ON public.financial_goals(priority);
```

FILE: supabase\migrations\20260519000000_sync_deployment_balances.sql

```
-- SYNC DEPLOYMENT BALANCES (v1.0)
-- Automated synchronization between deployments and account balances.
-- Goal: Ensure zero silent drift when capital is deployed from a specific account.

-- 1. Trigger Function to Update Account Balance
CREATE OR REPLACE FUNCTION public.sync_deployment_to_account_balance()
RETURNS TRIGGER AS $$
BEGIN
    -- Only proceed if an account_id is linked to the deployment
    IF NEW.account_id IS NOT NULL THEN
        -- Decrement the balance of the linked account
        UPDATE public.accounts
        SET
            current_balance = current_balance - NEW.amount,
            updated_at = now()
        WHERE id = NEW.account_id;

        -- Logic Safety Check: In Axiom, we allow negative balances (overdrafts),
        -- but we must ensure the account actually exists.
        IF NOT FOUND THEN
            RAISE WARNING 'Deployment % linked to non-existent account %', NEW.id, NEW.account_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Attach Trigger to deployments table
DROP TRIGGER IF EXISTS tr_sync_deployment_balance ON public.deployments;
CREATE TRIGGER tr_sync_deployment_balance
AFTER INSERT ON public.deployments
FOR EACH ROW
EXECUTE FUNCTION public.sync_deployment_to_account_balance();

-- 3. Documentation
COMMENT ON FUNCTION public.sync_deployment_to_account_balance() IS 'Automatically decrements account balance when a deployment is registered against an account.';
```

FILE: supabase\migrations\20260519000001_defensive_deployment_sync.sql

```
-- DEFENSIVE DEPLOYMENT SYNC (v1.1)
```

Axiom Repository Audit

```
-- Full lifecycle synchronization between deployments and account balances.
-- Goal: Prevent silent data corruption by handling INSERT, UPDATE, and DELETE.

-- 1. Upgraded Trigger Function
CREATE OR REPLACE FUNCTION public.handle_capital_deployment_lifecycle()
RETURNS TRIGGER AS $$
BEGIN
    -- CASE: DELETE - Restore the original amount to the account
    IF (TG_OP = 'DELETE') THEN
        IF OLD.account_id IS NOT NULL THEN
            UPDATE public.accounts
            SET
                current_balance = current_balance + OLD.amount,
                updated_at = now()
            WHERE id = OLD.account_id;
        END IF;
        RETURN OLD;

    -- CASE: UPDATE - Handle amount changes and account transfers
    ELSIF (TG_OP = 'UPDATE') THEN
        -- Scenario A: Account remains the same, amount changes
        IF (OLD.account_id = NEW.account_id) THEN
            IF (OLD.account_id IS NOT NULL AND OLD.amount != NEW.amount) THEN
                UPDATE public.accounts
                SET
                    current_balance = current_balance + (OLD.amount - NEW.amount),
                    updated_at = now()
                WHERE id = OLD.account_id;
            END IF;

        -- Scenario B: Account itself changed (Transfer)
        ELSE
            -- Restore original amount to the OLD account
            IF OLD.account_id IS NOT NULL THEN
                UPDATE public.accounts
                SET
                    current_balance = current_balance + OLD.amount,
                    updated_at = now()
                WHERE id = OLD.account_id;
            END IF;

            -- Subtract NEW amount from the NEW account
            IF NEW.account_id IS NOT NULL THEN
                UPDATE public.accounts
                SET
                    current_balance = current_balance - NEW.amount,
                    updated_at = now()
                WHERE id = NEW.account_id;
            END IF;
        END IF;
        RETURN NEW;

    -- CASE: INSERT - Standard deduction (Maintain existing logic)
    ELSIF (TG_OP = 'INSERT') THEN
        IF NEW.account_id IS NOT NULL THEN
            UPDATE public.accounts
            SET
                current_balance = current_balance - NEW.amount,
                updated_at = now()
            WHERE id = NEW.account_id;
        END IF;
        RETURN NEW;
    END IF;

    RETURN NULL;
```

```
END;
$$ LANGUAGE plpgsql;

-- 2. Attach Lifecycle Trigger
-- Drop the old restricted trigger if it exists
DROP TRIGGER IF EXISTS tr_sync_deployment_balance ON public.deployments;
DROP TRIGGER IF EXISTS tr_lifecycle_deployment_balance ON public.deployments;

CREATE TRIGGER tr_lifecycle_deployment_balance
AFTER INSERT OR UPDATE OR DELETE ON public.deployments
FOR EACH ROW
EXECUTE FUNCTION public.handle_capital_deployment_lifecycle();

-- 3. Documentation
COMMENT ON FUNCTION public.handle_capital_deployment_lifecycle() IS 'Ensures zero-drift financial integrity by syncing deployment changes (Insert/Update/Delete) with linked account balances.';
```

FILE: supabase\migrations\20260520000000_strategic_objectives_v1.sql

```
-- STRATEGIC OBJECTIVES ENGINE v1 (v1.0)
-- Establishing the directional financial intelligence layer.

-- 1. Strategic Objectives Table
CREATE TABLE IF NOT EXISTS public.strategic_objectives (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    objective_name TEXT NOT NULL,
    objective_type TEXT NOT NULL,
    target_amount NUMERIC(15, 2) NOT NULL,
    current_amount NUMERIC(15, 2) DEFAULT 0.00 NOT NULL,
    target_date DATE,
    priority_level TEXT NOT NULL,
    status TEXT NOT NULL,
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,

    CONSTRAINT strategic_objectives_type_check CHECK (
        objective_type IN (
            'emergency_reserve',
            'liquidity',
            'debt_elimination',
            'investment',
            'retirement',
            'education',
            'acquisition',
            'custom'
        )
    ),
    CONSTRAINT strategic_objectives_priority_check CHECK (
        priority_level IN ('critical', 'high', 'moderate', 'low')
    ),
    CONSTRAINT strategic_objectives_status_check CHECK (
        status IN ('active', 'paused', 'completed', 'abandoned')
    )
);

-- 2. Security Setup (RLS)
ALTER TABLE public.strategic_objectives ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS "Users can view their own objectives" ON public.strategic_objectives;
DROP POLICY IF EXISTS "Users can insert their own objectives" ON public.strategic_objectives;
DROP POLICY IF EXISTS "Users can update their own objectives" ON public.strategic_objectives;
```

```
DROP POLICY IF EXISTS "Users can delete their own objectives" ON public.strategic_objectives;

CREATE POLICY "Users can view their own objectives" ON public.strategic_objectives FOR SELECT TO authenticated USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own objectives" ON public.strategic_objectives FOR INSERT TO authenticated WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own objectives" ON public.strategic_objectives FOR UPDATE TO authenticated USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete their own objectives" ON public.strategic_objectives FOR DELETE TO authenticated USING (auth.uid() = user_id);

-- 3. Grants
GRANT ALL ON public.strategic_objectives TO authenticated;
GRANT ALL ON public.strategic_objectives TO service_role;

-- 4. Indexes
CREATE INDEX IF NOT EXISTS strategic_objectives_user_id_idx ON public.strategic_objectives(user_id);
CREATE INDEX IF NOT EXISTS strategic_objectives_priority_idx ON public.strategic_objectives(priority_level);

-- 5. Updated At Trigger
CREATE OR REPLACE FUNCTION public.handle_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tr_strategic_objectives_updated_at ON public.strategic_objectives;
CREATE TRIGGER tr_strategic_objectives_updated_at
BEFORE UPDATE ON public.strategic_objectives
FOR EACH ROW
EXECUTE FUNCTION public.handle_updated_at();

-- 6. Commenting
COMMENT ON TABLE public.strategic_objectives IS 'Directional financial objectives defining the users strategic intentions.';
```

FILE: supabase\migrations\20260521000000_operational_baseline_v1.sql

```
-- OPERATIONAL BASELINE ENGINE (v1.0)
-- Tracking recurring structural outflows and systemic allocations.

-- 1. Operational Baseline Table
CREATE TABLE IF NOT EXISTS public.operational_baseline (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    title TEXT NOT NULL,
    amount NUMERIC(15, 2) NOT NULL,
    category TEXT NOT NULL CHECK (category IN ('Asset', 'Skill', 'Leverage', 'Experience', 'Leakage')),
    cadence TEXT NOT NULL DEFAULT 'monthly' CHECK (cadence IN ('daily', 'weekly', 'biweekly', 'monthly', 'quarterly', 'yearly')),
    baseline_type TEXT NOT NULL DEFAULT 'expense' CHECK (baseline_type IN ('expense', 'allocation')),
    notes TEXT,
    is_active BOOLEAN DEFAULT true NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

-- 2. Security Setup (RLS)
ALTER TABLE public.operational_baseline ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS "Users can view their own baseline" ON public.operational_baseline;
DROP POLICY IF EXISTS "Users can insert their own baseline" ON public.operational_baseline;
```

```
DROP POLICY IF EXISTS "Users can update their own baseline" ON public.operational_baseline;
DROP POLICY IF EXISTS "Users can delete their own baseline" ON public.operational_baseline;

CREATE POLICY "Users can view their own baseline" ON public.operational_baseline FOR SELECT TO authenticated USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own baseline" ON public.operational_baseline FOR INSERT TO authenticated WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own baseline" ON public.operational_baseline FOR UPDATE TO authenticated USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete their own baseline" ON public.operational_baseline FOR DELETE TO authenticated USING (auth.uid() = user_id);

-- 3. Grants
GRANT ALL ON public.operational_baseline TO authenticated;
GRANT ALL ON public.operational_baseline TO service_role;

-- 4. Indexes
CREATE INDEX IF NOT EXISTS operational_baseline_user_id_idx ON public.operational_baseline(user_id);

-- 5. Updated At Trigger
DROP TRIGGER IF EXISTS tr_operational_baseline_updated_at ON public.operational_baseline;
CREATE TRIGGER tr_operational_baseline_updated_at
BEFORE UPDATE ON public.operational_baseline
FOR EACH ROW
EXECUTE FUNCTION public.handle_updated_at();

-- 6. Commenting
COMMENT ON TABLE public.operational_baseline IS 'Recurring structural outflows defining the users baseline burn and automated allocations.';
```

FILE: supabase\migrations\20260521000001_expand_taxonomy_maintenance.sql

```
-- TAXONOMY EXPANSION: MAINTENANCE (v1.1)
-- Introducing the 'Maintenance' category for structural survival requirements.

-- 1. Update Deployments Constraint
ALTER TABLE public.deployments DROP CONSTRAINT IF EXISTS deployments_category_valid_check;
ALTER TABLE public.deployments
ADD CONSTRAINT deployments_category_valid_check
CHECK (
    category IS NOT NULL
    AND category IN ('Asset', 'Skill', 'Leverage', 'Experience', 'Maintenance', 'Leakage')
) NOT VALID;
ALTER TABLE public.deployments VALIDATE CONSTRAINT deployments_category_valid_check;

-- 2. Update Operational Baseline Constraint
ALTER TABLE public.operational_baseline DROP CONSTRAINT IF EXISTS operational_baseline_category_check;
ALTER TABLE public.operational_baseline
ADD CONSTRAINT operational_baseline_category_check
CHECK (
    category IN ('Asset', 'Skill', 'Leverage', 'Experience', 'Maintenance', 'Leakage')
);

-- 3. Commenting
COMMENT ON COLUMN public.deployments.category IS 'Designated category of capital deployment. Expanded to include Maintenance (structural survival).';
COMMENT ON COLUMN public.operational_baseline.category IS 'Category of recurring outflow. Maintenance identifies survival-layer structural flows.';
```

FILE: supabase\migrations\20260529000000_enforce_strict_rls.sql

Axiom Repository Audit

```
-- Security Audit Fix ? Phase 2: Row Level Security (RLS) Enforcement
-- This migration ensures that PostgreSQL mathematically guarantees user data isolation.

-- 1. Table: deployments
ALTER TABLE public.deployments ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.deployments;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.deployments;
DROP POLICY IF EXISTS "Users can update their own data" ON public.deployments;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.deployments;

CREATE POLICY "Users can view their own data" ON public.deployments FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.deployments FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.deployments FOR UPDATE USING (auth.uid() = user_id) WITH CHECK
(auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.deployments FOR DELETE USING (auth.uid() = user_id);

-- 2. Table: accounts
ALTER TABLE public.accounts ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.accounts;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.accounts;
DROP POLICY IF EXISTS "Users can update their own data" ON public.accounts;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.accounts;

CREATE POLICY "Users can view their own data" ON public.accounts FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.accounts FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.accounts FOR UPDATE USING (auth.uid() = user_id) WITH CHECK
(auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.accounts FOR DELETE USING (auth.uid() = user_id);

-- 3. Table: liabilities
ALTER TABLE public.liabilities ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.liabilities;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.liabilities;
DROP POLICY IF EXISTS "Users can update their own data" ON public.liabilities;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.liabilities;

CREATE POLICY "Users can view their own data" ON public.liabilities FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.liabilities FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.liabilities FOR UPDATE USING (auth.uid() = user_id) WITH CHECK
(auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.liabilities FOR DELETE USING (auth.uid() = user_id);

-- 4. Table: income_streams
ALTER TABLE public.income_streams ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.income_streams;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.income_streams;
DROP POLICY IF EXISTS "Users can update their own data" ON public.income_streams;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.income_streams;

CREATE POLICY "Users can view their own data" ON public.income_streams FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.income_streams FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.income_streams FOR UPDATE USING (auth.uid() = user_id) WITH CHECK
(auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.income_streams FOR DELETE USING (auth.uid() = user_id);

-- 5. Table: financial_goals
ALTER TABLE public.financial_goals ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.financial_goals;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.financial_goals;
DROP POLICY IF EXISTS "Users can update their own data" ON public.financial_goals;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.financial_goals;

CREATE POLICY "Users can view their own data" ON public.financial_goals FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.financial_goals FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.financial_goals FOR UPDATE USING (auth.uid() = user_id) WITH CHECK
```

```
(auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.financial_goals FOR DELETE USING (auth.uid() = user_id);

-- 6. Table: strategic_objectives
ALTER TABLE public.strategic_objectives ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.strategic_objectives;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.strategic_objectives;
DROP POLICY IF EXISTS "Users can update their own data" ON public.strategic_objectives;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.strategic_objectives;

CREATE POLICY "Users can view their own data" ON public.strategic_objectives FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.strategic_objectives FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.strategic_objectives FOR UPDATE USING (auth.uid() = user_id) WITH
CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.strategic_objectives FOR DELETE USING (auth.uid() = user_id);

-- 7. Table: user_settings
ALTER TABLE public.user_settings ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.user_settings;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.user_settings;
DROP POLICY IF EXISTS "Users can update their own data" ON public.user_settings;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.user_settings;

CREATE POLICY "Users can view their own data" ON public.user_settings FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.user_settings FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.user_settings FOR UPDATE USING (auth.uid() = user_id) WITH CHECK
(auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.user_settings FOR DELETE USING (auth.uid() = user_id);

-- 8. Table: operational_baseline
ALTER TABLE public.operational_baseline ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.operational_baseline;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.operational_baseline;
DROP POLICY IF EXISTS "Users can update their own data" ON public.operational_baseline;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.operational_baseline;

CREATE POLICY "Users can view their own data" ON public.operational_baseline FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.operational_baseline FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.operational_baseline FOR UPDATE USING (auth.uid() = user_id) WITH
CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.operational_baseline FOR DELETE USING (auth.uid() = user_id);

-- 9. Table: kairos_insights
ALTER TABLE public.kairos_insights ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "Users can view their own data" ON public.kairos_insights;
DROP POLICY IF EXISTS "Users can insert their own data" ON public.kairos_insights;
DROP POLICY IF EXISTS "Users can update their own data" ON public.kairos_insights;
DROP POLICY IF EXISTS "Users can delete their own data" ON public.kairos_insights;

CREATE POLICY "Users can view their own data" ON public.kairos_insights FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert their own data" ON public.kairos_insights FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can update their own data" ON public.kairos_insights FOR UPDATE USING (auth.uid() = user_id) WITH CHECK
(auth.uid() = user_id);
CREATE POLICY "Users can delete their own data" ON public.kairos_insights FOR DELETE USING (auth.uid() = user_id);
```

FILE: supabase\migrations\20260529000001_add_soft_deletes.sql

```
-- Phase 1: Ledger Immutability (Soft Deletes)
-- Adds deleted_at column and index to core transactional tables.

-- Accounts
ALTER TABLE public.accounts ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMP WITH TIME ZONE DEFAULT NULL;
CREATE INDEX IF NOT EXISTS idx_accounts_deleted_at ON public.accounts(deleted_at);
```

```
-- Deployments
ALTER TABLE public.deployments ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMP WITH TIME ZONE DEFAULT NULL;
CREATE INDEX IF NOT EXISTS idx_deployments_deleted_at ON public.deployments(deleted_at);

-- Financial Goals
ALTER TABLE public.financial_goals ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMP WITH TIME ZONE DEFAULT NULL;
CREATE INDEX IF NOT EXISTS idx_financial_goals_deleted_at ON public.financial_goals(deleted_at);

-- Income Streams
ALTER TABLE public.income_streams ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMP WITH TIME ZONE DEFAULT NULL;
CREATE INDEX IF NOT EXISTS idx_income_streams_deleted_at ON public.income_streams(deleted_at);

-- Liabilities
ALTER TABLE public.liabilities ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMP WITH TIME ZONE DEFAULT NULL;
CREATE INDEX IF NOT EXISTS idx_liabilities_deleted_at ON public.liabilities(deleted_at);

-- Strategic Objectives
ALTER TABLE public.strategic_objectives ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMP WITH TIME ZONE DEFAULT NULL;
CREATE INDEX IF NOT EXISTS idx_strategic_objectives_deleted_at ON public.strategic_objectives(deleted_at);

-- Operational Baseline
ALTER TABLE public.operational_baseline ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMP WITH TIME ZONE DEFAULT NULL;
CREATE INDEX IF NOT EXISTS idx_operational_baseline_deleted_at ON public.operational_baseline(deleted_at);
```

FILE: supabase\migrations\20260529000002_add_telemetry_to_insights.sql

```
-- Phase 1 Observability: Instrumenting kairos_insights for telemetry
-- Adds rule-level tracking and performance metrics to the insights table.

ALTER TABLE public.kairos_insights ADD COLUMN IF NOT EXISTS rule_id TEXT;
ALTER TABLE public.kairos_insights ADD COLUMN IF NOT EXISTS evaluation_time_ms NUMERIC;
ALTER TABLE public.kairos_insights ADD COLUMN IF NOT EXISTS severity TEXT;

-- v1.2 Upgrade: Ensure existing business logic columns are nullable
-- to support non-insight evaluation events (telemetry-only rows).
ALTER TABLE public.kairos_insights ALTER COLUMN type DROP NOT NULL;
ALTER TABLE public.kairos_insights ALTER COLUMN category DROP NOT NULL;
ALTER TABLE public.kairos_insights ALTER COLUMN confidence DROP NOT NULL;
ALTER TABLE public.kairos_insights ALTER COLUMN message DROP NOT NULL;

-- Indexing for observability performance
CREATE INDEX IF NOT EXISTS insights_rule_id_idx ON public.kairos_insights(rule_id);
CREATE INDEX IF NOT EXISTS insights_severity_idx ON public.kairos_insights(severity);
```

FILE: supabase\migrations\20260601000000_add_income_execution_day.sql

```
-- Add execution_day to income_streams
ALTER TABLE income_streams ADD COLUMN execution_day smallint;
```

FILE: supabase\migrations\20260602000000_add_income_last_executed.sql

```
-- Add last_executed_at to income_streams
ALTER TABLE income_streams ADD COLUMN last_executed_at timestamp with time zone;
```

FILE: supabase\migrations\20260606000000_refactor_liabilities_cadence.sql

```
-- REFACTOR LIABILITIES: CADENCE-BASED PAYMENTS (2026-06-06)
-- Removing static minimum payment in favor of deterministic cadence schedules.

-- 1. Remove legacy minimum payment column
ALTER TABLE public.liabilities DROP COLUMN IF EXISTS minimum_payment;

-- 2. Add cadence-based scheduling columns
ALTER TABLE public.liabilities ADD COLUMN IF NOT EXISTS is_paid_in_cadences BOOLEAN DEFAULT false NOT NULL;
ALTER TABLE public.liabilities ADD COLUMN IF NOT EXISTS cadence TEXT CHECK (cadence IN ('weekly', 'monthly'));
ALTER TABLE public.liabilities ADD COLUMN IF NOT EXISTS cadence_day_date TEXT;

-- 3. Update comments for clarity
COMMENT ON COLUMN public.liabilities.is_paid_in_cadences IS 'Flag to determine if this liability follows a regular payment schedule.';
COMMENT ON COLUMN public.liabilities.cadence IS 'The frequency of payment: weekly or monthly.';
COMMENT ON COLUMN public.liabilities.cadence_day_date IS 'The specific day of the week (e.g., Monday) or date of the month (e.g., 15) for payments.';
```

FILE: supabase\migrations\20260606000001_track_liability_payments.sql

```
-- TRACK LIABILITY PAYMENTS (2026-06-06)
-- Adding last_executed_at to liabilities for verification tracking.

ALTER TABLE public.liabilities ADD COLUMN IF NOT EXISTS last_executed_at TIMESTAMPTZ;

COMMENT ON COLUMN public.liabilities.last_executed_at IS 'The timestamp of the last verified payment for this liability.';
```

FILE: supabase\migrations\20260606000002_add_cadence_amount.sql

```
-- ADD CADENCE AMOUNT TO LIABILITIES (2026-06-06)
-- Adding cadence_amount to track expected payment values.

ALTER TABLE public.liabilities ADD COLUMN IF NOT EXISTS cadence_amount NUMERIC(15, 2);

COMMENT ON COLUMN public.liabilities.cadence_amount IS 'The expected payment amount for each cadence cycle.';
```
